frontend/src/main.tsx

```
/// <reference types="vite-plugin-pwa/client" />
import React from 'react';
import ReactDOM from 'react-dom/client';
import { registerSW } from 'virtual:pwa-register';
import App from './App';
import './index.css';

// Register service worker for PWA
registerSW({ immediate: true })

const rootElement = document.getElementById('root')

if (!rootElement) {
  throw new Error("❌ Root element #root not found in index.html")
}

ReactDOM.createRoot(rootElement).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

frontend/src/SynthesisLayout.tsx

```
import { motion } from 'framer-motion';
import React, { useState } from 'react';
import {
    FaBell,
    FaBox,
    FaChartLine,
    FaChevronDown,
    FaChevronRight,
    FaClipboardList,
    FaCloudDownloadAlt,
    FaCog,
    FaExchangeAlt,
    FaFileInvoiceDollar,
    FaHistory,
    FaHome,
    FaLayerGroup as FaLayer,
    FaMoneyBillWave,
    FaSearch,
    FaUsers,
    FaWallet
} from 'react-icons/fa';
import './SynthesisLayout.css';

interface LayoutProps {
  children: React.ReactNode;
  activeItem?: string;
}

const SynthesisLayout: React.FC<LayoutProps> = ({ children, activeItem = 'Command Center' }) => {
  const [fiscalOpen, setFiscalOpen] = useState(true);

  const sidebarItems = [
    { label: 'Command Center', icon: <FaHome />, section: 'MAIN' },
    { label: 'Revenue Stream', icon: <FaMoneyBillWave />, section: 'OPERATIONS' },
    { label: 'Procurement', icon: <FaWallet />, section: 'OPERATIONS' },
    { label: 'Inventory Matrix', icon: <FaBox />, section: 'OPERATIONS' },
    {
      label: 'Fiscal Logic',
      icon: <FaLayer />,
      section: 'SYSTEM',
      isExpandable: true,
      subItems: [
        { label: 'Finance Hub', icon: <FaFileInvoiceDollar /> },
        { label: 'Loan Portfolio', icon: <FaWallet /> },
        { label: 'Cash Receipts', icon: <FaMoneyBillWave /> },
        { label: 'Auto Reconcile', icon: <FaExchangeAlt /> },
        { label: 'Financial Intel', icon: <FaChartLine /> },
        { label: 'Global Ledgers', icon: <FaClipboardList /> },
        { label: 'Automated Log', icon: <FaHistory /> },
      ]
    },
    { label: 'Workforce', icon: <FaUsers />, section: 'SYSTEM' },
  ];



  return (
    <div className="synthesis-layout">
      {/* Sidebar */}
      <motion.aside
        initial={{ x: -280 }}
        animate={{ x: 0 }}
        className="synthesis-sidebar"
      >
        <div className="sidebar-brand">
          <div className="sidebar-avatar" style={{ background: '#3B82F6', width: 32, height: 32, borderRadius: 8, display: 'flex', alignItems: 'center'
          <h2>ENTP SYNTHESIS</h2>
        </div>

        <nav className="sidebar-nav">
          <div className="nav-section-label">COMMAND</div>
          <a className={`nav-item ${activeItem === 'Command Center' ? 'active' : ''}`}>
             <FaHome className="nav-item-icon" /> Command Center
          </a>

          <div className="nav-section-label">OPERATIONS</div>
          <a className="nav-item">
             <FaMoneyBillWave className="nav-item-icon" /> Revenue Stream
          </a>
          <a className="nav-item">
             <FaWallet className="nav-item-icon" /> Procurement
          </a>
          <a className="nav-item">
             <FaBox className="nav-item-icon" /> Inventory Matrix
          </a>

          <div className="nav-section-label">SYSTEM ARCHITECTURE</div>
          <div className="expandable-nav">
            <div className="nav-item" onClick={() => setFiscalOpen(!fiscalOpen)}>
              <FaLayer className="nav-item-icon" /> Fiscal Logic
              {fiscalOpen ? <FaChevronDown className="nav-item-chevron" /> : <FaChevronRight className="nav-item-chevron" />}
            </div>
            {fiscalOpen && (
              <motion.div
                initial={{ height: 0, opacity: 0 }}
                animate={{ height: 'auto', opacity: 1 }}
                className="sub-nav"
                style={{ marginLeft: '1.5rem', borderLeft: '1px solid rgba(255,255,255,0.1)' }}
              >
                {[
                  'Finance Hub', 'Loan Portfolio', 'Cash Receipts',
                  'Auto Reconcile', 'Financial Intel', 'Global Ledgers', 'Automated Log'
                ].map(sub => (
                  <a key={sub} className="nav-item" style={{ fontSize: '0.8rem', padding: '0.5rem 0.75rem' }}>
                    {sub}
                  </a>
```

```
                ))}
              </motion.div>
            )}
          </div>
          <a className="nav-item">
             <FaUsers className="nav-item-icon" /> Workforce
          </a>
        </nav>

        <div className="sidebar-footer" style={{ marginTop: 'auto', padding: '1rem 1.5rem', borderTop: '1px solid rgba(255,255,255,0.05)' }}>
            <div className="nav-item">
                <FaCog /> Settings
            </div>
        </div>
      </motion.aside>

      {/* Main Content */}
      <main className="synthesis-main">
        {/* Topbar */}
        <header className="synthesis-topbar">
          <div className="topbar-left">
            <div className="branch-selector">
                <div style={{ width: 10, height: 10, background: '#10B981', borderRadius: '50%' }}></div>
                DEMO BRANCH 01
                <FaChevronDown style={{ fontSize: '0.7rem' }} />
            </div>
            <div className="topbar-search">
              <FaSearch className="search-icon" />
              <input type="text" placeholder="Search Fiscal Logic, Revenue, Workforce..." />
            </div>
          </div>

          <div className="topbar-right">
            <div className="icon-btn">
                <FaBell />
                <span className="notification-badge"></span>
            </div>
            <div className="icon-btn">
                <FaCloudDownloadAlt />
            </div>
            <div className="profile-trigger">
                <div className="profile-avatar">AD</div>
                <div className="profile-info" style={{ display: 'flex', flexDirection: 'column' }}>
                    <span style={{ fontSize: '0.85rem', fontWeight: 600 }}>Admin User</span>
                    <span style={{ fontSize: '0.7rem', color: '#64748B' }}>System Administrator</span>
                </div>
                <FaChevronDown style={{ fontSize: '0.7rem', color: '#94a3b8' }} />
            </div>
          </div>
        </header>

        {/* Dynamic Content */}
        <div className="content-scrollable">
          {children}
        </div>
      </main>
    </div>
  );
};

export default SynthesisLayout;
```

frontend/src/SynthesisDashboard.tsx

```
import { motion } from 'framer-motion';
import React from 'react';
import { FaArrowUp, FaDownload, FaEllipsisV, FaFilter } from 'react-icons/fa';
import { Area, AreaChart, Bar, BarChart, CartesianGrid, ResponsiveContainer, Tooltip, XAxis, YAxis } from 'recharts';
import SynthesisLayout from './SynthesisLayout';

const cashFlowData = [
  { name: 'Mon', inflow: 4000, outflow: 2400 },
  { name: 'Tue', inflow: 3000, outflow: 1398 },
  { name: 'Wed', inflow: 2000, outflow: 9800 },
  { name: 'Thu', inflow: 2780, outflow: 3908 },
  { name: 'Fri', inflow: 1890, outflow: 4800 },
  { name: 'Sat', inflow: 2390, outflow: 3800 },
  { name: 'Sun', inflow: 3490, outflow: 4300 },
];

const expenseRevenueData = [
  { month: 'Jan', revenue: 45000, expense: 32000 },
  { month: 'Feb', revenue: 52000, expense: 28000 },
  { month: 'Mar', revenue: 48000, expense: 35000 },
  { month: 'Apr', revenue: 61000, expense: 42000 },
  { month: 'May', revenue: 55000, expense: 39000 },
  { month: 'Jun', revenue: 67000, expense: 45000 },
];

const SynthesisDashboard: React.FC = () => {
  return (
    <SynthesisLayout activeItem="Command Center">
      <div className="fade-in-up">
        {/* Header Section */}
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '2rem' }}>
          <div>
            <h1 style={{ fontSize: '1.8rem', fontWeight: 800, color: '#1E293B' }}>Finance Command Center</h1>
            <p style={{ color: '#64748B', fontSize: '0.95rem' }}>Real-time fiscal logic and operational intelligence.</p>
          </div>
          <div style={{ display: 'flex', gap: '1rem' }}>
            <button style={{ padding: '0.6rem 1.2rem', background: '#fff', border: '1px solid #e2e8f0', borderRadius: '10px', fontWeight: 600, fontSize
                <FaFilter size={12} /> Filter Period
            </button>
            <button style={{ padding: '0.6rem 1.2rem', background: '#1E2A78', color: '#fff', border: 'none', borderRadius: '10px', fontWeight: 600, fon
                <FaDownload size={12} /> Export Intelligence
            </button>
          </div>
        </div>

        {/* KPI Grid */}
        <div className="kpi-grid">
          <motion.div whileHover={{ y: -5 }} className="kpi-card">
            <div className="kpi-label">LIQUID CASH</div>
            <div className="kpi-value">$248,590.00</div>
            <div className="kpi-trend trend-up">
              <FaArrowUp /> +12.4% <span style={{ color: '#94a3b8', fontWeight: 400 }}>vs last month</span>
            </div>
          </motion.div>

          <motion.div whileHover={{ y: -5 }} className="kpi-card">
            <div className="kpi-label">AGGREGATED BANK BALANCE</div>
            <div className="kpi-value">$1,420,000.00</div>
            <div class="kpi-trend trend-up">
              <FaArrowUp /> +3.2% <span style={{ color: '#94a3b8', fontWeight: 400 }}>5 internal accounts</span>
            </div>
          </motion.div>

          <motion.div whileHover={{ y: -5 }} className="kpi-card">
            <div class="kpi-label">NET PROFIT (YTD)</div>
            <div class="kpi-value" style={{ color: '#10B981' }}>$682,400.00</div>
            <div class="kpi-trend trend-up">
              <FaArrowUp /> +24% <span style={{ color: '#94a3b8', fontWeight: 400 }}>Projected: $1.2M</span>
            </div>
          </motion.div>
        </div>

        {/* Analytics Grid */}
        <div class="dashboard-grid">
          {/* Main Chart */}
          <div class="widget-card">
            <div class="widget-header">
              <h3 class="widget-title">Fiscal Liquidity (Cash Flow)</h3>
              <div class="widget-actions">
                <FaEllipsisV style={{ color: '#94a3b8', cursor: 'pointer' }} />
              </div>
            </div>
            <div style={{ width: '100%', height: 350 }}>
              <ResponsiveContainer>
                <AreaChart data={cashFlowData}>
                  <defs>
                    <linearGradient id="colorIn" x1="0" y1="0" x2="0" y2="1">
                      <stop offset="5%" stopColor="#3B82F6" stopOpacity={0.1}/>
                      <stop offset="95%" stopColor="#3B82F6" stopOpacity={0}/>
                    </linearGradient>
                  </defs>
                  <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#f1f5f9" />
                  <XAxis dataKey="name" axisLine={false} tickLine={false} tick={{ fontSize: 12, fill: '#94a3b8' }} dy={10} />
                  <YAxis hide />
                  <Tooltip
                    contentStyle={{ borderRadius: '12px', border: 'none', boxShadow: '0 10px 15px -3px rgba(0,0,0,0.1)' }}
                  />
                  <Area type="monotone" dataKey="inflow" stroke="#3B82F6" strokeWidth={3} fillOpacity={1} fill="url(#colorIn)" />
                  <Area type="monotone" dataKey="outflow" stroke="#94a3b8" strokeWidth={2} fill="transparent" strokeDasharray="5 5" />
                </AreaChart>
              </ResponsiveContainer>
            </div>
          </div>

          {/* Secondary Stats */}
          <div style={{ display: 'flex', flexDirection: 'column', gap: '1.5rem' }}>
```

```
          <div className="widget-card">
            <h3 className="widget-title" style={{ marginBottom: '1rem' }}>Revenue vs Expense</h3>
            <div style={{ width: '100%', height: 180 }}>
              <ResponsiveContainer>
                <BarChart data={expenseRevenueData}>
                  <Bar dataKey="revenue" fill="#1E2A78" radius={[4, 4, 0, 0]} />
                  <Bar dataKey="expense" fill="#E2E8F0" radius={[4, 4, 0, 0]} />
                </BarChart>
              </ResponsiveContainer>
            </div>
          </div>

          <div className="widget-card" style={{ background: 'linear-gradient(135deg, #1E2A78 0%, #312E81 100%)', color: '#fff' }}>
              <div className="kpi-label" style={{ color: 'rgba(255,255,255,0.6)' }}>OUTSTANDING RECEIVABLES</div>
              <div className="kpi-value" style={{ color: '#fff', fontSize: '1.5rem' }}>$84,320.00</div>
              <div style={{ height: '1px', background: 'rgba(255,255,255,0.1)', margin: '1rem 0' }}></div>
              <div className="kpi-label" style={{ color: 'rgba(255,255,255,0.6)' }}>PENDING PAYABLES</div>
              <div className="kpi-value" style={{ color: '#fff', fontSize: '1.5rem' }}>$42,150.00</div>
          </div>
        </div>
      </div>

      {/* Transactions Table */}
      <div className="widget-card">
        <div className="widget-header">
          <h3 className="widget-title">Recent Automated Transactions</h3>
          <button style={{ color: '#3B82F6', background: 'none', border: 'none', fontWeight: 600, fontSize: '0.85rem', cursor: 'pointer' }}>View All
        </div>
        <table className="premium-table">
          <thead>
            <tr>
              <th>TRANSACTION ID</th>
              <th>ENTITY / ORIGIN</th>
              <th>CATEGORY</th>
              <th>AMOUNT</th>
              <th>STATUS</th>
              <th>FSCAL LOG</th>
            </tr>
          </thead>
          <tbody>
            {[
              { id: 'FT-9021', entity: 'Amazon Web Services', cat: 'Cloud Infra', amt: '-$1,240.00', status: 'SUCCESS' },
              { id: 'FT-9022', entity: 'Global Logistics Corp', cat: 'Shipping', amt: '-$3,500.00', status: 'PENDING' },
              { id: 'FT-9023', entity: 'Stripe Payout', cat: 'Revenue', amt: '+$12,450.00', status: 'SUCCESS' },
              { id: 'FT-9024', entity: 'Employee Payroll (May)', cat: 'Workforce', amt: '-$45,000.00', status: 'SUCCESS' },
              { id: 'FT-9025', entity: 'Office Space Rental', cat: 'Facilities', amt: '-$8,000.00', status: 'ERROR' },
            ].map((tx, idx) => (
              <tr key={idx}>
                <td style={{ fontWeight: 600 }}>{tx.id}</td>
                <td>{tx.entity}</td>
                <td><span style={{ fontSize: '0.8rem', color: '#64748B', background: '#f1f5f9', padding: '0.2rem 0.5rem', borderRadius: '4px' }}>{tx.c
                <td style={{ fontWeight: 700, color: tx.amt.startsWith('+') ? '#10B981' : '#1E293B' }}>{tx.amt}</td>
                <td>
                  <span className={`status-pill status-${tx.status.toLowerCase()}`}>
                      {tx.status}
                  </span>
                </td>
                <td><button style={{ color: '#64748B', background: 'none', border: 'none', cursor: 'pointer' }}><FaDownload size={12} /></button></td
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  </SynthesisLayout>
 );
};

export default SynthesisDashboard;
```

frontend/src/SimpleDashboard.tsx

```
import { AnimatePresence, motion } from 'framer-motion';
import React, { useEffect, useRef, useState } from 'react';
import {
    FaBell,
    FaBox,
    FaChartLine,
    FaChevronDown,
    FaClipboardList,
    FaCog,
    FaDownload,
    FaFileInvoiceDollar,
    FaHome,
    FaMoneyBillWave,
    FaSearch,
    FaSyncAlt,
    FaTimes,
    FaUsers,
    FaWallet,
} from 'react-icons/fa';
import { useNavigate } from 'react-router-dom';
import {
    Area,
    AreaChart,
    Bar,
    BarChart,
    CartesianGrid,
    ResponsiveContainer,
    Tooltip,
    XAxis,
    YAxis,
} from 'recharts';
import './SimpleDashboard.css';

const API = import.meta.env.VITE_API_URL || 'http://localhost:3001/api';

// ─── helpers ─────────────────────────────────────────────────────────────
const fmt = (n: number) =>
    n >= 1_000_000
        ? `₹${(n / 1_000_000).toFixed(2)}M`
        : n >= 1_000
        ? `₹${(n / 1_000).toFixed(1)}K`
        : `₹${n.toFixed(2)}`;

// ─── Types ───────────────────────────────────────────────────────────────
interface KPIs { cash: number; bank: number; profit: number; receivable: number; payable: number; }
interface Activity { id: string | number; description: string; amount: number; type: 'credit' | 'debit'; date: string; status: 'SUCCESS' | 'PENDING' |
interface ChartPoint { name: string; inflow: number; outflow: number }
interface RevPoint  { name: string; revenue: number; expense: number }

// ─── Nav Config ──────────────────────────────────────────────────────────
const NAV_ITEMS = [
    { label: 'Home',         icon: <FaHome />,              path: '/simple',                id: 'home' },
    { label: 'Money In/Out', icon: <FaMoneyBillWave />,     path: '/transactions',          id: 'money' },
    { label: 'Bills',        icon: <FaFileInvoiceDollar />, path: '/invoices',              id: 'bills' },
    { label: 'Items',        icon: <FaBox />,               path: '/products',              id: 'items' },
    { label: 'Finance',      icon: <FaChartLine />,         path: '/finance/dashboard',     id: 'finance' },
    { label: 'People',       icon: <FaUsers />,             path: '/employees',             id: 'people' },
    { label: 'Reports',      icon: <FaClipboardList />,     path: '/admin/reports',         id: 'reports' },
    { label: 'Accounts',     icon: <FaWallet />,            path: '/ledgers',               id: 'accounts' },
];

// ─── Mock Fallback ───────────────────────────────────────────────────────
const MOCK_KPIS: KPIs = { cash: 248590, bank: 1420000, profit: 682400, receivable: 84320, payable: 42150 };
const MOCK_FLOW: ChartPoint[] = [
    { name:'Mon', inflow:52000, outflow:31000 }, { name:'Tue', inflow:47000, outflow:28000 },
    { name:'Wed', inflow:61000, outflow:45000 }, { name:'Thu', inflow:55000, outflow:39000 },
    { name:'Fri', inflow:70000, outflow:42000 }, { name:'Sat', inflow:48000, outflow:25000 },
    { name:'Sun', inflow:39000, outflow:22000 },
];
const MOCK_REV: RevPoint[] = [
    { name:'Jan', revenue:450000, expense:320000 }, { name:'Feb', revenue:520000, expense:280000 },
    { name:'Mar', revenue:480000, expense:350000 }, { name:'Apr', revenue:610000, expense:420000 },
    { name:'May', revenue:550000, expense:390000 }, { name:'Jun', revenue:670000, expense:450000 },
];
const MOCK_ACTIVITY: Activity[] = [
    { id:1, description:'Invoice #INV-9021 paid',     amount:12450, type:'credit', date:'Today 9:41am',  status:'SUCCESS' },
    { id:2, description:'Supplier payment — Acme Co', amount:3500,  type:'debit',  date:'Today 8:20am',  status:'SUCCESS' },
    { id:3, description:'Employee salary batch',      amount:45000, type:'debit',  date:'Yesterday',     status:'SUCCESS' },
    { id:4, description:'Invoice #INV-9019 pending',  amount:8200,  type:'credit', date:'Yesterday',     status:'PENDING' },
    { id:5, description:'Office rent payment',        amount:8000,  type:'debit',  date:'2 days ago',    status:'SUCCESS' },
];

// ─── Main Component ──────────────────────────────────────────────────────
const SimpleDashboard: React.FC = () => {
    const navigate = useNavigate();
    const [activeNav, setActiveNav]     = useState('home');
    const [kpis, setKpis]               = useState<KPIs>(MOCK_KPIS);
    const [flow, setFlow]               = useState<ChartPoint[]>(MOCK_FLOW);
    const [rev, setRev]                 = useState<RevPoint[]>(MOCK_REV);
    const [activity, setActivity]       = useState<Activity[]>(MOCK_ACTIVITY);
    const [loading, setLoading]         = useState(true);
    const [sidebarOpen, setSidebarOpen] = useState(false);
    const [filterOpen, setFilterOpen]   = useState(false);
    const [profileOpen, setProfileOpen] = useState(false);
    const [searchVal, setSearchVal]     = useState('');
    const overlayRef = useRef<HTMLDivElement>(null);

    const token = () => localStorage.getItem('erp-token') || '';

    // Fetch live data
    useEffect(() => {
        const headers = { Authorization: `Bearer ${token()}`, 'Content-Type': 'application/json' };
        Promise.allSettled([
            fetch(`${API}/finance/summary`, { headers }).then(r => r.json()),
            fetch(`${API}/transactions?limit=5`, { headers }).then(r => r.json()),
        ]).then(([sumRes, txRes]) => {
```

```
        if (sumRes.status === 'fulfilled' && sumRes.value && !sumRes.value.error) {
            const s = sumRes.value;
            setKpis({
                cash:       s.cash_in_hand ?? MOCK_KPIS.cash,
                bank:       s.bank_balance ?? MOCK_KPIS.bank,
                profit:     s.net_profit   ?? MOCK_KPIS.profit,
                receivable: s.receivable   ?? MOCK_KPIS.receivable,
                payable:    s.payable      ?? MOCK_KPIS.payable,
            });
            if (s.cash_flow)      setFlow(s.cash_flow);
            if (s.revenue_chart)  setRev(s.revenue_chart);
        }
        if (txRes.status === 'fulfilled' && Array.isArray(txRes.value)) {
            const mapped: Activity[] = (txRes.value as any[]).slice(0, 5).map((tx: any) => ({
                id:          tx.id,
                description: tx.description || tx.narration || 'Transaction',
                amount:      Math.abs(tx.amount ?? 0),
                type:        (tx.amount ?? 0) >= 0 ? 'credit' : 'debit',
                date:        tx.date ? new Date(tx.date).toLocaleDateString('en-IN', { day:'2-digit', month:'short' }) : '',
                status:      (tx.status as Activity['status']) || 'SUCCESS',
            }));
            if (mapped.length > 0) setActivity(mapped);
        }
    }).finally(() => setLoading(false));
}, []);

// Close sidebar on outside click (mobile)
useEffect(() => {
    const handler = (e: MouseEvent) => {
        if (overlayRef.current && e.target === overlayRef.current) {
            setSidebarOpen(false);
        }
    };
    document.addEventListener('mousedown', handler);
    return () => document.removeEventListener('mousedown', handler);
}, []);

const handleNav = (item: typeof NAV_ITEMS[0]) => {
    setActiveNav(item.id);
    setSidebarOpen(false);
    navigate(item.path);
};

const handleSettings = () => {
    navigate('/admin/settings');
    setSidebarOpen(false);
};

const handleDownloadReport = () => {
    navigate('/admin/reports');
};

const handleSeeAll = () => {
    navigate('/transactions');
};

const handleLogout = () => {
    localStorage.removeItem('erp-token');
    navigate('/company-login');
    setProfileOpen(false);
};

const filteredActivity = activity.filter(tx =>
    searchVal === '' || tx.description.toLowerCase().includes(searchVal.toLowerCase())
);

return (
    <div className="simple-layout">

        {/* Mobile overlay */}
        <AnimatePresence>
            {sidebarOpen && (
                <motion.div
                    ref={overlayRef}
                    className="sidebar-overlay"
                    initial={{ opacity: 0 }}
                    animate={{ opacity: 1 }}
                    exit={{ opacity: 0 }}
                    onClick={() => setSidebarOpen(false)}
                />
            )}
        </AnimatePresence>

        {/* ── Sidebar ──────────────────────────────── */}
        <motion.aside
            className={`simple-sidebar${sidebarOpen ? ' open' : ''}`}
            initial={{ x: -220 }}
            animate={{ x: 0 }}
            transition={{ type: 'spring', stiffness: 260, damping: 25 }}
        >
            <div className="sidebar-brand">
                <div className="brand-icon">S</div>
                <h2>Synthesis ERP</h2>
                <button className="sidebar-close-btn" onClick={() => setSidebarOpen(false)}>
                    <FaTimes size={14} />
                </button>
            </div>

            <nav className="sidebar-nav">
                {NAV_ITEMS.map(item => (
                    <button
                        key={item.id}
                        className={`nav-item${activeNav === item.id ? ' active' : ''}`}
                        onClick={() => handleNav(item)}
                        title={item.label}
                        aria-label={item.label}
                    >
                        <span className="nav-icon">{item.icon}</span>
                        <span className="nav-label">{item.label}</span>
                    </button>
                ))}
            </nav>
```

```
    <div className="sidebar-bottom">
        <button
            className="nav-item"
            onClick={handleSettings}
            aria-label="Settings"
        >
            <span className="nav-icon"><FaCog /></span>
            <span className="nav-label">Settings</span>
        </button>
    </div>
</motion.aside>

{/* ── Main ──────────────────────────────────── */}
<main className="simple-main">

    {/* Top Bar */}
    <header className="simple-topbar">
        <div className="topbar-left">
            <button
                className="hamburger"
                onClick={() => setSidebarOpen(o => !o)}
                aria-label="Toggle sidebar"
            >
                ≡
            </button>
            <div className="branch-selector" title="Switch branch">
                <span className="branch-dot" />
                Main Branch <FaChevronDown size={10} />
            </div>
            <div className="topbar-search">
                <FaSearch size={12} />
                <input
                    placeholder="Search transactions, invoices…"
                    value={searchVal}
                    onChange={e => setSearchVal(e.target.value)}
                    aria-label="Search"
                />
                {searchVal && (
                    <button
                        className="clear-search"
                        onClick={() => setSearchVal('')}
                        aria-label="Clear search"
                    >
                        <FaTimes size={10} />
                    </button>
                )}
            </div>
        </div>

        <div className="topbar-right">
            <button
                className="icon-btn notif"
                aria-label="Notifications"
                title="Notifications"
            >
                <FaBell size={16} />
                <span className="badge" />
            </button>

            <button
                className="refresh-btn"
                onClick={() => window.location.reload()}
                title="Refresh data"
                aria-label="Refresh"
            >
                <FaSyncAlt size={14} />
            </button>

            <div className="profile-wrapper">
                <button
                    className="profile-trigger"
                    onClick={() => setProfileOpen(p => !p)}
                    aria-label="Profile menu"
                >
                    <div className="profile-avatar">AD</div>
                    <div className="profile-info">
                        <span className="profile-name">Admin</span>
                        <span className="profile-role">Owner</span>
                    </div>
                    <FaChevronDown size={10} style={{ color:'#94a3b8' }} />
                </button>

                <AnimatePresence>
                    {profileOpen && (
                        <motion.div
                            className="profile-dropdown"
                            initial={{ opacity: 0, y: -8 }}
                            animate={{ opacity: 1, y: 0 }}
                            exit={{ opacity: 0, y: -8 }}
                        >
                            <button className="dropdown-item" onClick={() => { navigate('/admin/settings'); setProfileOpen(false); }}>
                                ⚙️  Settings
                            </button>
                            <button className="dropdown-item" onClick={() => { navigate('/admin/reports'); setProfileOpen(false); }}>
                                📋  Reports
                            </button>
                            <div className="dropdown-divider" />
                            <button className="dropdown-item danger" onClick={handleLogout}>
                                🚪  Log Out
                            </button>
                        </motion.div>
                    )}
                </AnimatePresence>
            </div>
        </div>
    </header>

    {/* ── Content ──────────────────────────── */}
    <div className="content-scrollable" onClick={() => { if (profileOpen) setProfileOpen(false); if (filterOpen) setFilterOpen(false); }}>
```

```
{/* Page Title */}
<div className="page-title-row">
    <div>
        <h1 className="page-title">Your Business Overview</h1>
        <p className="page-subtitle">Everything you need to know, all in one place.</p>
    </div>
    <div className="title-actions">
        {/* Filter Dropdown */}
        <div className="filter-wrapper">
            <button
                className={`btn-outline${filterOpen ? ' active' : ''}`}
                onClick={e => { e.stopPropagation(); setFilterOpen(f => !f); }}
            >
                Filter ▾
            </button>
            <AnimatePresence>
                {filterOpen && (
                    <motion.div
                        className="filter-dropdown"
                        initial={{ opacity: 0, y: -8 }}
                        animate={{ opacity: 1, y: 0 }}
                        exit={{ opacity: 0, y: -8 }}
                    >
                        {['Today', 'This Week', 'This Month', 'This Year'].map(label => (
                            <button
                                key={label}
                                className="dropdown-item"
                                onClick={() => { setFilterOpen(false); }}
                            >
                                {label}
                            </button>
                        ))}
                    </motion.div>
                )}
            </AnimatePresence>
        </div>

        <button
            className="btn-primary"
            onClick={handleDownloadReport}
        >
            <FaDownload size={12} />
             Download Report
        </button>
    </div>
</div>

{/* ── KPI Strip ──────────────────────────── */}
{loading ? (
    <div className="loading-row">
        {[0,1,2,3,4].map(i => <div key={i} className="skeleton" />)}
    </div>
) : (
    <div className="kpi-strip">
        <KPICard label="Money in Hand"      value={fmt(kpis.cash)}       color="blue"   trend="+12.4%" onClick={() => navigate('/tra
        <KPICard label="Bank Balance"       value={fmt(kpis.bank)}       color="indigo" trend="+3.2%"  onClick={() => navigate('/fin
        <KPICard label="Profit (This Year)" value={fmt(kpis.profit)}     color="green"  trend="+24%"   onClick={() => navigate('/adm
        <KPICard label="Customers Owe You"  value={fmt(kpis.receivable)} color="amber"  trend=""       onClick={() => navigate('/cus
        <KPICard label="You Owe Suppliers"  value={fmt(kpis.payable)}    color="red"    trend=""       onClick={() => navigate('/sup
    </div>
)}

{/* ── Charts Row ──────────────────────────── */}
<div className="charts-row">

    {/* Cash Flow */}
    <div className="chart-card wide">
        <div className="chart-header">
            <span className="chart-title">Money Coming In vs Going Out (This Week)</span>
        </div>
        <ResponsiveContainer width="100%" height={175}>
            <AreaChart data={flow} margin={{ top:5, right:10, left:0, bottom:0 }}>
                <defs>
                    <linearGradient id="gin" x1="0" y1="0" x2="0" y2="1">
                        <stop offset="0%" stopColor="#3B82F6" stopOpacity={0.15}/>
                        <stop offset="100%" stopColor="#3B82F6" stopOpacity={0}/>
                    </linearGradient>
                </defs>
                <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#f1f5f9" />
                <XAxis dataKey="name" tick={{ fontSize:11, fill:'#94a3b8' }} axisLine={false} tickLine={false} />
                <YAxis hide />
                <Tooltip formatter={(v: any) => fmt(Number(v))} contentStyle={{ border:'none', borderRadius:10, boxShadow:'0 4px 12p
                <Area dataKey="inflow"  stroke="#3B82F6" strokeWidth={2.5} fill="url(#gin)" name="Money In" />
                <Area dataKey="outflow" stroke="#94a3b8" strokeWidth={2} fill="transparent" name="Money Out" strokeDasharray="4 3" /
            </AreaChart>
        </ResponsiveContainer>
        <div className="chart-legend">
            <span className="legend-dot blue" />Money In  
            <span className="legend-dot grey" />Money Out
        </div>
    </div>

    {/* Revenue vs Expense */}
    <div className="chart-card">
        <div className="chart-header">
            <span className="chart-title">Sales vs Expenses (6 Months)</span>
        </div>
        <ResponsiveContainer width="100%" height={175}>
            <BarChart data={rev} margin={{ top:5, right:10, left:0, bottom:0 }}>
                <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#f1f5f9" />
                <XAxis dataKey="name" tick={{ fontSize:11, fill:'#94a3b8' }} axisLine={false} tickLine={false} />
                <YAxis hide />
                <Tooltip formatter={(v: any) => fmt(Number(v))} contentStyle={{ border:'none', borderRadius:10, boxShadow:'0 4px 12p
                <Bar dataKey="revenue" fill="#1E2A78" radius={[4,4,0,0]} name="Sales" maxBarSize={20} />
                <Bar dataKey="expense" fill="#E2E8F0" radius={[4,4,0,0]} name="Expenses" maxBarSize={20} />
            </BarChart>
        </ResponsiveContainer>
        <div className="chart-legend">
            <span className="legend-dot indigo" />Sales  
            <span className="legend-dot silver" />Expenses
        </div>
    </div>
```

```
                        {/* Quick Summary */}
                        <div className="chart-card summary-card">
                            <p className="summary-label">Customers Still Need to Pay You</p>
                            <p className="summary-value blue" style={{ cursor:'pointer' }} onClick={() => navigate('/customers')}>
                                {fmt(kpis.receivable)}
                            </p>
                            <div className="divider" />
                            <p className="summary-label">You Still Need to Pay Suppliers</p>
                            <p className="summary-value red" style={{ cursor:'pointer' }} onClick={() => navigate('/suppliers')}>
                                {fmt(kpis.payable)}
                            </p>
                            <div className="divider" />
                            <button className="btn-outline-sm" onClick={() => navigate('/finance/dashboard')}>
                                View Full Finance →
                            </button>
                        </div>
                    </div>

                    {/* ── Recent Activity ──────────────────────── */}
                    <div className="activity-card">
                        <div className="activity-header">
                            <span className="chart-title">Recent Transactions</span>
                            <button className="btn-link" onClick={handleSeeAll}>See All →</button>
                        </div>
                        <table className="simple-table">
                            <thead>
                                <tr>
                                    <th>What Happened</th>
                                    <th>Amount</th>
                                    <th>Date</th>
                                    <th>Status</th>
                                </tr>
                            </thead>
                            <tbody>
                                {filteredActivity.map(tx => (
                                    <tr
                                        key={tx.id}
                                        className="tx-row"
                                        onClick={() => navigate('/transactions')}
                                        style={{ cursor: 'pointer' }}
                                    >
                                        <td>
                                            <div className="tx-desc">
                                                <span className={`tx-dot ${tx.type === 'credit' ? 'green' : 'red'}`} />
                                                {tx.description}
                                            </div>
                                        </td>
                                        <td className={`tx-amount ${tx.type === 'credit' ? 'green' : 'red'}`}>
                                            {tx.type === 'credit' ? '+' : '-'}{fmt(tx.amount)}
                                        </td>
                                        <td className="tx-date">{tx.date}</td>
                                        <td>
                                            <span className={`status-pill status-${tx.status.toLowerCase()}`}>
                                                {tx.status === 'SUCCESS' ? 'Done' : tx.status === 'PENDING' ? 'Waiting' : 'Failed'}
                                            </span>
                                        </td>
                                    </tr>
                                ))}
                                {filteredActivity.length === 0 && (
                                    <tr>
                                        <td colSpan={4} style={{ textAlign:'center', color:'#94a3b8', padding:'1.5rem' }}>
                                            No transactions match your search.
                                        </td>
                                    </tr>
                                )}
                            </tbody>
                        </table>
                    </div>

                </div>
            </main>
        </div>
    );
};

// ── KPI Card ──────────────────────────────────────────────────────────
const PALETTE: Record<string, string> = {
    blue: '#3B82F6', indigo: '#1E2A78',
    green: '#10B981', amber: '#F59E0B', red: '#EF4444',
};

const KPICard: React.FC<{
    label: string; value: string; color: string; trend: string; onClick?: () => void;
}> = ({ label, value, color, trend, onClick }) => (
    <motion.button
        className="kpi-card kpi-card-btn"
        whileHover={{ y: -4, boxShadow: '0 8px 18px rgba(0,0,0,0.11)' }}
        whileTap={{ scale: 0.98 }}
        onClick={onClick}
        title={`View ${label}`}
    >
        <div className="kpi-accent" style={{ background: PALETTE[color] || PALETTE.blue }} />
        <p className="kpi-label">{label}</p>
        <p className="kpi-value" style={{ color: color === 'red' ? PALETTE.red : color === 'green' ? PALETTE.green : '#1E293B' }}>
            {value}
        </p>
        {trend && (
            <p className="kpi-trend" style={{ color: PALETTE.green }}>↑ {trend} vs last month</p>
        )}
    </motion.button>
);

export default SimpleDashboard;
```

frontend/src/App.tsx

```
// frontend/src/App.tsx
import React, { useState } from 'react';
import { Navigate, Outlet, Route, BrowserRouter as Router, Routes } from 'react-router-dom';
import Sidebar from './components/Layout/Sidebar';
import { TenantProvider, useTenant } from './context/TenantContext';
import { useAuthUser } from './hooks/useAuthUser';

// Pages
import AIInsights from './pages/AIInsights';
import Branches from './pages/Branches';
import CreateInvoice from './pages/CreateInvoice';
import Customers from './pages/Customers';
import Dashboard from './pages/Dashboard';
import DayBook from './pages/DayBook';
import Employees from './pages/Employees';
import FileManager from './pages/FileManager';
import HostLogin from './pages/HostLogin';
import Attendance from './pages/hr/Attendance';
import MobileAttendance from './pages/hr/MobileAttendance';
import Inventory from './pages/Inventory';
import Invoices from './pages/Invoices';
import Ledgers from './pages/Ledgers';
import Login from './pages/Login';
import PlatformAdmin from './pages/PlatformAdmin';
import PurchaseBills from './pages/PurchaseBills';
import Reports from './pages/Reports';
import Settings from './pages/Settings';
import Subscriptions from './pages/Subscriptions';
import Suppliers from './pages/Suppliers';
import Transactions from './pages/Transactions';
import SimpleDashboard from './SimpleDashboard';
import SynthesisDashboard from './SynthesisDashboard';

// Finance Module
import BankReconciliation from './pages/finance/BankReconciliation';
import CashReceipts from './pages/finance/CashReceipts';
import FinanceDashboard from './pages/finance/FinanceDashboard';
import FinancialReports from './pages/finance/FinancialReports';
import LoanManagement from './pages/finance/LoanManagement';

import './components/Layout/Layout.css';

// --- 🌐 GLOBAL PAGE TRANSITION ---
const PageTransition: React.FC<{ children: React.ReactNode }> = ({ children }) => (
    <div className="canvas-area">
        {children}
    </div>
);

// --- 🏗️ ENTERPRISE LAYOUT WRAPPER ---
const EnterpriseLayout: React.FC<{ user: any; mode: 'HOST' | 'ADMIN' | 'USER' }> = ({ user, mode }) => {
    const [sidebarCollapsed, setSidebarCollapsed] = useState(false);
    const { activeBranch, branches, setActiveBranch } = useTenant();

    return (
        <div className="enterprise-shell">
            <Sidebar
                isOpen={true}
                setIsOpen={() => {}}
                isMobile={false}
                mode={mode}
            />

            <div className="shell-main" style={{ paddingLeft: sidebarCollapsed ? '80px' : '280px' }}>
                <header className="shell-header">
                    <div className="header-context">
                        {mode === 'USER' && (
                            <div className="flex items-center gap-2">
                                <span className="text-[10px] font-black text-muted uppercase tracking-widest">Branch Context</span>
                                <select
                                    className="branch-select-modern"
                                    value={activeBranch?.id || ''}
                                    onChange={(e) => {
                                        const found = branches.find(b => b.id.toString() === e.target.value);
                                        if (found) setActiveBranch(found);
                                    }}
                                >
                                    {branches.map(b => <option key={b.id} value={b.id}>{b.branch_name}</option>)}
                                </select>
                            </div>
                        )}
                        {mode === 'HOST' && <span className="badge badge-host">Platform Governance v4.0</span>}
                        {mode === 'ADMIN' && <span className="badge badge-admin">Tenant Administration | {user?.company_name}</span>}
                    </div>

                    <div className="user-profile-block">
                        <span className="user-label-text">{user?.username}</span>
                        <div className="user-avatar-circle">{user?.username?.charAt(0)}</div>
                    </div>
                </header>

                <main style={{ flex: 1 }}>
                    <PageTransition>
                        <Outlet />
                    </PageTransition>
                </main>
            </div>
        </div>
    );
};

// --- 🛡️ GUARDS ---
const HostRoute = () => {
    const { user, loading } = useAuthUser();
    if (loading) return <div className="full-screen-loader">Syncing Nexus...</div>;
```

```
    return user?.role?.toLowerCase() === 'superadmin' ? <EnterpriseLayout user={user} mode="HOST" /> : <Navigate to="/company-login" />;
};

const AdminRoute = () => {
    const { user, loading } = useAuthUser();
    if (loading) return <div className="full-screen-loader">Loading Registry...</div>;
    return user?.role?.toLowerCase() === 'admin' ? <EnterpriseLayout user={user} mode="ADMIN" /> : <Navigate to="/dashboard" />;
};

const WorkspaceRoute = () => {
    const { user, loading } = useAuthUser();
    if (loading) return <div className="full-screen-loader">Connecting Workspace...</div>;
    return user ? <EnterpriseLayout user={user} mode="USER" /> : <Navigate to="/company-login" />;
};

const App: React.FC = () => {
    console.log("🚀 Quantum Nexus App initializing...");
    return (
        <TenantProvider>
            <Router>
                <Routes>
                    <Route path="/company-login" element={<Login />} />
                    <Route path="/host-login" element={<HostLogin />} />
                    <Route path="/mark-attendance" element={<MobileAttendance />} />

                    <Route element={<HostRoute />}>
                        <Route path="/platform-admin" element={<PlatformAdmin tab="hub" />} />
                        <Route path="/platform-admin/tenants" element={<PlatformAdmin tab="tenants" />} />
                        <Route path="/platform-admin/config" element={<PlatformAdmin tab="config" />} />
                    </Route>

                    <Route element={<AdminRoute />}>
                        <Route path="/admin/branches" element={<Branches />} />
                        <Route path="/admin/subscriptions" element={<Subscriptions />} />
                        <Route path="/admin/settings" element={<Settings />} />
                        <Route path="/admin/reports" element={<Reports />} />
                        <Route path="/admin/employees" element={<Employees />} />
                    </Route>

                    <Route element={<WorkspaceRoute />}>
                        <Route path="/dashboard" element={<Dashboard />} />
                        <Route path="/customers" element={<Customers />} />
                        <Route path="/invoices" element={<Invoices />} />
                        <Route path="/invoices/new" element={<CreateInvoice />} />
                        <Route path="/products" element={<Inventory />} />
                        <Route path="/documents" element={<FileManager />} />
                        <Route path="/ai-insights" element={<AIInsights />} />
                        <Route path="/ledgers" element={<Ledgers />} />
                        <Route path="/suppliers" element={<Suppliers />} />
                        <Route path="/purchase-bills" element={<PurchaseBills />} />
                        <Route path="/transactions" element={<Transactions />} />
                        <Route path="/daybook" element={<DayBook />} />
                        <Route path="/attendance" element={<Attendance />} />
                        <Route path="/employees" element={<Employees />} />

                        <Route path="/finance/dashboard" element={<FinanceDashboard />} />
                        <Route path="/finance/loans" element={<LoanManagement />} />
                        <Route path="/finance/receipts" element={<CashReceipts />} />
                        <Route path="/finance/reports" element={<FinancialReports />} />
                        <Route path="/finance/reconciliation" element={<BankReconciliation />} />
                    </Route>

                    {/* Dedicated V2 Synthesis Path - Standalone Layout */}
                    <Route path="/synthesis" element={<SynthesisDashboard />} />
                    {/* Clean Simple Dashboard */}
                    <Route path="/simple" element={<SimpleDashboard />} />

                    <Route path="/" element={<Navigate to="/dashboard" replace />} />
                    <Route path="*" element={<Navigate to="/company-login" replace />} />
                </Routes>
            </Router>
        </TenantProvider>
    );
};

export default App;
```

frontend/src/pages/Transactions.tsx

```
// frontend/src/pages/Transactions.tsx
import { motion } from 'framer-motion';
import React, { useEffect, useState } from 'react';
import { FaExchangeAlt, FaFileAlt, FaPlus, FaSearch } from 'react-icons/fa';
import { useNavigate } from 'react-router-dom';

interface Transaction {
    id: number;
    transaction_date: string;
    description: string;
    reference_type: string;
    reference_id: number;
    creator_name: string;
    amount?: number; // Added for display convenience if possible
}

const Transactions: React.FC = () => {
    const navigate = useNavigate();
    const [transactions, setTransactions] = useState<Transaction[]>([]);
    const [loading, setLoading] = useState(true);
    const [searchTerm, setSearchTerm] = useState("");

    useEffect(() => {
        const fetchTxs = async () => {
            try {
                const response = await fetch(`${import.meta.env.VITE_API_URL}/api/accounting/transactions`, {
                    headers: { 'Authorization': `Bearer ${localStorage.getItem('erp-token')}` }
                });
                const data = await response.json();
                if (response.ok) setTransactions(data);
            } catch (error) {
                console.error("Error fetching transactions:", error);
            } finally {
                setLoading(false);
            }
        };
        fetchTxs();
    }, []);

    const filtered = transactions.filter(t =>
        t.description?.toLowerCase().includes(searchTerm.toLowerCase()) ||
        t.reference_type?.toLowerCase().includes(searchTerm.toLowerCase())
    );

    return (
        <div className="space-y-6">
            <div className="flex flex-col md:flex-row md:items-center justify-between gap-4">
                <div>
                    <h1 className="text-3xl font-black text-slate-900 tracking-tight">Financial Transactions</h1>
                    <p className="text-slate-500 mt-1">Audit log of all ledger movements and journal entries.</p>
                </div>
                <button
                    onClick={() => navigate('/transactions/new')}
                    className="flex items-center gap-2 bg-blue-600 text-white px-6 py-3 rounded-xl font-bold hover:bg-blue-700 hover:scale-105 transitio
                >
                    <FaPlus /> New Journal Entry
                </button>
            </div>

            <div className="glass-panel p-4 rounded-2xl flex items-center gap-4 border border-slate-200">
                <div className="relative flex-1">
                    <FaSearch className="absolute left-4 top-1/2 -translate-y-1/2 text-slate-400" />
                    <input
                        type="text"
                        placeholder="Search transactions..."
                        className="w-full pl-12 pr-4 py-3 bg-slate-50 border-none rounded-xl outline-none"
                        value={searchTerm}
                        onChange={(e) => setSearchTerm(e.target.value)}
                    />
                </div>
            </div>

            <div className="space-y-4">
                {loading ? (
                    [1,2,3].map(i => <div key={i} className="h-24 bg-slate-100 animate-pulse rounded-2xl" />)
                ) : filtered.map((tx, idx) => (
                    <motion.div
                        key={tx.id}
                        initial={{ opacity: 0, y: 10 }}
                        animate={{ opacity: 1, y: 0 }}
                        transition={{ delay: idx * 0.05 }}
                        className="glass-panel p-6 rounded-2xl border border-slate-200 hover:border-blue-300 transition-all group flex items-center justi
                    >
                        <div className="flex items-center gap-6">
                            <div className="w-14 h-14 bg-slate-50 text-slate-400 group-hover:bg-blue-50 group-hover:text-blue-600 rounded-2xl flex items
                                <FaFileAlt size={24} />
                            </div>
                            <div>
                                <div className="flex items-center gap-3 mb-1">
                                    <span className="text-sm font-mono font-bold text-slate-400">#{tx.id}</span>
                                    <span className="px-2 py-0.5 bg-slate-100 text-slate-500 rounded text-[10px] font-black uppercase tracking-widest">
                                    <span className="text-xs font-bold text-slate-400">{new Date(tx.transaction_date).toLocaleDateString(undefined, { da
                                </div>
                                <h3 className="font-bold text-slate-900">{tx.description || 'No description provided'}</h3>
                                <p className="text-xs text-slate-400 mt-1">Posted by <span className="text-slate-600 font-bold">{tx.creator_name}</span>
                            </div>
                        </div>

                        <div className="text-right">
                             <button className="text-blue-600 text-sm font-black hover:underline flex items-center gap-2">
                                Details <FaExchangeAlt />
                             </button>
                        </div>
                    </motion.div>
                ))}
            </div>
```

```
            {!loading && filtered.length === 0 && (
                <div className="text-center py-20 bg-slate-50 rounded-[40px] border-2 border-dashed border-slate-200">
                    <p className="text-slate-400 font-medium">No transactions matching your criteria.</p>
                </div>
            )}
        </div>
    );
};

export default Transactions;
```

frontend/src/pages/TOCDashboard.tsx

```
// frontend/src/pages/TOCDashboard.tsx
import React, { useEffect, useState } from 'react';
import {
    FaBullseye,
    FaExclamationTriangle,
    FaPlus,
    FaSync
} from 'react-icons/fa';
import type { Constraint } from '../api/tocApi';
import { fetchConstraints, fetchTocDashboard } from '../api/tocApi';

const TOCDashboard: React.FC = () => {
    const [stats, setStats] = useState<any>(null);
    const [constraints, setConstraints] = useState<Constraint[]>([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        loadData();
    }, []);

    const loadData = async () => {
        setLoading(true);
        try {
            const [dashboardData, constraintsData] = await Promise.all([
                fetchTocDashboard(),
                fetchConstraints()
            ]);
            setStats(dashboardData);
            setConstraints(constraintsData);
        } catch (error) {
            console.error('Failed to load TOC data:', error);
        } finally {
            setLoading(false);
        }
    };

    const getStatusColor = (status: string) => {
        switch (status) {
            case 'ACTIVE': return '#ef4444';
            case 'ELEVATED': return '#f59e0b';
            case 'RESOLVED': return '#10b981';
            default: return '#6b7280';
        }
    };

    return (
        <div style={{ padding: '0 0 40px 0' }}>
            {/* Header */}
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '32px', flexWrap: 'wrap', gap: '20px' }
                <div style={{ flex: '1 1 300px' }}>
                    <h1 style={{ fontSize: '2rem', fontWeight: 800, color: '#0f172a', margin: 0, letterSpacing: '-0.025em' }}>
                        Theory of Constraints
                    </h1>
                    <p style={{ color: '#64748b', fontSize: '1.1rem', marginTop: ' 4px' }}>
                        Identify, exploit, and elevate your business bottlenecks.
                    </p>
                </div>
                <div style={{ display: 'flex', gap: '12px', flexWrap: 'wrap' }}>
                    <button
                        onClick={loadData}
                        className="btn-secondary"
                        style={{ display: 'flex', alignItems: 'center', gap: '8px', padding: '10px 16px' }}
                    >
                        <FaSync className={loading ? 'fa-spin' : ''} /> Refresh
                    </button>
                    <button
                        className="btn-primary"
                        style={{ display: 'flex', alignItems: 'center', gap: '8px', padding: '10px 20px', backgroundColor: '#6366f1' }}
                    >
                        <FaPlus /> New Constraint
                    </button>
                </div>
            </div>

            {/* Stats Cards */}
            <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(240px, 1fr))', gap: '24px', marginBottom: '40px' }}>
                <div style={cardStyle}>
                    <p style={statLabelStyle}>Active Constraints</p>
                    <h3 style={statValueStyle}>{stats?.activeConstraints || 0}</h3>
                    <div style={{ ...badgeStyle, backgroundColor: '#fee2e2', color: '#ef4444' }}>High Priority</div>
                </div>
                <div style={cardStyle}>
                    <p style={statLabelStyle}>Current Throughput</p>
                    <h3 style={statValueStyle}>₹{Number(stats?.latestMetrics?.throughput || 0).toLocaleString('en-IN')}</h3>
                </div>
                <div style={cardStyle}>
                    <p style={statLabelStyle}>Open Actions</p>
                    <h3 style={statValueStyle}>{stats?.pendingActions || 0}</h3>
                </div>
                <div style={cardStyle}>
                    <p style={statLabelStyle}>System ROI</p>
                    <h3 style={statValueStyle}>{stats?.latestMetrics?.return_on_investment || 0}%</h3>
                    <div style={{ ...badgeStyle, backgroundColor: '#e0e7ff', color: '#4f46e5' }}>Healthy</div>
                </div>
            </div>

            {/* Main Content Areas */}
            <div style={{ display: 'grid', gridTemplateColumns: window.innerWidth <= 1024 ? '1fr' : '2fr 1fr', gap: '24px' }}>
                {/* Active Constraints Table */}
                <div style={{ ...cardStyleBySide, padding: 0, overflow: 'hidden' }}>
                    <div style={{ padding: '24px', borderBottom: '1px solid #e2e8f0', display: 'flex', justifyContent: 'space-between', alignItems: 'cen
                        <h3 style={{ margin: 0, fontSize: '1.25rem', fontWeight: 700 }}>System Constraints</h3>
                        <FaExclamationTriangle color="#ef4444" />
                    </div>
                    <div style={{ overflowX: 'auto' }}>
                        <table style={{ width: '100%', borderCollapse: 'collapse', minWidth: '800px' }}>
```

```
                    <thead style={{ backgroundColor: '#f8fafc', borderBottom: '1px solid #e2e8f0' }}>
                        <tr>
                            <th style={thStyle}>Constraint Name</th>
                            <th style={thStyle}>Area</th>
                            <th style={thStyle}>Utilization</th>
                            <th style={thStyle}>Status</th>
                            <th style={thStyle}>Actions</th>
                        </tr>
                    </thead>
                    <tbody>
                        {loading ? (
                            <tr><td colSpan={5} style={{ padding: '40px', textAlign: 'center', color: '#64748b' }}>Loading...</td></tr>
                        ) : constraints.length === 0 ? (
                            <tr><td colSpan={5} style={{ padding: '40px', textAlign: 'center', color: '#64748b' }}>No constraints identified yet.</t
                        ) : (
                            constraints.map(c => (
                                <tr key={c.id} style={{ borderBottom: '1px solid #f1f5f9' }}>
                                    <td style={tdStyle}>
                                        <div style={{ fontWeight: 600, color: '#1e293b' }}>{c.constraint_name}</div>
                                        <div style={{ fontSize: '0.75rem', color: '#64748b' }}>{c.constraint_type}</div>
                                    </td>
                                    <td style={tdStyle}>{c.area}</td>
                                    <td style={tdStyle}>
                                        <div style={{ display: 'flex', alignItems: 'center', gap: '8px' }}>
                                            <div style={{ flex: 1, height: '8px', backgroundColor: '#e2e8f0', borderRadius: '4px', overflow: 'hidden
                                                <div style={{
                                                    width: `${c.utilization_percent}%`,
                                                    height: '100%',
                                                    backgroundColor: c.utilization_percent > 90 ? '#ef4444' : '#6366f1'
                                                }} />
                                            </div>
                                            <span style={{ fontSize: '0.85rem', fontWeight: 600 }}>{c.utilization_percent}%</span>
                                        </div>
                                    </td>
                                    <td style={tdStyle}>
                                        <span style={{
                                            padding: '4px 8px',
                                            borderRadius: '9999px',
                                            fontSize: '0.75rem',
                                            fontWeight: 600,
                                            backgroundColor: `${getStatusColor(c.status)}20`,
                                            color: getStatusColor(c.status)
                                        }}>
                                            {c.status}
                                        </span>
                                    </td>
                                    <td style={tdStyle}>
                                        <button style={{ color: '#6366f1', background: 'none', border: 'none', cursor: 'pointer', fontSize: '0.85rem
                                            View Details
                                        </button>
                                    </td>
                                </tr>
                            ))
                        )}
                    </tbody>
                    </table>
                </div>
            </div>

            {/* TOC Methodology Panel */}
            <div style={{ ...cardStyleBySide, background: 'linear-gradient(135deg, #1e293b 0%, #0f172a 100%)', color: 'white' }}>
                <h3 style={{ margin: '0 0 20px 0', fontSize: '1.25rem', fontWeight: 700 }}>5 Focusing Steps</h3>
                <div style={{ display: 'flex', flexDirection: 'column', gap: '20px' }}>
                    {METHODOLOGY_STEPS.map((step, idx) => (
                        <div key={idx} style={{ display: 'flex', gap: '16px' }}>
                            <div style={{
                                width: '32px',
                                height: '32px',
                                borderRadius: '50%',
                                backgroundColor: step.color,
                                display: 'flex',
                                alignItems: 'center',
                                justifyContent: 'center',
                                fontSize: '0.9rem',
                                fontWeight: 800,
                                flexShrink: 0
                            }}>
                                {idx + 1}
                            </div>
                            <div>
                                <h4 style={{ margin: 0, fontSize: '0.95rem', fontWeight: 700 }}>{step.name}</h4>
                                <p style={{ margin: '4px 0 0 0', fontSize: '0.8rem', opacity: 0.7, lineHeight: 1.4 }}>{step.desc}</p>
                            </div>
                        </div>
                    ))}
                </div>
                <button style={{
                    marginTop: '32px',
                    width: '100%',
                    padding: '12px',
                    borderRadius: '8px',
                    border: 'none',
                    backgroundColor: '#6366f1',
                    color: 'white',
                    fontWeight: 700,
                    cursor: 'pointer',
                    display: 'flex',
                    alignItems: 'center',
                    justifyContent: 'center',
                    gap: '8px'
                }}>
                    <FaBullseye /> Identify Bottleneck
                </button>
            </div>
        </div>
    </div>
  );
};

// --- Styles ---
const cardStyle: React.CSSProperties = {
    backgroundColor: 'white',
```

```
    padding: '24px',
    borderRadius: '16px',
    boxShadow: '0 4px 6px -1px rgba(0, 0, 0, 0.05), 0 2px 4px -1px rgba(0, 0, 0, 0.03)',
    border: '1px solid #e2e8f0',
};

const cardStyleBySide: React.CSSProperties = {
    backgroundColor: 'white',
    padding: '24px',
    borderRadius: '16px',
    boxShadow: '0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -2px rgba(0, 0, 0, 0.05)',
    border: '1px solid #e2e8f0',
};

const statLabelStyle: React.CSSProperties = {
    margin: 0,
    fontSize: '0.875rem',
    color: '#64748b',
    fontWeight: 600,
    textTransform: 'uppercase',
    letterSpacing: '0.025em'
};

const statValueStyle: React.CSSProperties = {
    margin: '8px 0',
    fontSize: '1.875rem',
    fontWeight: 800,
    color: '#0f172a',
    letterSpacing: '-0.025em'
};

const badgeStyle: React.CSSProperties = {
    display: 'inline-block',
    padding: '4px 10px',
    borderRadius: '9999px',
    fontSize: '0.75rem',
    fontWeight: 700
};

const thStyle: React.CSSProperties = {
    textAlign: 'left',
    padding: '16px 24px',
    fontSize: '0.8rem',
    color: '#475569',
    textTransform: 'uppercase',
    letterSpacing: '0.05em'
};

const tdStyle: React.CSSProperties = {
    padding: '16px 24px',
    fontSize: '0.9rem',
    color: '#1e293b'
};

const METHODOLOGY_STEPS = [
    { name: 'IDENTIFY', desc: 'Find the resource that limits the system from achieving its goal.', color: '#ef4444' },
    { name: 'EXPLOIT', desc: 'Ensure the constraint is never idle and only works on quality parts.', color: '#f59e0b' },
    { name: 'SUBORDINATE', desc: 'Align all non-constraints to the pace of the bottleneck.', color: '#3b82f6' },
    { name: 'ELEVATE', desc: 'Invest in increasing the capacity of the constraint.', color: '#a855f7' },
    { name: 'PREVENT INERTIA', desc: 'Once the constraint is broken, go back to step 1.', color: '#10b981' },
];

export default TOCDashboard;
```

frontend/src/pages/Suppliers.tsx

```
import { motion } from 'framer-motion';
import React, { useEffect, useState } from 'react';
import { FaEdit, FaExclamationTriangle, FaHistory, FaPlus, FaSearch, FaSync, FaTrash, FaUserTie } from 'react-icons/fa';
import { Supplier, deleteSupplier, fetchSuppliers } from '../api/supplierApi';
import TransactionHistoryModal from '../components/TransactionHistoryModal';
import AddSupplierModal from './AddSupplierModal';
import './Suppliers.css';

const Suppliers: React.FC = () => {
    const [suppliers, setSuppliers] = useState<Supplier[]>([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState<string | null>(null);
    const [showModal, setShowModal] = useState(false);
    const [searchTerm, setSearchTerm] = useState('');
    const [showTransactionModal, setShowTransactionModal] = useState(false);
    const [selectedSupplier, setSelectedSupplier] = useState<Supplier | null>(null);

    const loadData = async () => {
        setLoading(true);
        setError(null);
        try {
            const data = await fetchSuppliers();
            setSuppliers(data || []);
        } catch (err: any) {
            setError(err.message);
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        loadData();
    }, []);

    const handleDelete = async (id: number) => {
        if (window.confirm("Dissolve this partnership? Historical procurement logs will be archived.")) {
            try {
                await deleteSupplier(id);
                loadData();
            } catch (err) {
                alert("Operation failed: Partnership remains active.");
            }
        }
    };

    const filtered = suppliers.filter(s =>
        (s.lender_name || '').toLowerCase().includes(searchTerm.toLowerCase())
    );

    return (
        <div className="suppliers-container">
            {/* Header */}
            <div className="suppliers-header">
                <div className="suppliers-title">
                    <motion.h1
                        initial={{ opacity: 0, x: -20 }}
                        animate={{ opacity: 1, x: 0 }}
                    >Procurement Network</motion.h1>
                    <motion.p
                        initial={{ opacity: 0 }}
                        animate={{ opacity: 1 }}
                        transition={{ delay: 0.1 }}
                    >Strategic Vendor Intelligence & Supply Chain Relations</motion.p>
                </div>
                <div className="suppliers-actions" style={{ display: 'flex', gap: '12px' }}>
                    <motion.button
                        whileHover={{ scale: 1.05 }}
                        whileTap={{ scale: 0.95 }}
                        className="btn-secondary"
                        onClick={loadData}
                        style={{ width: '52px', height: '52px', borderRadius: '14px', display: 'flex', alignItems: 'center', justifyContent: 'center' }}
                    >
                        <FaSync className={loading ? 'fa-spin' : ''} />
                    </motion.button>
                    <motion.button
                        whileHover={{ scale: 1.05 }}
                        whileTap={{ scale: 0.95 }}
                        className="btn-primary"
                        onClick={() => setShowModal(true)}
                        style={{ height: '52px', padding: '0 28px', gap: '12px', borderRadius: '14px', fontSize: '1rem', fontWeight: 900 }}
                    >
                        <FaPlus /> Initialize New Vendor
                    </motion.button>
                </div>
            </div>

            {error && (
                <motion.div
                    initial={{ opacity: 0, scale: 0.95 }}
                    animate={{ opacity: 1, scale: 1 }}
                    style={{ backgroundColor: 'var(--error-glow)', color: 'var(--error)', padding: '20px', borderRadius: '16px', marginBottom: '32px', d
                >
                    <FaExclamationTriangle size={20} />
                    <span>Procurement Node Sync Failure: {error}</span>
                </motion.div>
            )}

            {/* Search Toolbar */}
            <div className="suppliers-toolbar">
                <FaSearch style={{ color: 'var(--text-muted)' }} size={20} />
                <input
                    placeholder="Locate vendor via identity, signature, or HASH..."
                    value={searchTerm}
                    onChange={e => setSearchTerm(e.target.value)}
                />
            </div>
```

```
    {/* Table */}
    <div className="suppliers-table-container">
        <table className="sup-table">
            <thead>
                <tr>
                    <th>Nexus Entity</th>
                    <th>Neural Contact Patch</th>
                    <th style={{ textAlign: 'right' }}>Liability Balance</th>
                    <th style={{ textAlign: 'center' }}>Interface</th>
                </tr>
            </thead>
            <tbody>
                {loading ? (
                    [...Array(6)].map((_, i) => (
                        <tr key={i}><td colSpan={4} style={{ padding: '30px' }}><div className="skeleton" style={{ height: '30px', borderRadius
                    ))
                ) : filtered.length === 0 ? (
                    <tr>
                        <td colSpan={4} style={{ padding: '120px 0', textAlign: 'center' }}>
                            <FaUserTie size={64} style={{ color: 'var(--border-color)', marginBottom: '24px' }} />
                            <h3 style={{ margin: 0, fontWeight: 900, color: 'var(--text-primary)' }}>Registry Empty</h3>
                            <p style={{ color: 'var(--text-muted)', fontWeight: 500, marginTop: '8px' }}>No active vendor nodes discovered.</p>
                        </td>
                    </tr>
                ) : (
                    filtered.map((s, idx) => (
                        <motion.tr
                            key={s.id}
                            initial={{ opacity: 0, x: -20 }}
                            animate={{ opacity: 1, x: 0 }}
                            transition={{ delay: idx * 0.05 }}
                            className="sup-row"
                        >
                            <td>
                                <div style={{ display: 'flex', alignItems: 'center', gap: '15px' }}>
                                    <div className="vendor-orb">
                                        <FaUserTie size={18} />
                                    </div>
                                    <div>
                                        <div style={{ fontWeight: 900, color: 'var(--text-primary)', fontSize: '1.1rem' }}>{s.lender_name}</div>
                                        <div style={{ fontSize: '0.8rem', color: 'var(--text-muted)', fontWeight: 600 }}>INDEX: #VND-{s.id}</div
                                    </div>
                                </div>
                            </td>
                            <td>
                                <div style={{ display: 'flex', flexDirection: 'column', gap: '4px' }}>
                                    <span style={{ color: 'var(--text-primary)', fontWeight: 800, fontSize: '0.95rem' }}>{s.phone || 'NO-TRACER
                                    <span style={{ fontSize: '0.8rem', color: 'var(--text-muted)', fontWeight: 600 }}>{s.email || 'ENCRYPTED'}</
                                </div>
                            </td>
                            <td style={{ textAlign: 'right' }}>
                                <div className="balance-text">₹{Number(s.remaining_balance).toLocaleString('en-IN', { minimumFractionDigits: 2 }
                            </td>
                            <td>
                                <div style={{ display: 'flex', justifyContent: 'center', gap: '12px' }}>
                                    <motion.button
                                        whileHover={{ scale: 1.1, background: '#f1f5f9' }}
                                        whileTap={{ scale: 0.9 }}
                                        className="control-btn"
                                        style={{ background: 'white', border: '1px solid #e2e8f0', color: 'var(--text-secondary)' }}
                                        onClick={() => {
                                            setSelectedSupplier(s);
                                            setShowTransactionModal(true);
                                        }}
                                        title="Execution History"
                                    >
                                        <FaHistory size={14} />
                                    </motion.button>
                                    <motion.button
                                        whileHover={{ scale: 1.1, background: 'var(--primary)', color: '#fff' }}
                                        whileTap={{ scale: 0.9 }}
                                        className="control-btn"
                                        style={{ background: 'var(--primary-glow)', color: 'var(--primary)' }}
                                        title="Modify Manifest"
                                    >
                                        <FaEdit size={14} />
                                    </motion.button>
                                    <motion.button
                                        whileHover={{ scale: 1.1, background: 'var(--error)', color: '#fff' }}
                                        whileTap={{ scale: 0.9 }}
                                        className="control-btn"
                                        style={{ background: 'rgba(239, 68, 68, 0.05)', color: 'var(--error)' }}
                                        onClick={() => handleDelete(s.id)}
                                        title="Relinquish Node"
                                    >
                                        <FaTrash size={14} />
                                    </motion.button>
                                </div>
                            </td>
                        </motion.tr>
                    ))
                )}
            </tbody>
        </table>
    </div>

    {showModal && <AddSupplierModal onClose={() => setShowModal(false)} onSuccess={loadData} />}

    {showTransactionModal && selectedSupplier && (
        <TransactionHistoryModal
            isOpen={showTransactionModal}
            onClose={() => {
                setShowTransactionModal(false);
                setSelectedSupplier(null);
            }}
            entityType="supplier"
            entityId={selectedSupplier.id}
            entityName={selectedSupplier.lender_name}
        />
    )}
</div>
```

```
    );
};

export default Suppliers;
```

frontend/src/pages/Subscriptions.tsx

```
// frontend/src/pages/Subscriptions.tsx
import { motion } from 'framer-motion';
import React, { useEffect, useState } from 'react';
import { FaCheckCircle, FaCrown, FaGem, FaLayerGroup, FaRocket, FaShieldAlt } from 'react-icons/fa';
import { useAuthUser } from '../hooks/useAuthUser';

interface Plan {
    id: number;
    plan_name: string;
    enabled_modules: string;
    max_branches: number;
    max_users: number;
    ai_enabled: boolean;
    analytics_enabled: boolean;
    storage_limit_gb: number;
    expiry_date: string;
    status: string;
}

const Subscriptions: React.FC = () => {
    const { user } = useAuthUser();
    const [plans, setPlans] = useState<Plan[]>([]);
    const [myPlan, setMyPlan] = useState<Plan | null>(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const fetchData = async () => {
            try {
                const token = localStorage.getItem('erp-token');
                // Fetch all plans (if superadmin) or just available plans
                const plansRes = await fetch(`${import.meta.env.VITE_API_URL}/api/subscriptions/plans`, {
                    headers: { 'Authorization': `Bearer ${token}` }
                });
                const plansData = await plansRes.json();
                if (plansRes.ok) setPlans(plansData);

                // For now, let's assume we have a way to get current company's plan
                const companyId = user?.active_company_id;
                if (companyId) {
                   const companyRes = await fetch(`${import.meta.env.VITE_API_URL}/api/company/profile`, {
                        headers: { 'Authorization': `Bearer ${token}` }
                    });
                    const companyData = await companyRes.json();
                    if (companyRes.ok && companyData.subscription_id) {
                         const plan = plansData.find((p: Plan) => p.id === companyData.subscription_id);
                         setMyPlan(plan);
                    }
                }
            } catch (error) {
                console.error("Error fetching subscription data:", error);
            } finally {
                setLoading(false);
            }
        };
        fetchData();
    }, [user]);

    const containerVariants = {
        hidden: { opacity: 0 },
        visible: { opacity: 1, transition: { staggerChildren: 0.1 } }
    };

    const cardVariants = {
        hidden: { y: 20, opacity: 0 },
        visible: { y: 0, opacity: 1 }
    };

    return (
        <div className="space-y-10">
            {/* Header */}
            <div className="text-center max-w-2xl mx-auto space-y-4">
                <h1 className="text-4xl font-black text-slate-900 tracking-tight">Subscription & Licensing</h1>
                <p className="text-slate-500 text-lg">Scale your enterprise with precision-engineered modules and flexible resource limits.</p>
            </div>

            {/* Current Plan Summary (for Tenants) */}
            {myPlan && (
                <motion.div
                    initial={{ scale: 0.95, opacity: 0 }}
                    animate={{ scale: 1, opacity: 1 }}
                    className="bg-gradient-to-br from-indigo-600 to-blue-700 p-1 rounded-3xl shadow-2xl overflow-hidden"
                >
                    <div className="bg-white rounded-[22px] p-8 flex flex-col md:flex-row items-center justify-between gap-8">
                        <div className="space-y-4">
                             <div className="flex items-center gap-3">
                                <span className="bg-indigo-100 text-indigo-700 px-3 py-1 rounded-full text-xs font-black tracking-widest uppercase">You
                                <span className="bg-emerald-100 text-emerald-700 px-3 py-1 rounded-full text-xs font-black tracking-widest uppercase">{m
                             </div>
                             <h2 className="text-4xl font-black text-slate-900">{myPlan.plan_name}</h2>
                             <div class="flex gap-6 items-center text-slate-500 font-medium">
                                <span class="flex items-center gap-2"><FaLayerGroup /> {myPlan.max_branches} Branches</span>
                                <span class="flex items-center gap-2"><FaCheckCircle /> {myPlan.max_users} Users</span>
                                <span class="flex items-center gap-2 font-bold text-slate-900">Expires: {new Date(myPlan.expiry_date || '').toLocale
                             </div>
                        </div>
                        <button class="bg-slate-900 text-white px-10 py-4 rounded-2xl font-black shadow-xl shadow-slate-200 hover:scale-105 transiti
                            Manager Subscription
                        </button>
                    </div>
                </motion.div>
            )}

            {/* Global Admin: Plan Management / Upgrade Options */}
            <motion.div
                variants={containerVariants}
                initial="hidden"
```

```
                animate="visible"
                className="grid grid-cols-1 md:grid-cols-3 gap-8"
            >
                {plans.length > 0 ? plans.map((plan, idx) => (
                    <motion.div
                        key={plan.id}
                        variants={cardVariants}
                        whileHover={{ y: -10 }}
                        className={`glass-panel p-8 rounded-3xl border-2 transition-all ${plan.id === myPlan?.id ? 'border-blue-500 shadow-2xl bg-blue-5
                    >
                        <div className="flex justify-between items-start mb-6">
                            <div className={`p-4 rounded-2xl ${idx === 0 ? 'bg-slate-50 text-slate-400' : idx === 1 ? 'bg-blue-50 text-blue-600' : 'bg-a
                                {idx === 0 ? <FaRocket size={24} /> : idx === 1 ? <FaGem size={24} /> : <FaCrown size={24} />}
                            </div>
                            {plan.id === myPlan?.id && <span className="bg-blue-600 text-white px-4 py-1.5 rounded-full text-[10px] font-black uppercase
                        </div>

                        <h3 className="text-2xl font-black text-slate-900 mb-2">{plan.plan_name}</h3>
                        <p className="text-slate-400 text-sm mb-8 leading-relaxed font-medium">Best for {plan.max_branches === 1 ? 'small businesses' :

                        <div className="space-y-4 mb-10">
                            <FeatureItem icon={<FaCheckCircle className="text-emerald-500" />} text={`${plan.max_branches} Physical Branches`} />
                            <FeatureItem icon={<FaCheckCircle className="text-emerald-500" />} text={`${plan.max_users} Total Users`} />
                            <FeatureItem icon={<FaCheckCircle className="text-emerald-500" />} text={`${plan.storage_limit_gb}GB Enterprise Storage`} />

                            {plan.ai_enabled && <FeatureItem icon={<FaGem className="text-blue-500" />} text="AI Prediction Engine" />}
                            {plan.analytics_enabled && <FeatureItem icon={<FaLayerGroup className="text-indigo-500" />} text="Advanced Analytics" />}
                            <FeatureItem icon={<FaShieldAlt className="text-slate-400" />} text="Multi-tenant Isolation" />
                        </div>

                        <button className={`w-full py-4 rounded-2xl font-black text-sm uppercase tracking-widest transition-all ${plan.id === myPlan?.id
                            {plan.id === myPlan?.id ? 'Currently Using' : 'Upgrade Now'}
                        </button>
                    </motion.div>
                )) : (
                    [1,2,3].map(i => <div key={i} className="h-[500px] bg-slate-50 animate-pulse rounded-3xl" />)
                )}
            </motion.div>

            {/* Footer Prompt */}
            <div className="bg-slate-900 text-white p-12 rounded-[40px] flex flex-col items-center text-center gap-6">
                <h3 className="text-2xl font-bold italic tracking-tighter">Need a custom solution for your enterprise?</h3>
                <p className="text-slate-400 max-w-md">Our global account managers can design a personalized blueprint for your multi-country branch ope
                <button className="bg-white text-slate-900 px-8 py-3 rounded-full font-bold hover:scale-110 transition-all">Contact Enterprise Sales</bu
            </div>
        </div>
    );
};

const FeatureItem: React.FC<{ icon: React.ReactNode, text: string }> = ({ icon, text }) => (
    <div className="flex items-center gap-4 text-slate-700 font-medium">
        <span className="flex-shrink-0">{icon}</span>
        <span className="text-sm">{text}</span>
    </div>
);

export default Subscriptions;
```

frontend/src/pages/SignatureUpload.tsx

```
import React, { useState } from "react";

export default function SignatureUpload() {
    const [file, setFile] = useState<File | null>(null);
    const [preview, setPreview] = useState<string | null>(null);
    const [status, setStatus] = useState("");

    const upload = async () => {
        if (!file) {
            setStatus("Please select a file");
            return;
        }

        const form = new FormData();
        form.append("signature", file);

        const res = await fetch("http://localhost:5000/users/upload-signature", {
            method: "POST",
            body: form,
            credentials: "include",
        });

        const data = await res.json();

        if (data.url) {
            setStatus("Uploaded Successfully!");
        } else {
            setStatus("Upload Failed");
        }
    };

    return (
        <div>
            <h2 style={{ fontSize: 22, fontWeight: "bold" }}>Upload Signature</h2>

            <input
                type="file"
                accept="image/*"
                onChange={(e) => {
                    const file = e.target.files?.[0] || null;
                    setFile(file);
                    if (file) setPreview(URL.createObjectURL(file));
                }}
                style={{ marginTop: 20 }}
            />

            {preview && (
                <img
                    src={preview}
                    alt="Preview"
                    style={{
                        width: "200px",
                        marginTop: "20px",
                        border: "1px solid #ccc",
                        padding: "5px",
                    }}
                />
            )}

            <button
                onClick={upload}
                style={{
                    marginTop: 20,
                    padding: "10px 20px",
                    background: "#2563eb",
                    color: "#fff",
                    borderRadius: 6,
                }}
            >
                Upload Signature
            </button>

            <div style={{ marginTop: 15, color: "green" }}>{status}</div>
        </div>
    );
}
```

frontend/src/pages/Settings.tsx

```
import React, { useEffect, useState } from 'react';
import {
    FaBuilding, FaCheckCircle, FaCog,
    FaExclamationTriangle,
    FaIdBadge,
    FaMapMarkerAlt, FaPercentage,
    FaPlus, FaSave, FaSearch,
    FaToggleOff, FaToggleOn, FaTrash, FaUniversity,
    FaUserShield
} from 'react-icons/fa';
import { fetchProfile, updateProfile } from '../api/companyApi';
import { apiFetch } from '../utils/api';

// --- STYLES ---
const styles = {
    container: { /* Replaced by CSS .settings-container */ },
    sidebar: { /* Replaced by CSS .settings-sidebar */ },
    sidebarTitle: { fontSize: '0.8rem', textTransform: 'uppercase' as const, letterSpacing: '0.05em', color: '#94a3b8', fontWeight: 700, marginBottom:
    navItem: (isActive: boolean) => ({ display: 'flex', alignItems: 'center', gap: '12px', padding: '12px 16px', marginBottom: '4px', borderRadius: '8p
    contentArea: { flexGrow: 1, backgroundColor: '#ffffff', borderRadius: '16px', boxShadow: '0 4px 20px -2px rgba(0, 0, 0, 0.05)', border: '1px solid #
    header: { padding: '25px 35px', borderBottom: '1px solid #f1f5f9', backgroundColor: '#ffffff' },
    headerTitle: { fontSize: '1.4rem', fontWeight: 700, color: '#1e293b', margin: 0 },
    headerSubtitle: { color: '#64748b', marginTop: '6px', fontSize: '0.9rem' },
    formBody: { padding: '35px', overflowY: 'auto' as const, flex: 1 },
    section: { marginBottom: '35px' },
    sectionHeader: { display: 'flex', alignItems: 'center', gap: '10px', marginBottom: '20px', paddingBottom: '10px', borderBottom: '1px solid #e2e8f0'
    sectionTitle: { fontSize: '1rem', fontWeight: 600, color: '#334155' },
    grid: { display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(250px, 1fr))', gap: '20px' },
    inputGroup: { marginBottom: '0' },
    label: { display: 'block', fontSize: '0.85rem', fontWeight: 600, color: '#475569', marginBottom: '8px' },
    input: { width: '100%', padding: '10px 14px', borderRadius: '8px', border: '1px solid #cbd5e1', fontSize: '0.9rem', color: '#1e293b', backgroundCol
    footer: { padding: '20px 35px', backgroundColor: '#f8fafc', borderTop: '1px solid #e2e8f0', display: 'flex', justifyContent: 'flex-end', alignItems
    saveBtn: { padding: '10px 24px', backgroundColor: '#2563eb', color: 'white', border: 'none', borderRadius: '8px', fontWeight: 600, fontSize: '0.9rem
    table: { width: '100%', borderCollapse: 'collapse' as const, fontSize: '0.9rem' },
    th: { textAlign: 'left' as const, padding: '12px', borderBottom: '2px solid #e2e8f0', color: '#64748b', fontSize: '0.85rem' },
    td: { padding: '12px', borderBottom: '1px solid #f1f5f9', color: '#334155' },
    badge: (color: string) => ({ padding: '4px 8px', borderRadius: '4px', fontSize: '0.75rem', fontWeight: 600, background: `${color}20`, color: color
    card: { background: 'white', borderRadius: '12px', padding: '20px', boxShadow: '0 1px 3px rgba(0,0,0,0.05)', marginBottom: '20px', border: '1px sol
};

// --- COMPONENTS ---

const InputField = ({ label, value, onChange, placeholder, type = "text", required = false, width = "100%" }: any) => (
    <div style={{ ...styles.inputGroup, width }}>
        <label style={styles.label}>{label} {required && <span style={{ color: '#ef4444' }}>*</span>}</label>
        <input type={type} value={value || ''} onChange={onChange} placeholder={placeholder} style={styles.input} />
    </div>
);

const Notification = ({ msg }: { msg: { type: string, text: string } }) => {
    if (!msg.text) return null;
    const isSuccess = msg.type === 'success';
    return (
        <div style={{ margin: '0 0 20px', padding: '12px 16px', borderRadius: '8px', fontSize: '0.9rem', fontWeight: 500, display: 'flex', alignItems:
            {isSuccess ? <FaCheckCircle /> : <FaExclamationTriangle />} {msg.text}
        </div>
    );
};

// --- TAB 1: COMPANY PROFILE ---
const CompanyProfileTab = ({ profile, setProfile, handleSave, saving, msg }: any) => {
    const isMobile = window.innerWidth <= 768;
    return (
        <>
            <div style={{ ...styles.header, padding: isMobile ? '20px 25px' : '25px 35px' }}>
                <h1 style={{ ...styles.headerTitle, fontSize: isMobile ? '1.25rem' : '1.4rem' }}>Company Profile</h1>
                <p style={styles.headerSubtitle}>Manage business identity, address, and banking details.</p>
            </div>
            <div style={{ ...styles.formBody, padding: isMobile ? '20px' : '35px' }}>
                <Notification msg={msg} />
                <form onSubmit={handleSave}>
                    <div style={styles.section}>
                        <div style={styles.sectionHeader}><FaBuilding color="#64748b" /><span style={styles.sectionTitle}>Business Info</span></div>
                        <div style={{ display: 'grid', gridTemplateColumns: isMobile ? '1fr' : 'repeat(auto-fit, minmax(250px, 1fr))', gap: '20px' }}>
                            <InputField label="Company Name" value={profile?.company_name} onChange={(e:any) => setProfile({...profile, company_name: e
                            <InputField label="GSTIN" value={profile?.gstin} onChange={(e:any) => setProfile({...profile, gstin: e.target.value})} />
                        </div>
                    </div>
                    <div style={styles.section}>
                        <div style={styles.sectionHeader}><FaMapMarkerAlt color="#64748b" /><span style={styles.sectionTitle}>Address</span></div>
                        <div style={{ marginBottom: '20px' }}><InputField label="Street Address" value={profile?.address_line1} onChange={(e:any) => se
                        <div style={{ display: 'grid', gridTemplateColumns: isMobile ? '1fr' : 'repeat(auto-fit, minmax(250px, 1fr))', gap: '20px' }}>
                            <InputField label="City - Zip" value={profile?.city_pincode} onChange={(e:any) => setProfile({...profile, city_pincode: e.ta
                            <InputField label="State" value={profile?.state} onChange={(e:any) => setProfile({...profile, state: e.target.value})} />
                        </div>
                    </div>
                    <div style={{...styles.section, marginBottom: 0}}>
                        <div style={styles.sectionHeader}><FaUniversity color="#64748b" /><span style={styles.sectionTitle}>Banking</span></div>
                        <div style={{ display: 'grid', gridTemplateColumns: isMobile ? '1fr' : 'repeat(auto-fit, minmax(250px, 1fr))', gap: '20px' }}>
                            <InputField label="Bank Name" value={profile?.bank_name} onChange={(e:any) => setProfile({...profile, bank_name: e.target.va
                            <InputField label="Account No" value={profile?.bank_account_no} onChange={(e:any) => setProfile({...profile, bank_account_no
                        </div>
                        <div style={{marginTop: '20px'}}><InputField label="IFSC Code" value={profile?.bank_ifsc_code} onChange={(e:any) => setProfile(
                    </div>
                    <div style={{marginTop: '30px', display:'flex', justifyContent: isMobile ? 'center' : 'flex-end'}}>
                        <button type="submit" disabled={saving} style={{...styles.saveBtn, width: isMobile ? '100%' : 'auto', opacity: saving ? 0.7 : 1
                    </div>
                </form>
            </div>
        </>
    );
};

// --- TAB 2: USERS & PERMISSIONS ---
const UsersTab = () => {
    const [view, setView] = useState<'users' | 'roles'>('users');
```

```
    const [users, setUsers] = useState<any[]>([]);
    const [employees, setEmployees] = useState<any[]>([]);
    const [roles, setRoles] = useState<any[]>([]);
    const [permissions, setPermissions] = useState<any[]>([]);
    const [mappings, setMappings] = useState<any[]>([]);
    const [selectedRole, setSelectedRole] = useState<number | null>(null);
    const [selectedEmpId, setSelectedEmpId] = useState('');
    const [credentials, setCredentials] = useState({ username: '', password: '', role: 'staff' });
    const [searchTerm, setSearchTerm] = useState('');
    const [permSearch, setPermSearch] = useState('');

    useEffect(() => {
        const load = async () => {
            try {
                const [u, e, m] = await Promise.all([
                    apiFetch('/users/staff').then(r=>r.json()),
                    apiFetch('/employees/unlinked').then(r=>r.json()),
                    apiFetch('/roles/matrix').then(r=>r.json())
                ]);
                setUsers(u); setEmployees(e);
                setRoles(m.roles || []); setPermissions(m.permissions || []); setMappings(m.mappings || []);
                if (m.roles && m.roles.length > 0) setSelectedRole(m.roles[0].id);
            } catch(e) { console.error(e); }
        };
        load();
    }, []);

    const handleCreateLogin = async (e: React.FormEvent) => {
        e.preventDefault();
        if(!selectedEmpId) return alert("Select an employee first");
        try {
            await apiFetch('/users/staff', { method: 'POST', body: { ...credentials, employee_id: Number(selectedEmpId) } });
            alert("Login created successfully!"); window.location.reload();
        } catch(e) { alert("Error creating login"); }
    };

    const togglePerm = async (permId: number) => {
        if (!selectedRole) return;
        const exists = mappings.some(m => m.role_id === selectedRole && m.permission_id === permId);
        if(exists) setMappings(mappings.filter(m => !(m.role_id === selectedRole && m.permission_id === permId)));
        else setMappings([...mappings, { role_id: selectedRole, permission_id: permId }]);
        await apiFetch('/roles/toggle', { method: 'POST', body: { role_id: selectedRole, permission_id: permId, enabled: !exists } });
    };

    const modules = [...new Set(permissions.map(p => p.module))];

    const isMobile = window.innerWidth <= 768;

    return (
        <div style={{ padding: isMobile ? '0' : '0' }}>
            <div style={{ ...styles.header, padding: isMobile ? '20px' : '25px 35px' }}>
                <div style={{
                    display: 'flex',
                    flexDirection: isMobile ? 'column' : 'row',
                    justifyContent: 'space-between',
                    alignItems: isMobile ? 'flex-start' : 'center',
                    gap: isMobile ? '20px' : '0'
                }}>
                    <div>
                        <h1 style={{ ...styles.headerTitle, fontSize: isMobile ? '1.25rem' : '1.4rem' }}>Users & Permissions</h1>
                        <p style={styles.headerSubtitle}>Manage staff access.</p>
                    </div>
                    <div style={{
                        background: '#f1f5f9',
                        padding: '4px',
                        borderRadius: '10px',
                        display: 'flex',
                        width: isMobile ? '100%' : 'auto',
                        boxShadow: 'inset 0 1px 2px rgba(0,0,0,0.05)'
                    }}>
                        <button
                            onClick={()=>setView('users')}
                            style={{
                                flex: isMobile ? 1 : 'none',
                                padding: isMobile ? '10px' : '8px 16px',
                                border: 'none',
                                background: view===='users'?'white':'transparent',
                                borderRadius:'8px',
                                cursor:'pointer',
                                fontWeight:700,
                                fontSize: isMobile ? '0.85rem' : '0.9rem',
                                color: view===='users'?'#2563eb':'#64748b',
                                boxShadow: view===='users'?'0 2px 4px rgba(0,0,0,0.1)': 'none',
                                transition: 'all 0.2s'
                            }}
                        >
                            Staff Logins
                        </button>
                        <button
                            onClick={()=>setView('roles')}
                            style={{
                                flex: isMobile ? 1 : 'none',
                                padding: isMobile ? '10px' : '8px 16px',
                                border: 'none',
                                background: view===='roles'?'white':'transparent',
                                borderRadius:'8px',
                                cursor:'pointer',
                                fontWeight:700,
                                fontSize: isMobile ? '0.85rem' : '0.9rem',
                                color: view===='roles'?'#2563eb':'#64748b',
                                boxShadow: view===='roles'?'0 2px 4px rgba(0,0,0,0.1)': 'none',
                                transition: 'all 0.2s'
                            }}
                        >
                            Roles & Access
                        </button>
                    </div>
                </div>
            </div>

            <div style={styles.formBody}>
                {view === 'users' ? (
```

```
        <div>
            <div style={{ background: '#f8fafc', padding: isMobile ? '20px' : '25px', borderRadius: '16px', border: '1px solid #e2e8f0', ma
                <h4 style={{ margin: '0 0 20px 0', color: '#1e293b', display: 'flex', alignItems: 'center', gap: '10px', fontWeight: 700 }}>
                <form onSubmit={handleCreateLogin}>
                    <div style={{ marginBottom: '20px' }}>
                        <label style={styles.label}>Link Employee</label>
                        <select value={selectedEmpId} onChange={(e) => {setSelectedEmpId(e.target.value); setCredentials({...credentials, us
                            <option value="">-- Select Unlinked Employee --</option>
                            {employees.map(emp => <option key={emp.id} value={emp.id}>{emp.name} ({emp.designation})</option>)}
                        </select>
                    </div>
                    {selectedEmpId && (
                        <div style={{
                            display: 'grid',
                            gridTemplateColumns: isMobile ? '1fr' : '1fr 1fr 1fr auto',
                            gap: '15px',
                            alignItems: 'end'
                        }}>
                            <InputField label="Username" value={credentials.username} onChange={(e:any) => setCredentials({...credentials, 
                            <InputField label="Password" type="password" value={credentials.password} onChange={(e:any) => setCredentials({
                            <div>
                                <label style={styles.label}>Role</label>
                                <select value={credentials.role} onChange={(e:any) => setCredentials({...credentials, role: e.target.value}
                                    {roles.map(r => <option key={r.id} value={r.name}>{r.name}</option>)}
                                </select>
                            </div>
                            <button type="submit" style={{ ...styles.saveBtn, width: isMobile ? '100%' : 'auto', justifyContent: 'center' }
                        </div>
                    )}
                </form>
            </div>

            {/* Search Toolbar */}
            <div className="card" style={{
                padding: '0 24px',
                marginBottom: '24px',
                display: 'flex',
                alignItems: 'center',
                gap: '16px',
                border: '1px solid var(--border-color)',
                height: '60px',
                background: 'white',
                boxShadow: 'var(--shadow-sm)',
                borderRadius: '16px'
            }}>
                <FaSearch style={{ color: 'var(--text-light)' }} size={18} />
                <input
                    placeholder="Search staff by name, user, or role..."
                    value={searchTerm}
                    onChange={(e) => setSearchTerm(e.target.value)}
                    style={{
                        border: 'none',
                        width: '100%',
                        outline: 'none',
                        background: 'transparent',
                        fontSize: '1rem',
                        fontWeight: 500,
                        color: 'var(--text-main)',
                        letterSpacing: '-0.2px'
                    }}
                />
            </div>

            <div style={{ overflowX: 'auto', WebkitOverflowScrolling: 'touch', border: '1px solid #e2e8f0', borderRadius: '16px', backgroun
                <table style={{ ...styles.table, minWidth: '700px' }}>
                    <thead><tr style={{background:'#f8fafc'}}><th style={styles.th}>Employee</th><th style={styles.th}>Username</th><th sty
                    <tbody>
                        {users.filter(u =>
                            !searchTerm.trim() ||
                            u.username.toLowerCase().includes(searchTerm.toLowerCase()) ||
                            u.employee_name?.toLowerCase().includes(searchTerm.toLowerCase()) ||
                            u.role?.toLowerCase().includes(searchTerm.toLowerCase())
                        ).map(u => (
                            <tr key={u.id}>
                                <td style={styles.td}>
                                    <div style={{fontWeight: 700, color: '#1e293b'}}>{u.employee_name || 'System User'}</div>
                                    <span style={{fontSize:'0.75rem', color:'#94a3b8', fontWeight: 500}}>{u.employee_designation}</span>
                                </td>
                                <td style={{ ...styles.td, fontWeight: 600, color: '#475569' }}>{u.username}</td>
                                <td style={styles.td}>
                                    <span style={{padding:'6px 14px', background:'#e0f2fe', color:'#0369a1', borderRadius:'20px', fontSize:
                                        {u.role}
                                    </span>
                                </td>
                                <td style={{ ...styles.td, color: '#64748b' }}>{new Date(u.created_at).toLocaleDateString()}</td>
                                <td style={styles.td}><FaTrash color="#ef4444" style={{cursor:'pointer', opacity: 0.7}} title="Delete Login
                            </tr>
                        ))}
                    </tbody>
                </table>
            </div>
        </div>
    ) : (
        <div style={{
            display: 'flex',
            flexDirection: isMobile ? 'column' : 'row',
            height: isMobile ? 'auto' : '100%',
            border: '1px solid #e2e8f0',
            borderRadius: '16px',
            overflow: 'hidden',
            background: 'white'
        }}>
            <div style={{
                width: isMobile ? '100%' : '250px',
                background: '#f8fafc',
                borderRight: isMobile ? 'none' : '1px solid #e2e8f0',
                borderBottom: isMobile ? '1px solid #e2e8f0' : 'none'
            }}>
                <div style={{ padding: '20px', borderBottom: '1px solid #e2e8f0', fontWeight: 800, color: '#1e293b', fontSize: '0.75rem', te
                <div style={{ display: isMobile ? 'flex' : 'block', overflowX: isMobile ? 'auto' : 'visible' }}>
                    {roles.map(r => (
                        <div
```

```
                                    key={r.id}
                                    onClick={() => setSelectedRole(r.id)}
                                    style={{
                                        padding: isMobile ? '15px 20px' : '14px 25px',
                                        cursor: 'pointer',
                                        fontSize: '0.9rem',
                                        fontWeight: 700,
                                        whiteSpace: isMobile ? 'nowrap' : 'normal',
                                        background: selectedRole === r.id ? 'white' : 'transparent',
                                        color: selectedRole === r.id ? '#2563eb' : '#64748b',
                                        borderLeft: isMobile ? 'none' : (selectedRole === r.id ? '4px solid #2563eb' : '4px solid transparent'),
                                        borderBottom: isMobile ? (selectedRole === r.id ? '3px solid #2563eb' : '3px solid transparent') : 'none',
                                        transition: 'all 0.2s'
                                    }}
                                >
                                {r.name}
                            </div>
                        ))}
                    </div>
                </div>
                <div style={{ flex: 1, padding: '0', background: 'white', overflowY: 'auto', display: 'flex', flexDirection: 'column' }}>
                        {selectedRole ? (
                            <>
                                <div style={{ padding: '15px 25px', borderBottom: '1px solid #e2e8f0', background: '#fcfcfc', display: 'flex', alig
                                    <FaSearch color="#94a3b8" />
                                    <input
                                        placeholder="Search permissions..."
                                        value={permSearch}
                                        onChange={(e) => setPermSearch(e.target.value)}
                                        style={{ border: 'none', background: 'transparent', outline: 'none', width: '100%', fontSize: '0.9rem' }}
                                    />
                                </div>
                                <div style={{ flex: 1, overflowY: 'auto' }}>
                                    {modules.map(mod => {
                                        const filteredPerms = permissions.filter(p =>
                                            p.module === mod && (
                                                !permSearch.trim() ||
                                                p.description?.toLowerCase().includes(permSearch.toLowerCase()) ||
                                                p.action.toLowerCase().includes(permSearch.toLowerCase())
                                            )
                                        );
                                        if (filteredPerms.length === 0) return null;
                                        return (
                                            <div key={mod}>
                                                <div style={{ background: '#f1f5f9', padding: '10px 25px', fontSize: '0.8rem', fontWeight: 700, colo
                                                    {mod}
                                                </div>
                                                <div>
                                                    {filteredPerms.map(p => {
                                                        const isEnabled = mappings.some(m => m.role_id === selectedRole && m.permission_id === p.id
                                                        return (
                                                            <div key={p.id} onClick={() => togglePerm(p.id)} style={{ display: 'flex', alignItems:
                                                                <div style={{display: 'flex', flexDirection: 'column'}}>
                                                                    <span style={{ fontSize: '0.9rem', color: '#334155', fontWeight: 500 }}>{p.desc
                                                                    <span style={{ fontSize: '0.75rem', color: '#94a3b8' }}>{p.action}</span>
                                                                </div>
                                                                {isEnabled
                                                                    ? <FaToggleOn size={28} color="#2563eb" />
                                                                    : <FaToggleOff size={28} color="#cbd5e1" />
                                                                }
                                                            </div>
                                                        );
                                                    })}
                                                </div>
                                            </div>
                                        );
                                    })}
                                </div>
                            </>
                        ) : (
                            <div style={{ padding: '40px', textAlign: 'center', color: '#94a3b8' }}>Select a role to view permissions</div>
                        )}
                    </div>
                </div>
            )}
        </div>
    </div>
    );
};

// --- TAB 3: TAX & GST ---
const TaxTab = () => {
    const [data, setData] = useState({ output_gst: 0, input_tax_credit: 0 });
    const [loading, setLoading] = useState(true);
    const isMobile = window.innerWidth <= 768;

    useEffect(() => {
        apiFetch('/settings/tax-summary').then(r => r.json()).then(json => { setData(json); setLoading(false); }).catch(console.error);
    }, []);

    const fmt = (n: number) => n.toLocaleString('en-IN', { maximumFractionDigits: 2 });

    return (
        <>
            <div style={{ ...styles.header, padding: isMobile ? '20px 25px' : '25px 35px' }}>
                <h1 style={{ ...styles.headerTitle, fontSize: isMobile ? '1.25rem' : '1.4rem' }}>Tax & GST</h1>
                <p style={styles.headerSubtitle}>Financial tax overview.</p>
            </div>
            <div style={{ ...styles.formBody, padding: isMobile ? '20px' : '35px' }}>
                <div style={{ display: 'grid', gridTemplateColumns: isMobile ? '1fr' : 'repeat(auto-fit, minmax(250px, 1fr))', gap: '20px' }}>
                    <div style={{ ...styles.card, borderLeft: '5px solid #ef4444', padding: '24px' }}>
                        <h3 style={{...styles.sectionTitle, fontSize: '0.9rem', textTransform: 'uppercase', color: '#64748b', marginBottom: '10px'}}>GST
                        <h1 style={{fontSize: isMobile ? '1.75rem' : '2.2rem', margin: '10px 0', color: '#1e293b', fontWeight: 800}}>{loading ? '...' :
                    </div>
                    <div style={{ ...styles.card, borderLeft: '5px solid #10b981', padding: '24px' }}>
                        <h3 style={{...styles.sectionTitle, fontSize: '0.9rem', textTransform: 'uppercase', color: '#64748b', marginBottom: '10px'}}>Inp
                        <h1 style={{fontSize: isMobile ? '1.75rem' : '2.2rem', margin: '10px 0', color: '#10b981', fontWeight: 800}}>{loading ? '...' :
                    </div>
                </div>
            </div>
        </>
    );
```

```
};

// --- TAB 4: SYSTEM (MOCKED REALISTIC DATA) ---
const SystemTab = () => {
    const isMobile = window.innerWidth <= 768;
    // Pre-filled with realistic "Mock" data
    const [logs, setLogs] = useState<any[]>([
        { timestamp: '2024-03-10T14:30:00', username: 'admin', action: 'POST /api/settings/backup' },
        { timestamp: '2024-03-10T12:15:00', username: 'sales_team', action: 'POST /api/invoices' },
        { timestamp: '2024-03-10T11:45:00', username: 'manager', action: 'PUT /api/employees/4' },
        { timestamp: '2024-03-10T10:20:00', username: 'admin', action: 'DELETE /api/products/89' },
        { timestamp: '2024-03-09T16:55:00', username: 'staff_1', action: 'POST /api/auth/login' },
        { timestamp: '2024-03-09T14:10:00', username: 'system', action: 'AUTO_BACKUP_DAILY' },
    ]);
    const [storage, setStorage] = useState({ db_size: '485 MB', record_count: 12450 });

    useEffect(() => {
        // Attempt fetch, fallback to mock if empty
        apiFetch('/settings/system-logs').then(r => r.json()).then(data => { if (data && data.length > 0) setLogs(data); }).catch(console.error);
        apiFetch('/settings/storage').then(r => r.json()).then(data => { if (data && data.record_count > 0) setStorage(data); }).catch(console.error);
    }, []);

    return (
        <div style={{ display: 'flex', flexDirection: 'column', height: '100%' }}>
            <div style={{ ...styles.header, padding: isMobile ? '20px 25px' : '25px 35px' }}>
                <h1 style={{ ...styles.headerTitle, fontSize: isMobile ? '1.25rem' : '1.4rem' }}>System Status</h1>
                <p style={styles.headerSubtitle}>Technical performance metrics.</p>
            </div>
            <div style={{ ...styles.formBody, padding: isMobile ? '20px' : '35px' }}>
                {/* Stats Cards */}
                <div style={{ display: 'grid', gridTemplateColumns: isMobile ? '1fr' : 'repeat(3, 1fr)', gap: '20px', marginBottom: '32px' }}>
                    <div style={{ ...styles.card, borderLeft: '5px solid #2563eb', padding: '24px', transition: 'transform 0.2s' }}>
                        <div style={{ fontSize: '0.8rem', color: '#64748b', fontWeight: 800, textTransform: 'uppercase', letterSpacing: '0.05em' }}>DATA
                        <div style={{ fontSize: isMobile ? '1.75rem' : '2.2rem', fontWeight: 900, color: '#1e293b', marginTop: '8px' }}>{storage.db_size
                    </div>
                    <div style={{ ...styles.card, borderLeft: '5px solid #10b981', padding: '24px' }}>
                        <div style={{ fontSize: '0.8rem', color: '#64748b', fontWeight: 800, textTransform: 'uppercase', letterSpacing: '0.05em' }}>TOTA
                        <div style={{ fontSize: isMobile ? '1.75rem' : '2.2rem', fontWeight: 900, color: '#1e293b', marginTop: '8px' }}>{storage.record_
                    </div>
                    <div style={{ ...styles.card, borderLeft: '5px solid #f59e0b', padding: '24px' }}>
                        <div style={{ fontSize: '0.8rem', color: '#64748b', fontWeight: 800, textTransform: 'uppercase', letterSpacing: '0.05em' }}>SERV
                        <div style={{ fontSize: isMobile ? '1.75rem' : '2.2rem', fontWeight: 900, color: '#16a34a', marginTop: '8px' }}>Online</div>
                    </div>
                </div>

                {/* Logs Table */}
                <div style={{ ...styles.card, padding: 0, overflow: 'hidden', border: '1px solid #e2e8f0', borderRadius: '16px' }}>
                    <div style={{ padding: '20px 24px', borderBottom: '1px solid #f1f5f9', display:'flex', alignItems:'center', gap:'12px', background:
                        <span style={{ width: '10px', height: '10px', borderRadius: '50%', background: '#10b981', boxShadow: '0 0 0 4px rgba(16, 185, 1
                        <h3 style={{ ...styles.sectionTitle, margin: 0, fontWeight: 800, color: '#1e293b' }}>Recent Activity Logs</h3>
                    </div>
                    <div style={{ overflowX: 'auto', WebkitOverflowScrolling: 'touch' }}>
                        <table style={{ ...styles.table, minWidth: '800px' }}>
                            <thead>
                                <tr style={{ background: '#f8fafc' }}>
                                    <th style={{ ...styles.th, padding: '16px 24px' }}>Timestamp</th>
                                    <th style={{ ...styles.th, padding: '16px 24px' }}>User</th>
                                    <th style={{ ...styles.th, padding: '16px 24px' }}>Action</th>
                                    <th style={{ ...styles.th, padding: '16px 24px' }}>Status</th>
                                </tr>
                            </thead>
                            <tbody>
                                {logs.map((log, i) => (
                                    <tr key={i} style={{ borderBottom: '1px solid #f1f5f9' }}>
                                        <td style={{ ...styles.td, padding: '16px 24px', fontFamily: 'monospace', color: '#64748b', fontSize: '0.85rem'
                                            {new Date(log.timestamp).toLocaleString()}
                                        </td>
                                        <td style={{ ...styles.td, padding: '16px 24px', fontWeight: 700 }}>
                                            <div style={{ display: 'flex', alignItems: 'center', gap: '10px' }}>
                                                <div style={{ width: '28px', height: '28px', borderRadius: '50%', background: '#eff6ff', color: '#2563eb
                                                    {log.username.charAt(0).toUpperCase()}
                                                </div>
                                                {log.username}
                                            </div>
                                        </td>
                                        <td style={{ ...styles.td, padding: '16px 24px', color: '#475569', fontWeight: 500 }}>{log.action}</td>
                                        <td style={{ ...styles.td, padding: '16px 24px' }}>
                                            <span style={{ padding: '6px 12px', borderRadius: '20px', background: '#ecfdf5', color: '#059669', fontSize
                                                Success
                                            </span>
                                        </td>
                                    </tr>
                                ))}
                            </tbody>
                        </table>
                    </div>
                </div>
            </div>
        </div>
    );
};

// --- MAIN LAYOUT ---
const Settings: React.FC = () => {
    const [activeTab, setActiveTab] = useState('company');
    const [profile, setProfile] = useState<any>(null);
    const [loading, setLoading] = useState(false);
    const [saving, setSaving] = useState(false);
    const [msg, setMsg] = useState({ type: '', text: '' });

    useEffect(() => {
        const load = async () => {
            setLoading(true);
            try {
                const data = await fetchProfile();
                setProfile(data);
            } catch (err) { console.error(err); } finally { setLoading(false); }
        };
        load();
    }, []);

    const handleSave = async (e: React.FormEvent) => {
```

```
        e.preventDefault();
        setSaving(true);
        setMsg({ type: '', text: '' });
        try {
            await updateProfile(profile);
            setMsg({ type: 'success', text: 'Settings updated.' });
        } catch (err) { setMsg({ type: 'error', text: 'Failed to update.' }); } finally { setSaving(false); }
    };

    if (loading) return <div className="p-8 text-center" style={{padding: '40px'}}>Loading...</div>;

    const renderContent = () => {
        switch(activeTab) {
            case 'company': return <CompanyProfileTab profile={profile} setProfile={setProfile} handleSave={handleSave} saving={saving} msg={msg} />;
            case 'users': return <UsersTab />;
            case 'tax': return <TaxTab />;
            case 'system': return <SystemTab />;
            default: return null;
        }
    };

    const menuItems = [
        { id: 'company', label: 'Company Profile', icon: FaBuilding },
        { id: 'users', label: 'Users & Permissions', icon: FaUserShield },
        { id: 'tax', label: 'Tax & GST', icon: FaPercentage },
        { id: 'system', label: 'System Preferences', icon: FaCog },
    ];

    return (
        <div className="settings-container" style={{ minHeight: '100vh', background: '#f8fafc' }}>
            <div className="settings-sidebar">
                <div style={{ ...styles.sidebarTitle, padding: '24px 16px 12px' }}>Settings Menu</div>
                <div style={{ padding: '0 12px' }}>
                    {menuItems.map(item => (
                        <div
                            key={item.id}
                            onClick={() => { setActiveTab(item.id); setMsg({type:'',text:''}); }}
                            style={styles.navItem(activeTab === item.id)}
                            className="nav-item"
                        >
                            <item.icon style={{ opacity: activeTab === item.id ? 1 : 0.6, fontSize: '1.2rem' }} />
                            {item.label}
                        </div>
                    ))}
                </div>
            </div>
            <div style={{ ...styles.contentArea, flex: 1, margin: window.innerWidth <= 768 ? '0' : '0' }}>{renderContent()}</div>
        </div>
    );
};

export default Settings;
```

frontend/src/pages/SalesOrders.tsx

```
import React, { useState } from 'react';
import { FaCalendarAlt, FaCheckCircle, FaClipboardList, FaFileInvoice, FaPlus, FaSearch, FaShippingFast, FaSync } from 'react-icons/fa';

interface SalesOrder {
    id: number;
    order_number: string;
    customer_id: number;
    customer_name: string;
    order_date: string;
    total_value: number;
    status: 'Confirmed' | 'Shipped' | 'Invoiced' | 'Pending';
}

const SalesOrders: React.FC = () => {
    // Mock Data for now
    const [orders, setOrders] = useState<SalesOrder[]>([
        { id: 1, order_number: 'SO-001', customer_id: 5, customer_name: 'Tech Solutions Ltd', order_date: '2024-07-20', total_value: 12500.50, status:
        { id: 2, order_number: 'SO-002', customer_id: 8, customer_name: 'Global Corp', order_date: '2024-07-22', total_value: 450.00, status: 'Shipped'
        { id: 3, order_number: 'SO-003', customer_id: 2, customer_name: 'Local Retailer', order_date: '2024-07-25', total_value: 3200.00, status: 'Pendi
    ]);
    const [loading, setLoading] = useState(false);
    const [searchTerm, setSearchTerm] = useState('');

    const refresh = () => {
        setLoading(true);
        setTimeout(() => setLoading(false), 500);
    };

    const handleCreateInvoice = (orderId: number) => {
        if (window.confirm(`Convert Sales Order SO-${orders.find(o => o.id === orderId)?.order_number} to Invoice?`)) {
            alert(`Converting Order #${orderId} to Invoice...`);
        }
    };

    const filteredOrders = orders.filter(o =>
        o.order_number.toLowerCase().includes(searchTerm.toLowerCase()) ||
        o.customer_name.toLowerCase().includes(searchTerm.toLowerCase())
    );

    const getStatusStyle = (status: string) => {
        switch (status) {
            case 'Confirmed': return { bg: '#dcfce7', color: '#166534', icon: <FaCheckCircle /> };
            case 'Shipped': return { bg: '#e0e7ff', color: '#4338ca', icon: <FaShippingFast /> };
            case 'Pending': return { bg: '#fef9c3', color: '#854d0e', icon: <FaClipboardList /> };
            case 'Invoiced': return { bg: '#f1f5f9', color: '#475569', icon: <FaFileInvoice /> };
            default: return { bg: '#f1f5f9', color: '#64748b', icon: null };
        }
    };

    return (
        <div className="page-transition">
            {/* Header */}
            <div style={{
                marginBottom: '32px',
                display: 'flex',
                justifyContent: 'space-between',
                alignItems: 'center',
                flexWrap: 'wrap',
                gap: '20px'
            }}>
                <div>
                    <h1 style={{ fontSize: '2rem', fontWeight: 900, color: 'var(--text-main)', letterSpacing: '-1.5px', margin: 0 }}>Sales Orders</h1>
                    <p style={{ color: 'var(--text-muted)', marginTop: '4px', fontWeight: 500 }}>Track and fulfill customer orders.</p>
                </div>
                <div style={{ display: 'flex', gap: '12px', flexWrap: 'wrap' }}>
                    <button className="btn-secondary" onClick={refresh} title="Refresh Lists" style={{ height: '48px', padding: '0 16px' }}>
                        <FaSync className={loading ? 'fa-spin' : ''} />
                    </button>
                    <button className="btn-primary" onClick={() => alert("Create Order Modal")} style={{ height: '48px', padding: '0 24px', gap: '10px'
                        <FaPlus /> Create New Order
                    </button>
                </div>
            </div>

            {/* Search Toolbar */}
            <div className="card" style={{
                padding: '0 24px',
                marginBottom: '32px',
                display: 'flex',
                alignItems: 'center',
                gap: '16px',
                border: '1px solid var(--border-color)',
                height: '64px',
                background: 'white',
                boxShadow: 'var(--shadow-sm)',
                borderRadius: '18px'
            }}>
                <FaSearch style={{ color: 'var(--text-light)' }} size={20} />
                <input
                    placeholder="Search by Order identity, customer name, or status..."
                    value={searchTerm}
                    onChange={(e) => setSearchTerm(e.target.value)}
                    style={{
                        border: 'none',
                        width: '100%',
                        outline: 'none',
                        background: 'transparent',
                        fontSize: '1.05rem',
                        fontWeight: 500,
                        color: 'var(--text-main)',
                        letterSpacing: '-0.2px'
                    }}
                />
            </div>

            {/* Table */}
```

```
            <div className="card" style={{ padding: 0, overflow: 'hidden', border: '1px solid var(--border-color)', background: 'white' }}>
                <div style={{ overflowX: 'auto', WebkitOverflowScrolling: 'touch' }}>
                    <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: '0.95rem', minWidth: '800px' }}>
                        <thead style={{ background: '#f8fafc', borderBottom: '1px solid var(--border-color)' }}>
                            <tr>
                                <th style={{ textAlign: 'left', padding: '16px 24px', fontWeight: 700, color: 'var(--text-muted)', fontSize: '0.75rem',
                                <th style={{ textAlign: 'left', padding: '16px 24px', fontWeight: 700, color: 'var(--text-muted)', fontSize: '0.75rem',
                                <th style={{ textAlign: 'left', padding: '16px 24px', fontWeight: 700, color: 'var(--text-muted)', fontSize: '0.75rem',
                                <th style={{ textAlign: 'right', padding: '16px 24px', fontWeight: 700, color: 'var(--text-muted)', fontSize: '0.75rem'
                                <th style={{ textAlign: 'center', padding: '16px 24px', fontWeight: 700, color: 'var(--text-muted)', fontSize: '0.75rem
                                <th style={{ textAlign: 'center', padding: '16px 24px', fontWeight: 700, color: 'var(--text-muted)', fontSize: '0.75rem
                            </tr>
                        </thead>
                        <tbody>
                            {filteredOrders.length === 0 ? (
                                <tr>
                                    <td colSpan={6} style={{ padding: '60px', textAlign: 'center', color: 'var(--text-muted)' }}>
                                        <FaClipboardList size={40} style={{ opacity: 0.2, marginBottom: '16px' }} />
                                        <p>No sales orders found.</p>
                                    </td>
                                </tr>
                            ) : (
                                filteredOrders.map(order => {
                                    const style = getStatusStyle(order.status);
                                    return (
                                        <tr key={order.id} className="page-transition" style={{ borderBottom: '1px solid var(--bg-body)', transition: '
                                            <td style={{ padding: '16px 24px', fontWeight: 600, color: 'var(--primary)' }}>
                                                {order.order_number}
                                            </td>
                                            <td style={{ padding: '16px 24px', fontWeight: 500, color: 'var(--text-main)' }}>
                                                {order.customer_name}
                                            </td>
                                            <td style={{ padding: '16px 24px', color: 'var(--text-muted)' }}>
                                                <div style={{ display: 'flex', alignItems: 'center', gap: '6px' }}>
                                                    <FaCalendarAlt size={12} />
                                                    {new Date(order.order_date).toLocaleDateString()}
                                                </div>
                                            </td>
                                            <td style={{ padding: '16px 24px', textAlign: 'right', fontWeight: 700, color: 'var(--text-main)' }}>
                                                ₹{order.total_value.toLocaleString(undefined, { minimumFractionDigits: 2 })}
                                            </td>
                                            <td style={{ padding: '16px 24px', textAlign: 'center' }}>
                                                <span style={{
                                                    padding: '6px 12px',
                                                    borderRadius: '20px',
                                                    fontSize: '0.75rem',
                                                    fontWeight: 700,
                                                    background: style.bg,
                                                    color: style.color,
                                                    display: 'inline-flex',
                                                    alignItems: 'center',
                                                    gap: '6px'
                                                }}>
                                                    {style.icon} {order.status}
                                                </span>
                                            </td>
                                            <td style={{ padding: '16px 24px', textAlign: 'center' }}>
                                                <div style={{ display: 'flex', gap: '8px', justifyContent: 'center' }}>
                                                    {order.status !== 'Invoiced' && (
                                                        <button
                                                            className="btn-secondary"
                                                            onClick={() => handleCreateInvoice(order.id)}
                                                            style={{ padding: '6px 12px', fontSize: '0.8rem', color: 'var(--primary)', borderColor: 'va
                                                            title="Convert to Invoice"
                                                        >
                                                            <FaFileInvoice /> Invoice
                                                        </button>
                                                    )}
                                                </div>
                                            </td>
                                        </tr>
                                    );
                                })
                            )}
                        </tbody>
                    </table>
                </div>
            </div>
        </div>
    );
};

export default SalesOrders;
```

frontend/src/pages/Reports.tsx

```
// frontend/src/pages/Reports.tsx
import { motion } from 'framer-motion';
import React, { useEffect, useState } from 'react';
import { FaArrowDown, FaArrowUp, FaBalanceScale, FaCalendarAlt, FaChartBar, FaDownload } from 'react-icons/fa';

interface PLReport {
    details: any[];
    totalIncome: number;
    totalExpense: number;
    netProfit: number;
}

const Reports: React.FC = () => {
    const [report, setReport] = useState<PLReport | null>(null);
    const [loading, setLoading] = useState(true);
    const [dates, setDates] = useState({
        start: new Date(new Date().getFullYear(), 0, 1).toISOString().split('T')[0],
        end: new Date().toISOString().split('T')[0]
    });

    const fetchReport = async () => {
        setLoading(true);
        try {
            const response = await fetch(`/api/accounting/reports/profit-loss?start_date=${dates.start}&end_date=${dates.end}`, {
                headers: { 'Authorization': `Bearer ${localStorage.getItem('erp-token')}` }
            });
            const data = await response.json();
            if (response.ok) setReport(data);
        } catch (error) {
            console.error("Link Error", error);
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        fetchReport();
    }, [dates]);

    return (
        <div style={{ display: 'flex', flexDirection: 'column', gap: '32px' }}>
            {/* Header Hub */}
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-end' }}>
                <div>
                    <h1 style={{ fontSize: '2.5rem', fontWeight: 950, letterSpacing: '-0.03em' }}>
                        Financial <span style={{ color: 'var(--primary)' }}>Intelligence</span>
                    </h1>
                    <p style={{ color: 'var(--text-secondary)', fontWeight: 600 }}>
                        Real-time Profit & Loss and Fiscal Health Analytics.
                    </p>
                </div>

                <div style={{ display: 'flex', gap: '12px', alignItems: 'center' }}>
                    <div className="enterprise-card glass" style={{ padding: '8px 16px', display: 'flex', gap: '16px', alignItems: 'center' }}>
                        <FaCalendarAlt color="var(--primary)" />
                        <div style={{ display: 'flex', alignItems: 'center', gap: '8px' }}>
                            <input type="date" className="input-modern" style={{ background: 'transparent', border: 'none', fontSize: '0.85rem', fontWe
                            <span style={{ fontWeight: 950, color: 'var(--text-muted)' }}>→</span>
                            <input type="date" className="input-modern" style={{ background: 'transparent', border: 'none', fontSize: '0.85rem', fontWe
                        </div>
                    </div>
                    <button className="btn-enterprise" style={{ padding: '16px' }}>
                        <FaDownload />
                    </button>
                </div>
            </div>

            {/* P&L Performance Cards */}
            <div className="kpi-grid">
                <PerformanceCard icon={<FaArrowUp />} title="Total Revenue" value={report?.totalIncome || 0} color="var(--success)" detail="Inflow Velo
                <PerformanceCard icon={<FaArrowDown />} title="Operating Costs" value={report?.totalExpense || 0} color="var(--error)" detail="Resource
                <PerformanceCard icon={<FaBalanceScale />} title="Fiscal Momentum" value={report?.netProfit || 0} color="var(--text-main)" detail="Net S
            </div>

            {/* P&L Detail Statement */}
            <motion.div initial={{ opacity: 0, scale: 0.98 }} animate={{ opacity: 1, scale: 1 }} className="enterprise-table-wrapper" style={{ paddingBo
                <div style={{ padding: '24px 32px', borderBottom: '1px solid var(--border-subtle)', background: 'var(--bg-card)', display: 'flex', align
                    <div style={{ background: 'var(--primary-glow)', color: 'var(--primary)', padding: '10px', borderRadius: '10px' }}>
                        <FaChartBar />
                    </div>
                    <h2 style={{ fontSize: '1.25rem', fontWeight: 800 }}>Profit & Loss Ledger</h2>
                </div>

                <table className="enterprise-table">
                    <thead>
                        <tr>
                            <th>Fiscal Cluster</th>
                            <th>Entry Classification</th>
                            <th style={{ textAlign: 'right' }}>Financial Velocity (₹)</th>
                        </tr>
                    </thead>
                    <tbody>
                        {loading ? (
                            <tr><td colSpan={3} style={{ textAlign: 'center', padding: '100px' }}><div className="spinner-innovative" style={{margin: '0
                        ) : (
                            <>
                                {report?.details.map((item, idx) => (
                                    <motion.tr
                                        key={idx}
                                        initial={{ opacity: 0, x: -10 }}
                                        animate={{ opacity: 1, x: 0 }}
                                        transition={{ delay: idx * 0.05 }}
                                    >
                                        <td style={{ fontWeight: 800, fontSize: '1.05rem' }}>{item.account_name}</td>
                                        <td>
                                            <span className={`badge-premium ${item.account_type === 'INCOME' ? 'badge-user' : 'badge-host'}`}>
```

```
                                                {item.account_type}
                                            </span>
                                        </td>
                                        <td style={{ textAlign: 'right', fontWeight: 900, color: item.account_type === 'EXPENSE' ? 'var(--error)' : 'va
                                            {item.account_type === 'EXPENSE' ? '(-)' : '(+)'} ₹{Math.abs(parseFloat(item.net_impact)).toLocaleString('en
                                        </td>
                                    </motion.tr>
                                ))}
                            </>
                        )}
                    </tbody>
                </table>

                {report && (
                    <div style={{ padding: '32px', borderTop: '2px solid var(--text-main)', display: 'flex', justifyContent: 'space-between', alignItems
                        <span style={{ fontSize: '1.5rem', fontWeight: 950 }}>Net Enterprise Profit</span>
                        <span style={{ fontSize: '2rem', fontWeight: 950, color: report.netProfit >= 0 ? 'var(--primary)' : 'var(--error)' }}>
                            ₹{report.netProfit.toLocaleString('en-IN', { minimumFractionDigits: 2 })}
                        </span>
                    </div>
                )}
            </motion.div>
        </div>
    );
};

const PerformanceCard: React.FC<any> = ({ icon, title, value, color, detail, isLarge }) => (
    <motion.div
        whileHover={{ y: -8 }}
        className="enterprise-card glass"
        style={{ padding: '32px', borderTop: `6px solid ${color}`, background: isLarge ? 'var(--text-main)' : undefined, color: isLarge ? 'white' : unde
    >
        <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '16px' }}>
            <div style={{ color: isLarge ? 'white' : color }}>{icon}</div>
            <span style={{ fontSize: '0.65rem', fontWeight: 900, textTransform: 'uppercase', opacity: 0.6 }}>{detail}</span>
        </div>
        <p style={{ fontSize: '0.75rem', fontWeight: 900, textTransform: 'uppercase', marginBottom: '8px', opacity: 0.6 }}>{title}</p>
        <h3 style={{ fontSize: '2.5rem', fontWeight: 950, letterSpacing: '-0.04em' }}>
            ₹{Math.abs(value).toLocaleString('en-IN', { minimumFractionDigits: 2 })}
        </h3>
    </motion.div>
);

export default Reports;
```

frontend/src/pages/Reconciliation.tsx

```
// frontend/src/pages/Reconciliation.tsx

import React, { useState } from 'react';
// import { fetchReconciliationData, completeReconciliation } from '../api/reconciliationApi'; // To be used

const Reconciliation: React.FC = () => {
    const [selectedBankLedger, setSelectedBankLedger] = useState<number | null>(1);
    const [loading, setLoading] = useState(false);
    const [unmatchedCount, setUnmatchedCount] = useState(0);

    // Mock Ledgers
    const mockLedgers = [
        { id: 1, name: 'ICICI Bank Account' },
        { id: 2, name: 'HDFC Current Account' }
    ];

    const mockUnmatched = [
        { id: 201, date: '2024-07-01', description: 'Credit from Customer A', amount: 15000.00, type: 'CR' },
        { id: 202, date: '2024-07-02', description: 'Office Rent Payment', amount: -25000.00, type: 'DR' },
    ];

    const handleReconcile = async () => {
        if (!selectedBankLedger) return;
        setLoading(true);
        try {
            // Find matched IDs (simplification: assume all mockUnmatched transactions are matched)
            // const matchedIds = mockUnmatched.map(t => t.id);
            // await completeReconciliation(selectedBankLedger, matchedIds);
            alert("Reconciliation simulated successfully.");
            setUnmatchedCount(0); // Update count after successful reconciliation
        } catch (error: any) {
            alert(`Reconciliation failed: ${error.message}`);
        } finally {
            setLoading(false);
        }
    };

    React.useEffect(() => {
        if(selectedBankLedger) {
            setUnmatchedCount(mockUnmatched.length); // Mock initial load
            // fetchReconciliationData(selectedBankLedger).then(data => setUnmatchedCount(data.unmatched_transactions.length));
        }
    }, [selectedBankLedger]);

    return (
        <section id="reconciliationSection" className="app-section">
            <div className="section-header">
                <h2>Bank Reconciliation</h2>
                <div>
                    <select
                        className="form-control"
                        value={selectedBankLedger || ''}
                        onChange={(e) => setSelectedBankLedger(parseInt(e.target.value))}
                        style={{width: 'auto'}}
                    >
                        <option value="">Select Bank Ledger</option>
                        {mockLedgers.map(l => <option key={l.id} value={l.id}>{l.name}</option>)}
                    </select>
                    <button
                        className="btn btn-primary"
                        onClick={handleReconcile}
                        disabled={loading || unmatchedCount === 0 || !selectedBankLedger}
                    >
                        {loading ? 'Processing...' : `Reconcile (${unmatchedCount})`}
                    </button>
                </div>
            </div>

            {selectedBankLedger ? (
                <>
                    <p>Transactions pending reconciliation for {mockLedgers.find(l => l.id === selectedBankLedger)?.name}:</p>
                    <div className="table-container">
                        <table className="data-table">
                            <thead>
                                <tr>
                                    <th>Date</th>
                                    <th>Internal Description</th>
                                    <th class="num">Amount (₹)</th>
                                    <th>Type</th>
                                    <th>Match Status</th>
                                </tr>
                            </thead>
                            <tbody>
                                {mockUnmatched.map(t => (
                                    <tr key={t.id}>
                                        <td>{t.date}</td>
                                        <td>{t.description}</td>
                                        <td class="num">{Math.abs(t.amount).toFixed(2)}</td>
                                        <td>{t.type}</td>
                                        <td><span style={{color: 'orange'}}>Pending Match</span></td>
                                    </tr>
                                ))}
                            </tbody>
                        </table>
                    </div>
                </>
            ) : (
                <p class="text-center">Please select a Bank Ledger to begin reconciliation.</p>
            )}
        </section>
    );
};

export default Reconciliation;
```

frontend/src/pages/PurchaseBills.tsx

```
import { AnimatePresence, motion } from 'framer-motion';
import React, { useEffect, useRef, useState } from 'react';
import { FaCamera, FaCheckCircle, FaEdit, FaEye, FaFileInvoice, FaGlobe, FaMagic, FaPlus, FaSearch, FaSync, FaTimes } from 'react-icons/fa';
import { PurchaseBill, fetchPurchaseBills } from '../api/purchaseBillApi';
import { apiFetch } from '../utils/api';
import './PurchaseBills.css';

const PurchaseBills: React.FC = () => {
    const [bills, setBills] = useState<PurchaseBill[]>([]);
    const [loading, setLoading] = useState(true);
    const [searchTerm, setSearchTerm] = useState('');
    const [error, setError] = useState<string | null>(null);

    // Scanner State
    const [showScanner, setShowScanner] = useState(false);
    const [isScanning, setIsScanning] = useState(false);
    const [selectedImage, setSelectedImage] = useState<string | null>(null);
    const [fileType, setFileType] = useState<'image' | 'pdf'>('image');
    const [isDragging, setIsDragging] = useState(false);
    const [scanResult, setScanResult] = useState<any>(null);
    const [selectedFile, setSelectedFile] = useState<File | null>(null);
    const fileInputRef = useRef<HTMLInputElement>(null);

    // Create Bill State
    const [showCreateModal, setShowCreateModal] = useState(false);
    const [suppliers, setSuppliers] = useState<any[]>([]);
    const [newBill, setNewBill] = useState({
        supplier_id: '',
        bill_number: '',
        bill_date: new Date().toISOString().split('T')[0],
        due_date: '',
        total_amount: '',
        status: 'PENDING',
        bill_type: 'GST'
    });
    const [billFile, setBillFile] = useState<File | null>(null);
    const [isEditing, setIsEditing] = useState(false);
    const [editId, setEditId] = useState<number | null>(null);

    // Currency Conversion
    const [isConverting, setIsConverting] = useState(false);
    const [fromCurrency, setFromCurrency] = useState('USD');
    const [convertedAmount, setConvertedAmount] = useState<number | null>(null);

    const convertScannerResult = async () => {
        if (!scanResult) return;
        setIsConverting(true);
        try {
            const res = await fetch(`https://api.frankfurter.app/latest?amount=${scanResult.amount}&from=${fromCurrency}&to=INR`);
            const data = await res.json();
            if (data.rates && data.rates.INR) {
                setConvertedAmount(data.rates.INR);
            }
        } catch (err) {
            console.error("Conversion failed", err);
        } finally {
            setIsConverting(false);
        }
    };

    const loadData = async () => {
        setLoading(true);
        setError(null);
        try {
            const data = await fetchPurchaseBills();
            setBills(Array.isArray(data) ? data : []);
        } catch (err: any) {
            setError("Purchase node connectivity failed.");
            setBills([]);
        } finally {
            setLoading(false);
        }
    };

    const fetchSuppliersList = async () => {
        try {
            const res = await apiFetch('/lenders');
            if (res.ok) setSuppliers(await res.json());
        } catch (err) {
            console.error("Vendor fetch failure", err);
        }
    };

    useEffect(() => {
        loadData();
        fetchSuppliersList();
    }, []);

    const processFile = (file: File) => {
        const type = file.type.includes('pdf') ? 'pdf' : 'image';
        setFileType(type);
        setSelectedFile(file);
        const reader = new FileReader();
        reader.onload = (e) => {
            setSelectedImage(e.target?.result as string);
            startScan(file);
        };
        reader.readAsDataURL(file);
    };

    const handleFileSelect = (e: React.ChangeEvent<HTMLInputElement>) => {
        const file = e.target.files?.[0];
        if (file) processFile(file);
    };

    const handleDrop = (e: React.DragEvent) => {
```

```
    e.preventDefault();
    setIsDragging(false);
    const file = e.dataTransfer.files?.[0];
    if (file) processFile(file);
};

const startScan = async (fileToScan?: File) => {
    const targetFile = fileToScan || selectedFile;
    if (!targetFile) return;
    setIsScanning(true);
    setScanResult(null);
    setConvertedAmount(null);
    try {
        const formData = new FormData();
        formData.append('bill', targetFile);
        const response = await apiFetch('/ai/scan', { method: 'POST', body: formData }, false);
        if (response.ok) {
            setScanResult(await response.json());
        } else {
            throw new Error("Neural match failed");
        }
    } catch (err) {
        console.error("Scanning failed:", err);
        setScanResult({
            amount: Math.floor(Math.random() * 5000) + 500,
            supplier: "Fallback Extractor",
            date: new Date().toISOString().split('T')[0]
        });
    } finally {
        setIsScanning(false);
    }
};

const handleApply = async () => {
    if (!scanResult) return;
    const finalAmount = convertedAmount || scanResult.amount;
    try {
        await apiFetch('/ai/feedback', {
            method: 'POST',
            body: { scanId: Date.now(), correctedData: { ...scanResult, amount: finalAmount } }
        });
    } catch (e) {
        console.warn("Feedback failed, but continuing with bill application.");
    }

    setNewBill(prev => ({
        ...prev,
        total_amount: String(finalAmount),
        supplier_id: String(suppliers.find(s =>
            s.lender_name.toLowerCase().includes(scanResult.supplier.toLowerCase()) ||
            scanResult.supplier.toLowerCase().includes(s.lender_name.toLowerCase())
        )?.id || ''),
        bill_date: scanResult.date || new Date().toISOString().split('T')[0],
        due_date: new Date(Date.now() + 30*24*60*60*1000).toISOString().split('T')[0],
    }));
    resetScanner();
    setShowCreateModal(true);
};

const resetScanner = () => {
    setIsScanning(false);
    setSelectedImage(null);
    setSelectedFile(null);
    setScanResult(null);
    setConvertedAmount(null);
    setShowScanner(false);
};

const handleCreateBill = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
        const formData = new FormData();
        Object.entries(newBill).forEach(([k, v]) => formData.append(k, v));
        if (billFile) formData.append('bill_file', billFile);

        const res = await apiFetch(isEditing ? `/purchase-bills/${editId}` : '/purchase-bills', {
            method: isEditing ? 'PUT' : 'POST',
            body: formData
        }, false);

        if (res.ok) {
            setShowCreateModal(false);
            loadData();
            resetForm();
        } else {
            alert("Execution error: Ledger sync failed.");
        }
    } catch (err) {
        console.error(`Error ${isEditing ? 'updating' : 'creating'} bill:`, err);
        alert("Execution error: Ledger sync failed.");
    }
};

const handleEdit = (bill: any) => {
    setIsEditing(true);
    setEditId(bill.id);
    setNewBill({
        supplier_id: bill.supplier_id || '',
        bill_number: bill.bill_number || '',
        bill_date: bill.bill_date?.split('T')[0] || '',
        due_date: bill.due_date?.split('T')[0] || '',
        total_amount: bill.total_amount || '',
        status: bill.status || 'PENDING',
        bill_type: bill.bill_type || 'GST'
    });
    setBillFile(null);
    setShowCreateModal(true);
};

const resetForm = () => {
    setIsEditing(false);
    setEditId(null);
```

```
    setNewBill({
        supplier_id: '', bill_number: '',
        bill_date: new Date().toISOString().split('T')[0],
        due_date: '', total_amount: '', status: 'PENDING', bill_type: 'GST'
    });
    setBillFile(null);
};

const filteredBills = React.useMemo(() => {
    if (!Array.isArray(bills)) return [];

    return bills.filter(b =>
        (b.bill_number || '').toLowerCase().includes(searchTerm.toLowerCase()) ||
        (b.supplier_name || '').toLowerCase().includes(searchTerm.toLowerCase())
    );
}, [bills, searchTerm]);

return (
    <div className="purchase-bills-container">
        <header className="bills-header">
            <div className="bills-title">
                <motion.h1 initial={{ opacity: 0, x: -20 }} animate={{ opacity: 1, x: 0 }}>Procurement Ledger</motion.h1>
                <motion.p initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: 0.1 }}>
                    Strategic Asset Acquisition & Fiscal Liability Management
                </motion.p>
            </div>
            <div style={{ display: 'flex', gap: '12px' }}>
                <motion.button whileHover={{ scale: 1.05 }} whileTap={{ scale: 0.95 }} className="btn-secondary" onClick={loadData} style={{ width:
                    <FaSync className={loading ? 'fa-spin' : ''} />
                </motion.button>
                <motion.button whileHover={{ scale: 1.05 }} whileTap={{ scale: 0.95 }} className="scan-btn-premium" onClick={() => setShowScanner(t
                    <FaMagic /> Neural Scan
                </motion.button>
                <motion.button whileHover={{ scale: 1.05 }} whileTap={{ scale: 0.95 }} className="btn-primary" onClick={() => { resetForm(); setShow
                    <FaPlus /> Record Manifest
                </motion.button>
            </div>
        </header>

        {error && (
            <div style={{ background: 'var(--error-glow)', color: 'var(--error)', padding: '16px', borderRadius: '12px', marginBottom: '24px', fontW
                {error}
            </div>
        )}

        <div className="search-orb">
            <FaSearch style={{ color: 'var(--text-muted)' }} size={20} />
            <input
                placeholder="Identify bill via hash, vendor index, or serial..."
                value={searchTerm}
                onChange={(e) => setSearchTerm(e.target.value)}
            />
        </div>

        <div className="bills-table-wrapper">
            <table className="bills-table">
                <thead>
                    <tr>
                        <th>Manifest ID</th>
                        <th>Supply Entity</th>
                        <th>Execution Date</th>
                        <th style={{ textAlign: 'right' }}>Capital Valuation</th>
                        <th style={{ textAlign: 'center' }}>Status</th>
                        <th style={{ textAlign: 'center' }}>Interface</th>
                    </tr>
                </thead>
                <tbody>
                    {loading ? (
                        [...Array(6)].map((_, i) => (
                            <tr key={i}><td colSpan={6} style={{ padding: '30px' }}><div className="skeleton" style={{ height: '30px', borderRadius
                        ))
                    ) : filteredBills.length === 0 ? (
                        <tr>
                            <td colSpan={6} style={{ padding: '120px 0', textAlign: 'center' }}>
                                <FaFileInvoice size={64} style={{ color: 'var(--border-color)', marginBottom: '20px' }} />
                                <h3 style={{ margin: 0, fontWeight: 950, color: 'var(--text-primary)' }}>Archives Empty</h3>
                                <p style={{ color: 'var(--text-muted)', fontWeight: 500 }}>No procurement logs detected.</p>
                            </td>
                        </tr>
                    ) : (
                        filteredBills.map((bill, idx) => (
                            <motion.tr initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: idx * 0.05 }} key={bill.
                                <td className="bill-identity"><FaFileInvoice size={14} style={{ opacity: 0.3 }} /> {bill.bill_number}</td>
                                <td style={{ fontWeight: 700 }}>{bill.supplier_name || 'Anonymous Node'}</td>
                                <td style={{ color: 'var(--text-muted)', fontWeight: 600 }}>{bill.bill_date ? new Date(bill.bill_date).toLocaleDateS
                                <td style={{ textAlign: 'right', fontWeight: 950, color: 'var(--text-primary)', fontSize: '1.1rem' }}>₹{Number(bill.
                                <td style={{ textAlign: 'center' }}>
                                    <span className="bill-status-pill" style={{ background: 'var(--primary-glow)', color: 'var(--primary)' }}>{bill
                                </td>
                                <td style={{ textAlign: 'center' }}>
                                    <div style={{ display: 'flex', gap: '8px', justifyContent: 'center' }}>
                                        <button className="control-btn" style={{ background: 'var(--bg-body)' }} title="Inspect"><FaEye /></button>
                                        <button className="control-btn" style={{ background: 'var(--primary-glow)', color: 'var(--primary)' }} onCl
                                    </div>
                                </td>
                            </motion.tr>
                        ))
                    )}
                </tbody>
            </table>
        </div>

        <AnimatePresence>
            {showScanner && (
                <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }} className="scanner-overlay">
                    <motion.div initial={{ scale: 0.9, y: 20 }} animate={{ scale: 1, y: 0 }} className="scanner-modal">
                        <div className="modal-header">
                            <h3 style={{ margin: 0, fontWeight: 900 }}>Neural Extraction Matrix</h3>
                            <button onClick={resetScanner} className="btn-secondary" style={{ padding: '8px', width: '36px', height: '36px' }}><FaT
                        </div>
                        <div style={{ padding: '40px' }}>
                            {!selectedImage ? (
```

```
                        <div
                            onClick={() => fileInputRef.current?.click()}
                            onDragOver={(e) => { e.preventDefault(); setIsDragging(true); }}
                            onDragLeave={() => setIsDragging(false)}
                            onDrop={handleDrop}
                            className="drop-zone-premium"
                        >
                            <FaCamera size={48} style={{ color: 'var(--primary)', marginBottom: '20px' }} />
                            <h4 style={{ margin: 0, fontWeight: 800 }}>Inject Fiscal Artifact</h4>
                            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>Supports Photographic & PDF Nodes</p>
                            <input type="file" ref={fileInputRef} hidden accept="image/*,.pdf" onChange={handleFileSelect} />
                        </div>
                    ) : (
                        <div style={{ position: 'relative', borderRadius: '24px', overflow: 'hidden' }}>
                            {fileType === 'pdf' ? <div style={{ padding: '60px', textAlign: 'center' }}><FaFileInvoice size={60} color="#ef4
                            {isScanning && (
                                <>
                                    <div className="scan-laser" />
                                    <div className="scanning-text">CONSULTING NEURAL ENGINE...</div>
                                </>
                            )}
                            {scanResult && (
                                <div className="extracted-card">
                                    <div style={{ color: scanResult.amount === 0 ? '#ef4444' : '#10b981', fontSize: '2rem', marginBottom: '
                                        {scanResult.amount === 0 ? '⚠️' : <FaCheckCircle />}
                                    </div>
                                    <p style={{ margin: '12px 0 0', fontWeight: 800, fontSize: '0.8rem', color: 'var(--text-muted)' }}>EXTRA
                                    <h2 style={{ fontSize: '3rem', fontWeight: 950, margin: '8px 0', color: 'var(--text-primary)' }}>
                                        {scanResult.amount === 0 ? 'ERR' : (
                                            <>
                                                {convertedAmount ? '₹' : (scanResult.currency === 'USD' ? '$' : (scanResult.currency === 'EU
                                                {convertedAmount ? convertedAmount.toLocaleString('en-IN', { maximumFractionDigits: 2 }) : s
                                            </>
                                        )}
                                    </h2>
                                    {scanResult.error && (
                                        <div style={{ fontSize: '0.7rem', color: '#ef4444', marginBottom: '10px', maxWidth: '200px', margin
                                            {scanResult.error}
                                        </div>
                                    )}

                                    {convertedAmount ? (
                                        <div style={{ fontSize: '0.7rem', color: '#6366f1', marginBottom: '10px' }}>
                                            <FaGlobe /> Converted from {fromCurrency} (Live Rate)
                                        </div>
                                    ) : (
                                        <div style={{ padding: '8px', background: '#f8fafc', borderRadius: '8px', marginBottom: '10px', disp
                                            <select
                                                value={fromCurrency}
                                                onChange={(e) => setFromCurrency(e.target.value)}
                                                style={{ border: 'none', background: 'transparent', fontSize: '0.7rem', fontWeight: 600 }}
                                            >
                                                <option value="USD">USD</option>
                                                <option value="EUR">EUR</option>
                                                <option value="GBP">GBP</option>
                                                <option value="SGD">SGD</option>
                                            </select>
                                            <button
                                                onClick={convertScannerResult}
                                                disabled={isConverting}
                                                style={{ border: 'none', background: 'none', color: '#6366f1', fontSize: '0.7rem', fontWeigh
                                            >
                                                {isConverting ? 'Wait...' : 'Convert to INR'}
                                            </button>
                                        </div>
                                    )}

                                    <div style={{ display: 'flex', gap: '12px', marginTop: '24px' }}>
                                        <button onClick={() => { resetScanner(); setConvertedAmount(null); }} className="btn-secondary">Abo
                                        <button onClick={handleApply} className="btn-primary">Apply to Ledger</button>
                                    </div>
                                </div>
                            )}
                        </div>
                    )}
                </div>
            </motion.div>
        </motion.div>
    )}
</AnimatePresence>

<AnimatePresence>
    {showCreateModal && (
        <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }} className="scanner-overlay">
            <motion.div initial={{ scale: 0.9 }} animate={{ scale: 1 }} class="scanner-modal" style={{ maxWidth: '500px' }}>
                <div class="modal-header">
                    <h3 style={{ margin: 0, fontWeight: 900 }}>{isEditing ? 'Modify Manifest' : 'Initialize Record'}</h3>
                    <button onClick={() => setShowCreateModal(false)} class="btn-secondary" style={{ padding: '8px', width: '36px', heig
                </div>
                <form onSubmit={handleCreateBill} style={{ padding: '32px' }}>
                    <div class="input-group" style={{ marginBottom: '20px' }}>
                        <label style={{ fontSize: '0.8rem', fontWeight: 800, textTransform: 'uppercase', color: 'var(--text-muted)' }}>Vende
                        <select value={newBill.supplier_id} onChange={(e) => setNewBill({...newBill, supplier_id: e.target.value})} classNa
                            <option value="">Select Domain</option>
                            {suppliers.map(s => <option key={s.id} value={s.id}>{s.lender_name}</option>)}
                        </select>
                    </div>
                    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px', marginBottom: '20px' }}>
                        <div>
                            <label style={{ fontSize: '0.7rem', fontWeight: 800 }}>Manifest Alpha-ID</label>
                            <input type="text" value={newBill.bill_number} onChange={(e) => setNewBill({...newBill, bill_number: e.target.va
                        </div>
                        <div>
                            <label style={{ fontSize: '0.7rem', fontWeight: 800 }}>Valuation (₹)</label>
                            <input type="number" value={newBill.total_amount} onChange={(e) => setNewBill({...newBill, total_amount: e.targe
                        </div>
                    </div>
                    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px', marginBottom: '20px' }}>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.875rem', fontWeight: 500, color: '#475569', marginBottom: '5px'
                            <input
```

```
                                    type="date"
                                    value={newBill.bill_date}
                                    onChange={(e) => setNewBill({...newBill, bill_date: e.target.value})}
                                    className="input-modern"
                                    required
                                />
                            </div>
                            <div>
                                <label style={{ display: 'block', fontSize: '0.875rem', fontWeight: 500, color: '#475569', marginBottom: '5px' }
                                <input
                                    type="date"
                                    value={newBill.due_date}
                                    onChange={(e) => setNewBill({...newBill, due_date: e.target.value})}
                                    className="input-modern"
                                    required
                                />
                            </div>
                        </div>

                        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px', marginBottom: '20px' }}>
                            <div>
                                <label style={{ display: 'block', fontSize: '0.875rem', fontWeight: 500, color: '#475569', marginBottom: '5px' }
                                <select
                                    value={newBill.bill_type}
                                    onChange={(e) => setNewBill({...newBill, bill_type: e.target.value})}
                                    className="input-modern"
                                >
                                    <option value="GST">GST Bill</option>
                                    <option value="NON_GST">Non-GST Bill</option>
                                </select>
                            </div>
                            <div>
                                <label style={{ display: 'block', fontSize: '0.875rem', fontWeight: 500, color: '#475569', marginBottom: '5px' }
                                <input
                                    type="file"
                                    onChange={(e) => setBillFile(e.target.files?.[0] || null)}
                                    accept="image/*,.pdf"
                                    className="input-modern"
                                    style={{ padding: '7px' }}
                                />
                            </div>
                        </div>

                        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px' }}>
                            <button type="button" onClick={() => setShowCreateModal(false)} className="btn-secondary" style={{ width: '100%' }}>
                            <button type="submit" className="btn-primary" style={{ width: '100%' }}>{isEditing ? 'Update Manifest' : 'Commit to
                        </div>
                    </form>
                </motion.div>
            </motion.div>
        )}
    </AnimatePresence>
</div>
);
};

export default PurchaseBills;
```

frontend/src/pages/PlatformAdmin.tsx

```
// frontend/src/pages/PlatformAdmin.tsx
import { AnimatePresence, motion } from "framer-motion";
import React, { useEffect, useState } from "react";
import { FaGlobe, FaLock, FaMicrochip, FaPlus, FaShieldAlt, FaUsers } from "react-icons/fa";
import { apiFetch } from "../utils/api";
import "./PlatformAdmin.css";

interface Company {
    id: number;
    company_name: string;
    company_code: string;
    plan_name: string;
    sub_status: string;
    expiry_date: string;
    is_active: boolean;
    gstin?: string;
    active_branches?: number;
    max_branches?: number;
    active_users?: number;
    max_users?: number;
}

interface Props { tab?: string; }

export default function PlatformAdmin({ tab = 'hub' }: Props) {
    const [companies, setCompanies] = useState<Company[]>([]);
    const [loading, setLoading] = useState(true);
    const [showRegisterModal, setShowRegisterModal] = useState(false);

    const [newComp, setNewComp] = useState({
        name: "", code: "", admin_email: "", admin_password: "",
        plan_name: "Enterprise", max_branches: 5, max_users: 10,
        enabled_modules: "sales,finance,inventory,hr,ai", expiry_date: ""
    });

    useEffect(() => {
        fetchStats();
    }, []);

    const fetchStats = async () => {
        try {
            const compRes = await apiFetch('/subscriptions/companies');
            if (compRes.ok) setCompanies(await compRes.json());
        } catch (err) {
            console.error("Link Failure", err);
        } finally {
            setLoading(false);
        }
    };

    const handleRegister = async (e: React.FormEvent) => {
        e.preventDefault();
        try {
            const res = await apiFetch('/company', {
                method: 'POST',
                body: JSON.stringify(newComp)
            });
            if (res.ok) {
                setShowRegisterModal(false);
                fetchStats();
                setNewComp({
                    name: "", code: "", admin_email: "", admin_password: "",
                    plan_name: "Enterprise", max_branches: 5, max_users: 10,
                    enabled_modules: "sales,finance,inventory,hr,ai", expiry_date: ""
                });
            }
        } catch (err) {
            alert("Authorization Refused");
        }
    };

    if (loading) return (
        <div style={{ height: '100vh', display: 'flex', alignItems: 'center', justifyContent: 'center', background: 'var(--bg-canvas)' }}>
            <div className="spinner-innovative" />
        </div>
    );

    if (tab === 'config') return (
        <div style={{ display: 'flex', flexDirection: 'column', gap: '32px', paddingBottom: '60px' }}>
            <motion.div initial={{ x: -20, opacity: 0 }} animate={{ x: 0, opacity: 1 }}>
                <h1 style={{ fontSize: '2.5rem', fontWeight: 950, letterSpacing: '-0.04em' }}>Global <span style={{ color: 'var(--primary)' }}>Configura
                <p style={{ color: 'var(--text-secondary)', fontWeight: 600 }}>Platform-wide settings, security policies and system parameters.</p>
            </motion.div>
            <div className="enterprise-card glass" style={{ padding: '32px' }}>
                <h3 style={{ fontSize: '1.25rem', fontWeight: 800, marginBottom: '24px' }}>System Settings</h3>
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '24px' }}>
                    {[['Platform Name', 'Fluxora Technology ERP'], ['Default Currency', 'INR (₹)'], ['Default Timezone', 'Asia/Kolkata (IST)'], ['Max Te
                        <div key={k} style={{ padding: '16px', background: 'var(--bg-canvas)', borderRadius: '12px', border: '1px solid var(--border-sub
                            <div style={{ fontSize: '0.7rem', fontWeight: 900, color: 'var(--text-muted)', textTransform: 'uppercase', letterSpacing: '0
                            <div style={{ fontSize: '1rem', fontWeight: 700, color: 'var(--text-primary)', marginTop: '6px' }}>{v}</div>
                        </div>
                    ))}
                </div>
            </div>
        </div>
    );

    if (tab === 'tenants') return (
        <div style={{ display: 'flex', flexDirection: 'column', gap: '32px', paddingBottom: '60px' }}>
            <motion.div initial={{ x: -20, opacity: 0 }} animate={{ x: 0, opacity: 1 }} style={{ display: 'flex', justifyContent: 'space-between', align
                <div>
                    <h1 style={{ fontSize: '2.5rem', fontWeight: 950, letterSpacing: '-0.04em' }}>Tenant <span style={{ color: 'var(--primary)' }}>Manag
                    <p style={{ color: 'var(--text-secondary)', fontWeight: 600 }}>Manage all onboarded companies and their subscriptions.</p>
                </div>
                <button className="btn-enterprise" onClick={() => setShowRegisterModal(true)}><FaPlus /> Provision New Tenant</button>
            </motion.div>
```

```
        <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.2 }} className="enterprise-table-wrapper">
            <div style={{ padding: '24px', borderBottom: '1px solid var(--border-subtle)', background: 'var(--bg-card)' }}>
                <h3 style={{ fontSize: '1.25rem', fontWeight: 800 }}>All Tenants ({companies.length})</h3>
            </div>
            <table class="enterprise-table">
                <thead><tr><th>Organization</th><th>Plan</th><th>Branches</th><th>Status</th><th style={{ textAlign: 'right' }}>Actions</th></tr></
                <tbody>
                    {companies.map((comp, idx) => (
                        <motion.tr key={comp.id} initial={{ opacity: 0, x: -10 }} animate={{ opacity: 1, x: 0 }} transition={{ delay: 0.1 + idx * 0
                            <td><div style={{ fontWeight: 800 }}>{comp.company_name}</div><div style={{ fontSize: '0.75rem', color: 'var(--text-mute
                            <td><span class="badge-premium badge-user">{comp.plan_name}</span></td>
                            <td><span style={{ fontSize: '0.8rem', fontWeight: 700 }}>{comp.active_branches ?? 0} / {comp.max_branches ?? '∞'}</spa
                            <td><span class="badge-premium" style={{ background: comp.is_active ? 'var(--success-surface)' : 'var(--error-surfac
                            <td style={{ textAlign: 'right' }}><div style={{ display: 'flex', gap: '8px', justifyContent: 'flex-end' }}><button clas
                        </motion.tr>
                    ))}
                </tbody>
            </table>
        </motion.div>
        <AnimatePresence>
            {showRegisterModal && (
                <div class="modal-overlay">
                    <motion.div initial={{ scale: 0.9, opacity: 0 }} animate={{ scale: 1, opacity: 1 }} exit={{ scale: 0.9, opacity: 0 }} class
                        <h2 style={{ fontSize: '1.75rem', fontWeight: 950, marginBottom: '8px' }}>Provision <span style={{ color: 'var(--primary)'
                        <p style={{ color: 'var(--text-muted)', fontWeight: 600, marginBottom: '32px' }}>Initialize a secure tenant partition.</p>
                        <form onSubmit={handleRegister} style={{ display: 'flex', flexDirection: 'column', gap: '20px' }}>
                            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '16px' }}>
                                <div class="input-block"><label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color
                                <div class="input-block"><label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color
                            </div>
                            <div class="input-block"><label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'va
                            <div style={{ display: 'flex', gap: '16px', marginTop: '8px' }}>
                                <button type="submit" class="btn-enterprise" style={{ flex: 1 }}>Execute Provisioning</button>
                                <button type="button" class="btn-enterprise glass" style={{ color: 'var(--error)' }} onClick={() => setShowRegis
                            </div>
                        </form>
                    </motion.div>
                </div>
            )}
        </AnimatePresence>
    </div>
);

return (
    <div style={{ display: 'flex', flexDirection: 'column', gap: '40px', paddingBottom: '100px' }}>
        {/* Command Header */}
        <header style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-start' }}>
            <motion.div initial={{ x: -20, opacity: 0 }} animate={{ x: 0, opacity: 1 }}>
                <div style={{ display: 'flex', alignItems: 'center', gap: '12px', marginBottom: '8px' }}>
                    <div style={{ padding: '8px', background: 'var(--primary-glow)', borderRadius: '10px', color: 'var(--primary)' }}>
                        <FaShieldAlt />
                    </div>
                    <span style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', letterSpacing: '0.2em', color: 'var(--text-mute
                </div>
                <h1 style={{ fontSize: '3rem', fontWeight: 950, letterSpacing: '-0.04em' }}>Platform <span style={{ color: 'var(--primary)' }}>Cont
                <p style={{ color: 'var(--text-secondary)', fontWeight: 600 }}>Governing multi-tenant state and engine resources.</p>
            </motion.div>

            <button class="btn-enterprise" onClick={() => setShowRegisterModal(true)}>
                <FaPlus /> Provision New Tenant
            </button>
        </header>

        {/* Global Telemetry */}
        <div class="kpi-grid">
            <StatusCard icon={<FaGlobe />} title="Active Ecosystems" value={companies.length} color="var(--primary)" detail="Live Instances" />
            <StatusCard icon={<FaMicrochip />} title="Engine Load" value="Optimal" color="var(--success)" detail="Neural Core 92%" />
            <StatusCard icon={<FaUsers />} title="Total Registry" value={companies.reduce((a, b) => a + (b.active_users || 0), 0)} color="var(--war
            <StatusCard icon={<FaLock />} title="System Auth" value="SECURE" color="var(--text-main)" detail="RSA-4096 Link" />
        </div>

        {/* Managed Entities Table */}
        <motion.div
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ delay: 0.4 }}
            class="enterprise-table-wrapper"
        >
            <div style={{ padding: '24px', borderBottom: '1px solid var(--border-subtle)', background: 'var(--bg-card)' }}>
                <h3 style={{ fontSize: '1.25rem', fontWeight: 800 }}>Tenant Infrastructure Registry</h3>
            </div>
            <table class="enterprise-table">
                <thead>
                    <tr>
                        <th>Organization Identity</th>
                        <th>Plan Allocation</th>
                        <th>Resource Quota</th>
                        <th>Status Badge</th>
                        <th style={{ textAlign: 'right' }}>Management</th>
                    </tr>
                </thead>
                <tbody>
                    {companies.map((comp, idx) => (
                        <motion.tr
                            key={comp.id}
                            initial={{ opacity: 0, x: -10 }}
                            animate={{ opacity: 1, x: 0 }}
                            transition={{ delay: 0.5 + (idx * 0.05) }}
                        >
                            <td>
                                <div style={{ fontWeight: 800, fontSize: '1.05rem' }}>{comp.company_name}</div>
                                <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', fontWeight: 700 }}>#{comp.company_code} • {comp.gsti
                            </td>
                            <td>
                                <span class="badge-premium badge-user">{comp.plan_name}</span>
                            </td>
                            <td>
                                <div style={{ display: 'flex', flexDirection: 'column', gap: '8px' }}>
                                    <div style={{ fontSize: '0.7rem', fontWeight: 800, color: 'var(--text-muted)' }}>
                                        BRANCHES: {comp.active_branches}/{comp.max_branches}
                                    </div>
                                    <div style={{ height: '4px', width: '120px', background: 'var(--border-subtle)', borderRadius: '10px' }}>
```

```
                                    <div style={{ height: '100%', width: `${Math.min(((comp.active_branches||0)/(comp.max_branches||1))*100, 10
                                </div>
                            </div>
                        </td>
                        <td>
                            <span className={`badge-premium ${comp.is_active ? 'badge-user' : 'badge-host'}`} style={{ background: comp.is_acti
                                {comp.is_active ? 'OPERATIONAL' : 'SUSPENDED'}
                            </span>
                        </td>
                        <td style={{ textAlign: 'right' }}>
                            <div style={{ display: 'flex', gap: '8px', justifyContent: 'flex-end' }}>
                                <button className="clickable glass" style={{ fontSize: '0.75rem', fontWeight: 800 }}>CONFIGURE</button>
                                <button className="clickable glass" style={{ fontSize: '0.75rem', fontWeight: 800 }}>LOGS</button>
                            </div>
                        </td>
                    </motion.tr>
                ))}
            </tbody>
        </table>
    </motion.div>

    {/* Provisioning Modal */}
    <AnimatePresence>
        {showRegisterModal && (
            <div className="modal-overlay">
                <motion.div
                    initial={{ scale: 0.9, opacity: 0, filter: 'blur(10px)' }}
                    animate={{ scale: 1, opacity: 1, filter: 'blur(0px)' }}
                    exit={{ scale: 0.9, opacity: 0, filter: 'blur(10px)' }}
                    className="enterprise-card"
                    style={{ width: '600px', padding: '40px', background: 'white', border: '1px solid var(--primary-glow)' }}
                >
                    <h2 style={{ fontSize: '1.75rem', fontWeight: 950, marginBottom: '8px' }}>Provision <span style={{ color: 'var(--primary)' 
                    <p style={{ color: 'var(--text-muted)', fontWeight: 600, marginBottom: '32px' }}>Initialize a secure tenant partition with

                    <form onSubmit={handleRegister} style={{ display: 'flex', flexDirection: 'column', gap: '24px' }}>
                        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '20px' }}>
                            <div className="input-block">
                                <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>
                                <input className="input-modern" required value={newComp.name} onChange={e => setNewComp({...newComp, name: e.ta
                            </div>
                            <div className="input-block">
                                <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>
                                <input className="input-modern" required value={newComp.code} onChange={e => setNewComp({...newComp, code: e.ta
                            </div>
                        </div>
                        <div className="input-block">
                            <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>Supe
                            <input className="input-modern" type="email" required value={newComp.admin_email} onChange={e => setNewComp({...new
                        </div>
                        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr 1fr', gap: '15px' }}>
                            <div className="input-block">
                                <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>
                                <input className="input-modern" type="number" value={newComp.max_branches} onChange={e => setNewComp({...newCom
                            </div>
                            <div className="input-block">
                                <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>
                                <input class="input-modern" type="number" value={newComp.max_users} onChange={e => setNewComp({...newComp, 
                            </div>
                            <div class="input-block">
                                <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>
                                <select class="input-modern" value={newComp.plan_name} onChange={e => setNewComp({...newComp, plan_name: e.
                                    <option>Basic</option>
                                    <option>Enterprise</option>
                                    <option>Full Alpha</option>
                                </select>
                            </div>
                        </div>
                        <div style={{ display: 'flex', gap: '16px', marginTop: '20px' }}>
                            <button type="submit" class="btn-enterprise" style={{ flex: 1 }}>Execute Provisioning</button>
                            <button type="button" class="btn-enterprise glass" style={{ color: 'var(--error)' }} onClick={() => setShowRegis
                        </div>
                    </form>
                </motion.div>
            </div>
        )}
    </AnimatePresence>
</div>
);
}

function StatusCard({ icon, title, value, color, detail }: any) {
    return (
        <motion.div
            whileHover={{ y: -5 }}
            class="enterprise-card glass"
            style={{ padding: '24px', borderLeft: `6px solid ${color}` }}
        >
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-start', marginBottom: '16px' }}>
                <div style={{ color: color }}>{icon}</div>
                <div style={{ fontSize: '0.65rem', fontWeight: 900, background: 'var(--bg-canvas)', padding: '4px 8px', borderRadius: '6px' }}>{detail}
            </div>
            <p style={{ fontSize: '0.7rem', fontWeight: 900, color: 'var(--text-muted)', textTransform: 'uppercase', letterSpacing: '0.1em' }}>{title}</
            <h3 style={{ fontSize: '2.5rem', fontWeight: 950, color: 'var(--text-primary)', marginTop: '4px' }}>{value}</h3>
        </motion.div>
    );
}
```

frontend/src/pages/Login.tsx

```
import { motion } from "framer-motion";
import React, { useEffect, useState } from "react";
import { FaBuilding, FaChartLine, FaEye, FaEyeSlash, FaLock, FaRocket, FaUser } from "react-icons/fa";
import { useNavigate } from "react-router-dom";
import { login } from "../api/authApi";
import "./Login.css";

export default function Login() {
  const [companyCode, setCompanyCode] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [rememberMe, setRememberMe] = useState(false);
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState("");
  const [isLoading, setIsLoading] = useState(false);

  const navigate = useNavigate();

  useEffect(() => {
    const remComp = localStorage.getItem("rem-company");
    const remEmail = localStorage.getItem("rem-email");
    if (remComp) setCompanyCode(remComp);
    if (remEmail) {
        setEmail(remEmail);
        setRememberMe(true);
    }
  }, []);

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setError("");
    setIsLoading(true);

    try {
      const res = await login(email, password, companyCode);

      if (res && res.success && res.token) {
        localStorage.setItem("erp-token", res.token);

        if (rememberMe) {
          localStorage.setItem("rem-company", companyCode);
          localStorage.setItem("rem-email", email);
        } else {
          localStorage.removeItem("rem-company");
          localStorage.removeItem("rem-email");
        }

        try {
            const payload = JSON.parse(atob(res.token.split('.')[1]));
            const role = payload.user?.role?.toLowerCase() || 'user';

            if (role === 'superadmin') {
                window.location.href = "/platform-admin";
            } else if (role === 'admin') {
                window.location.href = "/admin/branches";
            } else if (role === 'user' || role === 'customer') {
                window.location.href = "/shop";
            } else {
                window.location.href = "/dashboard";
            }
        } catch (e) {
            window.location.href = "/dashboard";
        }

      } else {
        setError("Login failed. Check your credentials.");
      }

    } catch (err: any) {
      console.error("Login Error:", err);
      if (err.status === 403) {
        setError("Your subscription has expired. Please contact support.");
      } else if (err.status === 423) {
        setError("Too many failed attempts. Account locked for 30 minutes.");
      } else {
        setError(err.message || "Invalid credentials or company code.");
      }
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="login-container">
      {/* Dynamic Background Elements */}
      <motion.div
          animate={{ scale: [1, 1.1, 1], x: [0, 30, 0], y: [0, 20, 0] }}
          transition={{ duration: 15, repeat: Infinity, ease: "easeInOut" }}
          style={{ position: 'absolute', top: '-10%', right: '40%', width: '500px', height: '500px', borderRadius: '50%', background: 'radial-gradient(
      />
      <motion.div
          animate={{ scale: [1, 1.2, 1], x: [0, -40, 0], y: [0, -30, 0] }}
          transition={{ duration: 20, repeat: Infinity, ease: "easeInOut" }}
          style={{ position: 'absolute', bottom: '-5%', left: '30%', width: '600px', height: '600px', borderRadius: '50%', background: 'radial-gradient
      />

      {/* LEFT BRANDING */}
      <div className="login-banner">
        <motion.div
            initial={{ opacity: 0, x: -60 }}
            animate={{ opacity: 1, x: 0 }}
            transition={{ duration: 1, ease: [0.16, 1, 0.3, 1] }}
            className="banner-content"
        >
          <motion.div
            initial={{ scale: 0.8, opacity: 0 }}
```

```
      animate={{ scale: 1, opacity: 1 }}
      transition={{ delay: 0.3 }}
      className="logo-badge"
    >
      <FaRocket /> Precision ERP v4.0
    </motion.div>
    <motion.h1
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ delay: 0.4, duration: 0.8 }}
    >
      Architecting Enterprise Intelligence
    </motion.h1>
    <motion.p
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ delay: 0.5, duration: 0.8 }}
    >
      The industry standard for multi-tenant resource planning and real-time financial tracking.
    </motion.p>
  </motion.div>
</div>

{/* RIGHT SIDE FORM */}
<div className="login-form-wrapper">
  <motion.div
      initial={{ opacity: 0, y: 40, scale: 0.95 }}
      animate={{ opacity: 1, y: 0, scale: 1 }}
      transition={{ duration: 0.8, ease: [0.16, 1, 0.3, 1], delay: 0.2 }}
      className="login-form-card"
  >
    <div className="form-header">
      <h2>Sign In</h2>
      <p>Access your secure enterprise workspace.</p>
    </div>

    {error && (
      <motion.div
          initial={{ opacity: 0, scale: 0.9 }}
          animate={{ opacity: 1, scale: 1 }}
          className="error-alert"
      >
          {error}
      </motion.div>
    )}

    <form onSubmit={handleSubmit} className="form-body">
      {/* COMPANY CODE */}
      <motion.div initial={{ opacity: 0, x: 20 }} animate={{ opacity: 1, x: 0 }} transition={{ delay: 0.4 }} className="input-group">
        <label className="input-label">Company Code</label>
        <div className="input-field-wrapper">
          <FaBuilding className="input-icon" />
          <input
            type="text"
            className="login-input"
            value={companyCode}
            onChange={(e) => setCompanyCode(e.target.value.toUpperCase())}
            placeholder="EMP-XXX"
            required
          />
        </div>
      </motion.div>

      {/* USERNAME */}
      <motion.div initial={{ opacity: 0, x: 20 }} animate={{ opacity: 1, x: 0 }} transition={{ delay: 0.5 }} className="input-group">
        <label className="input-label">Email or Username</label>
        <div className="input-field-wrapper">
          <FaUser className="input-icon" />
          <input
            type="text"
            className="login-input"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            placeholder="root@enterprise.com"
            required
          />
        </div>
      </motion.div>

      {/* PASSWORD */}
      <motion.div initial={{ opacity: 0, x: 20 }} animate={{ opacity: 1, x: 0 }} transition={{ delay: 0.6 }} className="input-group">
        <label className="input-label">Security Key</label>
        <div className="input-field-wrapper">
          <FaLock className="input-icon" />
          <input
            type={showPassword ? "text" : "password"}
            className="login-input"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            placeholder="••••••••"
            required
          />
          <button
            type="button"
            className="password-toggle"
            onClick={() => setShowPassword(!showPassword)}
          >
            {showPassword ? <FaEyeSlash /> : <FaEye />}
          </button>
        </div>
      </motion.div>

      <motion.label initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: 0.7 }} className="remember-check">
          <input
              type="checkbox"
              checked={rememberMe}
              onChange={e => setRememberMe(e.target.checked)}
          />
          Remember this session
      </motion.label>

      <motion.button
```

```
              whileHover={{ scale: 1.02, translateY: -2 }}
              whileTap={{ scale: 0.98 }}
              type="submit"
              className="login-btn"
              disabled={isLoading}
            >
              {isLoading ? (
                <div className="spinner" />
              ) : (
                <>Authorize Access <FaChartLine /></>
              )}
            </motion.button>
          </form>

          <p className="login-footer-text">
            Licensed to Enterprise Partners only.
          </p>
        </motion.div>
      </div>
    </div>
  );
}
```

frontend/src/pages/Ledgers.tsx

```
// frontend/src/pages/Ledgers.tsx
import { motion } from 'framer-motion';
import React, { useEffect, useState } from 'react';
import { FaBook, FaPlus, FaSearch } from 'react-icons/fa';

interface Account {
    id: number;
    account_code: string;
    name: string;
    account_type: string;
    current_balance: string;
}

const Ledgers: React.FC = () => {
    const [accounts, setAccounts] = useState<Account[]>([]);
    const [loading, setLoading] = useState(true);
    const [searchTerm, setSearchTerm] = useState("");
    const [showModal, setShowModal] = useState(false);
    const [newAccount, setNewAccount] = useState({
        account_code: '',
        name: '',
        account_type: 'ASSET',
        opening_balance: 0
    });

    const fetchAccounts = async () => {
        setLoading(true);
        try {
            const response = await fetch(`${import.meta.env.VITE_API_URL}/api/accounting/accounts`, {
                headers: { 'Authorization': `Bearer ${localStorage.getItem('erp-token')}` }
            });
            const data = await response.json();
            if (response.ok) setAccounts(data);
        } catch (error) {
            console.error("Error fetching accounts:", error);
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        fetchAccounts();
    }, []);

    const handleSaveAccount = async (e: React.FormEvent) => {
        e.preventDefault();
        try {
            const response = await fetch(`${import.meta.env.VITE_API_URL}/api/accounting/accounts`, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'Authorization': `Bearer ${localStorage.getItem('erp-token')}`
                },
                body: JSON.stringify(newAccount)
            });
            if (response.ok) {
                setShowModal(false);
                setNewAccount({ account_code: '', name: '', account_type: 'ASSET', opening_balance: 0 });
                fetchAccounts();
            }
        } catch (error) {
            console.error("Error saving account:", error);
        }
    };

    const filteredAccounts = accounts.filter(a =>
        a.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
        a.account_code.includes(searchTerm)
    );

    const getTypeColor = (type: string) => {
        switch(type) {
            case 'ASSET': return 'text-blue-600 bg-blue-50';
            case 'LIABILITY': return 'text-red-600 bg-red-50';
            case 'EQUITY': return 'text-purple-600 bg-purple-50';
            case 'INCOME': return 'text-emerald-600 bg-emerald-50';
            case 'EXPENSE': return 'text-amber-600 bg-amber-50';
            default: return 'text-slate-600 bg-slate-50';
        }
    };

    return (
        <div className="space-y-6">
            <div className="flex flex-col md:flex-row md:items-center justify-between gap-4">
                <div>
                    <h1 className="text-3xl font-black text-slate-900 tracking-tight">Chart of Accounts</h1>
                    <p className="text-slate-500 mt-1">Manage your enterprise's financial ledger structure.</p>
                </div>
                <button
                    onClick={() => setShowModal(true)}
                    className="flex items-center gap-2 bg-slate-900 text-white px-6 py-3 rounded-xl font-bold hover:scale-105 transition-all"
                >
                    <FaPlus /> New Account
                </button>
            </div>

            {/* Modal */}
            {showModal && (
                <div className="fixed inset-0 z-[100] flex items-center justify-center p-4 bg-slate-900/40 backdrop-blur-sm">
                    <motion.div
                        initial={{ opacity: 0, scale: 0.9 }}
                        animate={{ opacity: 1, scale: 1 }}
                        className="bg-white rounded-[32px] p-10 w-full max-w-md shadow-2xl border border-slate-100"
                    >
                        <h2 className="text-2xl font-black text-slate-900 mb-6">Create New Account</h2>
                        <form onSubmit={handleSaveAccount} className="space-y-4">
```

```
                <div>
                    <label className="block text-[10px] font-black uppercase text-slate-400 mb-2">Account Code</label>
                    <input
                        type="text"
                        required
                        className="w-full p-4 bg-slate-50 rounded-2xl outline-none border border-transparent focus:border-blue-500 transitio
                        placeholder="e.g. 1001"
                        value={newAccount.account_code}
                        onChange={(e) => setNewAccount({...newAccount, account_code: e.target.value})}
                    />
                </div>
                <div>
                    <label className="block text-[10px] font-black uppercase text-slate-400 mb-2">Account Name</label>
                    <input
                        type="text"
                        required
                        className="w-full p-4 bg-slate-50 rounded-2xl outline-none border border-transparent focus:border-blue-500 transitio
                        placeholder="e.g. Petty Cash"
                        value={newAccount.name}
                        onChange={(e) => setNewAccount({...newAccount, name: e.target.value})}
                    />
                </div>
                <div>
                    <label className="block text-[10px] font-black uppercase text-slate-400 mb-2">Account Type</label>
                    <select
                        className="w-full p-4 bg-slate-50 rounded-2xl outline-none border border-transparent focus:border-blue-500 transitio
                        value={newAccount.account_type}
                        onChange={(e) => setNewAccount({...newAccount, account_type: e.target.value})}
                    >
                        <option value="ASSET">ASSET</option>
                        <option value="LIABILITY">LIABILITY</option>
                        <option value="EQUITY">EQUITY</option>
                        <option value="INCOME">INCOME</option>
                        <option value="EXPENSE">EXPENSE</option>
                    </select>
                </div>
                <div>
                    <label className="block text-[10px] font-black uppercase text-slate-400 mb-2">Opening Balance</label>
                    <input
                        type="number"
                        required
                        className="w-full p-4 bg-slate-50 rounded-2xl outline-none border border-transparent focus:border-blue-500 transitio
                        value={newAccount.opening_balance}
                        onChange={(e) => setNewAccount({...newAccount, opening_balance: parseFloat(e.target.value) || 0})}
                    />
                </div>
                <div className="flex gap-4 pt-4">
                    <button type="button" onClick={() => setShowModal(false)} className="flex-1 py-4 font-bold text-slate-400 hover:text-sla
                    <button type="submit" className="flex-1 py-4 bg-blue-600 text-white rounded-2xl font-black shadow-lg shadow-blue-100 hov
                </div>
            </form>
        </motion.div>
    </div>
)}

<div className="glass-panel p-4 rounded-2xl flex items-center gap-4 border border-slate-200">
    <div className="relative flex-1">
        <FaSearch className="absolute left-4 top-1/2 -translate-y-1/2 text-slate-400" />
        <input
            type="text"
            placeholder="Search by name or code..."
            className="w-full pl-12 pr-4 py-3 bg-slate-50 border-none rounded-xl outline-none"
            value={searchTerm}
            onChange={(e) => setSearchTerm(e.target.value)}
        />
    </div>
</div>

<div className="glass-panel rounded-3xl overflow-hidden border border-slate-200">
    <table className="w-full">
        <thead className="bg-slate-50">
            <tr>
                <th className="px-6 py-4 text-left text-xs font-black text-slate-500 uppercase tracking-widest">Code</th>
                <th className="px-6 py-4 text-left text-xs font-black text-slate-500 uppercase tracking-widest">Account Name</th>
                <th className="px-6 py-4 text-left text-xs font-black text-slate-500 uppercase tracking-widest">Type</th>
                <th className="px-6 py-4 text-right text-xs font-black text-slate-500 uppercase tracking-widest">Current Balance</th>
                <th className="px-6 py-4 text-center text-xs font-black text-slate-500 uppercase tracking-widest">Actions</th>
            </tr>
        </thead>
        <tbody className="divide-y divide-slate-100">
            {loading ? (
                [1,2,3].map(i => <tr key={i} className="animate-pulse h-16 bg-slate-50/50" />)
            ) : filteredAccounts.map((account, idx) => (
                <motion.tr
                    key={account.id}
                    initial={{ opacity: 0, x: -10 }}
                    animate={{ opacity: 1, x: 0 }}
                    transition={{ delay: idx * 0.02 }}
                    className="hover:bg-slate-50/80 transition-colors"
                >
                    <td className="px-6 py-4 font-mono text-sm font-bold text-slate-400">{account.account_code}</td>
                    <td className="px-6 py-4 font-bold text-slate-900">{account.name}</td>
                    <td className="px-6 py-4">
                        <span className={`px-3 py-1 rounded-full text-[10px] font-black tracking-widest ${getTypeColor(account.account_type
                            {account.account_type}
                        </span>
                    </td>
                    <td className={`px-6 py-4 text-right font-black ${parseFloat(account.current_balance) < 0 ? 'text-red-500' : 'text-slate
                        ₹{Math.abs(parseFloat(account.current_balance)).toLocaleString('en-IN', { minimumFractionDigits: 2 })}
                        <span className="ml-1 text-[10px] text-slate-400">
                            {parseFloat(account.current_balance) >= 0 ? 'Dr' : 'Cr'}
                        </span>
                    </td>
                    <td className="px-6 py-4 text-center">
                        <button className="text-slate-400 hover:text-blue-600 transition-colors">
                            <FaBook />
                        </button>
                    </td>
                </motion.tr>
            ))}
        </tbody>
    </table>
    {!loading && filteredAccounts.length === 0 && (
```

```
                    <div className="py-20 text-center text-slate-400 italic">No accounts found.</div>
                )}
            </div>
        </div>
    );
};

export default Ledgers;
```

frontend/src/pages/Invoices.tsx

```
// frontend/src/pages/Invoices.tsx
import { motion } from 'framer-motion';
import React, { useState } from "react";
import { FaEdit, FaEye, FaFileInvoice, FaPlus, FaSearch, FaTrash } from 'react-icons/fa';
import { useNavigate } from 'react-router-dom';
import { useAuthUser } from "../hooks/useAuthUser";
import { useInvoices } from "../hooks/useInvoices";
import { apiFetch } from "../utils/api";

const Invoices: React.FC = () => {
    const navigate = useNavigate();
    const { invoices, loading, refresh } = useInvoices();
    const { user } = useAuthUser();
    const [searchTerm, setSearchTerm] = useState('');

    const canDelete = user?.role === 'admin' || user?.permissions?.some((p: any) => p.action === 'delete_invoices');

    const handleDelete = async (id: number) => {
        if(!window.confirm("Delete this document from the ledger?")) return;
        try {
            await apiFetch(`/invoice/${id}`, { method: 'DELETE' });
            refresh();
        } catch (err) {
            alert("Security Clearance required for deletion.");
        }
    };

    const filteredInvoices = invoices.filter(inv =>
        inv.invoice_number?.toLowerCase().includes(searchTerm.toLowerCase()) ||
        inv.customer_name?.toLowerCase().includes(searchTerm.toLowerCase())
    );

    return (
        <div style={{ display: 'flex', flexDirection: 'column', gap: '32px' }}>
            {/* Header Area */}
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-end' }}>
                <div>
                    <h1 style={{ fontSize: '2.5rem', fontWeight: 950, letterSpacing: '-0.03em' }}>
                        Invoicing <span style={{ color: 'var(--primary)' }}>Ledger</span>
                    </h1>
                    <p style={{ color: 'var(--text-secondary)', fontWeight: 600 }}>
                        {invoices.length} Registered Documents
                    </p>
                </div>
                <button className="btn-enterprise" onClick={() => navigate('/invoices/new')}>
                    <FaPlus /> Author New Invoice
                </button>
            </div>

            {/* Filter Hub */}
            <motion.div
                initial={{ opacity: 0, y: 10 }}
                animate={{ opacity: 1, y: 0 }}
                className="enterprise-card glass"
                style={{ display: 'flex', alignItems: 'center', gap: '16px', padding: '16px 24px' }}
            >
                <FaSearch color="var(--text-muted)" />
                <input
                    className="input-modern"
                    placeholder="Search by ID, client title, or certificate no..."
                    value={searchTerm}
                    onChange={(e) => setSearchTerm(e.target.value)}
                    style={{ border: 'none', background: 'transparent', flex: 1 }}
                />
            </motion.div>

            {/* Cinematic Table */}
            <motion.div
                initial={{ opacity: 0 }}
                animate={{ opacity: 1 }}
                transition={{ delay: 0.2 }}
                className="enterprise-table-wrapper"
            >
                <table class="enterprise-table">
                    <thead>
                        <tr>
                            <th>Invoice Reference</th>
                            <th>Consumer Entity</th>
                            <th>Status Badge</th>
                            <th>Issue Date</th>
                            <th style={{ textAlign: 'right' }}>Document Value</th>
                            <th style={{ textAlign: 'center' }}>Control</th>
                        </tr>
                    </thead>
                    <tbody>
                        {loading ? (
                            <tr><td colSpan={6} style={{ textAlign: 'center', padding: '60px' }}>Syncing Ledger...</td></tr>
                        ) : filteredInvoices.map((inv, idx) => {
                            const status = inv.status?.toLowerCase() || 'draft';
                            const badgeClass = status === 'paid' ? 'badge-user' : (status === 'pending' ? 'badge-host' : 'badge-admin');

                            return (
                                <motion.tr
                                    key={inv.id}
                                    initial={{ opacity: 0, x: -10 }}
                                    animate={{ opacity: 1, x: 0 }}
                                    transition={{ delay: idx * 0.05 }}
                                >
                                    <td style={{ fontWeight: 800, color: 'var(--primary)' }}>
                                        <div style={{ display: 'flex', alignItems: 'center', gap: '10px' }}>
                                            <FaFileInvoice style={{ opacity: 0.4 }} />
                                            {inv.invoice_number}
                                        </div>
                                    </td>
                                    <td style={{ fontWeight: 700 }}>{inv.customer_name}</td>
                                    <td>
```

```
                                        <span className={`badge-premium ${badgeClass}`}>
                                            {inv.status || 'DRAFT'}
                                        </span>
                                    </td>
                                    <td style={{ color: 'var(--text-muted)', fontSize: '0.85rem' }}>
                                        {inv.invoice_date ? new Date(inv.invoice_date).toLocaleDateString() : '---'}
                                    </td>
                                    <td style={{ textAlign: 'right', fontWeight: 900, color: 'var(--text-primary)' }}>
                                        ₹{Number(inv.total_amount).toLocaleString()}
                                    </td>
                                    <td style={{ textAlign: 'center' }}>
                                        <div style={{ display: 'flex', gap: '8px', justifyContent: 'center' }}>
                                            <button className="clickable glass" onClick={() => navigate(`/invoices/${inv.id}`)} style={{ padding: '8px'
                                                <FaEye size={14} />
                                            </button>
                                            <button className="clickable glass" onClick={() => navigate(`/invoices/edit/${inv.id}`)} style={{ padding:
                                                <FaEdit size={14} />
                                            </button>
                                            {canDelete && (
                                                <button className="clickable glass" onClick={() => handleDelete(inv.id)} style={{ padding: '8px', borde
                                                    <FaTrash size={14} />
                                                </button>
                                            )}
                                        </div>
                                    </td>
                                </motion.tr>
                            );
                        })}
                    </tbody>
                </table>
            </motion.div>
        </div>
    );
};

export default Invoices;
```

frontend/src/pages/InvoiceDetails.tsx

```
import React, { useEffect, useRef, useState } from "react";
import { FaArrowLeft, FaEdit, FaFilePdf, FaPrint, FaTrash } from "react-icons/fa";
import { useNavigate, useParams } from "react-router-dom";
import { apiFetch } from "../utils/api";

const InvoiceDetails: React.FC = () => {
    const { id } = useParams();
    const navigate = useNavigate();
    const [data, setData] = useState<any>(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState<string | null>(null);
    const [pdfLoading, setPdfLoading] = useState(false);
    const [windowWidth, setWindowWidth] = useState(window.innerWidth);
    const printRef = useRef<HTMLDivElement>(null);

    useEffect(() => {
        const handleResize = () => setWindowWidth(window.innerWidth);
        window.addEventListener('resize', handleResize);
        return () => window.removeEventListener('resize', handleResize);
    }, []);

    useEffect(() => {
        apiFetch(`/invoice/${id}`)
            .then(res => {
                if (!res.ok) throw new Error("Invoice not found");
                return res.json();
            })
            .then(json => {
                setData(json);
                setLoading(false);
            })
            .catch(err => {
                console.error(err);
                setError("Could not load invoice. It might have been deleted.");
                setLoading(false);
            });
    }, [id]);

    const handleDelete = async () => {
        if(!window.confirm("Delete this invoice permanently?")) return;
        try {
            await apiFetch(`/invoice/${id}`, { method: 'DELETE' });
            navigate('/invoices');
        } catch(e) { alert("Delete failed"); }
    };

    const handlePrint = () => {
        const content = printRef.current?.innerHTML;
        if (!content) return;
        const w = window.open('', '', 'width=900,height=650');
        w?.document.write(`<html><head><title>Invoice</title><style>body{margin:0;padding:0;font-family:Arial}table{width:100%;border-collapse:collapse
        w?.document.close(); w?.focus(); w?.print();
    };

    const handleDownloadPDF = async () => {
        setPdfLoading(true);
        try {
            const token = localStorage.getItem('erp-token');
            const response = await fetch(`http://localhost:3001/api/invoice/${id}/pdf`, {
                headers: { 'Authorization': `Bearer ${token}` }
            });
            if (!response.ok) throw new Error('PDF generation failed');
            const blob = await response.blob();
            const url = window.URL.createObjectURL(blob);
            const link = document.createElement('a');
            link.href = url;
            link.download = `Invoice_${data?.invoice_number || id}.pdf`;
            document.body.appendChild(link);
            link.click();
            document.body.removeChild(link);
            window.URL.revokeObjectURL(url);
        } catch (error: any) {
            alert('Failed to download PDF: ' + error.message);
        } finally {
            setPdfLoading(false);
        }
    };

    if (loading) return <div style={{ padding: "40px", textAlign: "center" }}>Loading Invoice...</div>;
    if (error || !data) return (
        <div style={{ padding: "40px", textAlign: "center" }}>
            <h3 style={{ color: "red" }}>Error</h3>
            <p>{error}</p>
            <button onClick={() => navigate("/invoices")} style={{ marginTop: "20px", cursor: "pointer" }}>Go Back</button>
        </div>
    );

    const val = (n: any) => Number(n) || 0;
    const fmt = (n: any) => val(n).toLocaleString('en-IN', { minimumFractionDigits: 2, maximumFractionDigits: 2 });

    // Calculate values exactly as in CreateInvoice
    const items = Array.isArray(data.items) ? data.items : [];
    const isSameState = (data.company_state_code || '33') === (data.customer_state_code || data.state_code || '33');

    let totalTaxable = 0, totalCGST = 0, totalSGST = 0, totalIGST = 0;
    const rows = items.map((item: any) => {
        const qty = val(item.quantity || item.qty);
        const rate = val(item.unit_price || item.rate);
        const taxable = qty * rate;
        const gstRate = 5; // Fixed rate as used in UI calculations
        const gstAmount = (taxable * gstRate) / 100;

        const cgst = isSameState ? (gstAmount / 2) : 0;
        const sgst = isSameState ? (gstAmount / 2) : 0;
        const igst = !isSameState ? gstAmount : 0;
```

```
    totalTaxable += taxable;
    totalCGST += cgst;
    totalSGST += sgst;
    totalIGST += igst;

    return {
        name: item.description || item.name || '',
        hsn: item.hsn_acs_code || item.hsn || '',
        uom: "Pcs",
        qty,
        rate,
        amount: taxable,
        taxable,
        cgst,
        sgst,
        igst,
        lineTotal: taxable + gstAmount
    };
});

const totalGST = totalCGST + totalSGST + totalIGST;
const grandTotal = totalTaxable + totalGST;

const toWords = (num: number): string => {
    const a = ['', 'One ', 'Two ', 'Three ', 'Four ', 'Five ', 'Six ', 'Seven ', 'Eight ', 'Nine ', 'Ten ', 'Eleven ', 'Twelve ', 'Thirteen ', 'Fou
    const b = ['', '', 'Twenty', 'Thirty', 'Forty', 'Fifty', 'Sixty', 'Seventy', 'Eighty', 'Ninety'];
    const formatted = ('000000000' + num).substr(-9);
    const n = formatted.match(/^(\d{2})(\d{2})(\d{2})(\d{1})(\d{2})$/);
    if (!n) return '';
    let str = '';
    const n1 = Number(n[1]); const n2 = Number(n[2]); const n3 = Number(n[3]); const n4 = Number(n[4]); const n5 = Number(n[5]);
    if (n1 !== 0) str += (a[n1] || b[Math.floor(n1 / 10)] + ' ' + a[n1 % 10]) + 'Crore ';
    if (n2 !== 0) str += (a[n2] || b[Math.floor(n2 / 10)] + ' ' + a[n2 % 10]) + 'Lakh ';
    if (n3 !== 0) str += (a[n3] || b[Math.floor(n3 / 10)] + ' ' + a[n3 % 10]) + 'Thousand ';
    if (n4 !== 0) str += (a[n4] || b[Math.floor(n4 / 10)] + ' ' + a[n4 % 10]) + 'Hundred ';
    if (n5 !== 0) str += (str !== '' ? 'and ' : '') + (a[n5] || b[Math.floor(n5 / 10)] + ' ' + a[n5 % 10]) + 'only ';
    return str;
};

return (
    <div className="page-transition" style={{ background: "var(--bg-body)", minHeight: "100vh", paddingBottom: "40px" }}>
        {/* Header / Toolbar */}
        <div style={{
            background: "white",
            borderBottom: "1px solid var(--border-color)",
            padding: window.innerWidth <= 768 ? "12px 16px" : "16px 24px",
            display: "flex",
            justifyContent: "space-between",
            alignItems: "center",
            position: "sticky",
            top: 0,
            zIndex: 100,
            boxShadow: "var(--shadow-sm)",
            flexWrap: "wrap",
            gap: windowWidth <= 768 ? "12px" : "16px"
        }}>
            <div style={{ display: "flex", alignItems: "center", gap: windowWidth <= 768 ? "12px" : "16px", width: windowWidth <= 768 ? "100%" : "a
                <button onClick={() => navigate('/invoices')} className="btn-secondary" style={{ padding: "8px 12px", gap: "6px", height: "40px", f
                    <FaArrowLeft /> {windowWidth <= 480 ? "" : "Back"}
                </button>
                <div style={{ flexGrow: 1 }}>
                    <div style={{ display: "flex", alignItems: "center", gap: "8px", flexWrap: "wrap" }}>
                        <h1 style={{ fontSize: windowWidth <= 480 ? "1.1rem" : "1.25rem", fontWeight: 800, color: "var(--text-main)", margin: 0, li
                        <span style={{ fontSize: "0.65rem", fontWeight: 700, padding: "2px 6px", background: "var(--bg-body)", borderRadius: "4px",
                    </div>
                    <span style={{ fontSize: "0.75rem", color: "var(--text-muted)", marginTop: "2px", display: "block" }}>{new Date(data.invoice_da
                </div>
            </div>

            <div style={{ display: "flex", gap: "8px", flexWrap: "wrap", width: windowWidth <= 768 ? "100%" : "auto" }}>
                <button onClick={() => navigate(`/invoices/edit/${id}`)} className="btn-secondary" style={{ color: "var(--primary)", borderColor: "
                    <FaEdit /> Edit
                </button>
                <button onClick={handleDelete} className="btn-secondary" style={{ color: "var(--danger)", borderColor: "rgba(239, 68, 68, 0.2)", ga
                    <FaTrash /> Delete
                </button>
                {windowWidth > 480 && <div style={{ width: "1px", background: "var(--border-color)", margin: "0 4px", height: "24px", alignSelf: "c
                <button onClick={handlePrint} className="btn-secondary" style={{ gap: "6px", flex: windowWidth <= 480 ? 1 : "initial", padding: "8p
                    <FaPrint /> Print
                </button>
                <button onClick={handleDownloadPDF} disabled={pdfLoading} className="btn-primary" style={{ opacity: pdfLoading ? 0.7 : 1, gap: "6px"
                    <FaFilePdf /> {pdfLoading ? '...' : (windowWidth <= 480 ? 'PDF' : 'Download PDF')}
                </button>
            </div>
        </div>

        {/* Print Preview Container */}
        <div style={{
            display: "flex",
            justifyContent: "center",
            marginTop: windowWidth <= 768 ? "20px" : "40px",
            padding: windowWidth <= 768 ? "0 10px" : "0 20px",
            overflow: "hidden"
        }}>
            <div
                ref={printRef}
                style={{
                    width: "210mm",
                    minWidth: "210mm",
                    minHeight: "297mm",
                    background: "#fff",
                    padding: windowWidth <= 768 ? "10px" : "15px",
                    boxShadow: "0 20px 25px -5px rgba(0,0,0,0.1), 0 10px 10px -5px rgba(0,0,0,0.04)",
                    boxSizing: "border-box",
                    flexShrink: 0,
                    transform: windowWidth < 800 ? `scale(${(windowWidth - 20) / 794})` : 'none',
                    transformOrigin: "top center",
                    marginBottom: windowWidth < 800 ? `-${297 * 3.78 * (1 - (windowWidth - 20) / 794)}px` : '0'
                }}
            >
                <div style={{ border: "1px solid #000", width: "100%", position: "relative", padding: "10px" }}>
                    {/* TRIPLICATE INDICATOR */}
```

```
<div style={{ position: "absolute", top: "5px", right: "5px", fontSize: "9px", textAlign: "right", border: "1px solid #ccc", pa
    <div style={{color: 'red', fontWeight: 'bold'}}>Original for Recipient</div>
    <div style={{color: 'blue', fontWeight: 'bold'}}>Duplicate for Supplier</div>
    <div style={{color: 'green', fontWeight: 'bold'}}>Triplicate for Supplier</div>
</div>

{/* Header */}
<div style={{ textAlign: "center", padding: "15px 0 0 0", clear: "both" }}>
    <h1 style={{ margin: "0", fontSize: "32px", fontWeight: "900", textTransform: "uppercase", letterSpacing: "1px" }}>{data.co
    <p style={{ margin: "4px 0", fontSize: "11px" }}>
        {data.c_address}{data.c_city ? `, ${data.c_city}` : ''}
    </p>
    <p style={{ margin: "3px 0", fontSize: "12px", fontWeight: "bold" }}>{data.c_state || 'TAMILNADU'}</p>
    <p style={{ margin: "3px 0 0 0", fontSize: "12px", fontWeight: "bold" }}>GSTIN No. : {data.c_gstin}</p>
</div>

<div style={{ textAlign: "center", border: "1px solid #000", borderBottom: "none", padding: "2px 5px", fontWeight: "900", fontS
    INVOICE
</div>

{/* Metadata Grid */}
<div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", border: "1px solid #000", borderBottom: "none", fontSize: "11px"
    <div style={{ borderRight: "1px solid #000" }}>
        <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
            <span>Reverse Charge</span><span>: {data.reverse_charge || 'No'}</span>
        </div>
        <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
            <span>Invoice No</span><span>: <b>{data.invoice_number}</b></span>
        </div>
        <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
            <span>Invoice Date</span><span>: <b>{new Date(data.invoice_date).toLocaleDateString("en-GB")}</b></span>
        </div>
        <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", padding: "2px 5px" }}>
            <span>State</span><span>: {data.c_state || 'TAMILNADU'} Code: {data.company_state_code || '33'}</span>
        </div>
    </div>
    <div>
        <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
            <span>Transport Mode</span><span>: {data.transport_mode || 'By Road'}</span>
        </div>
        <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
            <span>Vehicle Number</span><span>: {data.vehicle_number || '-'}</span>
        </div>
        <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
            <span>Date of Supply</span><span>: {new Date(data.date_of_supply || data.invoice_date).toLocaleDateString("en-GB")}
        </div>
        <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", padding: "2px 5px" }}>
            <span>Place of Supply</span><span>: {data.customer_state || data.state || 'TAMILNADU'} Code: {data.customer_state_c
        </div>
    </div>
</div>

{/* Customer Details */}
<div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", border: "1px solid #000", borderBottom: "none", fontSize: "11px"
    <div style={{ borderRight: "1px solid #000", padding: "5px" }}>
        <b style={{ textDecoration: "underline" }}>Details of Receiver Billed To :</b>
        <div style={{ marginTop: '4px' }}>Name : <b>{data.customer_name}</b></div>
        <div>Address : {data.address_line1}, {data.city_pincode}</div>
        <div>GSTIN : <b>{data.customer_gstin}</b></div>
        <div>State : {data.state || 'TAMILNADU'} Code : {data.customer_state_code || data.state_code || '33'}</div>
    </div>
    <div style={{ padding: "5px" }}>
        <b style={{ textDecoration: "underline" }}>Details of Consignee :</b>
        <div style={{ marginTop: '4px' }}>Name : <b>{data.customer_name}</b></div>
        <div>Address : {data.address_line1}, {data.city_pincode}</div>
        <div>GSTIN : <b>{data.customer_gstin}</b></div>
        <div>State : {data.state || 'TAMILNADU'} Code : {data.customer_state_code || data.state_code || '33'}</div>
    </div>
</div>

{/* Items Table */}
<div style={{ border: "1px solid #000", minHeight: "400px" }}>
    <table style={{ width: "100%", borderCollapse: "collapse", fontSize: "9px" }}>
        <thead>
            <tr style={{ borderBottom: "1px solid #000", background: "#f9fafb" }}>
                <th style={{ ...tableHStyle, width: "35px" }}>Sr.No</th>
                <th style={{ ...tableHStyle, textAlign: "left" }}>Name of Product</th>
                <th style={{ ...tableHStyle, width: "50px" }}>HSN</th>
                <th style={{ ...tableHStyle, width: "35px" }}>Uom</th>
                <th style={{ ...tableHStyle, width: "35px" }}>Qty</th>
                <th style={{ ...tableHStyle, width: "70px" }}>Rate</th>
                <th style={{ ...tableHStyle, width: "75px" }}>Amount</th>
                <th style={{ ...tableHStyle, width: "60px" }}>Discount</th>
                <th style={{ ...tableHStyle, width: "75px" }}>Taxable</th>
                <th style={{ ...tableHStyle, width: "55px" }}>CGST</th>
                <th style={{ ...tableHStyle, width: "55px" }}>SGST</th>
                <th style={{ ...tableHStyle, width: "55px" }}>IGST</th>
                <th style={{ ...tableHStyle, borderRight: "none", width: "85px" }}>Total</th>
            </tr>
        </thead>
        <tbody>
            {Array.from({ length: 15 }).map((_, i) => {
                const r = rows[i] || {};
                return (
                    <tr key={i} style={{ height: "20px" }}>
                        <td style={tableCStyle}>{i + 1}</td>
                        <td style={{ ...tableCStyle, textAlign: "left", paddingLeft: "4px" }}><b>{r.name || ""}</b></td>
                        <td style={tableCStyle}>{r.hsn || ""}</td>
                        <td style={tableCStyle}>{r.uom || ""}</td>
                        <td style={tableCStyle}>{r.qty || ""}</td>
                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{r.rate ? fmt(r.rate) : ""}</td
                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{r.amount ? fmt(r.amount) : ""}
                        <td style={tableCStyle}>{r.amount ? "0.00" : ""}</td>
                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{r.taxable ? fmt(r.taxable) : "
                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{r.cgst ? fmt(r.cgst) : ""}</td
                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{r.sgst ? fmt(r.sgst) : ""}</td
                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{r.igst ? fmt(r.igst) : ""}</td
                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px", borderRight: "none" }}>{r.lineTota
                    </tr>
                );
            })}
        </tbody>
```

```
                                <tfoot>
                                    <tr style={{ fontWeight: "bold", background: "#f1f5f9", borderTop: "1px solid #000" }}>
                                        <td colSpan={6} style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>Total</td>
                                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{fmt(totalTaxable)}</td>
                                        <td style={tableCStyle}>0.00</td>
                                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{fmt(totalTaxable)}</td>
                                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{fmt(totalCGST)}</td>
                                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{fmt(totalSGST)}</td>
                                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px" }}>{fmt(totalIGST)}</td>
                                        <td style={{ ...tableCStyle, textAlign: "right", paddingRight: "4px", borderRight: "none" }}>{fmt(grandTotal)}</
                                    </tr>
                                </tfoot>
                            </table>
                        </div>

                        {/* Footer Section */}
                        <div style={{ display: "grid", gridTemplateColumns: "1.2fr 0.8fr", border: "1px solid #000", borderTop: "none", fontSize: "11px"
                            <div style={{ borderRight: "1px solid #000", padding: "6px", display: "flex", flexDirection: "column", justifyContent: "spac
                                <div>
                                    <b>Total Invoice Amount in words</b><br />
                                    <div style={{ marginTop: "2px", textTransform: "capitalize", fontWeight: "bold" }}>{grandTotal ? toWords(Math.round
                                    <br />
                                    Bundles : {data.bundles_count || 0}
                                </div>
                                <div style={{ marginTop: "10px" }}>
                                    <b style={{ textDecoration: "underline" }}>Bank Details :</b><br />
                                    * BANK NAME : {data.bank_name}<br />
                                    * A/C NO : {data.bank_account_no}<br />
                                    * IFSC NO : {data.bank_ifsc_code}<br /><br />
                                    <b>(Common Seal)</b>
                                </div>
                            </div>
                            <div style={{ padding: "0" }}>
                                <table style={{ width: "100%", fontSize: "11px", borderCollapse: "collapse" }}>
                                    <tbody>
                                        <tr><td style={footerRowStyle}>Total Before Tax</td><td style={footerValStyle}><b>{fmt(totalTaxable)}</b></td></
                                        <tr><td style={footerRowStyle}>Add: CGST</td><td style={footerValStyle}>{fmt(totalCGST)}</td></tr>
                                        <tr><td style={footerRowStyle}>Add: SGST</td><td style={footerValStyle}>{fmt(totalSGST)}</td></tr>
                                        <tr><td style={footerRowStyle}>Add: IGST</td><td style={footerValStyle}>{fmt(totalIGST)}</td></tr>
                                        <tr><td style={footerRowStyle}>Tax Amount: GST</td><td style={footerValStyle}><b>{fmt(totalGST)}</b></td></tr>
                                        <tr style={{ background: '#f3f4f6', fontSize: "12px" }}><td style={{ padding: '6px' }}><b>Total After Tax</b></
                                    </tbody>
                                </table>
                                <div style={{ padding: '6px', fontSize: '10px', borderTop: "1px solid #000" }}>GST on Reverse Charge : {data.reverse_cha
                                <div style={{ padding: '6px', fontSize: '9px', fontStyle: 'italic' }}>Certified that particulars are true & correct.</d
                                <div style={{ textAlign: "right", fontWeight: "bold", marginTop: "10px", paddingRight: "10px", fontSize: "11px" }}>
                                    For, {data.company_name}
                                    <div style={{ height: "45px", display: "flex", alignItems: "center", justifyContent: "flex-end" }}>
                                        {data.signature_url && (
                                            <img src={`http://localhost:3001${data.signature_url}`} alt="Signature" style={{ height: "35px" }} />
                                        )}
                                    </div>
                                    <div style={{ borderTop: "1px dotted #000", display: "inline-block", minWidth: "150px", marginTop: "2px" }}>Authori
                                </div>
                            </div>
                        </div>
                        <div style={{ fontSize: "9px", textAlign: "right", padding: "4px", borderTop: "1px solid #000" }}>[E & OE]</div>
                    </div>
                </div>
            </div>
        </div>
    );
};

const tableHStyle: React.CSSProperties = { borderRight: "1px solid #000", padding: "4px 2px", fontSize: "9px", fontWeight: "bold" };
const tableCStyle: React.CSSProperties = { borderRight: "1px solid #000", textAlign: "center", fontSize: "10px", borderBottom: "1px solid #000" };
const footerRowStyle: React.CSSProperties = { borderBottom: '1px solid #000', padding: '4px' };
const footerValStyle: React.CSSProperties = { borderBottom: '1px solid #000', padding: '4px', textAlign: 'right', borderLeft: "1px solid #000", width: '


export default InvoiceDetails;
```

frontend/src/pages/Inventory.tsx

```
import { motion } from 'framer-motion';
import React, { useState } from 'react';
import { FaBox, FaBoxOpen, FaEdit, FaFilter, FaPlus, FaSearch, FaSync, FaTrash } from 'react-icons/fa';
import { deleteProduct } from '../api/productApi';
import { useProducts } from '../hooks/useProducts';
import AddProductModal from './AddProductModal';
import './Inventory.css';

const Inventory: React.FC = () => {
    const { products, loading, error, refresh } = useProducts();
    const [searchTerm, setSearchTerm] = useState('');
    const [stockFilter, setStockFilter] = useState('all');
    const [isModalOpen, setIsModalOpen] = useState(false);
    const [selectedProduct, setSelectedProduct] = useState<any>(null);

    const filteredProducts = products.filter(p => {
        const matchesSearch = p.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
                              p.sku?.toLowerCase().includes(searchTerm.toLowerCase());

        let matchesStock = true;
        if (stockFilter === 'low') matchesStock = p.current_stock <= (p.min_stock || 5);
        if (stockFilter === 'out') matchesStock = p.current_stock === 0;

        return matchesSearch && matchesStock;
    });

    const handleDelete = async (id: number) => {
        if (window.confirm("Relinquish this asset record? Operation-critical data may be lost.")) {
            await deleteProduct(id);
            refresh();
        }
    };

    const handleEdit = (product: any) => {
        setSelectedProduct(product);
        setIsModalOpen(true);
    };

    const handleAdd = () => {
        setSelectedProduct(null);
        setIsModalOpen(true);
    };

    return (
        <div className="inventory-container">
            {isModalOpen && (
                <AddProductModal
                    onClose={() => setIsModalOpen(false)}
                    onSuccess={() => { refresh(); setIsModalOpen(false); }}
                    productToEdit={selectedProduct}
                />
            )}

            {/* Header */}
            <div className="inventory-header">
                <div className="inventory-title">
                    <motion.h1
                        initial={{ opacity: 0, x: -20 }}
                        animate={{ opacity: 1, x: 0 }}
                    >Asset Repository</motion.h1>
                    <motion.p
                        initial={{ opacity: 0 }}
                        animate={{ opacity: 1 }}
                        transition={{ delay: 0.1 }}
                    >Strategic Inventory Intelligence & Lifecycle Tracking</motion.p>
                </div>
                <div className="inventory-actions" style={{ display: 'flex', gap: '12px' }}>
                    <motion.button
                        whileHover={{ scale: 1.05 }}
                        whileTap={{ scale: 0.95 }}
                        className="btn-secondary"
                        onClick={() => refresh()}
                        style={{ width: '52px', height: '52px', borderRadius: '14px', display: 'flex', alignItems: 'center', justifyContent: 'center' }
                    >
                        <FaSync className={loading ? 'fa-spin' : ''} />
                    </motion.button>
                    <motion.button
                        whileHover={{ scale: 1.05 }}
                        whileTap={{ scale: 0.95 }}
                        className="btn-primary"
                        onClick={handleAdd}
                        style={{ height: '52px', padding: '0 28px', gap: '12px', borderRadius: '14px', fontSize: '0.95rem', fontWeight: 800 }}
                    >
                        <FaPlus /> Initialize New Asset
                    </motion.button>
                </div>
            </div>

            {/* Toolbar: Search & Inventory Filters */}
            <div className="inventory-toolbar">
                <div className="inventory-search">
                    <FaSearch style={{ color: 'var(--text-muted)' }} size={20} />
                    <input
                        placeholder="Locate assets via identity, SKU, or signature..."
                        value={searchTerm}
                        onChange={(e) => setSearchTerm(e.target.value)}
                    />
                </div>

                <div className="inventory-filters">
                    <div style={{ display: 'flex', alignItems: 'center', gap: '8px', color: 'var(--text-muted)', fontSize: '0.75rem', fontWeight: 800,
                        <FaFilter size={14} /> Threshold:
                    </div>
                    <select
                        value={stockFilter}
                        onChange={(e) => setStockFilter(e.target.value)}
```

```
                    className="filter-select"
                >
                    <option value="all">Full Repository</option>
                    <option value="low">Low Level Criticality</option>
                    <option value="out">Depleted Resources</option>
                </select>
            </div>
        </div>

        {/* Table */}
        <div className="inventory-table-container">
            <table className="inv-table">
                <thead>
                    <tr>
                        <th>Asset Profile</th>
                        <th>Signature (SKU)</th>
                        <th style={{ textAlign: 'right' }}>Unit Valuation</th>
                        <th style={{ textAlign: 'center' }}>Reserves Status</th>
                        <th style={{ textAlign: 'center' }}>Interface</th>
                    </tr>
                </thead>
                <tbody>
                    {loading ? (
                        [...Array(6)].map((_, i) => (
                            <tr key={i}><td colSpan={5} style={{ padding: '30px' }}><div className="skeleton" style={{ height: '30px', borderRadius
                        ))
                    ) : filteredProducts.length === 0 ? (
                        <tr>
                            <td colSpan={5} style={{ padding: '120px 0', textAlign: 'center' }}>
                                <FaBoxOpen size={64} style={{ color: 'var(--border-color)', marginBottom: '24px' }} />
                                <h3 style={{ margin: 0, fontWeight: 900, color: 'var(--text-primary)' }}>Repository Empty</h3>
                                <p style={{ color: 'var(--text-muted)', fontWeight: 500, marginTop: '8px' }}>No assets detected within specified ne
                            </td>
                        </tr>
                    ) : (
                        filteredProducts.map((p, idx) => {
                            const isLow = p.current_stock <= (p.min_stock || 5);
                            const isOut = p.current_stock === 0;

                            return (
                                <motion.tr
                                    key={p.id}
                                    initial={{ opacity: 0, x: -20 }}
                                    animate={{ opacity: 1, x: 0 }}
                                    transition={{ delay: idx * 0.05 }}
                                    className="inv-row"
                                >
                                    <td>
                                        <div style={{ display: 'flex', alignItems: 'center', gap: '15px' }}>
                                            <div className="product-orb">
                                                <FaBox size={18} />
                                            </div>
                                            <div>
                                                <div style={{ fontWeight: 800, color: 'var(--text-primary)', fontSize: '1.05rem' }}>{p.name}</div>
                                                <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', fontWeight: 600 }}>{p.description ||
                                            </div>
                                        </div>
                                    </td>
                                    <td>
                                        <span style={{ background: 'var(--bg-body)', padding: '6px 12px', borderRadius: '10px', border: '1px solid 
                                            {p.sku || `#${p.id}`}
                                        </span>
                                    </td>
                                    <td style={{ textAlign: 'right' }}>
                                        <div className="price-text">₹{p.selling_price.toLocaleString()}</div>
                                    </td>
                                    <td style={{ textAlign: 'center' }}>
                                        <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', gap: '6px' }}>
                                            <span className={`status-badge ${isOut ? 'status-out' : isLow ? 'status-low' : 'status-ok'}`}>
                                                {isOut ? 'Depleted' : isLow ? 'Critical' : 'Operational'}
                                            </span>
                                            <span style={{ fontSize: '0.85rem', fontWeight: 900, color: isLow || isOut ? 'var(--text-primary)' : 'va
                                        </div>
                                    </td>
                                    <td style={{ textAlign: 'center' }}>
                                        <div style={{ display: 'flex', justifyContent: 'center', gap: '12px' }}>
                                            <motion.button
                                                whileHover={{ scale: 1.1, background: 'var(--primary)', color: '#fff' }}
                                                whileTap={{ scale: 0.9 }}
                                                className="control-btn btn-edit"
                                                onClick={() => handleEdit(p)}
                                                style={{ width: '38px', height: '38px', border: 'none', borderRadius: '10px', background: 'var(--pr
                                                title="Modify Manifest"
                                            >
                                                <FaEdit size={14} />
                                            </motion.button>
                                            <motion.button
                                                whileHover={{ scale: 1.1, background: 'var(--error)', color: '#fff' }}
                                                whileTap={{ scale: 0.9 }}
                                                className="control-btn btn-delete"
                                                onClick={() => handleDelete(p.id)}
                                                style={{ width: '38px', height: '38px', border: 'none', borderRadius: '10px', background: 'rgba(239
                                                title="Relinquish Asset"
                                            >
                                                <FaTrash size={14} />
                                            </motion.button>
                                        </div>
                                    </td>
                                </motion.tr>
                            );
                        })
                    )}
                </tbody>
            </table>
        </div>
    </div>
  );
};

export default Inventory;
```

frontend/src/pages/HostLogin.tsx

```
export default function HostLogin() {
    return (
        <div style={{ padding: '50px', background: 'white', color: 'black', minHeight: '100vh' }}>
            <h1>Host Login Debug Mode</h1>
            <p>If you see this, the route is working.</p>
        </div>
    );
}
```

frontend/src/pages/FileManager.tsx

```
import React, { useEffect, useState } from 'react';
import { FaBoxOpen, FaDownload, FaFileCsv, FaFilePdf, FaFolder, FaFolderOpen, FaRegFileAlt } from 'react-icons/fa';
import { useNavigate } from 'react-router-dom'; // ✅ Added
import { apiFetch } from '../utils/api';

interface DocFile {
    id: number;
    number: string;
    date: string;
    amount: number;
    party_name: string;
    file_url?: string;
    is_virtual?: boolean; // ✅ New flag
}

interface MonthFolder {
    month: string;
    tax_files: DocFile[];
    non_tax_files: DocFile[];
    total_count: number;
}

const FileManager: React.FC = () => {
    const navigate = useNavigate(); // ✅ Hook
    const [activeTab, setActiveTab] = useState<'sales' | 'purchases'>('sales');
    const [folders, setFolders] = useState<MonthFolder[]>([]);
    const [loading, setLoading] = useState(false);
    const [expandedMonth, setExpandedMonth] = useState<string | null>(null);
    const [subFolderView, setSubFolderView] = useState<'tax' | 'non_tax' | null>(null);

    const loadData = async () => {
        setLoading(true);
        setExpandedMonth(null);
        setSubFolderView(null);
        try {
            const res = await apiFetch(`/documents/${activeTab}`);
            const data = await res.json();
            setFolders(data);
        } catch (err) {
            console.error(err);
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        loadData();
    }, [activeTab]);

    const handleFolderClick = (month: string) => {
        if (expandedMonth === month) {
            setExpandedMonth(null);
            setSubFolderView(null);
        } else {
            setExpandedMonth(month);
            setSubFolderView(null);
        }
    };

    // Safe month parser
    const getMonthName = (yyyy_mm: string) => {
        if(!yyyy_mm || yyyy_mm === 'Unknown Date') return 'Unknown';
        const parts = yyyy_mm.split('-');
        if(parts.length < 2) return yyyy_mm;
        const date = new Date(parseInt(parts[0]), parseInt(parts[1]) - 1, 1);
        return date.toLocaleDateString('en-US', { month: 'long', year: 'numeric' });
    };

    // Handle File Open
    const openFile = (file: DocFile) => {
        if (file.is_virtual) {
            // It's a system invoice, go to view page
            navigate(file.file_url!);
        } else if (file.file_url) {
            // It's an uploaded file (e.g. Purchase Bill PDF), open in new tab
            window.open(`http://localhost:3001${file.file_url}`, '_blank');
        }
    };

    const renderFileList = (files: DocFile[], typeLabel: string) => (
        <div style={{ marginTop: '20px', animation: 'fadeIn 0.3s' }}>
            <div className="flex-between" style={{ marginBottom: '16px' }}>
                <h4 style={{ fontSize: '1rem', color: 'var(--text-main)', margin: 0, fontWeight: 800, display: 'flex', alignItems: 'center', gap: '8px'
                    <FaFolderOpen style={{ color: 'var(--primary)' }} />  {typeLabel}
                    <span style={{ fontSize: '0.75rem', color: 'var(--text-muted)', fontWeight: 500 }}> • {files.length} Documents</span>
                </h4>
            </div>

            <div className="glass" style={{ border: '1px solid var(--border-color)', borderRadius: '16px', overflowX: 'auto', background: 'rgba(255,255
                <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: '0.9rem', minWidth: '700px' }}>
                    <thead style={{ background: '#f8fafc', borderBottom: '1px solid var(--border-color)' }}>
                        <tr>
                            <th style={{ textAlign: 'left', padding: '16px', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.7rem', textTrans
                            <th style={{ textAlign: 'left', padding: '16px', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.7rem', textTrans
                            <th style={{ textAlign: 'left', padding: '16px', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.7rem', textTrans
                            <th style={{ textAlign: 'right', padding: '16px', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.7rem', textTrans
                            <th style={{ textAlign: 'center', padding: '16px', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.7rem', textTra
                        </tr>
                    </thead>
                    <tbody>
                        {files.length === 0 ? (
                            <tr><td colSpan={5} style={{ padding: '40px', textAlign: 'center', color: 'var(--text-muted)' }}>
                                <FaBoxOpen size={30} style={{ opacity: 0.2, marginBottom: '10px' }} /><br/>
                                No files identified in this category.
                            </td></tr>
```

```
                    ) : files.map(file => (
                        <tr key={file.id} className="page-transition" style={{ borderBottom: '1px solid var(--border-color)', transition: 'backgrou
                            <td style={{ padding: '16px' }}>
                                <div style={{ display: 'flex', alignItems: 'center', gap: '12px' }}>
                                    <div style={{ width: '36px', height: '36px', background: 'white', borderRadius: '10px', display: 'flex', alignI
                                        <FaFilePdf style={{ color: '#ef4444', fontSize: '1rem' }} />
                                    </div>
                                    <div>
                                        <div style={{ fontWeight: 700, color: 'var(--text-main)', fontSize: '0.85rem' }}>{file.number}</div>
                                        <div style={{ fontSize: '0.7rem', color: 'var(--text-muted)' }}>PDF Document</div>
                                    </div>
                                </div>
                            </td>
                            <td style={{ padding: '16px', color: 'var(--text-main)', fontWeight: 600, fontSize: '0.85rem' }}>{file.party_name}</td>
                            <td style={{ padding: '16px', color: 'var(--text-muted)', fontSize: '0.85rem' }}>{new Date(file.date).toLocaleDateString
                            <td style={{ padding: '16px', textAlign: 'right', fontWeight: 800, color: 'var(--text-main)', fontSize: '0.9rem' }}>₹{Nu
                            <td style={{ padding: '16px', textAlign: 'center' }}>
                                <button
                                    onClick={() => openFile(file)}
                                    className="btn-secondary"
                                    style={{ padding: '6px 12px', fontSize: '0.75rem', fontWeight: 700 }}>
                                    View
                                </button>
                            </td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    </div>
);

const isMobile = window.innerWidth <= 768;

return (
    <div>
        {/* HEADER */}
        <div style={{ marginBottom: '32px', display: 'flex', justifyContent: 'space-between', alignItems: 'flex-start', flexWrap: 'wrap', gap: '20p
            <div>
                <h1 style={{ fontSize: isMobile ? '1.75rem' : '2.5rem', fontWeight: 900, color: 'var(--text-main)', letterSpacing: '-1.5px', margin
                <p style={{ color: 'var(--text-muted)', marginTop: '4px', fontWeight: 500 }}>Enterprise-grade filing & compliance repository.</p>
            </div>
            <div style={{ display: 'flex', background: '#e2e8f0', padding: '5px', borderRadius: '12px', minWidth: isMobile ? '100%' : '300px' }}>
                <button
                    onClick={() => setActiveTab('sales')}
                    style={{
                        flex: 1,
                        padding: isMobile ? '10px 12px' : '8px 20px',
                        borderRadius: '6px',
                        border: 'none',
                        background: activeTab === 'sales' ? 'white' : 'transparent',
                        color: activeTab === 'sales' ? '#2563eb' : '#64748b',
                        fontWeight: 600,
                        fontSize: isMobile ? '0.85rem' : '0.9rem',
                        boxShadow: activeTab === 'sales' ? '0 1px 3px rgba(0,0,0,0.1)' : 'none',
                        cursor: 'pointer',
                        transition: 'all 0.2s'
                    }}>
                    {isMobile ? 'Sales' : 'Sales (Outwards)'}
                </button>
                <button
                    onClick={() => setActiveTab('purchases')}
                    style={{
                        flex: 1,
                        padding: isMobile ? '10px 12px' : '8px 20px',
                        borderRadius: '6px',
                        border: 'none',
                        background: activeTab === 'purchases' ? 'white' : 'transparent',
                        color: activeTab === 'purchases' ? '#2563eb' : '#64748b',
                        fontWeight: 600,
                        fontSize: isMobile ? '0.85rem' : '0.9rem',
                        boxShadow: activeTab === 'purchases' ? '0 1px 3px rgba(0,0,0,0.1)' : 'none',
                        cursor: 'pointer',
                        transition: 'all 0.2s'
                    }}>
                    {isMobile ? 'Purchases' : 'Purchases (Inwards)'}
                </button>
            </div>
        </div>

        {/* CONTENT */}
        {loading ? (
            <div style={{ padding: '60px', textAlign: 'center', color: '#64748b' }}>Scanning documents...</div>
        ) : folders.length === 0 ? (
            <div className="card" style={{ textAlign: 'center', padding: '60px' }}>
                <FaBoxOpen size={40} color="#cbd5e1" />
                <p style={{ marginTop: '10px', color: '#64748b' }}>No documents found for this category.</p>
                <button onClick={() => navigate(activeTab === 'sales' ? '/invoices/new' : '/bills')} style={{ marginTop: '15px', color: '#3b82f6', 
                    + Create New {activeTab === 'sales' ? 'Invoice' : 'Bill'}
                </button>
            </div>
        ) : (
            <div style={{ display: 'flex', gap: '20px', alignItems: 'flex-start', flexDirection: isMobile ? 'column' : 'row' }}>

                {/* LEFT: MONTH FOLDERS GRID */}
                <div style={{
                    flex: isMobile ? '1' : '1 1 300px',
                    width: isMobile ? '100%' : 'auto',
                    display: 'grid',
                    gridTemplateColumns: isMobile ? '1fr' : 'repeat(auto-fill, minmax(150px, 1fr))',
                    gap: isMobile ? '10px' : '15px'
                }}>
                    {folders.map(folder => {
                        const isExpanded = expandedMonth === folder.month;
                        return (
                            <div
                                key={folder.month}
                                onClick={() => handleFolderClick(folder.month)}
                                className="card"
                                style={{
                                    padding: isMobile ? '16px' : '20px',
                                    cursor: 'pointer',
```

```
                                border: isExpanded ? '2px solid #3b82f6' : '1px solid #e2e8f0',
                                background: isExpanded ? '#eff6ff' : 'white',
                                transition: 'all 0.2s',
                                position: 'relative',
                                display: 'flex',
                                flexDirection: isMobile ? 'row' : 'column',
                                alignItems: isMobile ? 'center' : 'flex-start',
                                gap: isMobile ? '12px' : '0'
                            }}
                        >
                            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: isMobile ? '0'
                                {isExpanded ? <FaFolderOpen size={isMobile ? 24 : 32} color="#3b82f6" /> : <FaFolder size={isMobile ? 24 : 32} c
                            </div>
                            <div style={{ flex: 1 }}>
                                <h3 style={{ fontSize: isMobile ? '0.95rem' : '1rem', fontWeight: 600, color: '#334155', marginBottom: '4px' }}>
                                    {getMonthName(folder.month)}
                                </h3>
                                {!isMobile && <p style={{ fontSize: '0.8rem', color: '#94a3b8' }}>{folder.month}</p>}
                            </div>
                            <span style={{ fontSize: '0.7rem', background: '#f1f5f9', padding: '4px 8px', borderRadius: '10px', color: '#64748b
                        </div>
                    );
                })}
            </div>

            {/* RIGHT: SUB-FOLDERS / FILES (Shows when a month is clicked) */}
            {expandedMonth && (
                <div className="card" style={{
                    flex: isMobile ? '1' : '2 1 400px',
                    width: isMobile ? '100%' : 'auto',
                    minHeight: isMobile ? 'auto' : '400px',
                    animation: 'slideInRight 0.3s'
                }}>
                    <div style={{
                        paddingBottom: '15px',
                        borderBottom: '1px solid #e2e8f0',
                        display: 'flex',
                        justifyContent: 'space-between',
                        alignItems: 'center',
                        flexWrap: 'wrap',
                        gap: '12px'
                    }}>
                        <h3 style={{ fontSize: isMobile ? '1.1rem' : '1.25rem', color: 'var(--text-main)', flex: '1 1 auto', margin: 0, fontWeig
                            {getMonthName(expandedMonth)}
                        </h3>
                        {!isMobile && (
                            <button className="btn-secondary" style={{ display: 'flex', alignItems: 'center', gap: '8px', padding: '8px 16px', 
                                <FaDownload /> Bulk Export
                            </button>
                        )}
                    </div>

                    {/* SUB FOLDERS: TAX vs NON-TAX */}
                    {!subFolderView ? (
                        <div style={{
                            display: 'grid',
                            gridTemplateColumns: isMobile ? '1fr' : 'repeat(auto-fit, minmax(250px, 1fr))',
                            gap: isMobile ? '12px' : '20px',
                            marginTop: isMobile ? '20px' : '30px'
                        }}>
                            {/* TAX FOLDER */}
                            <div
                                onClick={() => setSubFolderView('tax')}
                                style={{ background: '#f8fafc', padding: isMobile ? '20px' : '25px', borderRadius: '12px', border: '1px dashed 
                                onMouseOver={e => e.currentTarget.style.background = '#f1f5f9'}
                                onMouseOut={e => e.currentTarget.style.background = '#f8fafc'}
                            >
                                <FaRegFileAlt size={40} color="#22c55e" style={{marginBottom: '10px'}} />
                                <h4 style={{ color: '#334155', marginBottom: '5px' }}>Tax Invoices</h4>
                                <p style={{ color: '#64748b', fontSize: '0.9rem' }}>GST Ready Files ({folders.find(f => f.month === expandedMon
                            </div>

                            {/* NON-TAX FOLDER */}
                            <div
                                onClick={() => setSubFolderView('non_tax')}
                                style={{ background: '#f8fafc', padding: isMobile ? '20px' : '25px', borderRadius: '12px', border: '1px dashed 
                                onMouseOver={e => e.currentTarget.style.background = '#f1f5f9'}
                                onMouseOut={e => e.currentTarget.style.background = '#f8fafc'}
                            >
                                <FaFileCsv size={40} color="#64748b" style={{marginBottom: '10px'}} />
                                <h4 style={{ color: '#334155', marginBottom: '5px' }}>Non-Tax / Others</h4>
                                <p style={{ color: '#64748b', fontSize: '0.9rem' }}>Bill of Supply, Receipts ({folders.find(f => f.month === exp
                            </div>
                        </div>
                    ) : (
                        /* FILE LIST VIEW */
                        <div>
                            <button
                                onClick={() => setSubFolderView(null)}
                                style={{ background: 'none', border: 'none', color: '#3b82f6', cursor: 'pointer', marginTop: '15px', fontWeight
                            >
                                ← Back to Folders
                            </button>
                            {renderFileList(
                                subFolderView === 'tax'
                                    ? folders.find(f => f.month === expandedMonth)?.tax_files || []
                                    : folders.find(f => f.month === expandedMonth)?.non_tax_files || [],
                                subFolderView === 'tax' ? 'Tax Invoices (GST)' : 'Non-Tax Documents'
                            )}
                        
                    )}
                </div>
            )}
        </div>
    )}
    </div>
    );
};

export default FileManager;
```

frontend/src/pages/EnhancedDashboard.tsx

```
// frontend/src/pages/EnhancedDashboard.tsx
import axios from "axios";
import React, { useEffect, useState } from "react";
import {
    FiAlertCircle, FiBriefcase,
    FiCreditCard,
    FiDollarSign,
    FiShoppingCart, FiUsers
} from "react-icons/fi";
import {
    Bar,
    BarChart,
    CartesianGrid,
    Legend, ResponsiveContainer,
    Tooltip,
    XAxis, YAxis
} from "recharts";

interface DashboardData {
    finance: {
        bank: { total_balance: number; accounts: any[] };
        loans: { summary: { total_borrowed: number; total_outstanding: number } };
        sales: { outstanding_invoices: { amount: number } };
    };
    operations: {
        inventory: { critical_count: number };
    };
    hr: { total_employees: number; attendance_today: any };
}

const EnhancedDashboard: React.FC = () => {
    const [dashboardData, setDashboardData] = useState<DashboardData | null>(null);
    const [financialData, setFinancialData] = useState<any>(null);
    const [kpis, setKPIs] = useState<any>(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState<string | null>(null);

    useEffect(() => {
        fetchDashboardData();
    }, []);

    const fetchDashboardData = async () => {
        try {
            setLoading(true);
            const token = localStorage.getItem("accessToken");
            const headers = { Authorization: `Bearer ${token}` };

            const [dashResponse, finResponse, kpiResponse] = await Promise.all([
                axios.get("/api/dashboard", { headers }),
                axios.get("/api/dashboard/finance", { headers }),
                axios.get("/api/dashboard/kpis", { headers })
            ]);

            setDashboardData(dashResponse.data.data);
            setFinancialData(finResponse.data.data);
            setKPIs(kpiResponse.data.data);
        } catch (err: any) {
            setError(err.response?.data?.error || "Failed to load dashboard");
        } finally {
            setLoading(false);
        }
    };

    if (loading) {
        return (
            <div className="flex items-center justify-center min-h-screen bg-gradient-to-br from-blue-50 to-indigo-100">
                <div className="text-center">
                    <div className="animate-spin rounded-full h-16 w-16 border-b-4 border-indigo-600 mx-auto mb-4"></div>
                    <p className="text-gray-600 text-lg">Loading Dashboard...</p>
                </div>
            </div>
        );
    }

    if (error) {
        return (
            <div className="p-6 bg-red-50 border border-red-200 rounded-lg m-4">
                <div className="flex items-center gap-3">
                    <FiAlertCircle className="text-red-600 text-xl" />
                    <p className="text-red-800">{error}</p>
                </div>
            </div>
        );
    }

    const SummaryCard = ({ icon: Icon, title, value, unit, color }: any) => (
        <div className={`bg-white rounded-lg shadow-md p-6 border-l-4 ${color}`}>
            <div className="flex items-center justify-between">
                <div>
                    <p className="text-gray-600 text-sm font-medium">{title}</p>
                    <p className="text-2xl font-bold text-gray-900 mt-2">
                        {typeof value === "number" ? value.toLocaleString("en-IN") : value}
                        <span className="text-lg text-gray-500 ml-2">{unit}</span>
                    </p>
                </div>
                <Icon className={`text-3xl ${color.replace("border", "text")}`} />
            </div>
        </div>
    );

    return (
        <div className="min-h-screen bg-gradient-to-br from-gray-50 to-gray-100 p-6">
            <div class="max-w-7xl mx-auto">
                {/* Header */}
                <div class="mb-8">
                    <h1 class="text-4xl font-bold text-gray-900">Dashboard</h1>
```

```
    <p className="text-gray-600 mt-2">
        {new Date().toLocaleDateString("en-IN", {
            weekday: "long",
            year: "numeric",
            month: "long",
            day: "numeric"
        })}
    </p>
</div>

{/* KPI Summary Cards */}
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4 mb-8">
    <SummaryCard
        icon={FiDollarSign}
        title="Total Bank Balance"
        value={dashboardData?.finance.bank.total_balance || 0}
        unit="₹"
        color="border-blue-500 text-blue-600"
    />
    <SummaryCard
        icon={FiCreditCard}
        title="Outstanding Receivables"
        value={kpis?.outstanding_receivables || 0}
        unit="₹"
        color="border-orange-500 text-orange-600"
    />
    <SummaryCard
        icon={FiBriefcase}
        title="Total Loan Exposure"
        value={kpis?.total_loan_exposure || 0}
        unit="₹"
        color="border-red-500 text-red-600"
    />
    <SummaryCard
        icon={FiUsers}
        title="Total Employees"
        value={dashboardData?.hr.total_employees || 0}
        unit=""
        color="border-green-500 text-green-600"
    />
</div>

{/* Financial Overview */}
{financialData && (
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-6 mb-8">
        <div className="lg:col-span-2 bg-white rounded-lg shadow-md p-6">
            <h2 className="text-xl font-bold text-gray-900 mb-4">Monthly Income vs Expense</h2>
            <ResponsiveContainer width="100%" height={300}>
                <BarChart data={financialData.monthly_trend || []}>
                    <CartesianGrid strokeDasharray="3 3" />
                    <XAxis dataKey="month" />
                    <YAxis />
                    <Tooltip />
                    <Legend />
                    <Bar dataKey="sales" fill="#3b82f6" name="Sales" />
                    <Bar dataKey="tax" fill="#10b981" name="Tax" />
                </BarChart>
            </ResponsiveContainer>
        </div>

        <div className="bg-white rounded-lg shadow-md p-6">
            <h2 className="text-xl font-bold text-gray-900 mb-4">Financial Summary</h2>
            <div className="space-y-4">
                <div>
                    <p className="text-gray-600 text-sm">Total Income</p>
                    <p className="text-2xl font-bold text-green-600">
                        ₹{financialData.summary?.total_income.toLocaleString("en-IN")}
                    </p>
                </div>
                <div>
                    <p className="text-gray-600 text-sm">Total Expenses</p>
                    <p className="text-2xl font-bold text-red-600">
                        ₹{financialData.summary?.total_expenses.toLocaleString("en-IN")}
                    </p>
                </div>
                <div className="border-t pt-4">
                    <p className="text-gray-600 text-sm">Net Profit</p>
                    <p className={`text-2xl font-bold ${financialData.summary?.net_profit >= 0 ? "text-green-600" : "text-red-600"}`}>
                        ₹{financialData.summary?.net_profit.toLocaleString("en-IN")}
                    </p>
                    <p className="text-gray-500 text-xs mt-2">
                        Profit Margin: {financialData.summary?.profit_margin_percent}%
                    </p>
                </div>
            </div>
        </div>
    </div>
)}

{/* Operational Metrics */}
<div className="grid grid-cols-1 lg:grid-cols-2 gap-6 mb-8">
    <div className="bg-white rounded-lg shadow-md p-6">
        <h2 className="text-xl font-bold text-gray-900 mb-4 flex items-center gap-2">
            <FiShoppingCart className="text-blue-600" />
            Inventory Status
        </h2>
        <div className="grid grid-cols-3 gap-4">
            <div className="text-center p-4 bg-green-50 rounded">
                <p className="text-gray-600 text-sm">OK Stock</p>
                <p className="text-2xl font-bold text-green-600">
                    {dashboardData?.operations.inventory ?
                        (dashboardData.operations.inventory as any).all_stocks?.length -
                        (dashboardData.operations.inventory as any).critical_count : 0}
                </p>
            </div>
            <div className="text-center p-4 bg-yellow-50 rounded">
                <p className="text-gray-600 text-sm">Low Stock</p>
                <p className="text-2xl font-bold text-yellow-600">
                    {(dashboardData?.operations.inventory as any)?.low_stock_items?.length || 0}
                </p>
            </div>
            <div className="text-center p-4 bg-red-50 rounded">
```

```
                        <p className="text-gray-600 text-sm">Out of Stock</p>
                        <p className="text-2xl font-bold text-red-600">
                            {(dashboardData?.operations.inventory as any)?.out_of_stock_items?.length || 0}
                        </p>
                    </div>
                </div>
            </div>

            <div className="bg-white rounded-lg shadow-md p-6">
                <h2 className="text-xl font-bold text-gray-900 mb-4 flex items-center gap-2">
                    <FiUsers className="text-purple-600" />
                    HR Status
                </h2>
                <div className="space-y-3">
                    <div className="flex justify-between items-center">
                        <span className="text-gray-600">Present Today</span>
                        <span className="font-bold text-green-600">
                            {dashboardData?.hr.attendance_today?.present || 0}
                        </span>
                    </div>
                    <div className="flex justify-between items-center">
                        <span className="text-gray-600">Absent Today</span>
                        <span className="font-bold text-red-600">
                            {dashboardData?.hr.attendance_today?.absent || 0}
                        </span>
                    </div>
                    <div className="flex justify-between items-center">
                        <span className="text-gray-600">Not Marked</span>
                        <span className="font-bold text-gray-600">
                            {dashboardData?.hr.attendance_today?.not_marked || 0}
                        </span>
                    </div>
                </div>
            </div>
        </div>

        {/* Alerts & Notifications */}
        {(dashboardData?.operations.inventory as any)?.critical_count > 0 && (
            <div className="bg-yellow-50 border border-yellow-200 rounded-lg p-4 mb-8">
                <div className="flex items-center gap-3">
                    <FiAlertCircle className="text-yellow-600 text-xl" />
                    <div>
                        <p className="font-bold text-yellow-900">
                            ⚠️ {(dashboardData?.operations.inventory as any)?.critical_count} Products Need Attention
                        </p>
                        <p className="text-yellow-800 text-sm">
                            Low stock or out of stock items require immediate attention
                        </p>
                    </div>
                </div>
            </div>
        )}
    </div>
  </div>
  );
};

export default EnhancedDashboard;
```

frontend/src/pages/Employees.tsx

```
import { AnimatePresence, motion } from 'framer-motion';
import React, { useEffect, useState } from 'react';
import {
    FaBook,
    FaCalculator,
    FaCalendarDay,
    FaCalendarWeek,
    FaEdit,
    FaMoneyBillWave,
    FaPlus,
    FaQrcode,
    FaSearch,
    FaSync,
    FaTimes,
    FaTrash,
    FaUser,
    FaUserClock
} from 'react-icons/fa';
import './Employees.css';
import AdvanceSalaryModal from './hr/AdvanceSalaryModal';
import EmployeeLedgerModal from './hr/EmployeeLedgerModal';
import EmployeeQRModal from './hr/EmployeeQRModal';

interface Employee {
    id: number;
    name: string;
    designation?: string;
    phone?: string;
    email?: string;
    salary: number;
    salary_type: 'Monthly' | 'Weekly' | 'Daily';
    status: string;
    portal_username?: string;
    advance_balance?: number;
    days_present?: number;
}

interface DailyWorkerPayout {
    employee: Employee;
    daysWorked: number;
    dailyRate: number;
    grossAmount: number;
    advanceDeduction: number;
    netPayout: number;
    isSelected: boolean;
}

const Employees: React.FC = () => {
    const [employees, setEmployees] = useState<Employee[]>([]);
    const [loading, setLoading] = useState(true);
    const [search, setSearch] = useState('');
    const [activeTab, setActiveTab] = useState<'ALL' | 'MONTHLY' | 'WEEKLY' | 'DAILY'>('ALL');

    // Modals
    const [showAddModal, setShowAddModal] = useState(false);
    const [showQRModal, setShowQRModal] = useState(false);
    const [showLedgerModal, setShowLedgerModal] = useState(false);
    const [showAdvanceModal, setShowAdvanceModal] = useState(false);
    const [selectedEmployee, setSelectedEmployee] = useState<Employee | null>(null);

    // Daily Workers Payout State
    const [showDailyPayoutModal, setShowDailyPayoutModal] = useState(false);
    const [dailyPayouts, setDailyPayouts] = useState<DailyWorkerPayout[]>([]);
    const [payoutDate, setPayoutDate] = useState(new Date().toISOString().split('T')[0]);

    useEffect(() => {
        fetchEmployees();
    }, []);

    const fetchEmployees = async () => {
        setLoading(true);
        try {
            const token = localStorage.getItem('erp-token');
            const res = await fetch('/api/employees', {
                headers: { 'Authorization': `Bearer ${token}` }
            });
            const data = await res.json();
            setEmployees(data || []);
        } catch (err) {
            console.error('Workforce sync failure', err);
        } finally {
            setLoading(false);
        }
    };

    const handleDelete = async (id: number) => {
        if (!confirm('Relinquish this human asset record? This is irreversible.')) return;
        try {
            const token = localStorage.getItem('erp-token');
            await fetch(`/api/employees/${id}`, {
                method: 'DELETE',
                headers: { 'Authorization': `Bearer ${token}` }
            });
            fetchEmployees();
        } catch (err) {}
    };

    const filteredEmployees = employees.filter(emp => {
        const matchesSearch = emp.name.toLowerCase().includes(search.toLowerCase()) ||
                              (emp.designation || '').toLowerCase().includes(search.toLowerCase());
        const matchesTab = activeTab === 'ALL' || emp.salary_type.toUpperCase() === activeTab;
        return matchesSearch && matchesTab;
    });

    const stats = {
        total: employees.length,
```

```
    monthly: employees.filter(e => e.salary_type === 'Monthly').length,
    weekly: employees.filter(e => e.salary_type === 'Weekly').length,
    daily: employees.filter(e => e.salary_type === 'Daily').length
};

const openDailyPayoutCalculator = () => {
    const dailyWorkers = employees.filter(e => e.salary_type === 'Daily');
    const payouts: DailyWorkerPayout[] = dailyWorkers.map(emp => ({
        employee: emp,
        daysWorked: emp.days_present || 1,
        dailyRate: emp.salary,
        grossAmount: (emp.days_present || 1) * emp.salary,
        advanceDeduction: 0,
        netPayout: (emp.days_present || 1) * emp.salary,
        isSelected: true
    }));
    setDailyPayouts(payouts);
    setShowDailyPayoutModal(true);
};

const updatePayout = (index: number, field: string, value: number | boolean) => {
    setDailyPayouts(prev => {
        const updated = [...prev];
        if (field === 'daysWorked') {
            updated[index].daysWorked = value as number;
            updated[index].grossAmount = (value as number) * updated[index].dailyRate;
        } else if (field === 'advanceDeduction') {
            const maxDeduction = Math.min(value as number, updated[index].employee.advance_balance || 0);
            updated[index].advanceDeduction = maxDeduction;
        } else if (field === 'isSelected') {
            updated[index].isSelected = value as boolean;
        }
        updated[index].netPayout = updated[index].grossAmount - updated[index].advanceDeduction;
        return updated;
    });
};

const selectedPayouts = dailyPayouts.filter(p => p.isSelected);
const totalNet = selectedPayouts.reduce((sum, p) => sum + p.netPayout, 0);
const totalDeductions = selectedPayouts.reduce((sum, p) => sum + p.advanceDeduction, 0);

const processDailyPayouts = async () => {
    if (!confirm(`Process payouts for ${selectedPayouts.length} workers?\n\nTotal: ₹${totalNet.toLocaleString()}`)) return;
    try {
        alert('✅ Daily payouts processed successfully!');
        setShowDailyPayoutModal(false);
        fetchEmployees();
    } catch (err) {
        alert('Failed to process payouts');
    }
};

return (
    <div className="employees-container">
        <header className="employees-header">
            <div className="employees-title">
                <motion.h1 initial={{ opacity: 0, x: -30 }} animate={{ opacity: 1, x: 0 }}>Workforce Registry</motion.h1>
                <motion.p initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: 0.2 }}>
                    Human Capital Allocation & Performance Surveillance Matrix
                </motion.p>
            </div>
            <div className="employees-actions">
                <motion.button whileHover={{ scale: 1.05 }} whileTap={{ scale: 0.95 }} className="btn-secondary" onClick={fetchEmployees} style={{ w
                    <FaSync className={loading ? 'fa-spin' : ''} />
                </motion.button>
                <motion.button whileHover={{ scale: 1.05 }} whileTap={{ scale: 0.95 }} className="btn-primary" onClick={() => { setSelectedEmployee
                    <FaPlus /> Initialize Onboarding
                </motion.button>
            </div>
        </header>

        <div className="stats-matrix">
            {[
                { label: 'Total Personnel', value: stats.total, icon: <FaUser />, color: 'var(--primary)' },
                { label: 'Retainer Base', value: stats.monthly, icon: <FaCalendarWeek />, color: '#6366f1' },
                { label: 'Weekly Operatives', value: stats.weekly, icon: <FaCalendarDay />, color: '#a855f7' },
                { label: 'Daily Tacticals', value: stats.daily, icon: <FaUserClock />, color: '#ec4899' }
            ].map((stat, i) => (
                <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: i * 0.1 }} key={i} className="matrix
                    <div className="matrix-icon" style={{ color: stat.color, opacity: 0.1, position: 'absolute', right: '20px', top: '20px', fontSi
                    <span className="matrix-label">{stat.label}</span>
                    <h3 className="matrix-value">{stat.value}</h3>
                </motion.div>
            ))}
        </div>

        <div className="hub-controls" style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '32px', gap: '2
            <div className="employees-tabs">
                {(['ALL', 'MONTHLY', 'WEEKLY', 'DAILY'] as const).map(tab => (
                    <button key={tab} className={`tab-btn ${activeTab === tab ? 'active' : ''}`} onClick={() => setActiveTab(tab)}>
                        {tab}
                    </button>
                ))}
            </div>

            {activeTab === 'DAILY' && stats.daily > 0 && (
                <motion.button initial={{ scale: 0.9, opacity: 0 }} animate={{ scale: 1, opacity: 1 }} whileHover={{ scale: 1.05 }} className="btn-d
                    <FaCalculator /> Daily Ledger Sync
                </motion.button>
            )}
        </div>

        <div className="search-orb">
            <FaSearch style={{ color: 'var(--text-muted)' }} size={20} />
            <input placeholder="Locate operative via name, designation, or ID code..." value={search} onChange={(e) => setSearch(e.target.value)} />
        </div>

        <div className="employee-table-container">
            <table className="emp-table">
                <thead>
                    <tr>
```

```
                <th>Operative Identity</th>
                <th>Protocol Designation</th>
                <th>Remuneration</th>
                <th style={{ textAlign: 'center' }}>Engagement</th>
                <th>Status</th>
                <th style={{ textAlign: 'center' }}>Interface</th>
            </tr>
        </thead>
        <tbody>
            {loading ? (
                [...Array(6)].map((_, i) => <tr key={i}><td colSpan={6} style={{ padding: '30px' }}><div className="skeleton" style={{ heig
            ) : filteredEmployees.length === 0 ? (
                <tr><td colSpan={6} style={{ padding: '100px 0', textAlign: 'center' }}><FaUser size={64} style={{ opacity: 0.1, marginBott
            ) : (
                filteredEmployees.map((emp, i) => (
                    <motion.tr initial={{ opacity: 0, x: -20 }} animate={{ opacity: 1, x: 0 }} transition={{ delay: i * 0.05 }} key={emp.id
                        <td>
                            <div style={{ display: 'flex', alignItems: 'center', gap: '16px' }}>
                                <div className="profile-orb">{(emp.name || 'U').charAt(0).toUpperCase()}</div>
                                <div>
                                    <div style={{ fontWeight: 800, color: 'var(--text-primary)' }}>{emp.name}</div>
                                    <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)' }}>ID: #{emp.id}</div>
                                </div>
                            </div>
                        </td>
                        <td style={{ fontWeight: 700, color: 'var(--primary)' }}>{emp.designation || 'Specialist'}</td>
                        <td>
                            <div style={{ fontWeight: 900 }}>₹{emp.salary.toLocaleString()}</div>
                            <div style={{ fontSize: '0.7rem', color: 'var(--text-muted)', textTransform: 'uppercase' }}>Per {emp.salary_type
                        </td>
                        <td style={{ textAlign: 'center' }}>
                            <div className="attendance-pill">{emp.days_present || 0} Cycles</div>
                        </td>
                        <td>
                            <span className={`status-pill ${emp.status?.toLowerCase()}`}>{emp.status || 'Active'}</span>
                        </td>
                        <td style={{ textAlign: 'center' }}>
                            <div style={{ display: 'flex', gap: '8px', justifyContent: 'center' }}>
                                <button className="control-btn" style={{ background: 'var(--bg-body)' }} onClick={() => { setSelectedEmploye
                                <button className="control-btn" style={{ background: 'var(--primary-glow)', color: 'var(--primary)' }} onCli
                                <button className="control-btn" style={{ background: 'var(--success-glow)', color: 'var(--success)' }} onCli
                                <button className="control-btn" style={{ background: 'var(--bg-body)' }} onClick={() => { setSelectedEmploye
                                <button className="control-btn" style={{ background: 'var(--error-glow)', color: 'var(--error)' }} onClick=
                            </div>
                        </td>
                    </motion.tr>
                ))
            )}
        </tbody>
    </table>
</div>

{/* Modals */}
<AnimatePresence>
    {showAddModal && (
        <AddEmployeeModalInline
            employee={selectedEmployee}
            onClose={() => setShowAddModal(false)}
            onSuccess={() => { setShowAddModal(false); fetchEmployees(); }}
        />
    )}

    {showQRModal && selectedEmployee && (
        <EmployeeQRModal employee={selectedEmployee} onClose={() => setShowQRModal(false)} />
    )}

    {showLedgerModal && selectedEmployee && (
        <EmployeeLedgerModal employee={selectedEmployee} onClose={() => setShowLedgerModal(false)} />
    )}

    {showAdvanceModal && selectedEmployee && (
        <AdvanceSalaryModal
            employeeId={selectedEmployee.id}
            employeeName={selectedEmployee.name}
            onClose={() => setShowAdvanceModal(false)}
            onSuccess={fetchEmployees}
        />
    )}

    {showDailyPayoutModal && (
        <motion.div initial={{ opacity: 0 }} animate={{ opacity: 1 }} exit={{ opacity: 0 }} className="premium-modal-overlay">
            <motion.div initial={{ scale: 0.9, y: 20 }} animate={{ scale: 1, y: 0 }} className="premium-modal" style={{ maxWidth: '900px' }
                <div className="modal-header">
                    <h3 style={{ margin: 0, fontWeight: 900 }}>Daily Payout Gateway</h3>
                    <button onClick={() => setShowDailyPayoutModal(false)} className="btn-secondary" style={{ padding: '8px', width: '36px'
                </div>
                <div className="modal-content" style={{ maxHeight: '70vh', overflowY: 'auto' }}>
                    <table className="emp-table">
                        <thead>
                            <tr>
                                <th>Operative</th>
                                <th style={{ textAlign: 'center' }}>Cycles</th>
                                <th style={{ textAlign: 'right' }}>Rate</th>
                                <th style={{ textAlign: 'right' }}>Deduction</th>
                                <th style={{ textAlign: 'right' }}>Disbursement</th>
                            </tr>
                        </thead>
                        <tbody>
                            {dailyPayouts.map((p, i) => (
                                <tr key={p.employee.id}>
                                    <td><div style={{ fontWeight: 800 }}>{p.employee.name}</div></td>
                                    <td style={{ textAlign: 'center' }}>
                                        <input type="number" value={p.daysWorked} onChange={e => updatePayout(i, 'daysWorked', Number(e.targ
                                    </td>
                                    <td style={{ textAlign: 'right' }}>₹{p.dailyRate}</td>
                                    <td style={{ textAlign: 'right' }}>
                                        <input type="number" value={p.advanceDeduction} onChange={e => updatePayout(i, 'advanceDeduction', N
                                    </td>
                                    <td style={{ textAlign: 'right', fontWeight: 950, color: 'var(--success)' }}>₹{p.netPayout.toLocaleStrin
                                </tr>
                            ))}
                        </tbody>
```

```
                                </table>
                            </div>
                            <div className="modal-header" style={{ borderBottom: 'none', borderTop: '1px solid var(--border-color)', background: '#f8fa
                                <div style={{ fontWeight: 900 }}>Total Disbursement: <span style={{ color: 'var(--success)', fontSize: '1.5rem' }}>₹{tot
                                <div style={{ display: 'flex', gap: '12px' }}>
                                    <button className="btn-secondary" onClick={() => setShowDailyPayoutModal(false)}>Abort</button>
                                    <button className="btn-primary" onClick={processDailyPayouts} style={{ background: 'var(--success)', border: 'none'
                                </div>
                            </div>
                        </motion.div>
                    </motion.div>
                )}
            </AnimatePresence>
        </div>
    );
};

// --- Subcomponent: Add/Edit Modal ---
const AddEmployeeModalInline: React.FC<{ employee: Employee | null; onClose: () => void; onSuccess: () => void; }> = ({ employee, onClose, onSuccess })
    const [formData, setFormData] = useState({
        name: employee?.name || '',
        designation: employee?.designation || '',
        phone: employee?.phone || '',
        salary: employee?.salary || 0,
        salary_type: employee?.salary_type || 'Monthly',
        status: employee?.status || 'Active'
    });

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        try {
            const token = localStorage.getItem('erp-token');
            const res = await fetch(employee ? `/api/employees/${employee.id}` : '/api/employees', {
                method: employee ? 'PUT' : 'POST',
                headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${token}` },
                body: JSON.stringify(formData)
            });
            if (res.ok) onSuccess();
        } catch (err) { alert("Execution error"); }
    };

    return (
        <div className="premium-modal-overlay">
            <motion.div initial={{ scale: 0.9 }} animate={{ scale: 1 }} className="premium-modal" style={{ maxWidth: '500px' }}>
                <div className="modal-header"><h3 style={{ margin: 0 }}>{employee ? 'Modify Asset' : 'Onboard Operative'}</h3></div>
                <form onSubmit={handleSubmit} style={{ padding: '32px' }}>
                    <div className="input-group">
                        <label>Personnel Name</label>
                        <input value={formData.name} onChange={e => setFormData({...formData, name: e.target.value})} className="premium-input" require
                    </div>
                    <div class="input-group">
                        <label>Protocol Role</label>
                        <input value={formData.designation} onChange={e => setFormData({...formData, designation: e.target.value})} class="premium-
                    </div>
                    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '20px' }}>
                        <div class="input-group">
                            <label>Remuneration</label>
                            <input type="number" value={formData.salary} onChange={e => setFormData({...formData, salary: Number(e.target.value)})} clas
                        </div>
                        <div class="input-group">
                            <label>Cycle</label>
                            <select value={formData.salary_type} onChange={e => setFormData({...formData, salary_type: e.target.value as any})} classNa
                                <option value="Monthly">Monthly</option>
                                <option value="Weekly">Weekly</option>
                                <option value="Daily">Daily</option>
                            </select>
                        </div>
                    </div>
                    <div style={{ display: 'flex', gap: '15px', marginTop: '10px' }}>
                        <button type="button" onClick={onClose} class="btn-secondary" style={{ width: '100%' }}>Abort</button>
                        <button type="submit" class="btn-primary" style={{ width: '100%' }}>Commit Record</button>
                    </div>
                </form>
            </motion.div>
        </div>
    );
};

export default Employees;
```

frontend/src/pages/EditInvoice.tsx

```
// frontend/src/pages/EditInvoice.tsx
import React, { useEffect, useRef, useState } from "react";
import { FaArrowLeft, FaCreditCard, FaExchangeAlt, FaGlobe, FaMobileAlt, FaMoneyBillWave, FaPrint, FaReceipt, FaSave, FaTrash, FaUniversity, FaWallet }
import { useNavigate, useParams } from "react-router-dom";
import { fetchProfile } from "../api/companyApi";
import { useAuthUser } from "../hooks/useAuthUser";
import { useUsers } from "../hooks/useUsers";
import { apiFetch } from "../utils/api";

const toWords = (num: number): string => {
    const a = ['', 'One ', 'Two ', 'Three ', 'Four ', 'Five ', 'Six ', 'Seven ', 'Eight ', 'Nine ', 'Ten ', 'Eleven ', 'Twelve ', 'Thirteen ', 'Fourteen
    const b = ['', '', 'Twenty', 'Thirty', 'Forty', 'Fifty', 'Sixty', 'Seventy', 'Eighty', 'Ninety'];
    const formatted = ('000000000' + num).substr(-9);
    const n = formatted.match(/^(\d{2})(\d{2})(\d{2})(\d{1})(\d{2})$/);
    if (!n) return '';
    let str = '';
    const n1 = Number(n[1]); const n2 = Number(n[2]); const n3 = Number(n[3]); const n4 = Number(n[4]); const n5 = Number(n[5]);
    if (n1 !== 0) str += (a[n1] || b[Math.floor(n1 / 10)] + ' ' + a[n1 % 10]) + 'Crore ';
    if (n2 !== 0) str += (a[n2] || b[Math.floor(n2 / 10)] + ' ' + a[n2 % 10]) + 'Lakh ';
    if (n3 !== 0) str += (a[n3] || b[Math.floor(n3 / 10)] + ' ' + a[n3 % 10]) + 'Thousand ';
    if (n4 !== 0) str += (a[n4] || b[Math.floor(n4 / 10)] + ' ' + a[n4 % 10]) + 'Hundred ';
    if (n5 !== 0) str += (str !== '' ? 'and ' : '') + (a[n5] || b[Math.floor(n5 / 10)] + ' ' + a[n5 % 10]) + 'only ';
    return str;
};

const fmt = (n: any) => Number(n || 0).toLocaleString('en-IN', { minimumFractionDigits: 2, maximumFractionDigits: 2 });

// Table cell styles
const tableHeaderStyle: React.CSSProperties = { border: "1px solid #000", padding: "4px 3px", fontWeight: "bold", background: "#f1f5f9", textAlign: "cen
const tableCellStyle: React.CSSProperties = { border: "1px solid #000", padding: "3px", textAlign: "center", height: "20px" };
const tableCellLeftStyle: React.CSSProperties = { ...tableCellStyle, textAlign: "left", paddingLeft: "4px" };
const tableCellRightStyle: React.CSSProperties = { ...tableCellStyle, textAlign: "right", paddingRight: "4px" };

// Payment method options
const PAYMENT_METHODS = [
    { value: 'CASH', label: 'Cash', icon: <FaMoneyBillWave />, color: '#10b981' },
    { value: 'BANK_TRANSFER', label: 'Bank', icon: <FaUniversity />, color: '#3b82f6' },
    { value: 'UPI', label: 'UPI', icon: <FaMobileAlt />, color: '#8b5cf6' },
    { value: 'CARD', label: 'Card', icon: <FaCreditCard />, color: '#f59e0b' },
    { value: 'CHEQUE', label: 'Cheque', icon: <FaReceipt />, color: '#64748b' },
    { value: 'WALLET', label: 'Wallet', icon: <FaWallet />, color: '#ec4899' }
];

// Interfaces
interface PaymentEntry {
    amount: number;
    payment_method: string;
    payment_date: string;
    reference_no: string;
}

interface InvoiceItem {
    name: string;
    hsn: string;
    uom: string;
    qty: number;
    rate: number;
}

const EditInvoice: React.FC = () => {
    const { id } = useParams();
    const navigate = useNavigate();
    const { customers } = useUsers();
    const { user } = useAuthUser();
    const printRef = useRef<HTMLDivElement>(null);
    const [windowWidth, setWindowWidth] = useState(window.innerWidth);

    useEffect(() => {
        const handleResize = () => setWindowWidth(window.innerWidth);
        window.addEventListener('resize', handleResize);
        return () => window.removeEventListener('resize', handleResize);
    }, []);

    const [loading, setLoading] = useState(true);
    const [company, setCompany] = useState<any>({});
    const [invoiceNo, setInvoiceNo] = useState("");
    const [items, setItems] = useState<InvoiceItem[]>([{ name: "", hsn: "", uom: "Pcs", qty: 0, rate: 0 }]);
    const [notes, setNotes] = useState("");
    const [amountPaidAlready, setAmountPaidAlready] = useState<number>(0);

    // Multiple payments array (for new payments)
    const [payments, setPayments] = useState<PaymentEntry[]>([
        { amount: 0, payment_method: 'CASH', payment_date: new Date().toISOString().slice(0, 10), reference_no: '' }
    ]);

    const [meta, setMeta] = useState({
        invoiceDate: new Date().toISOString().slice(0, 10),
        vehicleNumber: "", bundles: 0, placeOfSupply: "", supplyStateCode: "",
        transportMode: "", reverseCharge: "No", dateOfSupply: new Date().toISOString().slice(0, 10)
    });
    const [customerId, setCustomerId] = useState<number | null>(null);
    const [customer, setCustomer] = useState({ name: "", address: "", gstin: "", state: "", stateCode: "" });

    // Currency Converter State
    const [showConverter, setShowConverter] = useState(false);
    const [convertAmount, setConvertAmount] = useState<string>("1");
    const [fromCurrency, setFromCurrency] = useState<string>("USD");
    const [convertedINR, setConvertedINR] = useState<number | null>(null);
    const [isConverting, setIsConverting] = useState(false);

    const performConversion = async () => {
        if (!convertAmount || parseFloat(convertAmount) <= 0) return;
        setIsConverting(true);
        try {
            const res = await fetch(`https://api.frankfurter.app/latest?amount=${convertAmount}&from=${fromCurrency}&to=INR`);
            const data = await res.json();
```

```
        if (data.rates && data.rates.INR) {
            setConvertedINR(data.rates.INR);
        }
    } catch (err) {
        console.error("Conversion failed", err);
    } finally {
        setIsConverting(false);
    }
};

// Load Data
useEffect(() => {
    const load = async () => {
        try {
            // Fetch Profile and Invoice Data in parallel
            const [profile, invoiceRes] = await Promise.all([
                fetchProfile(),
                apiFetch(`/invoice/${id}`)
            ]);

            // Set Company Profile
            setCompany({
                name: profile.company_name,
                address: profile.address_line1,
                city_pincode: profile.city_pincode,
                state: profile.state,
                stateCode: "33",
                gstin: profile.gstin,
                bank_name: profile.bank_name,
                ac_no: profile.bank_account_no,
                ifsc: profile.bank_ifsc_code,
                signature_url: profile.signature_url || null
            });

            // Set Invoice Data
            const inv = await invoiceRes.json();

            setInvoiceNo(inv.invoice_number);
            setNotes(inv.notes || "");
            setAmountPaidAlready(Number(inv.paid_amount) || 0);

            // Meta
            setMeta({
                invoiceDate: inv.invoice_date ? inv.invoice_date.substring(0, 10) : new Date().toISOString().slice(0, 10),
                vehicleNumber: inv.vehicle_number || "",
                bundles: Number(inv.bundles_count) || 0,
                transportMode: inv.transportation_mode || "",
                reverseCharge: inv.reverse_charge || "No",
                dateOfSupply: inv.date_of_supply ? inv.date_of_supply.substring(0, 10) : new Date().toISOString().slice(0, 10),
                placeOfSupply: "", // Will be set when customer loads
                supplyStateCode: ""
            });

            // Items
            if (inv.items && inv.items.length > 0) {
               setItems(inv.items.map((i: any) => ({
                   name: i.description || "",
                   hsn: i.hsn_acs_code || "",
                   uom: "Pcs", // Default
                   qty: Number(i.quantity) || 0,
                   rate: Number(i.unit_price) || 0
               })));
            }

            // Customer
            if (inv.customer_id) {
                setCustomerId(inv.customer_id);
                // Find customer details from the customers list (which should be cached/loaded by useUsers)
                // Note: customers might not be loaded yet if useUsers is async.
                // However useEffect dependency on `customers` will handle the update.
            }

        } catch (err) {
            console.error("Failed to load edit details", err);
            alert("Could not load invoice details");
        } finally {
            setLoading(false);
        }
    };
    load();
}, [id]);

// Update Customer details when `customers` list or `customerId` changes
useEffect(() => {
    if (customerId && customers.length > 0) {
         const c = customers.find(x => x.id === customerId);
         if (c) {
            setCustomer({
                name: c.username.toUpperCase(),
                address: `${c.address_line1 || ''}, ${c.city_pincode || ''}`.toUpperCase(),
                gstin: c.gstin || "", state: c.state?.toUpperCase() || "", stateCode: c.state_code || ""
            });
            setMeta(prev => ({
                ...prev,
                placeOfSupply: c.state?.toUpperCase() || "",
                supplyStateCode: c.state_code || ""
            }));
         }
    }
}, [customerId, customers]);


const handleCustomerSelect = (e: React.ChangeEvent<HTMLSelectElement>) => {
    const id = parseInt(e.target.value);
    setCustomerId(id);
};

let totalTaxable = 0, totalCGST = 0, totalSGST = 0, totalIGST = 0;
const rows = items.map(it => {
    const amount = it.qty * it.rate;
    const taxable = amount;
    const isSameState = (customer.stateCode || "33") === (company.stateCode || "33");
```

```
    const cgstRate = isSameState ? 2.5 : 0;
    const sgstRate = isSameState ? 2.5 : 0;
    const igstRate = isSameState ? 0 : 5;
    const cgst = taxable * (cgstRate / 100);
    const sgst = taxable * (sgstRate / 100);
    const igst = taxable * (igstRate / 100);
    totalTaxable += taxable; totalCGST += cgst; totalSGST += sgst; totalIGST += igst;
    return { ...it, amount, taxable, cgstRate, sgstRate, igstRate, cgst, sgst, igst, lineTotal: taxable + cgst + sgst + igst };
});

const totalGST = totalCGST + totalSGST + totalIGST;
const grandTotal = totalTaxable + totalGST;

// Calculate total from all payments being made now (New payments)
const paymentNow = payments.reduce((sum, p) => sum + (p.amount || 0), 0);
const totalPaid = amountPaidAlready + paymentNow;
const balanceDue = grandTotal - totalPaid;
const paymentProgress = grandTotal > 0 ? (totalPaid / grandTotal) * 100 : 0;

const getPaymentStatus = () => {
    if (balanceDue <= 0 && grandTotal > 0) return { label: 'PAID', color: '#10b981', bg: '#dcfce7' };
    if (totalPaid > 0) return { label: 'PARTIAL', color: '#f59e0b', bg: '#fef3c7' };
    return { label: 'UNPAID', color: '#ef4444', bg: '#fee2e2' };
};
const status = getPaymentStatus();

const handleQuickPayment = (percent: number) => {
    const maxPayable = grandTotal - amountPaidAlready;
    const amount = Math.round((maxPayable * percent) / 100);
    const newPayments = [...payments];
    newPayments[0] = { ...newPayments[0], amount };
    setPayments(newPayments);
};

const addPayment = () => {
    setPayments([...payments, {
        amount: 0,
        payment_method: 'CASH',
        payment_date: new Date().toISOString().slice(0, 10),
        reference_no: ''
    }]);
};

const removePayment = (index: number) => {
    if (payments.length > 1) {
        const newPayments = [...payments];
        newPayments.splice(index, 1);
        setPayments(newPayments);
    }
};

const updatePayment = (index: number, field: keyof PaymentEntry, value: any) => {
    const newPayments = [...payments];
    newPayments[index] = { ...newPayments[index], [field]: value };
    setPayments(newPayments);
};

const handleUpdate = async () => {
    if (!customerId) {
        alert("Please select a customer");
        return;
    }

    if (!invoiceNo.trim()) {
        alert("Please enter an invoice number");
        return;
    }

    if (items.length === 0 || items.every(item => !item.name.trim())) {
        alert("Please add at least one item");
        return;
    }

    // Filter and format valid payments (only new payments)
    const validPayments = payments
        .filter(p => p.amount > 0)
        .map(p => ({
            amount: Number(p.amount),
            payment_method: p.payment_method,
            payment_date: p.payment_date,
            reference_no: p.reference_no || '',
            notes: `Payment via ${PAYMENT_METHODS.find(m => m.value === p.payment_method)?.label || p.payment_method}`
        }));

    const formattedItems = items
        .filter(item => item.name.trim())
        .map(item => ({
            description: item.name, // NOTE: Backend expects 'description', frontend input is 'name'
            hsn_acs_code: item.hsn || '',
            quantity: Number(item.qty) || 0,
            unit_price: Number(item.rate) || 0,
            gst_rate: 0 // Will be calculated by backend or ignored as layout does calc
        }));

    // Create payload
    const payload: any = {
        invoice_number: invoiceNo.trim(),
        customer_id: customerId,
        items: formattedItems,
        notes: notes.trim(),
        amount_paid: totalPaid,
        balance_due: balanceDue,
        payment_status: status.label,
        payments: validPayments,
        transport_details: {
            vehicle: meta.vehicleNumber || '',
            mode: meta.transportMode || '',
            supply_date: meta.dateOfSupply,
            reverse_charge: meta.reverseCharge
        },
        bundles_count: Number(meta.bundles) || 0
    };
```

```
    // Note: We might need to handle how new payments are appended to history, for now we update total paid

    try {
        const response = await apiFetch(`/invoice/${id}`, {
            method: "PUT",
            body: JSON.stringify(payload)
        });

        if (response.ok) {
            alert("Invoice updated successfully!");
            navigate("/invoices");
        } else {
            alert("Failed to update invoice");
        }
    } catch (err: any) {
        console.error('Exception while saving:', err);
        alert(`Failed to save invoice: ${err.message || 'Network error'}`);
    }
};

const printInvoice = () => {
    const printContents = printRef.current?.innerHTML;
    if (!printContents) return;
    const w = window.open('', '', 'width=900,height=650');
    w!.document.write(`<html><head><title>Invoice</title><style>body{margin:0;padding:0;font-family:Arial}table{width:100%;border-collapse:collapse
    w!.document.close(); w!.focus(); w!.print();
};

const sectionStyle: React.CSSProperties = { marginBottom: '24px' };
const sectionTitleStyle: React.CSSProperties = { fontSize: '0.85rem', fontWeight: 800, color: 'var(--text-muted)', marginBottom: '16px', display: '
const inputLabelStyle: React.CSSProperties = { fontWeight: 600, fontSize: '0.75rem', color: 'var(--text-main)', marginBottom: '8px', display: 'bloc
const inputStyle: React.CSSProperties = {
    width: '100%',
    padding: '10px 12px',
    borderRadius: '8px',
    border: '1px solid var(--border-color)',
    fontSize: '0.9rem',
    outline: 'none',
    transition: 'border-color 0.2s, box-shadow 0.2s'
};

if (loading) return <div style={{ padding: "40px", textAlign: "center" }}>Loading Invoice...</div>;

return (
    <div className="invoice-layout">
        <div className="invoice-form-side">
            <div className="glass-panel" style={{ padding: "16px 24px", borderBottom: "1px solid var(--border-color)", display: "flex", alignItems:
                <button onClick={() => navigate("/invoices")} className="btn-secondary" style={{ padding: '8px', width: '36px', height: '36px', dis
                    <FaArrowLeft size={14} />
                </button>
                <h2 style={{ fontSize: "1.25rem", fontWeight: 800, flex: 1, margin: 0, color: 'var(--text-main)', letterSpacing: '-0.5px' }}>Edit In
                <div style={{ display: 'flex', gap: '8px' }}>
                    <button
                        onClick={() => setShowConverter(!showConverter)}
                        className="btn-secondary"
                        style={{
                            padding: "8px 12px", fontSize: "0.8rem", gap: '6px', display: 'flex', alignItems: 'center', borderColor: showConverter
                        }}
                    >
                        <FaGlobe /> <span style={{ display: window.innerWidth > 1400 ? 'inline' : 'none' }}>Convert</span>
                    </button>
                    <button onClick={printInvoice} className="btn-secondary" style={{ padding: "8px 12px", fontSize: "0.8rem", gap: '6px', display:
                </div>
            </div>

            {/* Live Currency Converter Tool */}
            {showConverter && (
                <div className="page-transition" style={{ padding: '20px', background: 'var(--bg-body)', borderBottom: '1px solid var(--border-colo
                    <div className="card" style={{ border: '1px solid var(--primary-glow)', boxShadow: 'var(--shadow-sm)' }}>
                        <div style={{ fontSize: '0.75rem', fontWeight: 800, color: 'var(--primary)', marginBottom: '12px', textTransform: 'uppercase
                            <FaExchangeAlt /> Currency Converter (INR)
                        </div>
                        <div style={{ display: 'flex', gap: '10px' }}>
                            <div style={{ flex: 1 }}>
                                <input
                                    type="number"
                                    value={convertAmount}
                                    onChange={(e) => setConvertAmount(e.target.value)}
                                    placeholder="Amount"
                                    style={{ ...inputStyle, background: 'white' }}
                                />
                            </div>
                            <div style={{ width: '100px' }}>
                                <select
                                    value={fromCurrency}
                                    onChange={(e) => setFromCurrency(e.target.value)}
                                    style={{ ...inputStyle, background: 'white', appearance: 'auto' }}
                                >
                                    <option value="USD">USD</option>
                                    <option value="EUR">EUR</option>
                                    <option value="GBP">GBP</option>
                                    <option value="SGD">SGD</option>
                                    <option value="AED">AED</option>
                                    <option value="SAR">SAR</option>
                                </select>
                            </div>
                            <button
                                onClick={performConversion}
                                disabled={isConverting}
                                className="btn-primary"
                                style={{ padding: '0 20px', height: '42px' }}
                            >
                                {isConverting ? '...' : 'Convert'}
                            </button>
                        </div>
                        {convertedINR !== null && (
                            <div class="page-transition" style={{ marginTop: '16px', paddingTop: '16px', borderTop: '1px dashed var(--border-co
                                <div>
                                    <span style={{ fontSize: '0.7rem', color: 'var(--text-muted)', fontWeight: 600 }}>RESULT</span>
                                    <div style={{ fontSize: '1.5rem', fontWeight: 900, color: 'var(--text-main)' }}>₹ {fmt(convertedINR)}</div>
                                </div>
```

```
                        <button
                            onClick={() => {
                                const newItems = [...items];
                                newItems[newItems.length - 1].rate = Math.round(convertedINR * 100) / 100;
                                setItems(newItems);
                                setShowConverter(false);
                            }}
                            style={{ fontSize: '0.8rem', color: 'var(--primary)', background: 'var(--primary-glow)', border: 'none', fontWe
                        >
                            Apply to Item
                        </button>
                    </div>
                )}
            </div>
        </div>
    )}

    <div style={{ flex: 1, overflowY: "auto", padding: "20px" }}>
        {/* Basic Details Section */}
        <div style={sectionStyle}>
            <h3 style={sectionTitleStyle}>Basic Details</h3>
            <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fit, minmax(180px, 1fr))", gap: "15px", marginBottom: "15px" }}
                <div><label style={inputLabelStyle}>Invoice No</label><input value={invoiceNo} onChange={e => setInvoiceNo(e.target.value)}
                <div><label style={inputLabelStyle}>Invoice Date</label><input type="date" value={meta.invoiceDate} onChange={e => setMeta(
            </div>
            <div><label style={inputLabelStyle}>Customer</label>
                <select value={customerId || ""} onChange={handleCustomerSelect} style={inputStyle}>
                    <option value="">-- Select Customer --</option>
                    {customers.filter(c => c.username !== "admin").map(c => <option key={c.id} value={c.id}>{c.username}</option>)}
                </select>
            </div>
        </div>

        {/* Items Section */}
        <div style={sectionStyle}>
            <h3 style={sectionTitleStyle}>Items</h3>
            {items.map((it, i) => (
                <div key={i} style={{ background: "#f8fafc", padding: "10px", borderRadius: "6px", border: "1px solid #e2e8f0", marginBottom
                    <div style={{ paddingRight: "30px" }}>
                        <input placeholder="Item Name" value={it.name} onChange={e => { const t = [...items]; t[i].name = e.target.value; se
                    </div>
                    <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fit, minmax(80px, 1fr))", gap: "8px" }}>
                        <input placeholder="HSN" value={it.hsn} onChange={e => { const t = [...items]; t[i].hsn = e.target.value; setItems(
                        <input type="number" placeholder="Qty" value={it.qty || ""} onChange={e => { const t = [...items]; t[i].qty = Numbe
                        <input type="number" placeholder="Rate" value={it.rate || ""} onChange={e => { const t = [...items]; t[i].rate = Nu
                    </div>
                    <button onClick={() => { const t = [...items]; t.splice(i, 1); setItems(t); }} style={{ position: "absolute", top: "10p
                </div>
            ))}
            <button onClick={() => setItems([...items, { name: "", hsn: "", uom: "Pcs", qty: 0, rate: 0 }])} style={{ width: "100%", padding
        </div>

        {/* Payment & Notes Section */}
        <div style={sectionStyle}>
            <div style={{ display: "flex", alignItems: "center", gap: "10px", marginBottom: "15px" }}>
                <FaMoneyBillWave color="#10b981" size={18} />
                <h3 style={{ ...sectionTitleStyle, margin: 0 }}>Payment & Notes</h3>
                <span style={{ marginLeft: "auto", padding: "4px 10px", background: status.bg, color: status.color, borderRadius: "12px", fo
            </div>

            <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fit, minmax(100px, 1fr))", gap: "10px", marginBottom: "15px" }}
                <div style={{ padding: "10px", background: "#f8fafc", borderRadius: "8px", textAlign: "center" }}><div style={{ fontSize: "0
                <div style={{ padding: "10px", background: "#dcfce7", borderRadius: "8px", textAlign: "center" }}><div style={{ fontSize: "0
                <div style={{ padding: "10px", background: balanceDue > 0 ? "#fee2e2" : "#dcfce7", borderRadius: "8px", textAlign: "center"
            </div>

            <div style={{ marginBottom: "15px" }}>
                <label style={inputLabelStyle}>Amount Paid Already (₹)</label>
                <input type="number" value={amountPaidAlready || ""} onChange={e => setAmountPaidAlready(Number(e.target.value))} style={{
            </div>

            {/* New Payment Entry */}
             <div style={{ padding: "15px", background: "#f0fdf4", borderRadius: "8px", border: "1px solid #bbf7d0", marginBottom: "15px" }}
                 <label style={{ ...inputLabelStyle, color: "#166534" }}>Add New Payment</label>
                  <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fit, minmax(140px, 1fr))", gap: "8px", marginBottom: "8p
                     <input type="number" placeholder="Amount" value={payments[0].amount || ""} onChange={e => updatePayment(0, 'amount', Nu
                     <select style={inputStyle} value={payments[0].payment_method} onChange={e => updatePayment(0, 'payment_method', e.targe
                         {PAYMENT_METHODS.map(m => <option key={m.value} value={m.value}>{m.label}</option>)}
                     </select>
                 </div>
             </div>

            <div style={{ marginBottom: "15px" }}><label style={inputLabelStyle}>Notes / Terms</label><textarea rows={3} value={notes} onCha
            <div style={{ background: "#f8fafc", padding: "10px", borderRadius: "6px", border: "1px dashed #cbd5e1" }}><label style={{ fontS
        </div>

        {/* Additional Details Section */}
        <div style={sectionStyle}>
            <h3 style={sectionTitleStyle}>Additional Details</h3>
            <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fit, minmax(140px, 1fr))", gap: "15px", marginBottom: "15px" }}
                <div><label style={inputLabelStyle}>Reverse Charge</label><select value={meta.reverseCharge} onChange={e => setMeta({ ...met
                <div><label style={inputLabelStyle}>Transport Mode</label><input value={meta.transportMode} onChange={e => setMeta({ ...meta
            </div>
            <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fit, minmax(140px, 1fr))", gap: "15px", marginBottom: "15px" }}
                <div><label style={inputLabelStyle}>Vehicle No.</label><input value={meta.vehicleNumber} onChange={e => setMeta({ ...meta, V
                <div><label style={inputLabelStyle}>Date of Supply</label><input type="date" value={meta.dateOfSupply} onChange={e => setMet
            </div>
            <div><label style={inputLabelStyle}>No. of Bundles</label><input type="number" value={meta.bundles} onChange={e => setMeta({ ..
        </div>
    </div>

    {/* Footer with Save Button */}
    {/* Footer with Save Button */}
    <div className="glass-panel" style={{ padding: "15px", borderTop: "1px solid var(--border-color)", display: "flex", justifyContent: "spa
        <div><div style={{ fontSize: "0.7rem", color: "var(--text-muted)" }}>Grand Total</div><div style={{ fontSize: "1.2rem", fontWeight:
        <button className="btn-primary" onClick={handleUpdate} style={{ gap: "8px" }}><FaSave /> Update Invoice</button>
    </div>
</div>

{/* Print Preview Section */}
<div className="invoice-preview-side" style={{ overflow: 'hidden' }}>
    <div
```

```
        ref={printRef}
        style={{
            width: "210mm",
            minWidth: "210mm",
            minHeight: "297mm",
            background: "#fff",
            boxShadow: "0 0 12px rgba(0,0,0,0.4)",
            padding: "15px",
            boxSizing: "border-box",
            transform: windowWidth < 1024 ? `scale(${(windowWidth - 40) / 794})` : 'none',
            transformOrigin: "top center",
            marginBottom: windowWidth < 1024 ? `-${297 * 3.78 * (1 - (windowWidth - 40) / 794)}px` : '0',
            flexShrink: 0
        }}
    >
        <div style={{ border: "1px solid #000", width: "100%", position: "relative" }}>
            {/* TRIPLICATE INDICATOR */}
            <div style={{ position: "absolute", top: "0", right: "0", fontSize: "10px", textAlign: "right", border: "1px solid #ccc", paddi
                <div style={{color: 'red'}}>Original for Recipient</div>
                <div style={{color: 'blue'}}>Duplicate for Supplier</div>
                <div style={{color: 'green'}}>Triplicate for Supplier</div>
            </div>

            {/* Header */}
            <div style={{ textAlign: "center", padding: "15px 0 0 0", clear: "both" }}>
                <h1 style={{ margin: "0", fontSize: "32px", fontWeight: "900", textTransform: "uppercase", letterSpacing: "1px" }}>{company
                <p style={{ margin: "4px 0", fontSize: "11px" }}>
                    {company.address}{company.city_pincode ? `, ${company.city_pincode}` : ''}
                </p>
                <p style={{ margin: "3px 0", fontSize: "12px", fontWeight: "bold" }}>{company.state?.toUpperCase() || 'TAMILNADU'}</p>
                <p style={{ margin: "3px 0 0 0", fontSize: "12px", fontWeight: "bold" }}>GSTIN No. : {company.gstin}</p>
            </div>

            {/* Invoice Title */}
            <div style={{ textAlign: "center", border: "1px solid #000", borderBottom: "none", padding: "2px 5px", fontWeight: "900", fontS
                INVOICE
            </div>

            {/* Metadata Grid */}
            <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", border: "1px solid #000", borderBottom: "none", fontSize: "11px"
                <div style={{ borderRight: "1px solid #000" }}>
                    <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
                        <span>Reverse Charge</span><span>: {meta.reverseCharge}</span>
                    </div>
                    <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
                        <span>Invoice No</span><span>: <b>{invoiceNo}</b></span>
                    </div>
                    <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
                        <span>Invoice Date</span><span>: <b>{new Date(meta.invoiceDate).toLocaleDateString("en-GB")}</b></span>
                    </div>
                    <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", padding: "2px 5px" }}>
                        <span>State</span><span>: {company.state || 'TAMILNADU'} Code: {company.stateCode || '33'}</span>
                    </div>
                </div>
                <div>
                    <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
                        <span>Transport Mode</span><span>: {meta.transportMode}</span>
                    </div>
                    <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
                        <span>Vehicle Number</span><span>: {meta.vehicleNumber}</span>
                    </div>
                    <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", borderBottom: "1px solid #000", padding: "2px 5px" }}>
                        <span>Date of Supply</span><span>: {new Date(meta.dateOfSupply).toLocaleDateString("en-GB")}</span>
                    </div>
                    <div style={{ display: "grid", gridTemplateColumns: "160px 1fr", padding: "2px 5px" }}>
                        <span>Place of Supply</span><span>: {meta.placeOfSupply || 'TAMILNADU'} Code: {meta.supplyStateCode || '33'}</span>
                    </div>
                </div>
            </div>

            {/* Customer Details */}
            <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", border: "1px solid #000", borderBottom: "none", fontSize: "11px"
                <div style={{ borderRight: "1px solid #000", padding: "5px" }}>
                    <b style={{ textDecoration: "underline" }}>Details of Receiver Billed To :</b>
                    <div style={{ marginTop: '4px' }}>Name : <b>{customer.name}</b></div>
                    <div>Address : {customer.address}</div>
                    <div>GSTIN : <b>{customer.gstin}</b></div>
                    <div>State : {customer.state || 'TAMILNADU'} Code : {customer.stateCode || '33'}</div>
                </div>
                <div style={{ padding: "5px" }}>
                    <b style={{ textDecoration: "underline" }}>Details of Consignee :</b>
                    <div style={{ marginTop: '4px' }}>Name : <b>{customer.name}</b></div>
                    <div>Address : {customer.address}</div>
                    <div>GSTIN : <b>{customer.gstin}</b></div>
                    <div>State : {customer.state || 'TAMILNADU'} Code : {customer.stateCode || '33'}</div>
                </div>
            </div>

            {/* Items Table */}
            <div style={{ border: "1px solid #000", minHeight: "400px" }}>
                <table style={{ width: "100%", borderCollapse: "collapse", fontSize: "9px" }}>
                    <thead>
                        <tr style={{ borderBottom: "1px solid #000", background: "#f9fafb" }}>
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "35px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", textAlign: "left", fontSize: "9px", fontWeight:
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "50px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "35px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "35px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "70px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "75px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "60px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "75px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "55px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "55px", fontSize: "9px", fontWeight: "bo
                            <th style={{ borderRight: "1px solid #000", padding: "4px 2px", width: "55px", fontSize: "9px", fontWeight: "bo
                            <th style={{ padding: "4px 2px", width: "85px", fontSize: "9px", fontWeight: "bold" }}>Total</th>
                        </tr>
                    </thead>
                    <tbody>
                        {Array.from({ length: 15 }).map((_, i) => {
                            const r = rows[i] || {};
                            return (
```

```
                                <tr key={i}>
                                    <td style={tableCellStyle}>{i + 1}</td>
                                    <td style={tableCellLeftStyle}>{r.name || ""}</td>
                                    <td style={tableCellStyle}>{r.hsn || ""}</td>
                                    <td style={tableCellStyle}>{r.uom || ""}</td>
                                    <td style={tableCellStyle}>{r.qty || ""}</td>
                                    <td style={tableCellStyle}>{r.rate ? fmt(r.rate) : ""}</td>
                                    <td style={tableCellStyle}>{r.amount ? fmt(r.amount) : ""}</td>
                                    <td style={tableCellStyle}>0.00</td>
                                    <td style={tableCellStyle}>{r.taxable ? fmt(r.taxable) : ""}</td>
                                    <td style={tableCellStyle}>{r.cgst > 0 ? fmt(r.cgst) : ""}</td>
                                    <td style={tableCellStyle}>{r.sgst > 0 ? fmt(r.sgst) : ""}</td>
                                    <td style={tableCellStyle}>{r.igst > 0 ? fmt(r.igst) : ""}</td>
                                    <td style={tableCellStyle}>{r.lineTotal ? fmt(r.lineTotal) : ""}</td>
                                </tr>
                            );
                        })}
                    </tbody>
                    <tfoot>
                        <tr style={{ fontWeight: "bold", background: "#f1f5f9", borderTop: "1px solid #000" }}>
                            <td colSpan={6} style={tableCellRightStyle}>Total</td>
                            <td style={tableCellStyle}>{fmt(totalTaxable)}</td>
                            <td style={tableCellStyle}>0.00</td>
                            <td style={tableCellStyle}>{fmt(totalTaxable)}</td>
                            <td style={tableCellStyle}>{fmt(totalCGST)}</td>
                            <td style={tableCellStyle}>{fmt(totalSGST)}</td>
                            <td style={tableCellStyle}>{fmt(totalIGST)}</td>
                            <td style={tableCellStyle}>{fmt(grandTotal)}</td>
                        </tr>
                    </tfoot>
                </table>
            </div>

            {/* Footer Section */}
            <div style={{ display: "grid", gridTemplateColumns: "1.2fr 0.8fr", border: "1px solid #000", borderTop: "none", fontSize: "11px"
                <div style={{ borderRight: "1px solid #000", padding: "6px", display: "flex", flexDirection: "column", justifyContent: "spac
                    <div>
                        <b>Total Invoice Amount in words</b><br />
                        <div style={{ marginTop: "2px", textTransform: "capitalize", fontWeight: "bold" }}>{grandTotal ? toWords(Math.round
                        <br />
                        Bundles : {meta.bundles}
                    </div>
                    <div style={{ marginTop: "10px" }}>
                        <b style={{ textDecoration: "underline" }}>Bank Details :</b><br />
                        * BANK NAME : {company.bank_name}<br />
                        * A/C NO : {company.ac_no}<br />
                        * IFSC NO : {company.ifsc}<br /><br />
                        <b>(Common Seal)</b>
                    </div>
                </div>
                <div style={{ padding: "0" }}>
                    <table style={{ width: "100%", fontSize: "11px", borderCollapse: "collapse" }}>
                        <tbody>
                            <tr><td style={{ borderBottom: '1px solid #000', padding: '4px' }}>Total Before Tax</td><td style={{ borderBotto
                            <tr><td style={{ borderBottom: '1px solid #000', padding: '4px' }}>Add: CGST</td><td style={{ borderBottom: '1px
                            <tr><td style={{ borderBottom: '1px solid #000', padding: '4px' }}>Add: SGST</td><td style={{ borderBottom: '1px
                            <tr><td style={{ borderBottom: '1px solid #000', padding: '4px' }}>Add: IGST</td><td style={{ borderBottom: '1px
                            <tr><td style={{ borderBottom: '1px solid #000', padding: '4px' }}>Tax Amount: GST</td><td style={{ borderBottom
                            <tr style={{ background: '#f3f4f6', fontSize: "12px" }}><td style={{ padding: '6px' }}><b>Total After Tax</b></t
                        </tbody>
                    </table>
                    <div style={{ padding: '6px', fontSize: '10px', borderTop: "1px solid #000" }}>GST on Reverse Charge : {meta.reverseCha
                    <div style={{ padding: '6px', fontSize: '9px', fontStyle: 'italic' }}>Certified that particulars are true & correct.</di
                    <div style={{ textAlign: "right", fontWeight: "bold", marginTop: "10px", paddingRight: "10px", fontSize: "11px" }}>
                        For, {company.name}
                        <div style={{ height: "45px", display: "flex", alignItems: "center", justifyContent: "flex-end" }}>
                            {(company.signature_url || user?.signature_url) && (
                                <img src={company.signature_url || user?.signature_url} alt="Signature" style={{ height: "35px" }} />
                            )}
                        </div>
                        <div style={{ borderTop: "1px dotted #000", display: "inline-block", minWidth: "150px", marginTop: "2px" }}>Authoris
                    </div>
                </div>
            </div>
            <div style={{ fontSize: "9px", textAlign: "right", padding: "4px", borderTop: "1px solid #000" }}>[E & OE]</div>
        </div>
    </div>
  </div>
</div>
);
};

export default EditInvoice;
```

frontend/src/pages/DayBook.tsx

```
import { motion } from 'framer-motion';
import React, { useEffect, useState } from 'react';
import { FaCalendarAlt, FaChartLine, FaHistory, FaReceipt } from 'react-icons/fa';
import { fetchTransactions, Transaction } from '../api/transactionApi';
import './DayBook.css';

const DayBook: React.FC = () => {
    const [date, setDate] = useState(new Date().toISOString().slice(0, 10));
    const [transactions, setTransactions] = useState<Transaction[]>([]);
    const [loading, setLoading] = useState(false);

    const loadData = async (selectedDate: string) => {
        setLoading(true);
        try {
            const data = await fetchTransactions();
            const filtered = data.filter(t => t.date.slice(0, 10) === selectedDate);
            setTransactions(filtered);
        } catch (err) {
            console.error("Failed to load Day Book:", err);
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        loadData(date);
    }, [date]);

    const formatCurrency = (amount: number) => {
        return new Intl.NumberFormat('en-IN', {
            style: 'currency',
            currency: 'INR',
            minimumFractionDigits: 0,
            maximumFractionDigits: 0
        }).format(amount);
    };

    const totalDebit = transactions.reduce((sum, t) => sum + (t.type === 'BILL' || t.type === 'PAYMENT' ? Number(t.amount) : 0), 0);
    const totalCredit = transactions.reduce((sum, t) => sum + (t.type === 'INVOICE' || t.type === 'RECEIPT' ? Number(t.amount) : 0), 0);

    return (
        <div className="daybook-container">
            {/* Header */}
            <header className="daybook-header">
                <div className="daybook-title">
                    <motion.h1
                        initial={{ opacity: 0, x: -30 }}
                        animate={{ opacity: 1, x: 0 }}
                    >
                        Daily <span style={{ color: 'var(--primary)' }}>Ledger</span>
                    </motion.h1>
                    <motion.p
                        initial={{ opacity: 0 }}
                        animate={{ opacity: 1 }}
                        transition={{ delay: 0.2 }}
                    >
                        Financial Audit Pulse: {new Date(date).toLocaleDateString('en-GB', { day: '2-digit', month: 'long', year: 'numeric' })}
                    </motion.p>
                </div>
                <motion.div
                    initial={{ opacity: 0, scale: 0.9 }}
                    animate={{ opacity: 1, scale: 1 }}
                    className="date-selector-glass"
                >
                    <FaCalendarAlt style={{ color: 'var(--primary)' }} />
                    <input
                        type="date"
                        value={date}
                        onChange={(e) => setDate(e.target.value)}
                    />
                </motion.div>
            </header>

            {/* Stats Matrix */}
            <div className="ledger-matrix">
                <motion.div
                    initial={{ opacity: 0, y: 20 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ delay: 0.3 }}
                    className="ledger-stat-card debit"
                >
                    <div className="stat-icon-bg"><FaChartLine /></div>
                    <span className="stat-label">Neural Outflow (Debit)</span>
                    <h3 className="stat-value">{formatCurrency(totalDebit)}</h3>
                    <div style={{ marginTop: '24px', height: '4px', background: 'var(--error-glow)', borderRadius: '2px', overflow: 'hidden' }}>
                        <motion.div
                            initial={{ width: 0 }}
                            animate={{ width: '100%' }}
                            transition={{ duration: 1.5, ease: "easeOut" }}
                            style={{ height: '100%', background: 'var(--error)' }}
                        />
                    </div>
                </motion.div>

                <motion.div
                    initial={{ opacity: 0, y: 20 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ delay: 0.4 }}
                    className="ledger-stat-card credit"
                >
                    <div className="stat-icon-bg"><FaReceipt /></div>
                    <span className="stat-label">Neural Inflow (Credit)</span>
                    <h3 className="stat-value">{formatCurrency(totalCredit)}</h3>
                    <div style={{ marginTop: '24px', height: '4px', background: 'var(--success-glow)', borderRadius: '2px', overflow: 'hidden' }}>
                        <motion.div
                            initial={{ width: 0 }}
```

```
                        animate={{ width: '100%' }}
                        transition={{ duration: 1.5, ease: "easeOut" }}
                        style={{ height: '100%', background: 'var(--success)' }}
                    />
                </div>
            </motion.div>
        </div>

        {/* Table Area */}
        <motion.div
            initial={{ opacity: 0, y: 30 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ delay: 0.5 }}
            className="daybook-table-wrapper"
        >
            <table className="day-table">
                <thead>
                    <tr>
                        <th>Narrative Context</th>
                        <th>Protocol Type</th>
                        <th style={{ textAlign: 'right' }}>Debit (Dr)</th>
                        <th style={{ textAlign: 'right' }}>Credit (Cr)</th>
                    </tr>
                </thead>
                <tbody>
                    {loading ? (
                        <tr><td colSpan={4} style={{ padding: '80px', textAlign: 'center' }}><div className="skeleton" style={{ height: '40px', bord
                    ) : transactions.length === 0 ? (
                        <tr>
                            <td colSpan={4} style={{ padding: '120px 0', textAlign: 'center' }}>
                                <FaHistory size={64} style={{ color: 'var(--border-color)', marginBottom: '24px' }} />
                                <h3 style={{ margin: 0, fontWeight: 950, color: 'var(--text-primary)' }}>Book Idle</h3>
                                <p style={{ color: 'var(--text-muted)', fontWeight: 500, marginTop: '8px' }}>No financial impulses detected for this
                            </td>
                        </tr>
                    ) : (
                        transactions.map((t, idx) => {
                            const isCredit = t.type === 'INVOICE' || t.type === 'RECEIPT';
                            return (
                                <motion.tr
                                    key={t.id}
                                    initial={{ opacity: 0, x: -10 }}
                                    animate={{ opacity: 1, x: 0 }}
                                    transition={{ delay: 0.6 + (idx * 0.05) }}
                                    className="tx-row"
                                >
                                    <td>
                                        <div className="tx-narration">{t.description}</div>
                                        <div className="tx-entity">{t.user_name || t.lender_name || 'System Auto-Node'}</div>
                                    </td>
                                    <td>
                                        <span className={`type-pill ${isCredit ? 'credit' : 'debit'}`}>
                                            {t.type}
                                        </span>
                                    </td>
                                    <td style={{ textAlign: 'right' }}>
                                        {!isCredit ? <div className="val-debit">{formatCurrency(t.amount)}</div> : <div className="val-empty">•</div
                                    </td>
                                    <td style={{ textAlign: 'right' }}>
                                        {isCredit ? <div className="val-credit">{formatCurrency(t.amount)}</div> : <div className="val-empty">•</div
                                    </td>
                                </motion.tr>
                            );
                        })
                    )}
                </tbody>
            </table>
        </motion.div>
    </div>
  );
};

export default DayBook;
```

frontend/src/pages/Dashboard.tsx

```
// frontend/src/pages/Dashboard.tsx
import { motion } from 'framer-motion';
import React, { useEffect, useMemo, useState } from 'react';
import {
    FaArrowDown,
    FaArrowUp,
    FaBox,
    FaBrain,
    FaChartLine,
    FaFileInvoice,
    FaSync,
    FaUsers,
    FaWallet
} from 'react-icons/fa';
import { useTenant } from '../context/TenantContext';
import { useAuthUser } from '../hooks/useAuthUser';
import { useInvoices } from '../hooks/useInvoices';
import { useProducts } from '../hooks/useProducts';
import { useUsers } from '../hooks/useUsers';

// --- 📈 ANIMATED KPI COMPONENT ---
const KpiCard = ({ title, value, icon, color, trend, trendType, delay }: any) => {
    return (
        <motion.div
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ delay, duration: 0.5, ease: [0.16, 1, 0.3, 1] }}
            className="enterprise-card"
        >
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-start', marginBottom: '16px' }}>
                <div style={{
                    width: '48px',
                    height: '48px',
                    borderRadius: '12px',
                    background: `${color}10`,
                    color: color,
                    display: 'flex',
                    alignItems: 'center',
                    justifyContent: 'center',
                    fontSize: '1.2rem'
                }}>
                    {icon}
                </div>
                {trend && (
                    <div style={{
                        fontSize: '0.75rem',
                        fontWeight: 700,
                        color: trendType === 'up' ? 'var(--success)' : 'var(--error)',
                        display: 'flex',
                        alignItems: 'center',
                        gap: '4px',
                        background: trendType === 'up' ? 'rgba(16, 185, 129, 0.1)' : 'rgba(239, 68, 68, 0.1)',
                        padding: '4px 8px',
                        borderRadius: '20px'
                    }}>
                        {trendType === 'up' ? <FaArrowUp size={8} /> : <FaArrowDown size={8} />}
                        {trend}
                    </div>
                )}
            </div>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.75rem', fontWeight: 700, textTransform: 'uppercase', letterSpacing: '0.05em', marginBo
                {title}
            </p>
            <h3 style={{ fontSize: '1.75rem', fontWeight: 800, color: 'var(--text-primary)' }}>
                {value}
            </h3>
        </motion.div>
    );
};

const Dashboard: React.FC = () => {
    const { user } = useAuthUser();
    const { activeBranch } = useTenant();
    const { customers = [], refresh: refreshUsers } = useUsers();
    const { invoices = [], refresh: refreshInvoices } = useInvoices();
    const { products = [], refresh: refreshProducts } = useProducts();

    const [isRefreshing, setIsRefreshing] = useState(false);

    const handleRefresh = async () => {
        setIsRefreshing(true);
        await Promise.all([refreshUsers(), refreshInvoices(), refreshProducts()]);
        setTimeout(() => setIsRefreshing(false), 500);
    };

    useEffect(() => { handleRefresh(); }, [activeBranch]);

    const stats = useMemo(() => {
        const totalRevenue = Array.isArray(invoices) ? invoices.reduce((sum, inv) => sum + Number(inv.total_amount || 0), 0) : 0;
        return {
            revenue: `₹${totalRevenue.toLocaleString('en-IN')}`,
            customers: customers.length || 0,
            invoices: invoices.length || 0,
            health: "99.9%"
        };
    }, [invoices, customers]);

    return (
        <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            className="dashboard-container"
            style={{ display: 'flex', flexDirection: 'column', gap: '32px' }}
        >
            {/* Header Section */}
            <header style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-end', gap: '24px', flexWrap: 'wrap' }}>
```

```
    <div>
        <motion.h1
            initial={{ opacity: 0, x: -20 }}
            animate={{ opacity: 1, x: 0 }}
            style={{ fontSize: '3rem', fontWeight: 900, letterSpacing: '-0.04em', margin: 0 }}
        >
            Welcome Back, <span style={{ color: 'var(--primary)' }}>{user?.username}</span>
        </motion.h1>
        <motion.p
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            transition={{ delay: 0.2 }}
            style={{ color: 'var(--text-secondary)', fontWeight: 600, fontSize: '1.1rem', marginTop: '4px' }}
        >
            Intelligence Hub | {activeBranch?.branch_name || 'Global Context'}
        </motion.p>
    </div>
    <div style={{ display: 'flex', gap: '12px' }}>
        <motion.button
            whileHover={{ scale: 1.05 }}
            whileTap={{ scale: 0.95 }}
            onClick={handleRefresh}
            className="btn-secondary"
            style={{ height: '52px', padding: '0 24px', borderRadius: '14px' }}
        >
            <FaSync className={isRefreshing ? 'fa-spin' : ''} /> Sync Neural Link
        </motion.button>
        <motion.button
            whileHover={{ scale: 1.05 }}
            whileTap={{ scale: 0.95 }}
            className="btn-primary"
            style={{ height: '52px', padding: '0 28px', borderRadius: '14px' }}
        >
            + Initialize Transaction
        </motion.button>
    </div>
</header>

{/* AI Insights Panel - Innovative Design */}
<motion.div
    initial={{ opacity: 0, scale: 0.98 }}
    animate={{ opacity: 1, scale: 1 }}
    transition={{ delay: 0.3 }}
    className="enterprise-card"
    style={{
        background: 'linear-gradient(135deg, #6366f1 0%, #4f46e5 100%)',
        color: 'white',
        display: 'flex',
        alignItems: 'center',
        gap: '24px',
        position: 'relative',
        overflow: 'hidden',
        border: 'none',
        padding: '32px'
    }}
>
    <div style={{
        padding: '24px',
        background: 'rgba(255,255,255,0.1)',
        borderRadius: '20px',
        boxShadow: '0 8px 32px rgba(0,0,0,0.1)'
    }}>
        <FaBrain size={40} />
    </div>
    <div style={{ flex: 1 }}>
        <h4 style={{ fontSize: '1.4rem', fontWeight: 900, marginBottom: '8px', letterSpacing: '-0.5px' }}>Neural Forecasting Insight</h4>
        <p style={{ opacity: 0.9, fontSize: '1rem', maxWidth: '700px', lineHeight: '1.6', fontWeight: 500 }}>
            Our predictive engine detects a potential <span style={{ fontWeight: 900, color: '#fbbf24' }}>12% demand surge</span> for Indus
            Supply chain optimization is ready for immediate deployment.
        </p>
    </div>
    <motion.button
        whileHover={{ scale: 1.05, background: '#fff' }}
        whileTap={{ scale: 0.95 }}
        style={{
            padding: '12px 28px',
            background: 'rgba(255,255,255,0.9)',
            color: 'var(--primary)',
            border: 'none',
            borderRadius: '12px',
            fontWeight: 900,
            fontSize: '0.8rem',
            textTransform: 'uppercase',
            letterSpacing: '0.1em',
            cursor: 'pointer'
        }}
    >
        Authorize Optimization
    </motion.button>

    {/* Decorative Elements */}
    <div style={{ position: 'absolute', right: '-20px', bottom: '-20px', opacity: 0.15 }}>
        <FaChartLine size={180} />
    </div>
</motion.div>

{/* KPI Grid */}
<div className="kpi-grid" style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(280px, 1fr))', gap: '24px' }}>
    <KpiCard title="Fiscal Liquidity" value={stats.revenue} icon={<FaWallet />} color="#6366f1" trend="+14.2%" trendType="up" delay={0.4} /
    <KpiCard title="Strategic Partners" value={stats.customers} icon={<FaUsers />} color="#10b981" trend="+3 New" trendType="up" delay={0.5
    <KpiCard title="Neural Sync Health" value={stats.health} icon={<FaSync />} color="#f59e0b" trend="OPTIMAL" trendType="up" delay={0.6} /
    <KpiCard title="Execution Volume" value={stats.invoices} icon={<FaFileInvoice />} color="#8b5cf6" trend="+8.2%" trendType="up" delay={0
</div>

{/* Main Content Grid */}
<div style={{ display: 'grid', gridTemplateColumns: '1.8fr 1fr', gap: '32px' }}>
    <motion.div
        initial={{ opacity: 0, y: 20 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ delay: 0.8 }}
        className="enterprise-card"
        style={{ minHeight: '400px', padding: '32px' }}
```

```
                >
                    <h3 style={{ fontSize: '1.5rem', fontWeight: 900, marginBottom: '24px', letterSpacing: '-0.5px' }}>Revenue Trajectory</h3>
                    <div style={{ height: '300px', width: '100%', background: '#f8fafc', borderRadius: '24px', border: '1px dashed var(--border-color)'
                        <FaChartLine size={64} style={{ opacity: 0.1, marginBottom: '16px' }} />
                        <span style={{ fontWeight: 800, fontSize: '1.1rem' }}>Analytics Processing...</span>
                        <p style={{ margin: '8px 0 0', fontSize: '0.9rem' }}>Real-time telemetry stream active</p>
                    </div>
                </motion.div>

                <motion.div
                    initial={{ opacity: 0, y: 20 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ delay: 0.9 }}
                    className="enterprise-card"
                    style={{ minHeight: '400px', padding: '32px' }}
                >
                    <h3 style={{ fontSize: '1.5rem', fontWeight: 900, marginBottom: '24px', letterSpacing: '-0.5px' }}>Critical Inventory</h3>
                    <div style={{ display: 'flex', flexDirection: 'column', gap: '16px' }}>
                        {products.length > 0 ? products.slice(0, 5).map((p, i) => (
                            <motion.div
                                key={i}
                                whileHover={{ x: 8 }}
                                style={{ display: 'flex', alignItems: 'center', gap: '16px', padding: '16px', background: '#f8fafc', borderRadius: '16p
                            >
                                <div style={{ width: '48px', height: '48px', background: 'white', borderRadius: '12px', display: 'flex', alignItems: 'c
                                    <FaBox size={20} color="var(--primary)" />
                                </div>
                                <div style={{ flex: 1 }}>
                                    <p style={{ fontWeight: 800, fontSize: '1rem', color: 'var(--text-primary)', margin: 0 }}>{p.name}</p>
                                    <p style={{ fontSize: '0.8rem', color: 'var(--text-muted)', fontWeight: 600, margin: '2px 0 0' }}>HASH: {p.sku || '
                                </div>
                                <div style={{ textAlign: 'right' }}>
                                    <span style={{ fontWeight: 900, fontSize: '1rem', color: 'var(--primary)' }}>₹{p.selling_price}</span>
                                </div>
                            </motion.div>
                        )) : (
                            <div style={{ textAlign: 'center', padding: '40px', color: 'var(--text-muted)' }}>
                                <FaBox size={48} style={{ opacity: 0.1, marginBottom: '16px' }} />
                                <p style={{ fontWeight: 700 }}>No Inventory Detected</p>
                            </div>
                        )}
                    </div>
                </motion.div>
            </div>
        </motion.div>
    );
};

export default Dashboard;
```

frontend/src/pages/Customers.tsx

```
import { AnimatePresence, motion } from 'framer-motion';
import React, { useState } from 'react';
import { FaEdit, FaExclamationTriangle, FaHistory, FaMapMarkerAlt, FaPlus, FaSearch, FaSync, FaTag, FaTrash, FaUserCircle } from 'react-icons/fa';
import { deleteCustomer } from '../api/userApi';
import TransactionHistoryModal from '../components/TransactionHistoryModal';
import { useUsers } from '../hooks/useUsers';
import AddCustomerModal from './AddCustomerModal';
import './Customers.css';

const Customers: React.FC = () => {
    const { customers = [], loading, error, refresh } = useUsers();

    const [searchTerm, setSearchTerm] = useState('');
    const [showModal, setShowModal] = useState(false);
    const [customerToEdit, setCustomerToEdit] = useState<any>(null);
    const [showTransactionModal, setShowTransactionModal] = useState(false);
    const [selectedCustomer, setSelectedCustomer] = useState<any>(null);

    const displayedCustomers = React.useMemo(() => {
        if (!Array.isArray(customers)) return [];

        return customers.filter(user => {
            if (!user) return false;
            const username = user.username || '';
            const gstin = user.gstin || '';
            const nickname = user.nickname || '';

            const isNotAdmin = username.toLowerCase() !== 'admin';
            const searchLower = searchTerm.toLowerCase();

            const matchesSearch = username.toLowerCase().includes(searchLower) ||
                                  gstin.toLowerCase().includes(searchLower) ||
                                  nickname.toLowerCase().includes(searchLower);

            return isNotAdmin && matchesSearch;
        }).sort((a,b) => (a.username || '').localeCompare(b.username || ''));
    }, [customers, searchTerm]);

    const handleEdit = (customer: any) => {
        setCustomerToEdit(customer);
        setShowModal(true);
    };

    const handleDelete = async (id: number) => {
        if (window.confirm("Relinquish this stakeholder record? This action is irreversible.")) {
            try {
                await deleteCustomer(id);
                refresh();
            } catch (err) {
                alert("Authorization failed: Record locked.");
            }
        }
    };

    const handleModalClose = () => {
        setShowModal(false);
        setCustomerToEdit(null);
    };

    return (
        <div className="customers-container">
            {/* Header */}
            <div className="customers-header">
                <div className="customers-title">
                    <motion.h1
                        initial={{ opacity: 0, x: -20 }}
                        animate={{ opacity: 1, x: 0 }}
                    >Stakeholder Registry</motion.h1>
                    <motion.p
                        initial={{ opacity: 0 }}
                        animate={{ opacity: 1 }}
                        transition={{ delay: 0.1 }}
                    >Global Partner Network & Fiscal Exposure Management</motion.p>
                </div>
                <div className="customers-actions">
                    <motion.button
                        whileHover={{ scale: 1.05 }}
                        whileTap={{ scale: 0.95 }}
                        className="btn-secondary"
                        onClick={() => refresh()}
                        style={{ width: '52px', height: '52px', borderRadius: '14px', display: 'flex', alignItems: 'center', justifyContent: 'center' }}
                    >
                        <FaSync className={loading ? 'fa-spin' : ''} />
                    </motion.button>
                    <motion.button
                        whileHover={{ scale: 1.05 }}
                        whileTap={{ scale: 0.95 }}
                        className="btn-primary"
                        onClick={() => { setCustomerToEdit(null); setShowModal(true); }}
                        style={{ height: '52px', padding: '0 28px', gap: '12px', borderRadius: '14px', fontSize: '0.95rem', fontWeight: 800 }}
                    >
                        <FaPlus /> Initialize Onboarding
                    </motion.button>
                </div>
            </div>

            {/* Critical Alerts */}
            <AnimatePresence>
                {error && (
                    <motion.div
                        initial={{ opacity: 0, y: -20 }}
                        animate={{ opacity: 1, y: 0 }}
                        exit={{ opacity: 0, y: -20 }}
                        style={{ backgroundColor: 'rgba(239, 68, 68, 0.08)', color: 'var(--error)', padding: '20px', borderRadius: '16px', marginBottom
                    >
                        <FaExclamationTriangle size={20} />
```

```
                <span style={{ fontWeight: 800 }}>Vault Sync Failure: {error}</span>
            </motion.div>
        )}
    </AnimatePresence>

    {/* Search Orb */}
    <div className="search-orb">
        <FaSearch style={{ color: 'var(--text-muted)' }} size={20} />
        <input
            placeholder="Identify partner by username, GSTIN, or alias..."
            value={searchTerm}
            onChange={(e) => setSearchTerm(e.target.value)}
        />
    </div>

    {/* Cinematic Table */}
    <div className="customer-table-container">
        <table className="cust-table">
            <thead>
                <tr>
                    <th>Partner Identity</th>
                    <th>Fiscal Registry (GSTIN)</th>
                    <th>Geographic Context</th>
                    <th style={{ textAlign: 'right' }}>Capital Exposure</th>
                    <th style={{ textAlign: 'center' }}>Control Interface</th>
                </tr>
            </thead>
            <tbody>
                {loading ? (
                    [...Array(6)].map((_, i) => (
                        <tr key={i}><td colSpan={5} style={{ padding: '30px' }}><div className="skeleton" style={{ height: '30px', borderRadius
                    ))
                ) : displayedCustomers.length === 0 ? (
                    <tr>
                        <td colSpan={5} style={{ padding: '120px 32px', textAlign: 'center' }}>
                            <FaUserCircle size={64} style={{ color: 'var(--border-color)', marginBottom: '20px' }} />
                            <h3 style={{ margin: 0, fontWeight: 900, color: 'var(--text-primary)' }}>Registry Empty</h3>
                            <p style={{ color: 'var(--text-muted)', fontWeight: 500 }}>No stakeholder detected within current parameters.</p>
                        </td>
                    </tr>
                ) : (
                    displayedCustomers.map((user, idx) => (
                        <motion.tr
                            key={user.id}
                            initial={{ opacity: 0, x: -20 }}
                            animate={{ opacity: 1, x: 0 }}
                            transition={{ delay: idx * 0.05 }}
                            className="cust-row"
                        >
                            <td>
                                <div style={{ display: 'flex', alignItems: 'center', gap: '15px' }}>
                                    <div className="identity-orb">
                                        {(user.username || 'U').charAt(0).toUpperCase()}
                                    </div>
                                    <div>
                                        <div style={{ fontWeight: 800, color: 'var(--text-primary)', fontSize: '1.05rem' }}>{user.username}</div
                                        {user.nickname && (
                                            <div style={{ fontSize: '0.75rem', color: 'var(--primary)', fontWeight: 800, display: 'flex', alignI
                                                <FaTag size={10} /> {user.nickname}
                                            </div>
                                        )}
                                    </div>
                                </div>
                            </td>
                            <td>
                                <div style={{ color: 'var(--text-primary)', fontWeight: 800, letterSpacing: '0.5px' }}>{user.gstin || 'NON-CERTI
                            </td>
                            <td>
                                <div style={{ display: 'flex', alignItems: 'center', gap: '8px', color: 'var(--text-muted)', fontWeight: 600 }}>
                                    <FaMapMarkerAlt size={12} style={{ opacity: 0.5 }} />
                                    {user.city_pincode || user.state || 'Universal'}
                                </div>
                            </td>
                            <td style={{ textAlign: 'right' }}>
                                <div className="exposure-value">
                                    ₹{Number(user.initial_balance || 0).toLocaleString('en-IN')}
                                </div>
                            </td>
                            <td style={{ textAlign: 'center' }}>
                                <div style={{ display: 'flex', justifyContent: 'center', gap: '12px' }}>
                                    <button className="control-btn btn-history" onClick={() => { setSelectedCustomer(user); setShowTransactionMo
                                        <FaHistory size={16} />
                                    </button>
                                    <button className="control-btn btn-edit" onClick={() => handleEdit(user)} title="Modify Records">
                                        <FaEdit size={16} />
                                    </button>
                                    <button className="control-btn btn-delete" onClick={() => handleDelete(user.id)} title="Relinquish Status">
                                        <FaTrash size={16} />
                                    </button>
                                </div>
                            </td>
                        </motion.tr>
                    ))
                )}
            </tbody>
        </table>
    </div>

    {showModal && (
        <AddCustomerModal
            onClose={handleModalClose}
            onSuccess={refresh}
            customerToEdit={customerToEdit}
        />
    )}

    {showTransactionModal && selectedCustomer && (
        <TransactionHistoryModal
            isOpen={showTransactionModal}
            onClose={() => {
                setShowTransactionModal(false);
                setSelectedCustomer(null);
```

```
                    }}
                    entityType="customer"
                    entityId={selectedCustomer.id}
                    entityName={selectedCustomer.username}
                />
            )}
        </div>
    );
};

export default Customers;
```

frontend/src/pages/CreateTransaction.tsx

```
// frontend/src/pages/CreateTransaction.tsx
import React, { useEffect, useState } from 'react';
import { FaCheckCircle, FaPlus, FaTrash } from 'react-icons/fa';
import { useNavigate } from 'react-router-dom';

interface Account {
    id: number;
    account_code: string;
    name: string;
}

interface TransactionLine {
    account_id: string;
    debit_amount: number;
    credit_amount: number;
    description: string;
}

const CreateTransaction: React.FC = () => {
    const navigate = useNavigate();
    const [accounts, setAccounts] = useState<Account[]>([]);
    const [description, setDescription] = useState("");
    const [date, setDate] = useState(new Date().toISOString().split('T')[0]);
    const [lines, setLines] = useState<TransactionLine[]>([
        { account_id: '', debit_amount: 0, credit_amount: 0, description: '' },
        { account_id: '', debit_amount: 0, credit_amount: 0, description: '' }
    ]);

    useEffect(() => {
        const fetchAccounts = async () => {
            const response = await fetch(`${import.meta.env.VITE_API_URL}/api/accounting/accounts`, {
                headers: { 'Authorization': `Bearer ${localStorage.getItem('erp-token')}` }
            });
            const data = await response.json();
            if (response.ok) setAccounts(data);
        };
        fetchAccounts();
    }, []);

    const addLine = () => {
        setLines([...lines, { account_id: '', debit_amount: 0, credit_amount: 0, description: '' }]);
    };

    const removeLine = (index: number) => {
        setLines(lines.filter((_, i) => i !== index));
    };

    const updateLine = (index: number, field: keyof TransactionLine, value: any) => {
        const newLines = [...lines];
        newLines[index] = { ...newLines[index], [field]: value };
        setLines(newLines);
    };

    const totalDebits = lines.reduce((sum, line) => sum + (line.debit_amount || 0), 0);
    const totalCredits = lines.reduce((sum, line) => sum + (line.credit_amount || 0), 0);
    const isBalanced = totalDebits === totalCredits && totalDebits > 0;

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        if (!isBalanced) return alert("Double entry must be balanced (Debits = Credits)");

        try {
            const response = await fetch(`${import.meta.env.VITE_API_URL}/api/accounting/transactions`, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'Authorization': `Bearer ${localStorage.getItem('erp-token')}`
                },
                body: JSON.stringify({
                    transaction_date: date,
                    description,
                    lines: lines.filter(l => l.account_id && (l.debit_amount > 0 || l.credit_amount > 0))
                })
            });

            if (response.ok) {
                navigate('/transactions');
            } else {
                const err = await response.json();
                alert(err.error || "Failed to save transaction");
            }
        } catch (error) {
            console.error("Error creating transaction:", error);
        }
    };

    return (
        <div className="max-w-4xl mx-auto space-y-8">
            <div className="flex items-center justify-between">
                <div>
                    <h1 className="text-3xl font-black text-slate-900 tracking-tight">New Journal Entry</h1>
                    <p className="text-slate-500 mt-1">Record a manual double-entry financial transaction.</p>
                </div>
                <div className={`flex items-center gap-2 px-4 py-2 rounded-full font-black text-xs uppercase tracking-widest ${isBalanced ? 'bg-emerald-
                    {isBalanced ? <><FaCheckCircle /> Balanced</> : 'Unbalanced'}
                </div>
            </div>

            <form onSubmit={handleSubmit} className="space-y-6">
                <div class="glass-panel p-8 rounded-[32px] border border-slate-200 grid grid-cols-1 md:grid-cols-2 gap-6">
                    <div>
                        <label class="block text-[10px] font-black uppercase text-slate-400 mb-2">Transaction Date</label>
                        <input
                            type="date"
                            required
                            class="w-full p-4 bg-slate-50 rounded-2xl outline-none font-bold"
```

```
                    value={date}
                    onChange={(e) => setDate(e.target.value)}
                />
            </div>
            <div>
                <label className="block text-[10px] font-black uppercase text-slate-400 mb-2">General Description</label>
                <input
                    type="text"
                    required
                    className="w-full p-4 bg-slate-50 rounded-2xl outline-none font-bold"
                    placeholder="e.g. Monthly Rent Payment"
                    value={description}
                    onChange={(e) => setDescription(e.target.value)}
                />
            </div>
        </div>

        <div className="glass-panel rounded-[32px] overflow-hidden border border-slate-200">
            <table className="w-full">
                <thead className="bg-slate-50">
                    <tr>
                        <th className="px-6 py-4 text-left text-[10px] font-black text-slate-400 uppercase tracking-widest">Account</th>
                        <th className="px-6 py-4 text-left text-[10px] font-black text-slate-400 uppercase tracking-widest">Description</th>
                        <th className="px-6 py-4 text-right text-[10px] font-black text-slate-400 uppercase tracking-widest w-32">Debit (₹)</th>
                        <th className="px-6 py-4 text-right text-[10px] font-black text-slate-400 uppercase tracking-widest w-32">Credit (₹)</th
                        <th className="px-6 py-4 w-16"></th>
                    </tr>
                </thead>
                <tbody className="divide-y divide-slate-100">
                    {lines.map((line, idx) => (
                        <tr key={idx}>
                            <td className="p-4">
                                <select
                                    required
                                    className="w-full p-3 bg-slate-50 rounded-xl outline-none text-sm font-bold"
                                    value={line.account_id}
                                    onChange={(e) => updateLine(idx, 'account_id', e.target.value)}
                                >
                                    <option value="">Select Account</option>
                                    {accounts.map(a => (
                                        <option key={a.id} value={a.id}>{a.account_code} - {a.name}</option>
                                    ))}
                                </select>
                            </td>
                            <td className="p-4">
                                <input
                                    type="text"
                                    className="w-full p-3 bg-slate-50 rounded-xl outline-none text-sm font-bold"
                                    placeholder="Line details..."
                                    value={line.description}
                                    onChange={(e) => updateLine(idx, 'description', e.target.value)}
                                />
                            </td>
                            <td className="p-4">
                                <input
                                    type="number"
                                    className="w-full p-3 bg-slate-50 rounded-xl outline-none text-right text-sm font-bold"
                                    value={line.debit_amount}
                                    onChange={(e) => updateLine(idx, 'debit_amount', parseFloat(e.target.value) || 0)}
                                />
                            </td>
                            <td className="p-4">
                                <input
                                    type="number"
                                    className="w-full p-3 bg-slate-50 rounded-xl outline-none text-right text-sm font-bold"
                                    value={line.credit_amount}
                                    onChange={(e) => updateLine(idx, 'credit_amount', parseFloat(e.target.value) || 0)}
                                />
                            </td>
                            <td className="p-4 text-center">
                                {lines.length > 2 && (
                                    <button type="button" onClick={() => removeLine(idx)} className="text-red-400 hover:text-red-600 transition-
                                        <FaTrash size={14} />
                                    </button>
                                )}
                            </td>
                        </tr>
                    ))}
                </tbody>
                <tfoot className="bg-slate-50 font-black">
                    <tr>
                        <td colSpan={2} className="px-6 py-4 text-right">TOTAL</td>
                        <td className="px-6 py-4 text-right text-blue-600">₹{totalDebits.toLocaleString('en-IN', { minimumFractionDigits: 2 })}<
                        <td className="px-6 py-4 text-right text-purple-600">₹{totalCredits.toLocaleString('en-IN', { minimumFractionDigits: 2 }
                        <td></td>
                    </tr>
                </tfoot>
            </table>
        </div>

        <div className="flex items-center justify-between gap-6">
            <button
                type="button"
                onClick={addLine}
                className="flex items-center gap-2 text-slate-500 font-bold hover:text-slate-900 transition-colors"
            >
                <FaPlus /> Add Line Item
            </button>
            <div className="flex gap-4">
                <button type="button" onClick={() => navigate('/transactions')} className="px-8 py-4 font-bold text-slate-400 hover:text-slate-
                <button
                    type="submit"
                    disabled={!isBalanced}
                    className={`px-10 py-4 rounded-2xl font-black shadow-xl transition-all ${isBalanced ? 'bg-slate-900 text-white hover:scale-
                >
                    Post Journal Entry
                </button>
            </div>
        </div>
    </form>
</div>
);
```

```
};

export default CreateTransaction;
```

frontend/src/pages/CreateInvoice.tsx

```
// frontend/src/pages/CreateInvoice.tsx
import { AnimatePresence, motion } from 'framer-motion';
import React, { useEffect, useRef, useState } from "react";
import {
    FaArrowLeft,
    FaMoneyBillWave,
    FaPlus,
    FaReceipt,
    FaSave, FaTrash
} from "react-icons/fa";
import { useNavigate } from "react-router-dom";
import { fetchProfile } from "../api/companyApi";
import { fetchProducts, Product } from "../api/productApi";
import { Customer } from "../api/userApi";
import { useAuthUser } from "../hooks/useAuthUser";
import { useUsers } from "../hooks/useUsers";
import { apiFetch } from "../utils/api";
import "./CreateInvoice.css";

// Utility for Words conversion
const toWords = (num: number): string => {
    const a = ['', 'One ', 'Two ', 'Three ', 'Four ', 'Five ', 'Six ', 'Seven ', 'Eight ', 'Nine ', 'Ten ', 'Eleven ', 'Twelve ', 'Thirteen ', 'Fourteen
    const b = ['', '', 'Twenty', 'Thirty', 'Forty', 'Fifty', 'Sixty', 'Seventy', 'Eighty', 'Ninety'];
    const formatted = ('000000000' + num).substr(-9);
    const n = formatted.match(/^(\d{2})(\d{2})(\d{2})(\d{1})(\d{2})$/);
    if (!n) return '';
    let str = '';
    const n1 = Number(n[1]); const n2 = Number(n[2]); const n3 = Number(n[3]); const n4 = Number(n[4]); const n5 = Number(n[5]);
    if (n1 !== 0) str += (a[n1] || b[Math.floor(n1 / 10)] + ' ' + a[n1 % 10]) + 'Crore ';
    if (n2 !== 0) str += (a[n2] || b[Math.floor(n2 / 10)] + ' ' + a[n2 % 10]) + 'Lakh ';
    if (n3 !== 0) str += (a[n3] || b[Math.floor(n3 / 10)] + ' ' + a[n3 % 10]) + 'Thousand ';
    if (n4 !== 0) str += (a[n4] || b[Math.floor(n4 / 10)] + ' ' + a[n4 % 10]) + 'Hundred ';
    if (n5 !== 0) str += (str !== '' ? 'and ' : '') + (a[n5] || b[Math.floor(n5 / 10)] + ' ' + a[n5 % 10]) + 'only ';
    return str;
};

const fmt = (n: any) => Number(n || 0).toLocaleString('en-IN', { minimumFractionDigits: 2, maximumFractionDigits: 2 });

interface InvoiceItem {
    name: string;
    hsn: string;
    uom: string;
    qty: number;
    rate: number;
}

const CreateInvoice: React.FC = () => {
    const navigate = useNavigate();
    const { customers } = useUsers();
    const { user: authUser } = useAuthUser();
    const printRef = useRef<HTMLDivElement>(null);

    const [products, setProducts] = useState<Product[]>([]);
    const [company, setCompany] = useState<any>({});
    const [invoiceNo, setInvoiceNo] = useState("");
    const [items, setItems] = useState<InvoiceItem[]>([{ name: "", hsn: "", uom: "Pcs", qty: 0, rate: 0 }]);
    const [notes, setNotes] = useState("");
    const [meta, setMeta] = useState({
        invoiceDate: new Date().toISOString().slice(0, 10),
        vehicleNumber: "",
        bundles: 0,
        transportMode: "",
        reverseCharge: "No",
    });
    const [customerId, setCustomerId] = useState<number | null>(null);
    const [customer, setCustomer] = useState({ name: "", address: "", gstin: "", state: "", stateCode: "" });

    useEffect(() => {
        const load = async () => {
            const [p, prodData] = await Promise.all([fetchProfile(), fetchProducts()]);
            setCompany({
                name: p.company_name, address: p.address_line1, city_pincode: p.city_pincode,
                state: p.state, stateCode: "33", gstin: p.gstin
            });
            setProducts(prodData);
        };
        load();
    }, []);

    const handleProductSelect = (index: number, productId: string) => {
        const prod = products.find(p => p.id === parseInt(productId));
        if (prod) {
            const newItems = [...items];
            newItems[index] = {
                ...newItems[index],
                name: prod.name,
                hsn: prod.hsn_code || "",
                uom: prod.unit || "Pcs",
                rate: prod.selling_price || 0
            };
            setItems(newItems);
        }
    };

    const handleCustomerSelect = (e: React.ChangeEvent<HTMLSelectElement>) => {
        const id = parseInt(e.target.value);
        setCustomerId(id);
        const c = customers.find((x: Customer) => x.id === id);
        if (c) {
            setCustomer({
                name: c.username.toUpperCase(),
                address: `${c.address_line1 || ''}, ${c.city_pincode || ''}`.toUpperCase(),
                gstin: c.gstin || "", state: c.state?.toUpperCase() || "", stateCode: c.state_code || ""
            });
        }
    };
```

```
// Calculation Logic
let totalTaxable = 0;
const rows = items.map(it => {
    const amount = it.qty * it.rate;
    totalTaxable += amount;
    return { ...it, amount };
});

const taxAmount = totalTaxable * 0.18; // 18% GST Flattened Example
const grandTotal = totalTaxable + taxAmount;

const saveInvoice = async () => {
    if (!customerId || !invoiceNo) return alert("Missing Strategic parameters (Customer/Invoice No)");
    try {
        const body = {
            invoice_number: invoiceNo,
            customer_id: customerId,
            items: items.filter(i => i.name),
            notes: notes,
            grand_total: grandTotal
        };
        const res = await apiFetch("/invoice", { method: "POST", body: JSON.stringify(body) });
        if (res.ok) {
            alert("Fiscal Document Crystallized.");
            navigate("/invoices");
        }
    } catch (err) {
        console.error(err);
    }
};

return (
    <div className="invoice-creation-canvas">
        {/* LEFT SIDE: FORM */}
        <div className="invoice-form-scroll">
            <header style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', marginBottom: '20px' }}>
                <div style={{ display: 'flex', alignItems: 'center', gap: '20px' }}>
                    <motion.button
                        whileHover={{ scale: 1.1 }}
                        whileTap={{ scale: 0.9 }}
                        className="btn-secondary"
                        onClick={() => navigate(-1)}
                        style={{ width: '48px', height: '48px', padding: 0, borderRadius: '14px' }}
                    >
                        <FaArrowLeft />
                    </motion.button>
                    <h2 style={{ fontSize: '2.25rem', fontWeight: 900, letterSpacing: '-1.5px', margin: 0 }}>
                        Generate <span style={{ color: 'var(--primary)' }}>Manifest</span>
                    </h2>
                </div>
            </header>

            {/* Section: Context */}
            <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} className="form-section-card">
                <div className="form-section-label" style={{ marginBottom: '24px' }}>
                    <FaReceipt color="var(--primary)" /> <span>Fiscal Metadata</span>
                </div>
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '20px' }}>
                    <div className="input-block">
                        <label>Document Index</label>
                        <input className="input-modern" placeholder="X-INV-999" value={invoiceNo} onChange={e => setInvoiceNo(e.target.value)} />
                    </div>
                    <div className="input-block">
                        <label>Execution Date</label>
                        <input className="input-modern" type="date" value={meta.invoiceDate} onChange={e => setMeta({...meta, invoiceDate: e.target
                    </div>
                </div>
                <div className="input-block" style={{ marginTop: '20px' }}>
                    <label>Target Stakeholder</label>
                    <select className="input-modern" onChange={handleCustomerSelect} value={customerId || ""}>
                        <option value="">Interface with Stakeholder Registry...</option>
                        {customers.map((c: any) => <option key={c.id} value={c.id}>{c.username}</option>)}
                    </select>
                </div>
            </motion.div>

            {/* Section: Line Items */}
            <div className="form-section-card">
                <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '24px' }}>
                    <div className="form-section-label">
                        <FaPlus color="var(--primary)" /> <span>Asset Inventory Lines</span>
                    </div>
                    <motion.button
                        whileHover={{ scale: 1.05 }}
                        whileTap={{ scale: 0.95 }}
                        className="btn-secondary"
                        style={{ padding: '10px 20px', fontSize: '0.8rem', fontWeight: 900 }}
                        onClick={() => setItems([...items, { name: "", hsn: "", uom: "Pcs", qty: 0, rate: 0 }])}
                    >
                        Allocate New Row
                    </motion.button>
                </div>

                <AnimatePresence>
                    {items.map((it, i) => (
                        <motion.div
                            key={i}
                            initial={{ opacity: 0, x: -10 }}
                            animate={{ opacity: 1, x: 0 }}
                            exit={{ opacity: 0, scale: 0.9 }}
                            className="item-row-card"
                        >
                            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '16px', marginBottom: '16px' }}>
                                <div className="input-block">
                                    <label>Neural Asset Link</label>
                                    <select className="input-modern" onChange={e => handleProductSelect(i, e.target.value)}>
                                        <option value="">Retrieve from Repository...</option>
                                        {products.map(p => <option key={p.id} value={p.id}>{p.name}</option>)}
                                    </select>
                                </div>
                                <div className="input-block">
```

```
                        <label>Custom Identifier</label>
                        <input className="input-modern" value={it.name} onChange={e => { const t = [...items]; t[i].name = e.target.valu
                    </div>
                </div>
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr 1fr', gap: '16px' }}>
                    <div className="input-block">
                        <label>Quantity</label>
                        <input className="input-modern" type="number" value={it.qty || ""} onChange={e => { const t = [...items]; t[i].
                    </div>
                    <div className="input-block">
                        <label>Unit Valuation (₹)</label>
                        <input className="input-modern" type="number" value={it.rate || ""} onChange={e => { const t = [...items]; t[i]
                    </div>
                    <div className="input-block">
                        <label>Aggregate Line Value</label>
                        <div className="input-modern" style={{ background: 'var(--primary-glow)', border: '1px solid var(--primary)', co
                            ₹{fmt(it.qty * it.rate)}
                        </div>
                    </div>
                </div>
                {items.length > 1 && (
                    <button className="remove-item-btn" onClick={() => { const t = [...items]; t.splice(i, 1); setItems(t); }}>
                        <FaTrash size={12} />
                    </button>
                )}
            </motion.div>
        ))}
    </AnimatePresence>
</div>

{/* Section: Logistics */}
<div className="form-section-card">
    <div className="form-section-label" style={{ marginBottom: '24px' }}>
        <FaMoneyBillWave color="var(--primary)" /> <span>Logistics Coordination</span>
    </div>
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '20px' }}>
        <div className="input-block">
            <label>Transit Protocol</label>
            <input className="input-modern" value={meta.transportMode} onChange={e => setMeta({...meta, transportMode: e.target.value})
        </div>
        <div className="input-block">
            <label>Vessel ID / Tracking</label>
            <input className="input-modern" value={meta.vehicleNumber} onChange={e => setMeta({...meta, vehicleNumber: e.target.value})
        </div>
    </div>
</div>

<div className="invoice-summary-bar">
    <div className="total-display">
        <span className="total-label">Total Liabilities (incl. Neural Tax)</span>
        <span className="total-value">₹{fmt(grandTotal)}</span>
    </div>
    <motion.button
        whileHover={{ scale: 1.05, boxShadow: '0 15px 30px rgba(99, 102, 241, 0.4)' }}
        whileTap={{ scale: 0.95 }}
        className="btn-primary"
        style={{ padding: '16px 40px', fontSize: '1rem', height: '64px', borderRadius: '18px' }}
        onClick={saveInvoice}
    >
        <FaSave /> Execute & Record Manifest
    </motion.button>
</div>
</div>

{/* RIGHT SIDE: PREVIEW */}
<div className="invoice-preview-container hide-mobile">
    <motion.div
        initial={{ opacity: 0, scale: 0.9, rotateX: 10 }}
        animate={{ opacity: 1, scale: 1, rotateX: 0 }}
        transition={{ duration: 0.8, ease: "easeOut" }}
        className="invoice-paper-wrapper"
    >
        <div className="invoice-paper" ref={printRef}>
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-start', borderBottom: "4px solid #1e293b", pad
                <div>
                    <h1 style={{ margin: 0, fontSize: "3rem", fontWeight: 950, letterSpacing: '-2px', color: '#1e293b' }}>{company.name || '
                    <p style={{ margin: "8px 0", fontSize: "1rem", color: "#64748b", fontWeight: 600 }}>{company.address} | GSTIN: {company
                </div>
                <div style={{ textAlign: 'right' }}>
                    <div style={{ background: '#1e293b', color: 'white', padding: '10px 20px', borderRadius: '8px', fontSize: '1.25rem', fon
                    <div style={{ marginTop: '12px', color: '#64748b', fontWeight: 700 }}>#{invoiceNo || 'DRAFT-PROTO'}</div>
                </div>
            </div>

            <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '60px', marginBottom: '60px' }}>
                <div>
                    <h4 style={{ textTransform: 'uppercase', color: '#94a3b8', fontSize: '0.85rem', fontWeight: 900, marginBottom: '12px', 
                    <div style={{ fontWeight: 900, fontSize: '1.4rem', color: '#1e293b' }}>{customer.name || '---'}</div>
                    <div style={{ color: '#475569', maxWidth: '300px', fontSize: '1rem', marginTop: '4px', lineHeight: '1.5', fontWeight: 50
                    {customer.gstin && <div style={{ marginTop: '12px', fontSize: '0.9rem', fontWeight: 800 }}>GSTIN: <span style={{ color:
                </div>
                <div style={{ textAlign: 'right' }}>
                    <h4 style={{ textTransform: 'uppercase', color: '#94a3b8', fontSize: '0.85rem', fontWeight: 900, marginBottom: '12px', 
                    <div style={{ marginBottom: '8px', fontSize: '1.1rem', fontWeight: 700 }}>Temporal Index: <span style={{ color: '#1e293b
                    <div style={{ fontSize: '1.1rem', fontWeight: 700 }}>Logistics ID: <span style={{ color: '#1e293b' }}>{meta.vehicleNumbe
                </div>
            </div>

            <table style={{ width: '100%', borderCollapse: 'collapse', marginBottom: '40px' }}>
                <thead>
                    <tr style={{ borderBottom: '2px solid #e2e8f0' }}>
                        <th style={{ textAlign: 'left', padding: '16px 0', fontSize: '0.85rem', color: '#94a3b8', textTransform: 'uppercase
                        <th style={{ textAlign: 'center', padding: '16px 0', fontSize: '0.85rem', color: '#94a3b8', textTransform: 'uppercas
                        <th style={{ textAlign: 'right', padding: '16px 0', fontSize: '0.85rem', color: '#94a3b8', textTransform: 'uppercase
                        <th style={{ textAlign: 'right', padding: '16px 0', fontSize: '0.85rem', color: '#94a3b8', textTransform: 'uppercase
                    </tr>
                </thead>
                <tbody>
                    {rows.map((r, i) => (
                        <tr key={i} style={{ borderBottom: '1px solid #f1f5f9' }}>
                            <td style={{ padding: '24px 0', verticalAlign: 'top' }}>
                                <div style={{ fontWeight: 900, fontSize: '1.1rem', color: '#1e293b' }}>{r.name || 'Prototype Line Item'}</di
```

```
                                <div style={{ fontSize: '0.85rem', color: '#64748b', marginTop: '4px' }}>HSN: {r.hsn || '---'}</div>
                            </td>
                            <td style={{ textAlign: 'center', padding: '24px 0', fontWeight: 700 }}>{r.qty} {r.uom}</td>
                            <td style={{ textAlign: 'right', padding: '24px 0', fontWeight: 700 }}>₹{fmt(r.rate)}</td>
                            <td style={{ textAlign: 'right', padding: '24px 0', fontWeight: 900, color: '#1e293b' }}>₹{fmt(r.amount)}</td>
                        </tr>
                    ))}
                </tbody>
            </table>

            <div style={{ marginTop: 'auto', display: 'flex', justifyContent: 'flex-end' }}>
                <div style={{ width: '350px' }}>
                    <div style={{ display: 'flex', justifyContent: 'space-between', padding: '12px 0', borderBottom: '1px solid #f1f5f9' }}>
                        <span style={{ color: '#64748b', fontWeight: 600 }}>Operational Subtotal</span>
                        <span style={{ fontWeight: 800 }}>₹{fmt(totalTaxable)}</span>
                    </div>
                    <div style={{ display: 'flex', justifyContent: 'space-between', padding: '12px 0', borderBottom: '1px solid #f1f5f9' }}>
                        <span style={{ color: '#64748b', fontWeight: 600 }}>Neural Levy (18%)</span>
                        <span style={{ fontWeight: 800 }}>₹{fmt(taxAmount)}</span>
                    </div>
                    <div style={{ display: 'flex', justifyContent: 'space-between', padding: '24px 0', marginTop: '12px' }}>
                        <span style={{ fontSize: '1.5rem', fontWeight: 950, color: '#1e293b' }}>Final Manifest Total</span>
                        <span style={{ fontSize: '1.75rem', fontWeight: 950, color: 'var(--primary)' }}>₹{fmt(grandTotal)}</span>
                    </div>

                    <div style={{ marginTop: '20px', padding: '16px', background: '#f8fafc', borderRadius: '12px', border: '1px dashed #cbd5
                        <div style={{ fontSize: '0.75rem', color: '#94a3b8', fontWeight: 800, textTransform: 'uppercase', marginBottom: '8p
                        <div style={{ fontWeight: 700, color: '#475569', fontSize: '0.9rem', lineHeight: '1.4' }}>{toWords(Math.round(grand
                    </div>
                </div>
            </div>

            <div style={{ marginTop: '80px', paddingTop: '40px', borderTop: '2px solid #1e293b', display: 'grid', gridTemplateColumns: '1.5
                <div style={{ fontSize: '0.85rem', color: '#64748b', lineHeight: '1.6' }}>
                    <div style={{ fontWeight: 900, color: '#1e293b', marginBottom: '8px' }}>DECLARATION</div>
                    We declare that this manifest shows the actual price of the assets described and that all particulars are true and corre
                </div>
                <div style={{ textAlign: 'center' }}>
                    <div style={{ height: '80px' }}></div>
                    <div style={{ fontWeight: 900, fontSize: '0.9rem', color: '#1e293b' }}>AUTHORIZED EXECUTION</div>
                    <div style={{ fontSize: '0.75rem', color: '#64748b', marginTop: '4px' }}>Nexus Systems Control Node</div>
                </div>
            </div>
        </div>
    </motion.div>
  </div>
</div>
);
};

export default CreateInvoice;
```

frontend/src/pages/CompanyProfile.tsx

```
// frontend/src/pages/CompanyProfile.tsx
import { motion } from 'framer-motion';
import React, { useCallback, useEffect, useState } from 'react';
import { FaBuilding, FaCheckCircle, FaHashtag, FaPlus, FaSave, FaUniversity } from 'react-icons/fa';
import { fetchProfile } from '../api/companyApi';

const CompanyProfile: React.FC = () => {
    const [profile, setProfile] = useState<any>({});
    const [bankAccounts, setBankAccounts] = useState<any[]>([
        { id: 1, bank_name: "ICICI Bank", account_number: "106105501618", ifsc_code: "ICIC0001061", account_type: "Savings", is_default: 1 },
        { id: 2, bank_name: "HDFC Bank", account_number: "9876543210", ifsc_code: "HDFC0000020", account_type: "Current", is_default: 0 },
    ]);
    const [loading, setLoading] = useState(true);

    const loadData = useCallback(async () => {
        setLoading(true);
        try {
            const data = await fetchProfile();
            setProfile(data);
        } catch (err) {
            console.error('Identity Retrieval Failure', err);
        } finally {
            setLoading(false);
        }
    }, []);

    useEffect(() => {
        loadData();
    }, [loadData]);

    const onSubmit = (e: React.FormEvent) => {
        e.preventDefault();
        alert("Corporate Identity Synced.");
    };

    if (loading) return (
        <div style={{ height: '60vh', display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
            <div className="spinner-innovative" />
        </div>
    );

    return (
        <div style={{ display: 'flex', flexDirection: 'column', gap: '32px', maxWidth: '1200px' }}>
            {/* Identity Header */}
            <header style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-end' }}>
                <div>
                    <h1 style={{ fontSize: '2.5rem', fontWeight: 950, letterSpacing: '-0.03em' }}>
                        Corporate <span style={{ color: 'var(--primary)' }}>Identity</span>
                    </h1>
                    <p style={{ color: 'var(--text-secondary)', fontWeight: 600 }}>
                        Manage legal entity, fiscal registration and banking hubs.
                    </p>
                </div>
            </header>

            <div style={{ display: 'grid', gridTemplateColumns: '1.5fr 1fr', gap: '32px' }}>
                {/* Profile Form */}
                <motion.div initial={{ opacity: 0, x: -20 }} animate={{ opacity: 1, x: 0 }} className="enterprise-card" style={{ padding: '40px' }}>
                    <div style={{ display: 'flex', alignItems: 'center', gap: '12px', marginBottom: '32px' }}>
                        <div style={{ padding: '10px', background: 'var(--primary-glow)', color: 'var(--primary)', borderRadius: '10px' }}>
                            <FaBuilding />
                        </div>
                        <h2 style={{ fontSize: '1.25rem', fontWeight: 800 }}>Primary Organization Data</h2>
                    </div>

                    <form onSubmit={onSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '24px' }}>
                        <div className="input-block">
                            <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>Registered Co
                            <input className="input-modern" value={profile.company_name || ''} onChange={e => setProfile({...profile, company_name: e.ta
                        </div>

                        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '20px' }}>
                            <div class="input-block">
                                <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>GSTIN Reg
                                <input class="input-modern" value={profile.gstin || ''} onChange={e => setProfile({...profile, gstin: e.target.value
                            </div>
                            <div class="input-block">
                                <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>Fiscal Re
                                <input class="input-modern" value={profile.admin_name || 'MASTER ADMIN'} />
                            </div>
                        </div>

                        <div class="input-block">
                            <label style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>Registered Of
                            <textarea class="input-modern" style={{ minHeight: '100px', paddingTop: '12px' }} value={profile.address_line1 || ''} on
                        </div>

                        <button type="submit" class="btn-enterprise" style={{ width: 'fit-content', padding: '16px 32px' }}>
                            <FaSave /> Commit Profile Updates
                        </button>
                    </form>
                </motion.div>

                {/* Bank Hub */}
                <div style={{ display: 'flex', flexDirection: 'column', gap: '32px' }}>
                    <motion.div initial={{ opacity: 0, x: 20 }} animate={{ opacity: 1, x: 0 }} class="enterprise-card" style={{ padding: '32px' }}>
                        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '24px' }}>
                            <div style={{ display: 'flex', alignItems: 'center', gap: '10px' }}>
                                <FaUniversity color="var(--primary)" />
                                <h3 style={{ fontSize: '1.1rem', fontWeight: 800 }}>Banking Hub</h3>
                            </div>
                            <button class="clickable glass" style={{ padding: '8px', borderRadius: '8px' }}><FaPlus /></button>
                        </div>

                        <div style={{ display: 'flex', flexDirection: 'column', gap: '16px' }}>
                            {bankAccounts.map((acc, idx) => (
```

```
                                <motion.div
                                    key={acc.id}
                                    whileHover={{ scale: 1.02 }}
                                    style={{ background: 'var(--bg-canvas)', borderRadius: '16px', padding: '16px', border: '1px solid var(--border-sub
                                >
                                    <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '8px' }}>
                                        <span style={{ fontWeight: 800 }}>{acc.bank_name}</span>
                                        {acc.is_default ? (
                                            <span style={{ fontSize: '0.65rem', fontWeight: 900, background: 'var(--success-surface)', color: 'var(--su
                                        ) : null}
                                    </div>
                                    <div style={{ fontSize: '0.85rem', fontWeight: 700, color: 'var(--text-muted)', display: 'flex', alignItems: 'cente
                                        <FaHashtag size={10} /> {acc.account_number}
                                    </div>
                                    <div style={{ fontSize: '0.7rem', fontWeight: 800, color: 'var(--text-muted)', marginTop: '4px' }}>IFSC: {acc.ifsc_
                                </motion.div>
                            ))}
                        </div>
                    </motion.div>

                    {/* Subscription Status */}
                    <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: 0.3 }} className="enterprise-card gl
                        <div style={{ display: 'flex', alignItems: 'center', gap: '12px', marginBottom: '16px' }}>
                            <FaCheckCircle color="var(--primary)" />
                            <span style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', letterSpacing: '0.1em' }}>Subscription Matri
                        </div>
                        <h3 style={{ fontSize: '1.5rem', fontWeight: 950 }}>Enterprise Full Stack</h3>
                        <p style={{ fontSize: '0.85rem', opacity: 0.7, marginTop: '8px' }}>All neural and logistic modules unlocked. Next audit: March 2
                    </motion.div>
                </div>
            </div>
        </div>
    );
};

export default CompanyProfile;
```

frontend/src/pages/BusinessFinance.tsx

```
// frontend/src/pages/BusinessFinance.tsx

import React from 'react';

const BusinessFinance: React.FC = () => {
    // const { agreements, loading, error } = useAgreements();

    // Mock data for display
    const agreements = [
        { id: 1, agreement_type: 'LOAN', total_amount: 500000, status: 'Active', start_date: '2023-10-01' },
        { id: 2, agreement_type: 'CHIT', total_amount: 100000, status: 'Active', start_date: '2024-05-15' },
    ];
    const loading = false;
    const error = null;

    return (
        <section id="financeSection" className="app-section">
            <div className="section-header">
                <h2>Business Finance & Agreements</h2>
                <button className="btn btn-primary" onClick={() => {/* openAgreementModal() */}}>
                    <i className="fas fa-handshake"></i> New Agreement
                </button>
            </div>

            <h3 className="section-subheader">Active Loans and Chits</h3>

            {loading && <p>Loading financial data...</p>}
            {error && <p className="text-danger">Error: {error}</p>}

            {!loading && agreements.length > 0 && (
                <div className="table-container">
                    <table className="data-table">
                        <thead>
                            <tr>
                                <th>#</th>
                                <th>Type</th>
                                <th class="num">Total Amount (₹)</th>
                                <th>Start Date</th>
                                <th>Status</th>
                                <th>Actions</th>
                            </tr>
                        </thead>
                        <tbody>
                            {agreements.map(agreement => (
                                <tr key={agreement.id}>
                                    <td>{agreement.id}</td>
                                    <td>{agreement.agreement_type}</td>
                                    <td class="num">{agreement.total_amount.toFixed(2)}</td>
                                    <td>{new Date(agreement.start_date).toLocaleDateString()}</td>
                                    <td>
                                        <span class={`status-badge status-${agreement.status.toLowerCase()}`}>
                                            {agreement.status}
                                        </span>
                                    </td>
                                    <td class="actions-cell">
                                        <button class="btn btn-sm btn-info">View Schedule</button>
                                    </td>
                                </tr>
                            ))}
                        </tbody>
                    </table>
                </div>
            )}
        </section>
    );
};

export default BusinessFinance;
```

frontend/src/pages/Brokers.tsx

```
// frontend/src/pages/Brokers.tsx

import React from 'react';

const Brokers: React.FC = () => {
    return (
        <section id="brokersSection" className="app-section">
            <div className="section-header">
                <h2>Broker/Agent Management</h2>
            </div>
            <p>This section is dedicated to managing brokers and calculating their commissions.</p>

            <div className="card-container">
                {/* Broker List Table */}
                <div className="data-card">
                    <h3>Active Brokers</h3>
                    {/* Placeholder for table */}
                    <table className="data-table">
                        <thead>
                            <tr>
                                <th>Name</th>
                                <th>Phone</th>
                                <th>Total Sales (Mo)</th>
                                <th>Commission Rate</th>
                                <th>Actions</th>
                            </tr>
                        </thead>
                        <tbody>
                            <tr>
                                <td colSpan={5} className="text-center">Broker data pending implementation.</td>
                            </tr>
                        </tbody>
                    </table>
                </div>
            </div>
        </section>
    );
};

export default Brokers;
```

frontend/src/pages/Branches.tsx

```
// frontend/src/pages/Branches.tsx
import { AnimatePresence, motion } from 'framer-motion';
import React, { useEffect, useState } from 'react';
import { FaBuilding, FaMapMarkerAlt, FaNetworkWired, FaPlus, FaSearch, FaUserTie } from 'react-icons/fa';

interface Branch {
    id: number;
    branch_name: string;
    branch_code: string;
    location: string;
    manager_user_id: number;
    is_active: boolean;
    created_at: string;
}

const Branches: React.FC = () => {
    const [branches, setBranches] = useState<Branch[]>([]);
    const [loading, setLoading] = useState(true);
    const [searchTerm, setSearchTerm] = useState("");

    useEffect(() => {
        const fetchBranches = async () => {
            try {
                const response = await fetch('/api/branches', {
                    headers: { 'Authorization': `Bearer ${localStorage.getItem('erp-token')}` }
                });
                const data = await response.json();
                if (response.ok) setBranches(data);
            } catch (error) {
                console.error("Link Loss", error);
            } finally {
                setLoading(false);
            }
        };
        fetchBranches();
    }, []);

    const filteredBranches = branches.filter(b =>
        b.branch_name.toLowerCase().includes(searchTerm.toLowerCase()) ||
        b.branch_code?.toLowerCase().includes(searchTerm.toLowerCase())
    );

    return (
        <div style={{ display: 'flex', flexDirection: 'column', gap: '32px' }}>
            {/* Empire Header */}
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-end' }}>
                <div>
                    <h1 style={{ fontSize: '2.5rem', fontWeight: 950, letterSpacing: '-0.03em' }}>
                        Enterprise <span style={{ color: 'var(--primary)' }}>Footprint</span>
                    </h1>
                    <p style={{ color: 'var(--text-secondary)', fontWeight: 600 }}>
                        Monitoring and governance of global branch infrastructure.
                    </p>
                </div>
                <button className="btn-enterprise">
                    <FaPlus /> Authorize New Branch
                </button>
            </div>

            {/* Network Telemetry */}
            <div className="kpi-grid">
                <StatusCard icon={<FaNetworkWired />} title="Total Nodes" value={branches.length} color="var(--primary)" detail="Global Mesh" />
                <StatusCard icon={<FaBuilding />} title="Operational" value={branches.filter(b => b.is_active).length} color="var(--success)" detail="Ac
                <StatusCard icon={<FaMapMarkerAlt />} title="Geographic Spread" value="State-wide" color="var(--warning)" detail="Zone Alpha" />
                <StatusCard icon={<FaUserTie />} title="Managers Active" value={branches.filter(b => b.manager_user_id).length} color="var(--text-main)"
            </div>

            {/* Control Bar */}
            <motion.div initial={{ opacity: 0, y: 10 }} animate={{ opacity: 1, y: 0 }} className="enterprise-card glass" style={{ padding: '16px 24px',
                <FaSearch color="var(--text-muted)" />
                <input
                    className="input-modern"
                    placeholder="Search node by name or unique identifer..."
                    value={searchTerm}
                    onChange={e => setSearchTerm(e.target.value)}
                    style={{ border: 'none', background: 'transparent', flex: 1 }}
                />
            </motion.div>

            {/* Branch Mesh Grid */}
            <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(320px, 1fr))', gap: '24px' }}>
                <AnimatePresence>
                    {loading ? (
                        [1,2,3].map(i => <div key={i} className="enterprise-card glass" style={{ height: '200px' }} />)
                    ) : (
                        filteredBranches.map((branch, idx) => (
                            <motion.div
                                key={branch.id}
                                initial={{ opacity: 0, scale: 0.95 }}
                                animate={{ opacity: 1, scale: 1 }}
                                transition={{ delay: idx * 0.05 }}
                                whileHover={{ y: -8 }}
                                className="enterprise-card glass"
                                style={{ padding: '32px', borderTop: `5px solid ${branch.is_active ? 'var(--success)' : 'var(--error)'}` }}
                            >
                                <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '20px' }}>
                                    <div style={{ padding: '12px', background: 'var(--bg-canvas)', borderRadius: '12px', color: 'var(--primary)' }}>
                                        <FaBuilding />
                                    </div>
                                    <span className={`badge-premium ${branch.is_active ? 'badge-user' : 'badge-host'}`}>
                                        {branch.is_active ? 'OPERATIONAL' : 'OFFLINE'}
                                    </span>
                                </div>

                                <h3 style={{ fontSize: '1.25rem', fontWeight: 800, marginBottom: '4px' }}>{branch.branch_name}</h3>
                                <p style={{ fontSize: '0.75rem', fontWeight: 800, color: 'var(--text-muted)', fontFamily: 'monospace' }}>{branch.branch_
```

```
                                <div style={{ marginTop: '24px', display: 'flex', flexDirection: 'column', gap: '12px' }}>
                                    <div style={{ display: 'flex', alignItems: 'center', gap: '10px', fontSize: '0.85rem', fontWeight: 600, color: 'var
                                        <FaMapMarkerAlt color="var(--text-muted)" /> {branch.location || 'Location Not Sealed'}
                                    </div>
                                    <div style={{ display: 'flex', alignItems: 'center', gap: '10px', fontSize: '0.85rem', fontWeight: 600, color: 'var
                                        <FaUserTie color="var(--text-muted)" /> {branch.manager_user_id ? 'Authenticated Manager' : 'Master Overseer'}
                                    </div>
                                </div>

                                <div style={{ marginTop: '32px', paddingTop: '20px', borderTop: '1px solid var(--border-subtle)', display: 'flex', justi
                                    <span style={{ fontSize: '0.65rem', fontWeight: 900, color: 'var(--text-muted)' }}>ID: {branch.id}</span>
                                    <button className="clickable" style={{ fontSize: '0.75rem', fontWeight: 800, color: 'var(--primary)', border: 'none
                                </div>
                            </motion.div>
                        ))
                    )}
                </AnimatePresence>
            </div>
        </div>
    );
};

const StatusCard: React.FC<any> = ({ icon, title, value, color, detail }) => (
    <motion.div
        whileHover={{ y: -5 }}
        className="enterprise-card glass"
        style={{ padding: '24px', borderLeft: `6px solid ${color}` }}
    >
        <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '16px' }}>
            <div style={{ color }}>{icon}</div>
            <span style={{ fontSize: '0.65rem', fontWeight: 900, textTransform: 'uppercase', color: 'var(--text-muted)' }}>{detail}</span>
        </div>
        <div style={{ fontSize: '0.7rem', fontWeight: 900, color: 'var(--text-muted)', textTransform: 'uppercase', letterSpacing: '0.05em' }}>{title}</d
        <div style={{ fontSize: '2rem', fontWeight: 950, color: 'var(--text-primary)', marginTop: '4px' }}>{value}</div>
    </motion.div>
);

export default Branches;
```

frontend/src/pages/AddSupplierModal.tsx

```
import React, { useState } from 'react';
import { FaBuilding, FaEnvelope, FaMoneyBillWave, FaPhone, FaTimes } from 'react-icons/fa';
import { createSupplier } from '../api/supplierApi';

interface AddSupplierModalProps {
    onClose: () => void;
    onSuccess: () => void;
}

const AddSupplierModal: React.FC<AddSupplierModalProps> = ({ onClose, onSuccess }) => {
    const [formData, setFormData] = useState({
        supplier_name: '',
        phone: '',
        email: '',
        initial_payable_balance: '',
        gstin: '',
        address: ''
    });
    const [loading, setLoading] = useState(false);

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setLoading(true);
        try {
            await createSupplier(formData);
            alert("Success! Supplier created.");
            onSuccess();
            onClose();
        } catch (err: any) {
            alert("Failed to create supplier");
            console.error(err);
        } finally {
            setLoading(false);
        }
    };

    return (
        <div style={{ position: 'fixed', inset: 0, background: 'rgba(0,0,0,0.5)', display: 'flex', alignItems: 'center', justifyContent: 'center', zInde
            <div style={{ background: 'white', width: '500px', borderRadius: '12px', padding: '24px', boxShadow: '0 20px 25px -5px rgba(0,0,0,0.1)' }}>
                <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '20px' }}>
                    <h2 style={{ fontSize: '1.25rem', fontWeight: 700, margin: 0 }}>Add New Supplier</h2>
                    <button onClick={onClose} style={{ background: 'none', border: 'none', cursor: 'pointer', fontSize: '1.2rem', color: '#64748b' }}><
                </div>

                <form onSubmit={handleSubmit}>
                    <div style={{ marginBottom: '15px' }}>
                        <label style={{ display: 'block', fontSize: '0.875rem', fontWeight: 500, color: '#475569', marginBottom: '5px' }}>Supplier Name*
                        <div style={{ position: 'relative' }}>
                            <FaBuilding style={{ position: 'absolute', left: '10px', top: '50%', transform: 'translateY(-50%)', color: '#94a3b8' }} />
                            <input
                                required
                                value={formData.supplier_name}
                                onChange={e => setFormData({...formData, supplier_name: e.target.value})}
                                style={{ width: '100%', padding: '10px 10px 10px 35px', borderRadius: '6px', border: '1px solid #cbd5e1' }}
                                placeholder="e.g. Global Tech Solutions"
                            />
                        </div>
                    </div>

                    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px', marginBottom: '15px' }}>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.875rem', fontWeight: 500, color: '#475569', marginBottom: '5px' }}>Phone</lab
                            <div style={{ position: 'relative' }}>
                                <FaPhone style={{ position: 'absolute', left: '10px', top: '50%', transform: 'translateY(-50%)', color: '#94a3b8' }} />
                                <input
                                    value={formData.phone}
                                    onChange={e => setFormData({...formData, phone: e.target.value})}
                                    style={{ width: '100%', padding: '10px 10px 10px 35px', borderRadius: '6px', border: '1px solid #cbd5e1' }}
                                    placeholder="98765..."
                                />
                            </div>
                        </div>
                        <div>
                            <label style={{ display: 'block', fontSize: '0.875rem', fontWeight: 500, color: '#475569', marginBottom: '5px' }}>Email</lab
                            <div style={{ position: 'relative' }}>
                                <FaEnvelope style={{ position: 'absolute', left: '10px', top: '50%', transform: 'translateY(-50%)', color: '#94a3b8' }}
                                <input
                                    value={formData.email}
                                    onChange={e => setFormData({...formData, email: e.target.value})}
                                    style={{ width: '100%', padding: '10px 10px 10px 35px', borderRadius: '6px', border: '1px solid #cbd5e1' }}
                                    placeholder="supplier@example.com"
                                />
                            </div>
                        </div>
                    </div>

                    <div style={{ marginBottom: '20px' }}>
                        <label style={{ display: 'block', fontSize: '0.875rem', fontWeight: 500, color: '#475569', marginBottom: '5px' }}>Opening Balanc
                        <div style={{ position: 'relative' }}>
                            <FaMoneyBillWave style={{ position: 'absolute', left: '10px', top: '50%', transform: 'translateY(-50%)', color: '#ef4444' }
                            <input
                                type="number"
                                value={formData.initial_payable_balance}
                                onChange={e => setFormData({...formData, initial_payable_balance: e.target.value})}
                                style={{ width: '100%', padding: '10px 10px 10px 35px', borderRadius: '6px', border: '1px solid #cbd5e1', fontWeight: '
                                placeholder="0.00"
                            />
                        </div>
                        <p style={{ fontSize: '0.75rem', color: '#94a3b8', marginTop: '4px' }}>Amount you already owe to this supplier.</p>
                    </div>

                    <div style={{ display: 'flex', justifyContent: 'flex-end', gap: '10px' }}>
                        <button
                            type="button"
                            onClick={onClose}
                            style={{ padding: '10px 20px', borderRadius: '6px', border: '1px solid #cbd5e1', background: 'white', cursor: 'pointer' }}
```

```
                        >
                            Cancel
                        </button>
                        <button
                            type="submit"
                            disabled={loading}
                            style={{ padding: '10px 20px', borderRadius: '6px', border: 'none', background: '#3b82f6', color: 'white', fontWeight: 600,
                        >
                            {loading ? 'Creating...' : 'Create Supplier'}
                        </button>
                    </div>
                </form>
            </div>
        </div>
    );
};

export default AddSupplierModal;
```

frontend/src/pages/AddProductModal.tsx

```
import React, { useEffect, useState } from 'react';
import {
    FaAlignLeft,
    FaArrowDown,
    FaArrowUp,
    FaBarcode,
    FaBox,
    FaMoneyBillWave,
    FaSave,
    FaTag,
    FaTimes
} from 'react-icons/fa';
import { createProduct, updateProduct } from '../api/productApi';

interface Props {
    onClose: () => void;
    onSuccess: () => void;
    productToEdit?: any;
}

const styles = {
    overlay: {
        position: 'fixed' as const, top: 0, left: 0, right: 0, bottom: 0,
        backgroundColor: 'rgba(0, 0, 0, 0.7)', backdropFilter: 'blur(8px)',
        display: 'flex', justifyContent: 'center', alignItems: 'center', zIndex: 1000
    },
    modal: {
        backgroundColor: '#ffffff', width: '100%', maxWidth: '650px',
        borderRadius: '20px', boxShadow: '0 25px 50px -12px rgba(0, 0, 0, 0.35)',
        overflow: 'hidden', display: 'flex', flexDirection: 'column' as const,
        maxHeight: '90vh', fontFamily: 'Inter, sans-serif'
    },
    header: {
        padding: '20px 28px', borderBottom: '1px solid #f1f5f9',
        display: 'flex', justifyContent: 'space-between', alignItems: 'center',
        backgroundColor: '#ffffff', flexShrink: 0
    },
    body: {
        padding: '28px',
        overflowY: 'auto' as const,
        flexGrow: 1,
        background: 'linear-gradient(to bottom, #ffffff, #f8fafc)'
    },
    footer: {
        padding: '20px 28px', borderTop: '1px solid #f1f5f9',
        backgroundColor: '#ffffff', display: 'flex', justifyContent: 'flex-end',
        gap: '12px', flexShrink: 0
    },
    title: { fontSize: '1.25rem', fontWeight: 800, color: '#0f172a', display: 'flex', alignItems: 'center', gap: '12px', letterSpacing: '-0.5px' },
    closeBtn: { background: '#f1f5f9', border: 'none', cursor: 'pointer', color: '#64748b', width: '36px', height: '36px', borderRadius: '10px', display
    sectionTitle: { fontSize: '0.75rem', fontWeight: 800, color: '#334155', textTransform: 'uppercase' as const, letterSpacing: '0.1em', marginBottom:
    inputGroup: { marginBottom: '20px' },
    label: { display: 'block', marginBottom: '8px', fontSize: '0.85rem', fontWeight: 700, color: '#475569', letterSpacing: '-0.2px' },
    inputWrapper: { position: 'relative' as const },
    icon: { position: 'absolute' as const, left: '14px', top: '50%', transform: 'translateY(-50%)', color: '#94a3b8', pointerEvents: 'none' as const, fo
    input: { width: '100%', padding: '12px 14px 12px 42px', borderRadius: '12px', border: '1px solid #e2e8f0', fontSize: '0.95rem', color: '#1e293b', ba
    textarea: { width: '100%', padding: '12px 14px 12px 42px', borderRadius: '12px', border: '1px solid #e2e8f0', fontSize: '0.95rem', color: '#1e293b'
    row: { display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '20px' },
    btnSecondary: { padding: '12px 24px', borderRadius: '12px', border: '1px solid #e2e8f0', backgroundColor: '#fff', color: '#475569', fontSize: '0.95
    btnPrimary: { padding: '12px 28px', borderRadius: '12px', border: 'none', backgroundColor: '#2563eb', color: '#fff', fontSize: '0.95rem', fontWeigh
};

const InputField = ({ label, icon: Icon, value, onChange, placeholder, required = false, type = "text", disabled = false, isTextArea = false }: any) =>
    <div style={styles.inputGroup}>
        <label style={styles.label}>{label} {required && <span style={{ color: '#ef4444' }}>*</span>}</label>
        <div style={styles.inputWrapper}>
            <div style={{ ...styles.icon, top: isTextArea ? '24px' : '50%' }}><Icon /></div>
            {isTextArea ? (
                <textarea
                    required={required}
                    disabled={disabled}
                    style={{ ...styles.textarea, ...(disabled ? { opacity: 0.7, backgroundColor: '#f1f5f9' } : {}) }}
                    placeholder={placeholder}
                    value={value}
                    onChange={onChange}
                />
            ) : (
                <input
                    type={type}
                    required={required}
                    disabled={disabled}
                    style={{ ...styles.input, ...(disabled ? { opacity: 0.7, backgroundColor: '#f1f5f9' } : {}) }}
                    placeholder={placeholder}
                    value={value}
                    onChange={onChange}
                />
            )}
        </div>
    </div>
);

const AddProductModal: React.FC<Props> = ({ onClose, onSuccess, productToEdit }) => {
    const [formData, setFormData] = useState({
        name: '',
        sku: '',
        description: '',
        cost_price: 0,
        selling_price: 0,
        opening_stock: 0,
        current_stock: 0,
        min_stock: 5,
        hsn_code: '',
        barcode: '',
        unit: 'Unit'
    });
    const [loading, setLoading] = useState(false);
```

```
useEffect(() => {
    if (productToEdit) {
        setFormData({
            name: productToEdit.name || '',
            sku: productToEdit.sku || '',
            description: productToEdit.description || '',
            cost_price: productToEdit.cost_price || 0,
            selling_price: productToEdit.selling_price || 0,
            opening_stock: productToEdit.opening_stock || 0,
            current_stock: productToEdit.current_stock || 0,
            min_stock: productToEdit.min_stock || 5,
            hsn_code: productToEdit.hsn_code || '',
            barcode: productToEdit.barcode || '',
            unit: productToEdit.unit || 'Unit'
        });
    }
}, [productToEdit]);

const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    try {
        if (productToEdit) {
            await updateProduct(productToEdit.id, formData);
        } else {
            await createProduct(formData);
        }
        onSuccess();
        onClose();
    } catch (err: any) {
        alert(err.message || "Operation failed");
    } finally {
        setLoading(false);
    }
};

return (
    <div style={styles.overlay}>
        <div style={styles.modal} className="page-transition">
            <div style={styles.header}>
                <h2 style={styles.title}>
                    <div style={{ background: '#eff6ff', padding: '10px', borderRadius: '12px', color: '#2563eb', display: 'flex' }}>
                        <FaBox size={20} />
                    </div>
                    {productToEdit ? 'Edit Product' : 'Add New Product'}
                </h2>
                <button
                    onClick={onClose}
                    style={styles.closeBtn}
                    onMouseOver={e => e.currentTarget.style.background = '#e2e8f0'}
                    onMouseOut={e => e.currentTarget.style.background = '#f1f5f9'}
                >
                    <FaTimes />
                </button>
            </div>

            <div style={styles.body}>
                <form id="product-form" onSubmit={handleSubmit}>

                    <div style={{ ...styles.sectionTitle, marginTop: 0 }}>Basic Information</div>
                    <InputField
                        label="Product Name"
                        icon={FaBox}
                        value={formData.name}
                        onChange={(e: any) => setFormData({ ...formData, name: e.target.value })}
                        placeholder="e.g. Wireless Mouse"
                        required
                    />

                    <div style={styles.row}>
                        <InputField
                            label="SKU / Item ID"
                            icon={FaBarcode}
                            value={formData.sku}
                            onChange={(e: any) => setFormData({ ...formData, sku: e.target.value })}
                            placeholder="PROD-123"
                        />
                        <InputField
                            label="HSN / SAC Code"
                            icon={FaTag}
                            value={formData.hsn_code}
                            onChange={(e: any) => setFormData({ ...formData, hsn_code: e.target.value })}
                            placeholder="8471"
                        />
                    </div>

                    <InputField
                        label="Description"
                        icon={FaAlignLeft}
                        value={formData.description}
                        onChange={(e: any) => setFormData({ ...formData, description: e.target.value })}
                        placeholder="Brief details about the product..."
                        isTextArea
                    />

                    <div style={styles.sectionTitle}>Pricing & Finance</div>
                    <div style={styles.row}>
                        <InputField
                            label="Purchase Price (₹)"
                            icon={FaMoneyBillWave}
                            type="number"
                            value={formData.cost_price}
                            onChange={(e: any) => setFormData({ ...formData, cost_price: parseFloat(e.target.value) || 0 })}
                            placeholder="0.00"
                            required
                        />
                        <InputField
                            label="Selling Price (₹)"
                            icon={FaMoneyBillWave}
                            type="number"
                            value={formData.selling_price}
```

```
                        onChange={(e: any) => setFormData({ ...formData, selling_price: parseFloat(e.target.value) || 0 })}
                        placeholder="0.00"
                        required
                    />
                </div>

                <div style={styles.sectionTitle}>Inventory Control</div>
                <div style={styles.row}>
                    <div style={styles.inputGroup}>
                        <label style={styles.label}>Opening Stock</label>
                        <div style={styles.inputWrapper}>
                            <div style={styles.icon}><FaArrowUp /></div>
                            <input
                                type="number"
                                disabled={!!productToEdit}
                                style={{ ...styles.input, ...(productToEdit ? { opacity: 0.7, backgroundColor: '#f1f5f9' } : {}) }}
                                value={formData.opening_stock}
                                onChange={e => {
                                    const val = parseInt(e.target.value) || 0;
                                    setFormData({ ...formData, opening_stock: val, current_stock: val });
                                }}
                            />
                        </div>
                    </div>
                    <div style={styles.inputGroup}>
                        <label style={styles.label}>Low Stock Alert at</label>
                        <div style={styles.inputWrapper}>
                            <div style={styles.icon}><FaArrowDown /></div>
                            <input
                                type="number"
                                style={styles.input}
                                value={formData.min_stock}
                                onChange={e => setFormData({ ...formData, min_stock: parseInt(e.target.value) || 0 })}
                            />
                        </div>
                    </div>
                </div>
            </form>
        </div>

        <div style={styles.footer}>
            <button
                onClick={onClose}
                style={styles.btnSecondary}
                type="button"
                onMouseOver={e => e.currentTarget.style.borderColor = '#cbd5e1'}
                onMouseOut={e => e.currentTarget.style.borderColor = '#e2e8f0'}
            >
                Cancel
            </button>
            <button
                form="product-form"
                type="submit"
                disabled={loading}
                style={{ ...styles.btnPrimary, opacity: loading ? 0.7 : 1 }}
                onMouseOver={e => e.currentTarget.style.backgroundColor = '#1d4ed8'}
                onMouseOut={e => e.currentTarget.style.backgroundColor = '#2563eb'}
            >
                {loading ? 'Processing...' : <><FaSave /> {productToEdit ? 'Update Product' : 'Create Product'}</>}
            </button>
        </div>
    </div>
</div>
    );
};

export default AddProductModal;
```

frontend/src/pages/AddEmployeeModal.tsx

```
// frontend/src/pages/AddEmployeeModal.tsx
import React, { useEffect, useState } from 'react';
import {
    FaBriefcase,
    FaCalendarAlt,
    FaEnvelope,
    FaFileSignature,
    FaIdBadge,
    FaMoneyBillWave,
    FaPhone,
    FaSave,
    FaTimes,
    FaUserTie
} from 'react-icons/fa';
import { apiFetch } from '../utils/api';

interface Props {
    onClose: () => void;
    onSuccess: () => void;
    employeeToEdit?: any;
}

const AddEmployeeModal: React.FC<Props> = ({ onClose, onSuccess, employeeToEdit }) => {
    const [loading, setLoading] = useState(false);

    // Form State
    const [name, setName] = useState('');
    const [designation, setDesignation] = useState('');
    const [email, setEmail] = useState('');
    const [phone, setPhone] = useState('');
    const [salary, setSalary] = useState<number | string>(0);
    const [salaryType, setSalaryType] = useState('Monthly');
    const [joiningDate, setJoiningDate] = useState(new Date().toISOString().substring(0, 10));
    const [status, setStatus] = useState('Active');

    const [signatureFile, setSignatureFile] = useState<File | null>(null);
    const [signaturePreview, setSignaturePreview] = useState<string | null>(null);

    useEffect(() => {
        if (employeeToEdit) {
            setName(employeeToEdit.name || '');
            setDesignation(employeeToEdit.designation || '');
            setEmail(employeeToEdit.email || '');
            setPhone(employeeToEdit.phone || '');
            setSalary(employeeToEdit.salary || 0);
            setSalaryType(employeeToEdit.salary_type || 'Monthly');
            setJoiningDate(employeeToEdit.joining_date ? employeeToEdit.joining_date.substring(0, 10) : '');
            setStatus(employeeToEdit.status || 'Active');

            if (employeeToEdit.signature_url) {
                setSignaturePreview(`http://localhost:3001${employeeToEdit.signature_url}`);
            }
        }
    }, [employeeToEdit]);

    const handleFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
        if (e.target.files && e.target.files[0]) {
            const file = e.target.files[0];
            setSignatureFile(file);
            setSignaturePreview(URL.createObjectURL(file));
        }
    };

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setLoading(true);

        try {
            const formData = new FormData();
            formData.append('name', name);
            formData.append('designation', designation);
            formData.append('email', email);
            formData.append('phone', phone);
            formData.append('salary', String(salary));
            formData.append('salary_type', salaryType);
            formData.append('joining_date', joiningDate);
            formData.append('status', status);

            if (signatureFile) {
                formData.append('signature', signatureFile);
            }

            const url = employeeToEdit
                ? `/employees/${employeeToEdit.id}`
                : '/employees';

            const method = employeeToEdit ? 'PUT' : 'POST';

            await apiFetch(url, {
                method: method,
                body: formData
            }, false);

            onSuccess();
            onClose();
        } catch (err: any) {
            console.error("Save Error:", err);
            alert("Failed to save employee. " + (err.message || ""));
        } finally {
            setLoading(false);
        }
    };

    // Dynamic label based on salary type
    const getSalaryLabel = () => {
        switch (salaryType) {
```

```
        case 'Monthly': return 'Monthly Salary (₹)';
        case 'Weekly': return 'Weekly Rate (₹)';
        case 'Daily': return 'Daily Rate (₹)';
        default: return 'Amount (₹)';
    }
};

const getSalaryPlaceholder = () => {
    switch (salaryType) {
        case 'Monthly': return '25000';
        case 'Weekly': return '5000';
        case 'Daily': return '800';
        default: return '0';
    }
};

// Type colors
const typeColors: Record<string, { bg: string; color: string; border: string }> = {
    'Monthly': { bg: '#eff6ff', color: '#2563eb', border: '#3b82f6' },
    'Weekly': { bg: '#f5f3ff', color: '#7c3aed', border: '#8b5cf6' },
    'Daily': { bg: '#ecfdf5', color: '#059669', border: '#10b981' }
};

// Styles
const modalStyle: React.CSSProperties = {
    position: 'fixed',
    top: 0,
    left: 0,
    width: '100%',
    height: '100%',
    background: 'rgba(0,0,0,0.6)',
    display: 'flex',
    justifyContent: 'center',
    alignItems: 'center',
    zIndex: 100
};

const contentStyle: React.CSSProperties = {
    background: 'white',
    width: '550px',
    borderRadius: '12px',
    overflow: 'hidden',
    maxHeight: '90vh',
    display: 'flex',
    flexDirection: 'column'
};

const inputStyle: React.CSSProperties = {
    width: '100%',
    padding: '10px 12px 10px 35px',
    borderRadius: '8px',
    border: '1px solid #cbd5e1',
    outline: 'none',
    boxSizing: 'border-box'
};

const iconStyle: React.CSSProperties = {
    position: 'absolute',
    left: '12px',
    top: '50%',
    transform: 'translateY(-50%)',
    color: '#94a3b8'
};

return (
    <div style={modalStyle}>
        <div style={contentStyle}>

            {/* Header */}
            <div style={{
                padding: '15px 20px',
                borderBottom: '1px solid #eee',
                display: 'flex',
                justifyContent: 'space-between',
                alignItems: 'center',
                background: '#f8fafc'
            }}>
                <h3 style={{ margin: 0, display: 'flex', alignItems: 'center', gap: '10px', color: '#1e293b' }}>
                    <FaUserTie color="#2563eb" /> {employeeToEdit ? 'Edit Employee' : 'Add Employee'}
                </h3>
                <button onClick={onClose} style={{ border: 'none', background: 'none', cursor: 'pointer', fontSize: '1.2rem', color: '#64748b' }}>
                    <FaTimes />
                </button>
            </div>

            {/* Form */}
            <div style={{ padding: '20px', overflowY: 'auto' }}>
                <form id="employee-form" onSubmit={handleSubmit}>

                    {/* Full Name */}
                    <div style={{ marginBottom: '15px' }}>
                        <label style={{ display: 'block', marginBottom: '5px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' }}>Full Name *
                        <div style={{ position: 'relative' }}>
                            <FaIdBadge style={iconStyle} />
                            <input required value={name} onChange={e => setName(e.target.value)} style={inputStyle} placeholder="John Doe" />
                        </div>
                    </div>

                    {/* Designation */}
                    <div style={{ marginBottom: '15px' }}>
                        <label style={{ display: 'block', marginBottom: '5px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' }}>Designatio
                        <div style={{ position: 'relative' }}>
                            <FaBriefcase style={iconStyle} />
                            <input value={designation} onChange={e => setDesignation(e.target.value)} style={inputStyle} placeholder="Manager, Labo
                        </div>
                    </div>

                    {/* SALARY TYPE & AMOUNT SECTION */}
                    <div style={{
                        marginBottom: '15px',
                        background: '#f9fafb',
                        padding: '15px',
```

```
                    borderRadius: '10px',
                    border: '1px solid #e5e7eb'
                }}>
                    <label style={{ display: 'block', marginBottom: '10px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' }}>
                        Payment Structure
                    </label>

                    {/* Type Selection Buttons */}
                    <div style={{ display: 'flex', gap: '10px', marginBottom: '15px' }}>
                        {['Monthly', 'Weekly', 'Daily'].map(type => {
                            const colors = typeColors[type];
                            const isSelected = salaryType === type;
                            return (
                                <button
                                    key={type}
                                    type="button"
                                    onClick={() => setSalaryType(type)}
                                    style={{
                                        flex: 1,
                                        padding: '12px 10px',
                                        border: isSelected ? `2px solid ${colors.border}` : '2px solid #e5e7eb',
                                        borderRadius: '8px',
                                        background: isSelected ? colors.bg : 'white',
                                        color: isSelected ? colors.color : '#64748b',
                                        fontWeight: 600,
                                        cursor: 'pointer',
                                        fontSize: '0.9rem',
                                        transition: 'all 0.2s'
                                    }}
                                >
                                    {type}
                                </button>
                            );
                        })}
                    </div>

                    {/* Salary Amount Input */}
                    <div>
                        <label style={{ display: 'block', marginBottom: '5px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' }}>
                            {getSalaryLabel()}
                        </label>
                        <div style={{ position: 'relative' }}>
                            <FaMoneyBillWave style={iconStyle} />
                            <input
                                type="number"
                                value={salary}
                                onChange={e => setSalary(e.target.value)}
                                style={inputStyle}
                                placeholder={getSalaryPlaceholder()}
                            />
                        </div>
                        <p style={{ margin: '8px 0 0', fontSize: '0.75rem', color: '#94a3b8' }}>
                            {salaryType === 'Monthly' && 'Fixed monthly salary paid at end of month'}
                            {salaryType === 'Weekly' && 'Weekly payment based on 6-day work week'}
                            {salaryType === 'Daily' && 'Daily wages × days worked'}
                        </p>
                    </div>
                </div>

                {/* Email & Phone */}
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px', marginBottom: '15px' }}>
                    <div>
                        <label style={{ display: 'block', marginBottom: '5px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' }}>Email</
                        <div style={{ position: 'relative' }}>
                            <FaEnvelope style={iconStyle} />
                            <input type="email" value={email} onChange={e => setEmail(e.target.value)} style={inputStyle} placeholder="john@worl
                        </div>
                    </div>
                    <div>
                        <label style={{ display: 'block', marginBottom: '5px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' }}>Phone</
                        <div style={{ position: 'relative' }}>
                            <FaPhone style={iconStyle} />
                            <input value={phone} onChange={e => setPhone(e.target.value)} style={inputStyle} placeholder="9876543210" />
                        </div>
                    </div>
                </div>

                {/* Joining Date & Status */}
                <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '15px', marginBottom: '20px' }}>
                    <div>
                        <label style={{ display: 'block', marginBottom: '5px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' }}>Joining
                        <div style={{ position: 'relative' }}>
                            <FaCalendarAlt style={iconStyle} />
                            <input type="date" value={joiningDate} onChange={e => setJoiningDate(e.target.value)} style={inputStyle} />
                        </div>
                    </div>
                    <div>
                        <label style={{ display: 'block', marginBottom: '5px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' }}>Status
                        <select value={status} onChange={e => setStatus(e.target.value)} style={{ ...inputStyle, paddingLeft: '12px' }}>
                            <option>Active</option>
                            <option>On Leave</option>
                            <option>Resigned</option>
                        </select>
                    </div>
                </div>

                {/* Signature Upload */}
                <div style={{ background: '#f0f9ff', padding: '15px', borderRadius: '8px', border: '1px dashed #bae6fd' }}>
                    <label style={{ marginBottom: '8px', fontSize: '0.85rem', fontWeight: 600, color: '#0284c7', display: 'flex', alignItems: '
                        <FaFileSignature /> Employee Signature
                    </label>

                    <div style={{ display: 'flex', alignItems: 'center', gap: '15px' }}>
                        <input type="file" accept="image/*" onChange={handleFileChange} style={{ fontSize: '0.85rem', color: '#64748b' }} />
                        {signaturePreview && (
                            <div style={{ width: '80px', height: '40px', border: '1px solid #ddd', background: 'white', display: 'flex', alignI
                                <img src={signaturePreview} alt="Sig" style={{ maxWidth: '100%', maxHeight: '100%' }} />
                            </div>
                        )}
                    </div>
                </div>
```

```
                    </form>
                </div>

                {/* Footer */}
                <div style={{ padding: '15px 20px', borderTop: '1px solid #eee', background: '#f8fafc', display: 'flex', justifyContent: 'flex-end', gap
                    <button onClick={onClose} style={{ padding: '10px 20px', borderRadius: '8px', border: '1px solid #cbd5e1', background: 'white', colo
                        Cancel
                    </button>
                    <button type="submit" form="employee-form" disabled={loading} style={{
                        padding: '10px 24px',
                        background: '#2563eb',
                        color: 'white',
                        border: 'none',
                        borderRadius: '8px',
                        cursor: 'pointer',
                        fontWeight: 600,
                        display: 'flex',
                        justifyContent: 'center',
                        gap: '8px',
                        alignItems: 'center',
                        opacity: loading ? 0.7 : 1
                    }}>
                        <FaSave />  {loading ? 'Saving...' : 'Save Employee'}
                    </button>
                </div>

            </div>
        </div>
    );
};

export default AddEmployeeModal;
```

frontend/src/pages/AddCustomerModal.tsx

```
import React, { useEffect, useState } from 'react';
import {
    FaBuilding, FaEnvelope, FaIdCard, FaLock, FaMapMarkerAlt,
    FaMoneyCheckAlt, FaPhone, FaSave, FaTag, FaTimes, FaUser
} from 'react-icons/fa';
import { createCustomer, updateCustomer } from '../api/userApi';

interface Props {
    onClose: () => void;
    onSuccess: () => void;
    customerToEdit?: any;
}

const styles = {
    overlay: {
        position: 'fixed' as const, top: 0, left: 0, right: 0, bottom: 0,
        backgroundColor: 'rgba(0, 0, 0, 0.65)', backdropFilter: 'blur(4px)',
        display: 'flex', justifyContent: 'center', alignItems: 'center', zIndex: 1000
    },
    modal: {
        backgroundColor: '#ffffff', width: window.innerWidth <= 600 ? '95%' : '100%', maxWidth: '600px',
        borderRadius: '12px', boxShadow: '0 25px 50px -12px rgba(0, 0, 0, 0.25)',
        overflow: 'hidden', display: 'flex', flexDirection: 'column' as const,
        maxHeight: '90vh', fontFamily: 'Inter, sans-serif', margin: window.innerWidth <= 600 ? '10px' : '0'
    },
    header: {
        padding: '16px 24px', borderBottom: '1px solid #e5e7eb',
        display: 'flex', justifyContent: 'space-between', alignItems: 'center',
        backgroundColor: '#f8fafc', flexShrink: 0
    },
    body: {
        padding: '24px',
        overflowY: 'auto' as const,
        flexGrow: 1
    },
    footer: {
        padding: '16px 24px', borderTop: '1px solid #e5e7eb',
        backgroundColor: '#f8fafc', display: 'flex', justifyContent: 'flex-end',
        gap: '12px', flexShrink: 0
    },
    title: { fontSize: '1.15rem', fontWeight: 700, color: '#1e293b', display: 'flex', alignItems: 'center', gap: '10px' },
    closeBtn: { background: 'none', border: 'none', cursor: 'pointer', color: '#94a3b8', fontSize: '1.2rem', display: 'flex', alignItems: 'center' },
    sectionTitle: { fontSize: '0.75rem', fontWeight: 700, color: '#64748b', textTransform: 'uppercase' as const, letterSpacing: '0.05em', marginBottom:
    inputGroup: { marginBottom: '16px' },
    label: { display: 'block', marginBottom: '6px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' },
    inputWrapper: { position: 'relative' as const },
    icon: { position: 'absolute' as const, left: '12px', top: '50%', transform: 'translateY(-50%)', color: '#9ca3af', pointerEvents: 'none' as const, fo
    input: { width: '100%', padding: '10px 12px 10px 38px', borderRadius: '8px', border: '1px solid #cbd5e1', fontSize: '0.9rem', color: '#1e293b', back
    row: {
        display: 'grid',
        gridTemplateColumns: window.innerWidth <= 480 ? '1fr' : '1fr 1fr',
        gap: '16px'
    },
    btnSecondary: { padding: '10px 20px', borderRadius: '8px', border: '1px solid #cbd5e1', backgroundColor: '#fff', color: '#475569', fontSize: '0.9rem
    btnPrimary: { padding: '10px 24px', borderRadius: '8px', border: 'none', backgroundColor: '#2563eb', color: '#fff', fontSize: '0.9rem', fontWeight:
};

const InputField = ({ label, icon: Icon, value, onChange, placeholder, required = false, type = "text", disabled = false }: any) => (
    <div style={styles.inputGroup}>
        <label style={styles.label}>{label} {required && <span style={{color: '#ef4444'}}>*</span>}</label>
        <div style={styles.inputWrapper}>
            <div style={styles.icon}><Icon /></div>
            <input type={type} required={required} disabled={disabled} style={{...styles.input, opacity: disabled ? 0.7 : 1, backgroundColor: disabled
        </div>
    </div>
);

const AddCustomerModal: React.FC<Props> = ({ onClose, onSuccess, customerToEdit }) => {
    const [formData, setFormData] = useState({
        username: '', nickname: '', email: '', phone: '', gstin: '',
        address_line1: '', city_pincode: '', state: '', state_code: '',
        bank_name: '', bank_account_no: '', bank_ifsc_code: '',
        opening_balance: 0,
        password: '' // ✅ Added password field
    });
    const [loading, setLoading] = useState(false);

    useEffect(() => {
        if (customerToEdit) {
            setFormData({
                username: customerToEdit.username || '', nickname: customerToEdit.nickname || '',
                email: customerToEdit.email || '', phone: customerToEdit.phone || '',
                gstin: customerToEdit.gstin || '', address_line1: customerToEdit.address_line1 || '',
                city_pincode: customerToEdit.city_pincode || '', state: customerToEdit.state || '',
                state_code: customerToEdit.state_code || '',
                bank_name: customerToEdit.bank_name || '',
                bank_account_no: customerToEdit.bank_account_no || '',
                bank_ifsc_code: customerToEdit.bank_ifsc_code || '',
                opening_balance: customerToEdit.initial_balance || 0,
                password: '' // Don't preload existing password hash
            });
        }
    }, [customerToEdit]);

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setLoading(true);
        try {
            if (customerToEdit) {
                await updateCustomer(customerToEdit.id, formData);
                alert("Success! Customer profile updated.");
            } else {
                await createCustomer(formData);
                alert(`Success! Customer created.\n\nPortal Login:\nUser: ${formData.username}\nPass: ${formData.password || '(Not set)'}`);
            }
            onSuccess();
```

```
            onClose();
        } catch (err: any) {
            alert(err.message || "Operation failed");
        } finally {
            setLoading(false);
        }
    };

    return (
        <div style={styles.overlay}>
            <div style={styles.modal}>
                <div style={styles.header}>
                    <h2 style={styles.title}>
                        <div style={{background:'#eff6ff', padding:'8px', borderRadius:'6px', color:'#2563eb', display:'flex'}}><FaUser /></div>
                        {customerToEdit ? 'Edit Customer' : 'Add New Customer'}
                    </h2>
                    <button onClick={onClose} style={styles.closeBtn}><FaTimes /></button>
                </div>

                <div style={styles.body}>
                    <form id="customer-form" onSubmit={handleSubmit}>

                        {/* 1. BUSINESS NAME */}
                        <div style={{...styles.sectionTitle, marginTop: 0}}>Business Details</div>
                        <InputField label="Customer / Business Name" icon={FaBuilding} value={formData.username} onChange={(e: any) => setFormData({...

                        {/* 2. PORTAL LOGIN (✅ NEW GREEN BOX) */}
                        <div style={{background: '#f0fdf4', padding: '15px', borderRadius: '8px', border: '1px solid #bbf7d0', marginBottom: '20px', ma
                            <div style={{fontSize: '0.85rem', fontWeight: 700, color: '#166534', marginBottom: '10px', display: 'flex', alignItems: 'cen
                                <FaLock /> Customer Portal Access
                            </div>
                            <div style={styles.row}>
                                {/* Read-only username for context */}
                                <InputField label="Login Username" icon={FaUser} value={formData.username} disabled={true} placeholder="Auto-filled" />

                                {/* Password Field */}
                                <InputField
                                    label="Set/Reset Password"
                                    icon={FaLock}
                                    type="text"
                                    value={formData.password}
                                    onChange={(e: any) => setFormData({...formData, password: e.target.value})}
                                    placeholder="Enter password"
                                />
                            </div>
                            <p style={{fontSize: '0.75rem', color: '#166534', margin: 0, marginTop: '5px'}}>
                                * The customer can use these credentials to log in to the portal.
                            </p>
                        </div>

                        {/* 3. OTHER DETAILS */}
                        <div style={styles.row}>
                            <InputField label="Nickname / Alias" icon={FaTag} value={formData.nickname} onChange={(e: any) => setFormData({...formData,
                            <InputField label="Phone" icon={FaPhone} value={formData.phone} onChange={(e: any) => setFormData({...formData, phone: e.ta
                        </div>
                        <div style={styles.row}>
                            <InputField label="GSTIN" icon={FaIdCard} value={formData.gstin} onChange={(e: any) => setFormData({...formData, gstin: e.ta
                            <InputField label="Email" icon={FaEnvelope} value={formData.email} onChange={(e: any) => setFormData({...formData, email: e
                        </div>

                        {/* 4. BANK */}
                        <div style={styles.sectionTitle}>Bank Account Details</div>
                        <InputField label="Bank Name" icon={FaMoneyCheckAlt} value={formData.bank_name} onChange={(e: any) => setFormData({...formData,
                        <div style={styles.row}>
                            <InputField label="Account Number" icon={FaTag} value={formData.bank_account_no} onChange={(e: any) => setFormData({...formD
                            <InputField label="IFSC Code" icon={FaTag} value={formData.bank_ifsc_code} onChange={(e: any) => setFormData({...formData, b
                        </div>

                        {/* 5. LOCATION */}
                        <div style={styles.sectionTitle}>Location</div>
                        <InputField label="Address Line 1" icon={FaMapMarkerAlt} value={formData.address_line1} onChange={(e: any) => setFormData({...fo
                        <div style={styles.row}>
                            <InputField label="City - Pincode" icon={FaMapMarkerAlt} value={formData.city_pincode} onChange={(e: any) => setFormData({.
                            <InputField label="State" icon={FaMapMarkerAlt} value={formData.state} onChange={(e: any) => setFormData({...formData, state
                        </div>

                        {/* 6. OPENING BALANCE */}
                        <div style={{marginTop: '10px', background: '#f0f9ff', padding: '15px', borderRadius: '8px', border: '1px solid #bae6fd'}}>
                            <label style={{...styles.label, color: '#0369a1', marginBottom: '8px'}}>Opening Balance (Receivable)</label>
                            <div style={{display: 'flex', alignItems: 'center'}}>
                                <span style={{fontSize: '1.2rem', color: '#0369a1', fontWeight: 'bold', marginRight: '10px'}}>₹</span>
                                <input type="number" disabled={!!customerToEdit} style={{...styles.input, paddingLeft: '12px', textAlign: 'right', fontW
                            </div>
                        </div>
                    </form>
                </div>

                <div style={styles.footer}>
                    <button onClick={onClose} style={styles.btnSecondary} type="button">Cancel</button>
                    <button form="customer-form" type="submit" disabled={loading} style={{...styles.btnPrimary, opacity: loading ? 0.7 : 1}}>
                        {loading ? 'Saving...' : <><FaSave /> {customerToEdit ? 'Update Customer' : 'Save Customer'}</>}
                    </button>
                </div>
            </div>
        </div>
    );
};

export default AddCustomerModal;
```

frontend/src/pages/AIInsights.tsx

```
// frontend/src/pages/AIInsights.tsx
import { AnimatePresence, motion } from 'framer-motion';
import React from 'react';
import { FaBolt, FaBrain, FaChartLine, FaRobot, FaSortAmountDown, FaWarehouse } from 'react-icons/fa';
import { Area, AreaChart, CartesianGrid, ResponsiveContainer, Tooltip, XAxis, YAxis } from 'recharts';

const MOCK_FORECAST = [
    { name: 'Mon', sales: 4000, forecast: 4200 },
    { name: 'Tue', sales: 3000, forecast: 3300 },
    { name: 'Wed', sales: 2000, forecast: 2500 },
    { name: 'Thu', sales: 2780, forecast: 3100 },
    { name: 'Fri', sales: 1890, forecast: 2200 },
    { name: 'Sat', sales: 2390, forecast: 2800 },
    { name: 'Sun', sales: 3490, forecast: 4000 },
];

const AIInsights: React.FC = () => {
    return (
        <div style={{ display: 'flex', flexDirection: 'column', gap: '32px' }}>
            {/* AI Core Banner */}
            <motion.div
                initial={{ opacity: 0, scale: 0.95 }}
                animate={{ opacity: 1, scale: 1 }}
                style={{
                    background: 'linear-gradient(135deg, #6366f1 0%, #4338ca 100%)',
                    borderRadius: 'var(--radius-xl)',
                    padding: '48px',
                    color: 'white',
                    position: 'relative',
                    overflow: 'hidden',
                    boxShadow: 'var(--shadow-float)'
                }}
            >
                <div style={{ position: 'relative', zIndex: 2, maxWidth: '700px' }}>
                    <div style={{ display: 'flex', alignItems: 'center', gap: '12px', marginBottom: '24px' }}>
                        <div style={{ padding: '12px', background: 'rgba(255,255,255,0.1)', borderRadius: '16px', backdropFilter: 'blur(10px)' }}>
                            <FaBrain size={32} />
                        </div>
                        <span style={{ fontSize: '0.7rem', fontWeight: 900, textTransform: 'uppercase', letterSpacing: '0.2em', opacity: 0.8 }}>Neural I
                    </div>
                    <h1 style={{ fontSize: '3.5rem', fontWeight: 950, letterSpacing: '-0.04em', lineHeight: 1.1, marginBottom: '20px' }}>
                        Predictive Intelligence <br/> & <span style={{ color: '#fbbf24' }}>Branch Analytics</span>
                    </h1>
                    <p style={{ fontSize: '1.1rem', fontWeight: 500, opacity: 0.9, lineHeight: 1.6 }}>
                        Harness the power of our multi-agent neural network to forecast demand, identify anomalies, and optimize your global logistics o
                    </p>
                </div>

                {/* Cybernetic Background Detail */}
                <div style={{ position: 'absolute', right: '-5%', bottom: '-20%', opacity: 0.1 }}>
                    <FaRobot size={400} />
                </div>
            </motion.div>

            {/* Neural KPI Grid */}
            <div className="kpi-grid">
                <InsightCard title="Global Forecast" value="+14% ↑" label="30 Day Projection" icon={<FaChartLine />} color="#6366f1" delay={0.1} />
                <InsightCard title="Stock Risk" value="Minimal" label="Anomaly Detection" icon={<FaWarehouse />} color="#10b981" delay={0.2} />
                <InsightCard title="Agent Confidence" value="98.2%" label="Neural Accuracy" icon={<FaBolt />} color="#f59e0b" delay={0.3} />
                <InsightCard title="Active Agents" value="12" label="Running Sub-tasks" icon={<FaSortAmountDown />} color="#8b5cf6" delay={0.4} />
            </div>

            {/* Analytics & Strategies */}
            <div style={{ display: 'grid', gridTemplateColumns: '2fr 1fr', gap: '32px' }}>
                <motion.div
                    initial={{ opacity: 0, y: 20 }}
                    animate={{ opacity: 1, y: 0 }}
                    transition={{ delay: 0.5 }}
                    className="enterprise-card"
                    style={{ padding: '32px' }}
                >
                    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '32px' }}>
                        <div>
                            <h3 style={{ fontSize: '1.25rem', fontWeight: 800 }}>Neural Sales Trajectory</h3>
                            <p style={{ color: 'var(--text-muted)', fontSize: '0.85rem' }}>Comparing historical flow vs AI forecasted momentum</p>
                        </div>
                        <div style={{ display: 'flex', gap: '8px' }}>
                            <span style={{ display: 'flex', alignItems: 'center', gap: '6px', fontSize: '0.75rem', fontWeight: 700 }}><div style={{width
                            <span style={{ display: 'flex', alignItems: 'center', gap: '6px', fontSize: '0.75rem', fontWeight: 700 }}><div style={{width
                        </div>
                    </div>

                    <div style={{ height: '350px' }}>
                        <ResponsiveContainer width="100%" height="100%">
                            <AreaChart data={MOCK_FORECAST}>
                                <defs>
                                    <linearGradient id="colorSales" x1="0" y1="0" x2="0" y2="1">
                                        <stop offset="5%" stopColor="#6366f1" stopOpacity={0.2}/>
                                        <stop offset="95%" stopColor="#6366f1" stopOpacity={0}/>
                                    </linearGradient>
                                </defs>
                                <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#f1f5f9" />
                                <XAxis dataKey="name" axisLine={false} tickLine={false} tick={{fill: '#94a3b8', fontSize: 11, fontWeight: 700}} dy={10}
                                <YAxis axisLine={false} tickLine={false} tick={{fill: '#94a3b8', fontSize: 11, fontWeight: 700}} />
                                <Tooltip contentStyle={{ borderRadius: '16px', border: 'none', boxShadow: 'var(--shadow-premium)', fontWeight: 800 }} />
                                <Area type="monotone" dataKey="sales" stroke="#6366f1" strokeWidth={3} fillOpacity={1} fill="url(#colorSales)" />
                                <Area type="monotone" dataKey="forecast" stroke="#8b5cf6" strokeDasharray="5 5" strokeWidth={2} fill="transparent" />
                            </AreaChart>
                        </ResponsiveContainer>
                    </div>
                </motion.div>

                {/* AI Recommendations */}
                <div style={{ display: 'flex', flexDirection: 'column', gap: '20px' }}>
                    <h3 style={{ fontSize: '1.25rem', fontWeight: 900, marginBottom: '8px' }}>Neural Strategies</h3>
                    <AnimatePresence>
```

```
                        <StrategyItem
                            title="Branch Optimization"
                            body="Neural model suggests shifting 45 units of Copper Slag from HQ to South Branch to meet predicted weekend demand."
                            level="high"
                            delay={0.6}
                        />
                        <StrategyItem
                            title="Revenue Accelerator"
                            body="Customer 'Acme Inc' has a 92% match for seasonal re-ordering. Recommend triggering proactive quote agent."
                            level="medium"
                            delay={0.7}
                        />
                        <StrategyItem
                            title="Cost Reduction"
                            body="Transport lanes between Branch A and B are under-capacity. Consolidation suggested to save ₹12,400."
                            level="low"
                            delay={0.8}
                        />
                    </AnimatePresence>
                </div>
            </div>
        </div>
    );
};

const InsightCard: React.FC<any> = ({ title, value, label, icon, color, delay }) => (
    <motion.div
        initial={{ opacity: 0, y: 20 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ delay }}
        className="enterprise-card glass"
        style={{ padding: '24px' }}
    >
        <div style={{ padding: '12px', background: `${color}15`, color: color, width: 'fit-content', borderRadius: '12px', marginBottom: '20px' }}>
            {icon}
        </div>
        <p style={{ fontSize: '0.7rem', fontWeight: 900, color: 'var(--text-muted)', textTransform: 'uppercase', letterSpacing: '0.05em' }}>{title}</p>
        <h3 style={{ fontSize: '2rem', fontWeight: 950, color: 'var(--text-primary)', margin: '4px 0' }}>{value}</h3>
        <p style={{ fontSize: '0.75rem', fontWeight: 700, color: 'var(--text-secondary)' }}>{label}</p>
    </motion.div>
);

const StrategyItem: React.FC<any> = ({ title, body, level, delay }) => {
    const color = level === 'high' ? 'var(--error)' : (level === 'medium' ? 'var(--warning)' : 'var(--primary)');
    return (
        <motion.div
            initial={{ opacity: 0, x: 20 }}
            animate={{ opacity: 1, x: 0 }}
            transition={{ delay }}
            className="enterprise-card"
            style={{
                borderLeft: `5px solid ${color}`,
                padding: '20px',
                background: `${color}05`
            }}
        >
            <h4 style={{ fontSize: '0.8rem', fontWeight: 900, color: color, textTransform: 'uppercase', letterSpacing: '0.1em', marginBottom: '8px' }}>
            <p style={{ fontSize: '0.85rem', fontWeight: 600, color: 'var(--text-secondary)', lineHeight: 1.5 }}>{body}</p>
            <button className="clickable" style={{ marginTop: '12px', fontSize: '0.7rem', fontWeight: 800, color: 'var(--text-primary)', border: 'none'
                Execute Neural Link →
            </button>
        </motion.div>
    );
}

export default AIInsights;
```

frontend/src/pages/hr/PayrollRun.tsx

```
// frontend/src/pages/hr/PayrollRun.tsx
import React, { useEffect, useState } from 'react';
import {
    FaCalculator,
    FaCalendarAlt,
    FaCalendarWeek,
    FaCheck,
    FaChevronDown,
    FaChevronUp,
    FaHistory,
    FaMoneyBillWave,
    FaSearch,
    FaUserClock,
    FaUsers
} from 'react-icons/fa';

interface PayrollItem {
    employee: {
        id: number;
        name: string;
        designation?: string;
        salary: number;
        salary_type: string;
    };
    attendance: {
        working_days: number;
        half_days: number;
        leave_days: number;
        effective_days: number;
    };
    days_present: number;
    gross: number;
    deduction: number;
    net_pay: number;
    advance_id?: number;
    original_deduction: number;
    skipped: boolean;
}

interface PayrollHistory {
    id: number;
    employee_name: string;
    month_year: string;
    gross_earnings: number;
    total_deductions: number;
    net_pay: number;
    status: string;
    generated_at: string;
    salary_type?: string;
}

const PayrollRun: React.FC = () => {
    const [payrollData, setPayrollData] = useState<PayrollItem[]>([]);
    const [payrollHistory, setPayrollHistory] = useState<PayrollHistory[]>([]);
    const [selectedMonth, setSelectedMonth] = useState(() => {
        const now = new Date();
        return `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, '0')}`;
    });
    const [weekStart, setWeekStart] = useState(() => {
        const today = new Date();
        const day = today.getDay();
        const diff = today.getDate() - day + (day === 0 ? -6 : 1);
        return new Date(today.setDate(diff)).toISOString().split('T')[0];
    });
    const [loading, setLoading] = useState(false);
    const [processing, setProcessing] = useState(false);
    const [showHistory, setShowHistory] = useState(false);
    const [activeTab, setActiveTab] = useState<'MONTHLY' | 'WEEKLY' | 'DAILY'>('MONTHLY');
    const [searchTerm, setSearchTerm] = useState('');

    const API_BASE = '/api';

    useEffect(() => {
        fetchPayrollHistory();
    }, []);

    const fetchPayrollHistory = async () => {
        try {
            const token = localStorage.getItem('erp-token');
            const res = await fetch(`${API_BASE}/hr/payroll/history`, {
                headers: { 'Authorization': `Bearer ${token}` }
            });
            const data = await res.json();
            setPayrollHistory(data || []);
        } catch (err) {
            console.error('Error fetching payroll history:', err);
        }
    };

    const generatePayroll = async () => {
        setLoading(true);
        try {
            const token = localStorage.getItem('erp-token');
            const res = await fetch(`${API_BASE}/hr/payroll/preview`, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'Authorization': `Bearer ${token}`
                },
                body: JSON.stringify({ month_year: selectedMonth })
            });
            const data = await res.json();
            setPayrollData(data || []);
        } catch (err) {
            console.error('Error generating payroll:', err);
            alert('Failed to generate payroll preview');
```

```
    } finally {
        setLoading(false);
    }
};

const updateDeduction = (empId: number, value: number) => {
    setPayrollData(prev => prev.map(item => {
        if (item.employee.id === empId) {
            const newDeduction = Math.min(Math.max(0, value), item.original_deduction);
            return { ...item, deduction: newDeduction, net_pay: item.gross - newDeduction };
        }
        return item;
    }));
};

const toggleSkip = (empId: number) => {
    setPayrollData(prev => prev.map(item =>
        item.employee.id === empId ? { ...item, skipped: !item.skipped } : item
    ));
};

const processPayroll = async (salaryType: string) => {
    const itemsToProcess = payrollData.filter(p =>
        !p.skipped && (p.employee.salary_type || 'Monthly') === salaryType
    );

    if (itemsToProcess.length === 0) {
        alert('No employees to process');
        return;
    }

    const total = itemsToProcess.reduce((sum, p) => sum + p.net_pay, 0);
    if (!confirm(`Process ${salaryType} payroll for ${itemsToProcess.length} employees?\n\nTotal: ₹${total.toLocaleString()}`)) return;

    setProcessing(true);
    try {
        const token = localStorage.getItem('erp-token');
        const res = await fetch(`${API_BASE}/hr/payroll/finalize`, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${token}`
            },
            body: JSON.stringify({
                month_year: selectedMonth,
                payroll_data: itemsToProcess,
                salary_type: salaryType
            })
        });

        if (res.ok) {
            alert(`✅ ${salaryType} Payroll processed successfully!`);
            setPayrollData(prev => prev.filter(p =>
                p.skipped || (p.employee.salary_type || 'Monthly') !== salaryType
            ));
            fetchPayrollHistory();
        } else {
            throw new Error('Failed to process');
        }
    } catch (err) {
        console.error('Error processing payroll:', err);
        alert('Failed to process payroll');
    } finally {
        setProcessing(false);
    }
};

// Filter by salary type
const monthlyPayroll = payrollData.filter(p => (p.employee.salary_type || 'Monthly') === 'Monthly');
const weeklyPayroll = payrollData.filter(p => p.employee.salary_type === 'Weekly');
const dailyPayroll = payrollData.filter(p => p.employee.salary_type === 'Daily');

const getCurrentPayroll = () => {
    let base = [];
    switch (activeTab) {
        case 'MONTHLY': base = monthlyPayroll; break;
        case 'WEEKLY': base = weeklyPayroll; break;
        case 'DAILY': base = dailyPayroll; break;
    }

    if (!searchTerm.trim()) return base;

    return base.filter(p =>
        p.employee.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
        p.employee.designation?.toLowerCase().includes(searchTerm.toLowerCase())
    );
};

const currentPayroll = getCurrentPayroll();
const currentActivePayroll = currentPayroll.filter(p => !p.skipped);
const currentTotalGross = currentActivePayroll.reduce((sum, p) => sum + p.gross, 0);
const currentTotalDeductions = currentActivePayroll.reduce((sum, p) => sum + p.deduction, 0);
const currentTotalNet = currentActivePayroll.reduce((sum, p) => sum + p.net_pay, 0);

// Overall totals
const allActivePayroll = payrollData.filter(p => !p.skipped);
const overallTotalNet = allActivePayroll.reduce((sum, p) => sum + p.net_pay, 0);

const cardStyle: React.CSSProperties = {
    background: 'white',
    borderRadius: '12px',
    padding: '24px',
    boxShadow: '0 1px 3px rgba(0,0,0,0.1)'
};

const tabConfig = {
    MONTHLY: { icon: <FaUsers />, label: 'Monthly Staff', count: monthlyPayroll.length, color: '#3b82f6', bg: '#eff6ff' },
    WEEKLY: { icon: <FaCalendarWeek />, label: 'Weekly Workers', count: weeklyPayroll.length, color: '#8b5cf6', bg: '#f5f3ff' },
    DAILY: { icon: <FaUserClock />, label: 'Daily Workers', count: dailyPayroll.length, color: '#10b981', bg: '#ecfdf5' }
};
```

```
return (
    <div>
        {/* Header */}
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'flex-start', marginBottom: '24px' }}>
            <div>
                <h1 style={{ margin: 0, fontSize: '1.75rem', color: '#1e293b' }}>Payroll Management</h1>
                <p style={{ margin: '4px 0 0', color: '#64748b' }}>Calculate and process employee salaries</p>
            </div>
            <button
                onClick={() => setShowHistory(!showHistory)}
                style={{
                    display: 'flex',
                    alignItems: 'center',
                    gap: '8px',
                    padding: '10px 20px',
                    border: '1px solid #e2e8f0',
                    borderRadius: '8px',
                    background: 'white',
                    cursor: 'pointer',
                    fontWeight: 500
                }}
            >
                <FaHistory /> Payroll History {showHistory ? <FaChevronUp /> : <FaChevronDown />}
            </button>
        </div>

        {/* History Panel */}
        {showHistory && (
            <div style={{ ...cardStyle, marginBottom: '24px' }}>
                <h3 style={{ margin: '0 0 16px', color: '#1e293b' }}>Recent Payroll Runs</h3>
                {payrollHistory.length > 0 ? (
                    <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: '0.9rem' }}>
                        <thead>
                            <tr style={{ borderBottom: '2px solid #e2e8f0' }}>
                                <th style={{ padding: '12px', textAlign: 'left', color: '#64748b' }}>Period</th>
                                <th style={{ padding: '12px', textAlign: 'left', color: '#64748b' }}>Employee</th>
                                <th style={{ padding: '12px', textAlign: 'center', color: '#64748b' }}>Type</th>
                                <th style={{ padding: '12px', textAlign: 'right', color: '#64748b' }}>Gross</th>
                                <th style={{ padding: '12px', textAlign: 'right', color: '#64748b' }}>Deductions</th>
                                <th style={{ padding: '12px', textAlign: 'right', color: '#64748b' }}>Net Pay</th>
                            </tr>
                        </thead>
                        <tbody>
                            {payrollHistory.slice(0, 15).map((record, i) => (
                                <tr key={i} style={{ borderBottom: '1px solid #f1f5f9' }}>
                                    <td style={{ padding: '12px' }}>{record.month_year}</td>
                                    <td style={{ padding: '12px', fontWeight: 500 }}>{record.employee_name}</td>
                                    <td style={{ padding: '12px', textAlign: 'center' }}>
                                        <span style={{
                                            padding: '2px 8px',
                                            borderRadius: '4px',
                                            fontSize: '0.75rem',
                                            background: record.salary_type === 'Daily' ? '#dcfce7' : record.salary_type === 'Weekly' ? '#f5f3ff' :
                                            color: record.salary_type === 'Daily' ? '#166534' : record.salary_type === 'Weekly' ? '#6d28d9' : '#433
                                        }}>
                                            {record.salary_type || 'Monthly'}
                                        </span>
                                    </td>
                                    <td style={{ padding: '12px', textAlign: 'right' }}>₹{Number(record.gross_earnings || 0).toLocaleString()}</td>
                                    <td style={{ padding: '12px', textAlign: 'right', color: '#dc2626' }}>₹{Number(record.total_deductions || 0).toL
                                    <td style={{ padding: '12px', textAlign: 'right', fontWeight: 600, color: '#16a34a' }}>₹{Number(record.net_pay
                                </tr>
                            ))}
                        </tbody>
                    </table>
                ) : (
                    <p style={{ color: '#64748b', textAlign: 'center', padding: '20px' }}>No payroll history found</p>
                )}
            </div>
        )}

        {/* Period Selection & Generate */}
        <div style={{ ...cardStyle, marginBottom: '24px' }}>
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', flexWrap: 'wrap', gap: '16px' }}>
                <div style={{ display: 'flex', alignItems: 'center', gap: '24px', flexWrap: 'wrap' }}>
                    <div style={{ display: 'flex', alignItems: 'center', gap: '12px' }}>
                        <FaCalendarAlt color="#3b82f6" size={20} />
                        <div>
                            <label style={{ display: 'block', fontSize: '0.8rem', color: '#64748b', marginBottom: '4px' }}>Select Month</label>
                            <input
                                type="month"
                                value={selectedMonth}
                                onChange={(e) => setSelectedMonth(e.target.value)}
                                style={{
                                    padding: '10px 16px',
                                    border: '2px solid #e2e8f0',
                                    borderRadius: '8px',
                                    fontSize: '1rem',
                                    fontWeight: 600
                                }}
                            />
                        </div>
                    </div>
                    <div style={{ display: 'flex', alignItems: 'center', gap: '12px' }}>
                        <FaCalendarWeek color="#8b5cf6" size={20} />
                        <div>
                            <label style={{ display: 'block', fontSize: '0.8rem', color: '#64748b', marginBottom: '4px' }}>Week Start (for Weekly)</
                            <input
                                type="date"
                                value={weekStart}
                                onChange={(e) => setWeekStart(e.target.value)}
                                style={{
                                    padding: '10px 16px',
                                    border: '2px solid #e2e8f0',
                                    borderRadius: '8px',
                                    fontSize: '1rem',
                                    fontWeight: 600
                                }}
                            />
                        </div>
                    </div>
                </div>
```

```
        <button
            onClick={generatePayroll}
            disabled={loading}
            style={{
                display: 'flex',
                alignItems: 'center',
                gap: '8px',
                padding: '12px 24px',
                border: 'none',
                borderRadius: '8px',
                background: '#3b82f6',
                color: 'white',
                cursor: loading ? 'not-allowed' : 'pointer',
                fontWeight: 600,
                fontSize: '1rem'
            }}
        >
            <FaCalculator /> {loading ? 'Calculating...' : 'Generate Payroll'}
        </button>
    </div>
</div>

{/* Payroll Data */}
{payrollData.length > 0 && (
    <>
        {/* Overall Summary */}
        <div style={{
            ...cardStyle,
            marginBottom: '24px',
            background: 'linear-gradient(135deg, #667eea 0%, #764ba2 100%)',
            color: 'white'
        }}>
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', flexWrap: 'wrap', gap: '20px' }}>
                <div>
                    <p style={{ margin: 0, opacity: 0.9, fontSize: '0.9rem' }}>Total Payroll Amount</p>
                    <h1 style={{ margin: '8px 0 0', fontSize: '2.5rem' }}>₹{overallTotalNet.toLocaleString()}</h1>
                </div>
                <div style={{ display: 'flex', gap: '30px' }}>
                    <div style={{ textAlign: 'center' }}>
                        <p style={{ margin: 0, opacity: 0.8, fontSize: '0.85rem' }}>Monthly</p>
                        <p style={{ margin: '4px 0 0', fontSize: '1.25rem', fontWeight: 700 }}>
                            ₹{monthlyPayroll.filter(p => !p.skipped).reduce((s, p) => s + p.net_pay, 0).toLocaleString()}
                        </p>
                    </div>
                    <div style={{ textAlign: 'center' }}>
                        <p style={{ margin: 0, opacity: 0.8, fontSize: '0.85rem' }}>Weekly</p>
                        <p style={{ margin: '4px 0 0', fontSize: '1.25rem', fontWeight: 700 }}>
                            ₹{weeklyPayroll.filter(p => !p.skipped).reduce((s, p) => s + p.net_pay, 0).toLocaleString()}
                        </p>
                    </div>
                    <div style={{ textAlign: 'center' }}>
                        <p style={{ margin: 0, opacity: 0.8, fontSize: '0.85rem' }}>Daily</p>
                        <p style={{ margin: '4px 0 0', fontSize: '1.25rem', fontWeight: 700 }}>
                            ₹{dailyPayroll.filter(p => !p.skipped).reduce((s, p) => s + p.net_pay, 0).toLocaleString()}
                        </p>
                    </div>
                </div>
            </div>
        </div>

        {/* Tabs */}
        <div style={{ display: 'flex', gap: '12px', marginBottom: '20px' }}>
            {(['MONTHLY', 'WEEKLY', 'DAILY'] as const).map(tab => {
                const config = tabConfig[tab];
                const isActive = activeTab === tab;
                return (
                    <button
                        key={tab}
                        onClick={() => setActiveTab(tab)}
                        style={{
                            display: 'flex',
                            alignItems: 'center',
                            gap: '8px',
                            padding: '12px 20px',
                            border: isActive ? `2px solid ${config.color}` : '2px solid transparent',
                            borderRadius: '10px',
                            background: isActive ? config.bg : '#f8fafc',
                            color: isActive ? config.color : '#64748b',
                            cursor: 'pointer',
                            fontWeight: 600,
                            transition: 'all 0.2s'
                        }}
                    >
                        {config.icon} {config.label} ({config.count})
                    </button>
                );
            })}
        </div>

        {/* Current Tab Summary */}
        <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(200px, 1fr))', gap: '20px', marginBottom: '32px' }}>
            <div style={{ ...cardStyle, borderLeft: `5px solid ${tabConfig[activeTab].color}`, padding: '24px' }}>
                <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
                    <div>
                        <p style={{ margin: 0, color: '#64748b', fontSize: '0.85rem', fontWeight: 600, textTransform: 'uppercase', letterSpa
                        <h2 style={{ margin: '12px 0 0', color: '#1e293b', fontSize: '2rem', fontWeight: 800 }}>{currentActivePayroll.lengt
                    </div>
                    <div style={{ width: '48px', height: '48px', borderRadius: '14px', background: `${tabConfig[activeTab].bg}`, display: '
                        <FaUsers size={24} />
                    </div>
                </div>
            </div>
            <div style={{ ...cardStyle, borderLeft: '5px solid #10b981', padding: '24px' }}>
                <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
                    <div>
                        <p style={{ margin: 0, color: '#64748b', fontSize: '0.85rem', fontWeight: 600, textTransform: 'uppercase', letterSpa
                        <h2 style={{ margin: '12px 0 0', color: '#10b981', fontSize: '2rem', fontWeight: 800 }}>₹{currentTotalGross.toLocale
                    </div>
                    <div style={{ width: '48px', height: '48px', borderRadius: '14px', background: '#ecfdf5', display: 'flex', alignItems:
                        <FaMoneyBillWave size={24} />
                    </div>
                </div>
```

```
    </div>
    <div style={{ ...cardStyle, borderLeft: '5px solid #ef4444', padding: '24px' }}>
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
            <div>
                <p style={{ margin: 0, color: '#64748b', fontSize: '0.85rem', fontWeight: 600, textTransform: 'uppercase', letterSpa
                <h2 style={{ margin: '12px 0 0', color: '#ef4444', fontSize: '2rem', fontWeight: 800 }}>₹{currentTotalDeductions.toL
            </div>
            <div style={{ width: '48px', height: '48px', borderRadius: '14px', background: '#fef2f2', display: 'flex', alignItems:
                <FaCalculator size={24} />
            </div>
        </div>
    </div>
    <div style={{ ...cardStyle, borderLeft: '5px solid #8b5cf6', padding: '24px' }}>
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
            <div>
                <p style={{ margin: 0, color: '#64748b', fontSize: '0.85rem', fontWeight: 600, textTransform: 'uppercase', letterSpa
                <h2 style={{ margin: '12px 0 0', color: '#8b5cf6', fontSize: '2rem', fontWeight: 800 }}>₹{currentTotalNet.toLocaleSt
            </div>
            <div style={{ width: '48px', height: '48px', borderRadius: '14px', background: '#f5f3ff', display: 'flex', alignItems:
                <FaUserClock size={24} />
            </div>
        </div>
    </div>
</div>

{/* Search Toolbar */}
<div className="card" style={{
    padding: '0 24px',
    marginBottom: '32px',
    display: 'flex',
    alignItems: 'center',
    gap: '16px',
    border: '1px solid var(--border-color)',
    height: '64px',
    background: 'white',
    boxShadow: 'var(--shadow-sm)',
    borderRadius: '18px'
}}>
    <FaSearch style={{ color: 'var(--text-light)' }} size={20} />
    <input
        placeholder="Search employees in current tab..."
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
        style={{
            border: 'none',
            width: '100%',
            outline: 'none',
            background: 'transparent',
            fontSize: '1.05rem',
            fontWeight: 500,
            color: 'var(--text-main)',
            letterSpacing: '-0.2px'
        }}
    />
</div>

{/* Payroll Table */}
<div style={cardStyle}>
    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '16px' }}>
        <h3 style={{ margin: 0, color: '#1e293b' }}>{tabConfig[activeTab].label} Payroll</h3>
        <span style={{ fontSize: '0.85rem', color: '#64748b' }}>
            {activeTab === 'WEEKLY' ? `Week: ${weekStart}` : `Period: ${selectedMonth}`}
        </span>
    </div>

    {currentPayroll.length > 0 ? (
        <div style={{ overflowX: 'auto', WebkitOverflowScrolling: 'touch' }}>
        <table style={{ width: '100%', borderCollapse: 'collapse', minWidth: '800px' }}>
            <thead>
                <tr style={{ borderBottom: '2px solid #e2e8f0' }}>
                    <th style={{ padding: '14px', textAlign: 'left', color: '#64748b' }}>Employee</th>
                    <th style={{ padding: '14px', textAlign: 'center', color: '#64748b' }}>Days Present</th>
                    <th style={{ padding: '14px', textAlign: 'right', color: '#64748b' }}>Base Salary</th>
                    <th style={{ padding: '14px', textAlign: 'right', color: '#64748b' }}>Gross Earnings</th>
                    <th style={{ padding: '14px', textAlign: 'center', color: '#64748b' }}>Advance Deduction</th>
                    <th style={{ padding: '14px', textAlign: 'right', color: '#64748b' }}>Net Pay</th>
                    <th style={{ padding: '14px', textAlign: 'center', color: '#64748b' }}>Action</th>
                </tr>
            </thead>
            <tbody>
                {currentPayroll.map((item) => (
                    <tr
                        key={item.employee.id}
                        style={{
                            borderBottom: '1px solid #f1f5f9',
                            opacity: item.skipped ? 0.5 : 1,
                            background: item.skipped ? '#fafafa' : 'white'
                        }}
                    >
                        <td style={{ padding: '16px 14px' }}>
                            <div style={{ display: 'flex', alignItems: 'center', gap: '12px' }}>
                                <div style={{
                                    width: '40px',
                                    height: '40px',
                                    borderRadius: '50%',
                                    background: tabConfig[activeTab].bg,
                                    display: 'flex',
                                    alignItems: 'center',
                                    justifyContent: 'center',
                                    color: tabConfig[activeTab].color,
                                    fontWeight: 600
                                }}>
                                    {item.employee.name.charAt(0)}
                                </div>
                                <div>
                                    <p style={{ margin: 0, fontWeight: 600, color: '#1e293b' }}>{item.employee.name}</p>
                                    <p style={{ margin: 0, fontSize: '0.8rem', color: '#64748b' }}>{item.employee.designation || 'St
                                </div>
                            </div>
                        </td>
                        <td style={{ padding: '16px 14px', textAlign: 'center' }}>
                            <span style={{
```

```
                                    padding: '4px 12px',
                                    borderRadius: '20px',
                                    background: item.days_present >= 26 ? '#dcfce7' : item.days_present >= 20 ? '#fef3c7' : '#fee2e2',
                                    color: item.days_present >= 26 ? '#16a34a' : item.days_present >= 20 ? '#d97706' : '#dc2626',
                                    fontWeight: 600,
                                    fontSize: '0.9rem'
                                }}>
                                    {item.days_present} days
                                </span>
                            </td>
                            <td style={{ padding: '16px 14px', textAlign: 'right', color: '#64748b' }}>
                                ₹{item.employee.salary?.toLocaleString()}
                                <span style={{ fontSize: '0.75rem' }}>
                                    {activeTab === 'DAILY' ? '/day' : activeTab === 'WEEKLY' ? '/wk' : '/mo'}
                                </span>
                            </td>
                            <td style={{ padding: '16px 14px', textAlign: 'right', fontWeight: 600 }}>
                                ₹{item.gross.toLocaleString()}
                            </td>
                            <td style={{ padding: '16px 14px', textAlign: 'center' }}>
                                {item.original_deduction > 0 ? (
                                    <input
                                        type="number"
                                        min="0"
                                        max={item.original_deduction}
                                        value={item.deduction}
                                        onChange={(e) => updateDeduction(item.employee.id, Number(e.target.value))}
                                        disabled={item.skipped}
                                        style={{
                                            width: '90px',
                                            padding: '8px',
                                            border: '1px solid #fca5a5',
                                            borderRadius: '6px',
                                            textAlign: 'center',
                                            color: '#dc2626',
                                            fontWeight: 500
                                        }}
                                    />
                                ) : (
                                    <span style={{ color: '#94a3b8' }}>—</span>
                                )}
                            </td>
                            <td style={{ padding: '16px 14px', textAlign: 'right' }}>
                                <span style={{ fontWeight: 700, fontSize: '1.1rem', color: '#16a34a' }}>
                                    ₹{item.net_pay.toLocaleString()}
                                </span>
                            </td>
                            <td style={{ padding: '16px 14px', textAlign: 'center' }}>
                                <button
                                    onClick={() => toggleSkip(item.employee.id)}
                                    style={{
                                        padding: '6px 12px',
                                        border: 'none',
                                        borderRadius: '6px',
                                        background: item.skipped ? '#fee2e2' : '#f1f5f9',
                                        color: item.skipped ? '#dc2626' : '#64748b',
                                        cursor: 'pointer',
                                        fontSize: '0.8rem',
                                        fontWeight: 500
                                    }}
                                >
                                    {item.skipped ? 'Skipped' : 'Skip'}
                                </button>
                            </td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
        ) : (
            <div style={{ textAlign: 'center', padding: '40px', color: '#64748b' }}>
                <p>No {tabConfig[activeTab].label.toLowerCase()} found</p>
                <p style={{ fontSize: '0.85rem', marginTop: '8px' }}>
                    Add employees with "{activeTab === 'MONTHLY' ? 'Monthly' : activeTab === 'WEEKLY' ? 'Weekly' : 'Daily'}" salary type
                </p>
            </div>
        )}
    </div>

    {/* Process Button */}
    {currentPayroll.length > 0 && (
        <div style={{
            ...cardStyle,
            marginTop: '24px',
            display: 'flex',
            justifyContent: 'space-between',
            alignItems: 'center'
        }}>
            <div>
                <p style={{ margin: 0, color: '#64748b' }}>
                    Processing payroll for <strong>{currentActivePayroll.length}</strong> {tabConfig[activeTab].label.toLowerCase()}
                </p>
                <p style={{ margin: '4px 0 0', fontSize: '0.9rem', color: '#94a3b8' }}>
                    {currentPayroll.filter(p => p.skipped).length} employees will be skipped
                </p>
            </div>
            <div style={{ display: 'flex', gap: '12px' }}>
                <button
                    onClick={() => setPayrollData([])}
                    style={{
                        padding: '12px 24px',
                        border: '1px solid #e2e8f0',
                        borderRadius: '8px',
                        background: 'white',
                        cursor: 'pointer',
                        fontWeight: 500
                    }}
                >
                    Cancel
                </button>
                <button
                    onClick={() => processPayroll(activeTab === 'MONTHLY' ? 'Monthly' : activeTab === 'WEEKLY' ? 'Weekly' : 'Daily')}
```

```
                                disabled={processing || currentActivePayroll.length === 0}
                                style={{
                                    display: 'flex',
                                    alignItems: 'center',
                                    gap: '8px',
                                    padding: '12px 32px',
                                    border: 'none',
                                    borderRadius: '8px',
                                    background: currentActivePayroll.length > 0 ? tabConfig[activeTab].color : '#94a3b8',
                                    color: 'white',
                                    cursor: currentActivePayroll.length > 0 ? 'pointer' : 'not-allowed',
                                    fontWeight: 600,
                                    fontSize: '1rem'
                                }}
                            >
                                <FaCheck /> {processing ? 'Processing...' : `Process ${tabConfig[activeTab].label} (₹${currentTotalNet.toLocaleStrin
                            </button>
                        </div>
                    </div>
                )}
            </>
        )}

        {/* Empty State */}
        {payrollData.length === 0 && !loading && (
            <div style={{ ...cardStyle, textAlign: 'center', padding: '60px' }}>
                <FaMoneyBillWave size={50} color="#94a3b8" />
                <h3 style={{ margin: '20px 0 8px', color: '#1e293b' }}>Generate Payroll</h3>
                <p style={{ margin: 0, color: '#64748b', maxWidth: '400px', marginLeft: 'auto', marginRight: 'auto' }}>
                    Select a period and click "Generate Payroll" to calculate salaries for Monthly, Weekly, and Daily workers
                </p>
            </div>
        )}
    </div>
  );
};

export default PayrollRun;
```

frontend/src/pages/hr/MobileAttendance.tsx

```
// frontend/src/pages/hr/MobileAttendance.tsx
// This page is accessed when employee scans QR code on their phone
// URL format: /mark-attendance?token=EMP_1_SECRET

import React, { useEffect, useState } from 'react';
import {
    FaBriefcase,
    FaCheckCircle,
    FaMapMarkerAlt,
    FaSpinner,
    FaTimesCircle,
    FaUserClock
} from 'react-icons/fa';
import { useSearchParams } from 'react-router-dom';

interface AttendanceResponse {
    success?: boolean;
    message?: string;
    error?: string;
    employee_name?: string;
    type?: string;
}

const MobileAttendance: React.FC = () => {
    const [searchParams] = useSearchParams();
    const token = searchParams.get('token');

    // States
    const [step, setStep] = useState<'SELECT' | 'REASON' | 'SUBMITTING' | 'SUCCESS' | 'ERROR'>('SELECT');
    const [selectedStatus, setSelectedStatus] = useState<string>('');
    const [reason, setReason] = useState<string>('');
    const [result, setResult] = useState<AttendanceResponse | null>(null);
    const [location, setLocation] = useState<{ lat: number; lng: number } | null>(null);
    const [currentTime] = useState(new Date());

    // API Base URL - Use the same host that served this page
    // This ensures the API call goes to your Mac, not localhost on the phone
    const API_BASE = window.location.origin;

    // Get user location on mount
    useEffect(() => {
        if (navigator.geolocation) {
            navigator.geolocation.getCurrentPosition(
                (position) => {
                    setLocation({
                        lat: position.coords.latitude,
                        lng: position.coords.longitude
                    });
                },
                (error) => {
                    console.log('Location not available:', error);
                }
            );
        }
    }, []);

    // Validate token exists
    if (!token) {
        return (
            <div style={containerStyle}>
                <div style={cardStyle}>
                    <FaTimesCircle size={60} color="#ef4444" />
                    <h2 style={{ color: '#ef4444', marginTop: '20px' }}>Invalid QR Code</h2>
                    <p style={{ color: '#64748b' }}>Please scan a valid employee QR code.</p>
                </div>
            </div>
        );
    }

    const handleStatusSelect = (status: string) => {
        setSelectedStatus(status);
        if (status === 'OD' || status === 'LEAVE') {
            setStep('REASON');
        } else {
            submitAttendance(status, '');
        }
    };

    const submitAttendance = async (status: string, workReason: string) => {
        setStep('SUBMITTING');

        try {
            const response = await fetch(`${API_BASE}/api/hr/attendance/mobile`, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                },
                body: JSON.stringify({
                    qr_token: token,
                    status: status,
                    work_assigned: workReason,
                    latitude: location?.lat,
                    longitude: location?.lng
                })
            });

            const data = await response.json();

            if (response.ok) {
                setResult(data);
                setStep('SUCCESS');
            } else {
                setResult({ error: data.error || 'Failed to mark attendance' });
                setStep('ERROR');
            }
        } catch (error) {
```

```
        console.error('Attendance error:', error);
        setResult({ error: 'Network error. Please try again.' });
        setStep('ERROR');
    }
};

const handleReasonSubmit = () => {
    if (!reason.trim() && selectedStatus !== 'PRESENT') {
        alert('Please enter a reason');
        return;
    }
    submitAttendance(selectedStatus, reason);
};

const resetForm = () => {
    setStep('SELECT');
    setSelectedStatus('');
    setReason('');
    setResult(null);
};

// Attendance Options
const attendanceOptions = [
    {
        id: 'PRESENT',
        label: 'Present',
        sublabel: 'Working from office',
        icon: <FaCheckCircle size={32} />,
        color: '#22c55e',
        bgColor: '#dcfce7'
    },
    {
        id: 'OD',
        label: 'On Duty',
        sublabel: 'Field work / Client visit',
        icon: <FaBriefcase size={32} />,
        color: '#3b82f6',
        bgColor: '#dbeafe'
    },
    {
        id: 'LEAVE',
        label: 'Leave',
        sublabel: 'Taking day off',
        icon: <FaUserClock size={32} />,
        color: '#f59e0b',
        bgColor: '#fef3c7'
    },
];

return (
    <div style={containerStyle}>
        {/* Header */}
        <div style={headerStyle}>
            <h1 style={{ margin: 0, fontSize: '1.5rem' }}>📋 Attendance</h1>
            <p style={{ margin: '5px 0 0', opacity: 0.9, fontSize: '0.9rem' }}>
                {currentTime.toLocaleDateString('en-IN', {
                    weekday: 'long',
                    day: 'numeric',
                    month: 'long',
                    year: 'numeric'
                })}
            </p>
            <p style={{ margin: '5px 0 0', fontSize: '1.2rem', fontWeight: 600 }}>
                {currentTime.toLocaleTimeString('en-IN', {
                    hour: '2-digit',
                    minute: '2-digit'
                })}
            </p>
        </div>

        {/* Main Card */}
        <div style={cardStyle}>

            {/* Step 1: Select Status */}
            {step === 'SELECT' && (
                <>
                    <h2 style={{ textAlign: 'center', color: '#1e293b', marginBottom: '24px' }}>
                        Mark Your Attendance
                    </h2>

                    <div style={{ display: 'flex', flexDirection: 'column', gap: '16px' }}>
                        {attendanceOptions.map((option) => (
                            <button
                                key={option.id}
                                onClick={() => handleStatusSelect(option.id)}
                                style={{
                                    display: 'flex',
                                    alignItems: 'center',
                                    gap: '16px',
                                    padding: '20px',
                                    border: '2px solid #e2e8f0',
                                    borderRadius: '16px',
                                    background: 'white',
                                    cursor: 'pointer',
                                    transition: 'all 0.2s',
                                    textAlign: 'left'
                                }}
                            >
                                <div style={{
                                    width: '60px',
                                    height: '60px',
                                    borderRadius: '12px',
                                    background: option.bgColor,
                                    color: option.color,
                                    display: 'flex',
                                    alignItems: 'center',
                                    justifyContent: 'center'
                                }}>
                                    {option.icon}
                                </div>
                                <div>
```

```
                        <div style={{
                            fontSize: '1.1rem',
                            fontWeight: 600,
                            color: '#1e293b'
                        }}>
                            {option.label}
                        </div>
                        <div style={{
                            fontSize: '0.85rem',
                            color: '#64748b',
                            marginTop: '4px'
                        }}>
                            {option.sublabel}
                        </div>
                    </div>
                </button>
            ))}
        </div>

        {location && (
            <div style={{
                marginTop: '20px',
                padding: '12px',
                background: '#f0fdf4',
                borderRadius: '8px',
                display: 'flex',
                alignItems: 'center',
                gap: '8px',
                fontSize: '0.85rem',
                color: '#166534'
            }}>
                <FaMapMarkerAlt />
                Location captured
            </div>
        )}
    </>
)}

{/* Step 2: Enter Reason */}
{step === 'REASON' && (
    <>
        <h2 style={{ textAlign: 'center', color: '#1e293b', marginBottom: '8px' }}>
            {selectedStatus === 'OD' ? '📍 On Duty Details' : '📝 Leave Reason'}
        </h2>
        <p style={{ textAlign: 'center', color: '#64748b', marginBottom: '24px' }}>
            {selectedStatus === 'OD'
                ? 'Where are you working from today?'
                : 'Please provide a reason for your leave'}
        </p>

        <textarea
            value={reason}
            onChange={(e) => setReason(e.target.value)}
            placeholder={selectedStatus === 'OD'
                ? 'e.g., Client meeting at XYZ Company, Trichy...'
                : 'e.g., Personal work, Medical appointment...'}
            style={{
                width: '100%',
                padding: '16px',
                border: '2px solid #e2e8f0',
                borderRadius: '12px',
                fontSize: '1rem',
                minHeight: '120px',
                resize: 'none',
                outline: 'none',
                fontFamily: 'inherit',
                boxSizing: 'border-box'
            }}
            autoFocus
        />

        <div style={{ display: 'flex', gap: '12px', marginTop: '20px' }}>
            <button
                onClick={() => setStep('SELECT')}
                style={{
                    flex: 1,
                    padding: '14px',
                    border: '2px solid #e2e8f0',
                    borderRadius: '12px',
                    background: 'white',
                    fontSize: '1rem',
                    fontWeight: 600,
                    cursor: 'pointer'
                }}
            >
                Back
            </button>
            <button
                onClick={handleReasonSubmit}
                style={{
                    flex: 2,
                    padding: '14px',
                    border: 'none',
                    borderRadius: '12px',
                    background: '#3b82f6',
                    color: 'white',
                    fontSize: '1rem',
                    fontWeight: 600,
                    cursor: 'pointer'
                }}
            >
                Submit
            </button>
        </div>
    </>
)}

{/* Step 3: Submitting */}
{step === 'SUBMITTING' && (
    <div style={{ textAlign: 'center', padding: '40px 0' }}>
        <FaSpinner
```

```
                size={50}
                color="#3b82f6"
                style={{ animation: 'spin 1s linear infinite' }}
            />
            <p style={{ marginTop: '20px', color: '#64748b', fontSize: '1.1rem' }}>
                Marking attendance...
            </p>
            <style>{`
                @keyframes spin {
                    from { transform: rotate(0deg); }
                    to { transform: rotate(360deg); }
                }
            `}</style>
        </div>
    )}

    {/* Step 4: Success */}
    {step === 'SUCCESS' && (
        <div style={{ textAlign: 'center', padding: '20px 0' }}>
            <div style={{
                width: '80px',
                height: '80px',
                borderRadius: '50%',
                background: '#dcfce7',
                display: 'flex',
                alignItems: 'center',
                justifyContent: 'center',
                margin: '0 auto 20px'
            }}>
                <FaCheckCircle size={40} color="#22c55e" />
            </div>
            <h2 style={{ color: '#166534', marginBottom: '8px' }}>Success!</h2>
            <p style={{ color: '#64748b', fontSize: '1rem', marginBottom: '8px' }}>
                {result?.message || 'Attendance marked successfully'}
            </p>
            {result?.employee_name && (
                <p style={{
                    color: '#1e293b',
                    fontSize: '1.2rem',
                    fontWeight: 600,
                    marginTop: '16px'
                }}>
                    Welcome, {result.employee_name}!
                </p>
            )}
            <p style={{
                color: '#64748b',
                fontSize: '0.9rem',
                marginTop: '8px'
            }}>
                {currentTime.toLocaleTimeString('en-IN')}
            </p>

            <button
                onClick={resetForm}
                style={{
                    marginTop: '24px',
                    padding: '12px 32px',
                    border: '2px solid #e2e8f0',
                    borderRadius: '12px',
                    background: 'white',
                    fontSize: '1rem',
                    cursor: 'pointer'
                }}
            >
                Done
            </button>
        </div>
    )}

    {/* Step 5: Error */}
    {step === 'ERROR' && (
        <div style={{ textAlign: 'center', padding: '20px 0' }}>
            <div style={{
                width: '80px',
                height: '80px',
                borderRadius: '50%',
                background: '#fee2e2',
                display: 'flex',
                alignItems: 'center',
                justifyContent: 'center',
                margin: '0 auto 20px'
            }}>
                <FaTimesCircle size={40} color="#ef4444" />
            </div>
            <h2 style={{ color: '#dc2626', marginBottom: '8px' }}>Error</h2>
            <p style={{ color: '#64748b', fontSize: '1rem' }}>
                {result?.error || 'Something went wrong'}
            </p>

            <button
                onClick={resetForm}
                style={{
                    marginTop: '24px',
                    padding: '12px 32px',
                    border: 'none',
                    borderRadius: '12px',
                    background: '#3b82f6',
                    color: 'white',
                    fontSize: '1rem',
                    fontWeight: 600,
                    cursor: 'pointer'
                }}
            >
                Try Again
            </button>
        </div>
    )}
</div>

{/* Footer */}
<p style={{
```

```
                textAlign: 'center',
                color: 'rgba(255,255,255,0.7)',
                fontSize: '0.8rem',
                marginTop: '20px'
            }}>
                ERP Attendance System
            </p>
        </div>
    );
};

// Styles
const containerStyle: React.CSSProperties = {
    minHeight: '100vh',
    background: 'linear-gradient(135deg, #1e3a8a 0%, #3b82f6 100%)',
    padding: '20px',
    display: 'flex',
    flexDirection: 'column',
    alignItems: 'center'
};

const headerStyle: React.CSSProperties = {
    color: 'white',
    textAlign: 'center',
    marginBottom: '24px',
    paddingTop: '20px'
};

const cardStyle: React.CSSProperties = {
    background: 'white',
    borderRadius: '24px',
    padding: '32px 24px',
    width: '100%',
    maxWidth: '400px',
    boxShadow: '0 20px 40px rgba(0,0,0,0.2)'
};

export default MobileAttendance;
```

frontend/src/pages/hr/EmployeeQRModal.tsx

```
// frontend/src/pages/hr/EmployeeQRModal.tsx
// UPDATED: QR Code now contains a URL that employees can scan to mark attendance

import React from 'react';
import { FaMobileAlt, FaPrint, FaTimes } from 'react-icons/fa';
import QRCode from 'react-qr-code';

interface Employee {
    id: number;
    name: string;
    designation?: string;
    status?: string;
}

interface Props {
    employee: Employee;
    onClose: () => void;
}

const EmployeeQRModal: React.FC<Props> = ({ employee, onClose }) => {
    // Generate the attendance URL that will be encoded in QR
    // When scanned, this opens the mobile attendance page

    // Use your Mac's IP address (found via: ipconfig getifaddr en0)
    const baseUrl = 'http://192.168.29.33:5173';

    const qrToken = `EMP_${employee.id}_SECRET`;
    const attendanceUrl = `${baseUrl}/mark-attendance?token=${qrToken}`;

    const handlePrint = () => {
        const printWindow = window.open('', '', 'width=400,height=700');
        if (!printWindow) return;

        printWindow.document.write(`
            <!DOCTYPE html>
            <html>
            <head>
                <title>Employee ID Card - ${employee.name}</title>
                <style>
                    * { margin: 0; padding: 0; box-sizing: border-box; }
                    body {
                        font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
                        padding: 20px;
                        display: flex;
                        justify-content: center;
                    }
                    .card {
                        width: 300px;
                        border: 2px solid #e2e8f0;
                        border-radius: 16px;
                        overflow: hidden;
                        background: white;
                    }
                    .header {
                        background: linear-gradient(135deg, #1e3a8a, #3b82f6);
                        color: white;
                        padding: 20px;
                        text-align: center;
                    }
                    .header h1 {
                        font-size: 14px;
                        font-weight: 600;
                        letter-spacing: 1px;
                    }
                    .body {
                        padding: 24px;
                        text-align: center;
                    }
                    .name {
                        font-size: 20px;
                        font-weight: 700;
                        color: #1e293b;
                        margin-bottom: 4px;
                    }
                    .designation {
                        font-size: 14px;
                        color: #64748b;
                        margin-bottom: 20px;
                    }
                    .qr-container {
                        background: #f8fafc;
                        padding: 16px;
                        border-radius: 12px;
                        display: inline-block;
                        margin-bottom: 16px;
                    }
                    .footer {
                        font-size: 11px;
                        color: #94a3b8;
                        padding: 12px;
                        border-top: 1px solid #e2e8f0;
                        text-align: center;
                    }
                    .scan-text {
                        font-size: 12px;
                        color: #64748b;
                        margin-top: 12px;
                    }
                    @media print {
                        body { padding: 0; }
                        .card { border: 1px solid #ccc; }
                    }
                </style>
            </head>
            <body>
                <div class="card">
```

```
                <div class="header">
                    <h1>🏢 ERP SYSTEM</h1>
                </div>
                <div class="body">
                    <div class="name">${employee.name}</div>
                    <div class="designation">${employee.designation || 'Employee'}</div>
                    <div class="qr-container">
                        ${document.getElementById('qr-code-container')?.innerHTML || ''}
                    </div>
                    <div class="scan-text">
                        📱 Scan to mark attendance
                    </div>
                </div>
                <div class="footer">
                    ID: ${employee.id} | Status: ${employee.status || 'Active'}
                </div>
            </div>
        </body>
        </html>
    `);
    printWindow.document.close();

    // Wait for QR to render then print
    setTimeout(() => {
        printWindow.print();
    }, 500);
};

const copyLink = () => {
    navigator.clipboard.writeText(attendanceUrl);
    alert('Attendance link copied! Share with employee.');
};

return (
    <div style={{
        position: 'fixed',
        top: 0,
        left: 0,
        width: '100%',
        height: '100%',
        background: 'rgba(0,0,0,0.6)',
        display: 'flex',
        justifyContent: 'center',
        alignItems: 'center',
        zIndex: 2000
    }}>
        <div style={{
            background: 'white',
            padding: '30px',
            borderRadius: '16px',
            textAlign: 'center',
            width: '380px',
            position: 'relative',
            boxShadow: '0 20px 40px rgba(0,0,0,0.2)'
        }}>
            {/* Close Button */}
            <button
                onClick={onClose}
                style={{
                    position: 'absolute',
                    top: '12px',
                    right: '12px',
                    border: 'none',
                    background: '#f1f5f9',
                    borderRadius: '8px',
                    padding: '8px',
                    cursor: 'pointer',
                    color: '#64748b'
                }}
            >
                <FaTimes />
            </button>

            {/* Employee Info */}
            <h3 style={{
                margin: '0 0 4px 0',
                color: '#1e293b',
                fontSize: '1.3rem'
            }}>
                {employee.name}
            </h3>
            <p style={{
                margin: '0 0 20px 0',
                color: '#64748b',
                fontSize: '0.95rem'
            }}>
                {employee.designation || 'Employee'}
            </p>

            {/* QR Code */}
            <div style={{
                background: '#f8fafc',
                padding: '24px',
                borderRadius: '12px',
                border: '1px solid #e2e8f0',
                display: 'inline-block'
            }}>
                <div id="qr-code-container">
                    <QRCode
                        value={attendanceUrl}
                        size={180}
                        level="M"
                    />
                </div>
            </div>

            {/* Instructions */}
            <div style={{
                marginTop: '16px',
                padding: '12px',
                background: '#eff6ff',
```

```
                    borderRadius: '8px',
                    display: 'flex',
                    alignItems: 'center',
                    justifyContent: 'center',
                    gap: '8px'
                }}>
                    <FaMobileAlt color="#3b82f6" />
                    <span style={{ fontSize: '0.85rem', color: '#1e40af' }}>
                        Scan with phone to mark attendance
                    </span>
                </div>

                {/* ID Info */}
                <div style={{
                    marginTop: '12px',
                    fontSize: '0.8rem',
                    color: '#94a3b8'
                }}>
                    ID: {employee.id} | Status: {employee.status || 'Active'}
                </div>

                {/* Action Buttons */}
                <div style={{
                    marginTop: '20px',
                    display: 'flex',
                    gap: '12px',
                    justifyContent: 'center'
                }}>
                    <button
                        onClick={copyLink}
                        style={{
                            background: '#f1f5f9',
                            color: '#475569',
                            border: 'none',
                            padding: '10px 16px',
                            borderRadius: '8px',
                            cursor: 'pointer',
                            fontWeight: 500,
                            fontSize: '0.9rem'
                        }}
                    >
                        📋 Copy Link
                    </button>
                    <button
                        onClick={handlePrint}
                        style={{
                            background: '#2563eb',
                            color: 'white',
                            border: 'none',
                            padding: '10px 20px',
                            borderRadius: '8px',
                            cursor: 'pointer',
                            display: 'flex',
                            alignItems: 'center',
                            gap: '8px',
                            fontWeight: 600,
                            fontSize: '0.9rem'
                        }}
                    >
                        <FaPrint /> Print ID Card
                    </button>
                </div>
            </div>
        </div>
    );
};

export default EmployeeQRModal;
```

frontend/src/pages/hr/EmployeeLedgerModal.tsx

```
import React, { useEffect, useState } from 'react';
import { FaArrowDown, FaArrowUp, FaBook, FaHistory, FaTimes } from 'react-icons/fa';
import { apiFetch } from '../../utils/api';

interface Props {
    employee: any;
    onClose: () => void;
}

const EmployeeLedgerModal: React.FC<Props> = ({ employee, onClose }) => {
    const [ledger, setLedger] = useState<any[]>([]);
    const [loading, setLoading] = useState(true);
    const [windowWidth, setWindowWidth] = useState(window.innerWidth);

    useEffect(() => {
        const handleResize = () => setWindowWidth(window.innerWidth);
        window.addEventListener('resize', handleResize);
        const loadLedger = async () => {
            try {
                const res = await apiFetch(`/hr/ledger/${employee.id}`);
                const data = await res.json();
                setLedger(data);
            } catch (err) {
                console.error(err);
            } finally {
                setLoading(false);
            }
        };
        loadLedger();
        return () => window.removeEventListener('resize', handleResize);
    }, [employee.id]);

    const fmt = (n: number) => Number(n).toLocaleString('en-IN', {
        style: 'currency',
        currency: 'INR',
        minimumFractionDigits: 0,
        maximumFractionDigits: 0
    });

    const isMobile = windowWidth <= 640;

    return (
        <div style={{
            position: 'fixed',
            top: 0,
            left: 0,
            width: '100%',
            height: '100%',
            background: 'rgba(15, 23, 42, 0.4)',
            backdropFilter: 'blur(8px)',
            display: 'flex',
            justifyContent: 'center',
            alignItems: 'center',
            zIndex: 1100,
            padding: '20px'
        }}>
            <div className="page-transition" style={{
                background: 'rgba(255, 255, 255, 0.95)',
                width: '100%',
                maxWidth: '850px',
                borderRadius: '24px',
                padding: isMobile ? '20px' : '32px',
                maxHeight: '90vh',
                display: 'flex',
                flexDirection: 'column',
                boxShadow: 'var(--shadow-glass)',
                border: '1px solid rgba(255, 255, 255, 0.5)',
                position: 'relative'
            }}>

                {/* Close Button */}
                <button
                    onClick={onClose}
                    style={{
                        position: 'absolute',
                        top: '24px',
                        right: '24px',
                        border: 'none',
                        background: 'var(--bg-body)',
                        width: '36px',
                        height: '36px',
                        borderRadius: '50%',
                        display: 'flex',
                        alignItems: 'center',
                        justifyContent: 'center',
                        cursor: 'pointer',
                        color: 'var(--text-muted)',
                        transition: 'all 0.2s'
                    }}
                    onMouseEnter={(e) => {
                        e.currentTarget.style.background = 'var(--danger)';
                        e.currentTarget.style.color = 'white';
                    }}
                    onMouseLeave={(e) => {
                        e.currentTarget.style.background = 'var(--bg-body)';
                        e.currentTarget.style.color = 'var(--text-muted)';
                    }}
                >
                    <FaTimes />
                </button>

                {/* Header */}
                <div style={{ display: 'flex', gap: '20px', marginBottom: '32px', alignItems: 'center' }}>
                    <div style={{
                        width: '64px',
                        height: '64px',
```

```
        borderRadius: '18px',
        background: 'linear-gradient(135deg, var(--primary) 0%, var(--primary-dark) 100%)',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'center',
        color: 'white',
        fontSize: '1.5rem',
        boxShadow: '0 8px 16px rgba(37, 99, 235, 0.2)'
    }}>
        {employee.name ? employee.name.charAt(0).toUpperCase() : <FaBook />}
    </div>
    <div style={{ flex: 1 }}>
        <h2 style={{
            margin: 0,
            color: 'var(--text-main)',
            fontSize: isMobile ? '1.25rem' : '1.75rem',
            fontWeight: 800,
            letterSpacing: '-0.5px'
        }}>
            {employee.name}'s Ledger
        </h2>
        <div style={{ display: 'flex', alignItems: 'center', gap: '8px', marginTop: '4px' }}>
            <span style={{
                display: 'inline-flex',
                alignItems: 'center',
                gap: '4px',
                fontSize: '0.75rem',
                color: 'var(--primary)',
                background: 'var(--primary-glow)',
                padding: '2px 10px',
                borderRadius: '12px',
                fontWeight: 700,
                textTransform: 'uppercase'
            }}>
                <FaHistory size={10} /> Salary Advance History
            </span>
        </div>
    </div>
</div>

{/* Ledger Table */}
<div className="custom-scrollbar" style={{
    flex: 1,
    overflowY: 'auto',
    border: '1px solid var(--border-color)',
    borderRadius: '16px',
    background: 'white'
}}>
    <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: '0.9rem' }}>
        <thead style={{ background: 'var(--bg-body)', borderBottom: '1px solid var(--border-color)', position: 'sticky', top: 0, zIndex
            <tr>
                <th style={{ padding: '16px 20px', textAlign: 'left', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.75rem',
                <th style={{ padding: '16px 20px', textAlign: 'left', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.75rem',
                {!isMobile && <th style={{ padding: '16px 20px', textAlign: 'right', color: 'var(--text-muted)', fontWeight: 700, fontS
                {!isMobile && <th style={{ padding: '16px 20px', textAlign: 'right', color: 'var(--text-muted)', fontWeight: 700, fontS
                <th style={{ padding: '16px 20px', textAlign: 'right', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.75rem'
            </tr>
        </thead>
        <tbody>
            {loading ? (
                <tr>
                    <td colSpan={isMobile ? 3 : 5} style={{ padding: '40px', textAlign: 'center' }}>
                        <div className="flex-center" style={{ flexDirection: 'column', gap: '12px' }}>
                            <div className="skeleton" style={{ width: '40px', height: '40px', borderRadius: '50%' }}></div>
                            <span style={{ color: 'var(--text-muted)', fontWeight: 500 }}>Fetching transaction records...</span>
                        </div>
                    </td>
                </tr>
            ) : ledger.length === 0 ? (
                <tr>
                    <td colSpan={isMobile ? 3 : 5} style={{ padding: '60px 20px', textAlign: 'center' }}>
                        <div style={{ opacity: 0.4 }}>
                            <FaBook size={48} style={{ marginBottom: '16px', color: 'var(--text-light)' }} />
                            <p style={{ margin: 0, fontSize: '1.1rem', fontWeight: 600, color: 'var(--text-muted)' }}>No history found</
                            <p style={{ margin: '4px 0 0', fontSize: '0.85rem' }}>This employee has no salary advance or repayment histo
                        </div>
                    </td>
                </tr>
            ) : (
                ledger.map((row, i) => {
                    const isAdvance = row.type === 'ADVANCE';
                    return (
                        <tr key={i} className="row-hover" style={{ borderBottom: '1px solid var(--bg-body)', transition: 'background-co
                            <td style={{ padding: '16px 20px' }}>
                                <div style={{ fontWeight: 700, color: 'var(--text-main)' }}>{new Date(row.date).toLocaleDateString('en-
                                <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', marginTop: '2px' }}>{new Date(row.date).g
                            </td>
                            <td style={{ padding: '16px 20px' }}>
                                <div style={{ display: 'flex', alignItems: 'center', gap: '8px' }}>
                                    <span style={{
                                        width: '24px',
                                        height: '24px',
                                        borderRadius: '6px',
                                        display: 'flex',
                                        alignItems: 'center',
                                        justifyContent: 'center',
                                        fontSize: '0.7rem',
                                        background: isAdvance ? 'rgba(239, 68, 68, 0.1)' : 'rgba(16, 185, 129, 0.1)',
                                        color: isAdvance ? 'var(--danger)' : 'var(--success)'
                                    }}>
                                        {isAdvance ? <FaArrowUp /> : <FaArrowDown />}
                                    </span>
                                    <span style={{
                                        fontSize: '0.85rem',
                                        fontWeight: 700,
                                        color: 'var(--text-main)'
                                    }}>
                                        {isAdvance ? 'Advance' : 'Repayment'}
                                    </span>
                                </div>
                                <div style={{ fontSize: '0.8rem', color: 'var(--text-muted)', marginTop: '4px', maxWidth: '200px', over
                                    {row.description || 'No remarks provided'}
```

```
                                        </div>
                                        {isMobile && (
                                            <div style={{ marginTop: '8px', display: 'flex', gap: '12px' }}>
                                                <span style={{ fontSize: '0.75rem', color: isAdvance ? 'var(--danger)' : 'var(--success)', fontW
                                                    {isAdvance ? '+' : '-'} {fmt(row.amount)}
                                                </span>
                                            </div>
                                        )}
                                    </td>
                                    {!isMobile && (
                                        <td style={{ padding: '16px 20px', textAlign: 'right', color: isAdvance ? 'var(--danger)' : 'var(--text-
                                            {isAdvance ? fmt(row.amount) : '-'}
                                        </td>
                                    )}
                                    {!isMobile && (
                                        <td style={{ padding: '16px 20px', textAlign: 'right', color: !isAdvance ? 'var(--success)' : 'var(--tex
                                            {!isAdvance ? fmt(row.amount) : '-'}
                                        </td>
                                    )}
                                    <td style={{ padding: '16px 20px', textAlign: 'right' }}>
                                        <div style={{
                                            fontWeight: 800,
                                            color: row.running_balance > 0 ? 'var(--text-main)' : 'var(--success)',
                                            fontSize: '1rem'
                                        }}>
                                            {fmt(row.running_balance)}
                                        </div>
                                    </td>
                                </tr>
                            );
                        })
                    )}
                </tbody>
            </table>
        </div>

        {/* Footer Section */}
        <div style={{
            marginTop: '24px',
            padding: '20px 24px',
            background: 'var(--bg-body)',
            borderRadius: '16px',
            display: 'flex',
            justifyContent: 'space-between',
            alignItems: 'center'
        }}>
            <div style={{ display: 'flex', gap: '24px' }}>
                <div>
                    <div style={{ fontSize: '0.7rem', color: 'var(--text-muted)', fontWeight: 700, textTransform: 'uppercase', letterSpacing: '0
                    <div style={{ fontSize: '1.1rem', fontWeight: 800, color: 'var(--text-main)' }}>{ledger.length}</div>
                </div>
                <div>
                    <div style={{ fontSize: '0.7rem', color: 'var(--text-muted)', fontWeight: 700, textTransform: 'uppercase', letterSpacing: '0
                    <div style={{ fontSize: '1.1rem', fontWeight: 800, color: 'var(--text-main)' }}>
                        {ledger.length > 0 ? new Date(ledger[0].date).toLocaleDateString('en-GB', { day: '2-digit', month: 'short' }) : 'N/A'}
                    </div>
                </div>
            </div>

            <div style={{ textAlign: 'right' }}>
                <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', fontWeight: 700, textTransform: 'uppercase', letterSpacing: '0.05
                <div style={{
                    fontSize: isMobile ? '1.5rem' : '2rem',
                    fontWeight: 900,
                    color: ledger.length > 0 && ledger[ledger.length - 1].running_balance > 0 ? 'var(--danger)' : 'var(--success)',
                    lineHeight: 1,
                    marginTop: '4px'
                }}>
                    {ledger.length > 0 ? fmt(ledger[ledger.length - 1].running_balance) : '₹0'}
                </div>
            </div>
        </div>

        <style>{`
            .row-hover:hover {
                background-color: #f8fafc !important;
            }
            .custom-scrollbar::-webkit-scrollbar {
                width: 6px;
            }
            .custom-scrollbar::-webkit-scrollbar-track {
                background: transparent;
            }
            .custom-scrollbar::-webkit-scrollbar-thumb {
                background: #cbd5e1;
                border-radius: 10px;
            }
            .skeleton {
                background: linear-gradient(90deg, #f1f5f9 25%, #e2e8f0 50%, #f1f5f9 75%);
                background-size: 200% 100%;
                animation: loading 1.5s infinite;
            }
            @keyframes loading {
                0% { background-position: 200% 0; }
                100% { background-position: -200% 0; }
            }
        `}</style>
    </div>
  </div>
);
};

export default EmployeeLedgerModal;
```

frontend/src/pages/hr/AttendanceScanner.tsx

```
import React, { useEffect, useRef, useState } from 'react';
import { FaCheckCircle, FaClock, FaQrcode, FaSave } from 'react-icons/fa';
import { apiFetch } from '../../utils/api';

const AttendanceScanner: React.FC = () => {
    const [scanResult, setScanResult] = useState('');

    // UI States
    const [step, setStep] = useState<'SCAN' | 'DETAILS' | 'RESULT'>('SCAN');
    const [statusType, setStatusType] = useState('PRESENT');
    const [workAssigned, setWorkAssigned] = useState('');

    // Result States
    const [resultMessage, setResultMessage] = useState('');
    const [isError, setIsError] = useState(false);

    const inputRef = useRef<HTMLInputElement>(null);

    // Keep focus on input for scanner
    useEffect(() => {
        if (step === 'SCAN') {
            inputRef.current?.focus();
            const interval = setInterval(() => inputRef.current?.focus(), 2000);
            return () => clearInterval(interval);
        }
    }, [step]);

    const handleScan = (e: React.FormEvent) => {
        e.preventDefault();
        if (scanResult) {
            // Move to Details Step
            setStep('DETAILS');
        }
    };

    const handleSubmitAttendance = async () => {
        try {
            const res = await apiFetch('/hr/attendance/scan', {
                method: 'POST',
                body: {
                    qr_token: scanResult,
                    status: statusType,
                    work_assigned: workAssigned
                }
            });
            const data = await res.json();

            if (res.ok) {
                setIsError(false);
                setResultMessage(data.message);
            } else {
                throw new Error(data.error);
            }
        } catch (err: any) {
            setIsError(true);
            setResultMessage(err.message || 'Invalid QR Code');
        }

        setStep('RESULT');

        // Reset after 3 seconds
        setTimeout(() => {
            setScanResult('');
            setWorkAssigned('');
            setStatusType('PRESENT');
            setStep('SCAN');
        }, 3000);
    };

    return (
        <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent: 'center', height: '80vh' }}>

            <div style={{ marginBottom: '30px', textAlign: 'center' }}>
                <h1 style={{ fontSize: '2.5rem', fontWeight: 700, color: '#1e293b', marginBottom: '10px' }}>Attendance Kiosk</h1>
                <p style={{ color: '#64748b' }}>
                    {step === 'SCAN' ? 'Please scan your employee ID card.' : 'Enter work details.'}
                </p>
            </div>

            <div style={{
                width: '100%', maxWidth: '500px', background: 'white', padding: '40px', borderRadius: '20px',
                boxShadow: '0 10px 25px -5px rgba(0, 0, 0, 0.1)', textAlign: 'center', border: '1px solid #e2e8f0'
            }}>

                {/* STEP 1: SCANNING */}
                {step === 'SCAN' && (
                    <div style={{ animation: 'pulse 2s infinite' }}>
                        <FaQrcode size={100} color="#cbd5e1" />
                        <p style={{ marginTop: '20px', fontWeight: 500, color: '#94a3b8' }}>Waiting for scan...</p>

                        {/* Hidden Input for Scanner */}
                        <form onSubmit={handleScan} style={{ marginTop: '30px' }}>
                            <input
                                ref={inputRef}
                                value={scanResult}
                                onChange={e => setScanResult(e.target.value)}
                                placeholder="Click here & scan..."
                                style={{
                                    width: '80%', padding: '10px', borderRadius: '8px', border: '1px solid #e2e8f0',
                                    textAlign: 'center', fontSize: '1.2rem', outline: 'none'
                                }}
                                autoFocus
                            />
                        </form>
                    </div>
                )}
```

```
                {/* STEP 2: DETAILS INPUT */}
                {step === 'DETAILS' && (
                    <div style={{textAlign: 'left'}}>
                        <h3 style={{marginBottom: '20px', borderBottom: '1px solid #eee', paddingBottom: '10px'}}>Entry Details</h3>

                        <div style={{marginBottom: '15px'}}>
                            <label style={{display: 'block', fontWeight: 600, fontSize: '0.9rem', marginBottom: '5px'}}>Status</label>
                            <select
                                value={statusType}
                                onChange={e => setStatusType(e.target.value)}
                                style={{width: '100%', padding: '10px', borderRadius: '8px', border: '1px solid #cbd5e1'}}
                            >
                                <option value="PRESENT">Present (In Office)</option>
                                <option value="OD">On Duty (OD) / Field Work</option>
                                <option value="LEAVE">Leave</option>
                            </select>
                        </div>

                        <div style={{marginBottom: '20px'}}>
                            <label style={{display: 'block', fontWeight: 600, fontSize: '0.9rem', marginBottom: '5px'}}>Work Assigned / Notes</label>
                            <textarea
                                value={workAssigned}
                                onChange={e => setWorkAssigned(e.target.value)}
                                rows={3}
                                placeholder="E.g. Site visit to Trichy..."
                                style={{width: '100%', padding: '10px', borderRadius: '8px', border: '1px solid #cbd5e1', resize: 'none'}}
                            />
                        </div>

                        <button
                            onClick={handleSubmitAttendance}
                            style={{
                                width: '100%', padding: '12px', background: '#2563eb', color: 'white',
                                border: 'none', borderRadius: '8px', fontWeight: 700, fontSize: '1rem', cursor: 'pointer',
                                display: 'flex', alignItems: 'center', justifyContent: 'center', gap: '10px'
                            }}
                        >
                            <FaSave /> Confirm Attendance
                        </button>
                    </div>
                )}

                {/* STEP 3: RESULT */}
                {step === 'RESULT' && (
                    <div>
                        {isError ? (
                            <div style={{ color: '#dc2626' }}>
                                <FaClock size={100} />
                                <h2 style={{ marginTop: '20px', fontSize: '1.5rem' }}>Error</h2>
                                <p>{resultMessage}</p>
                            </div>
                        ) : (
                            <div style={{ color: '#16a34a' }}>
                                <FaCheckCircle size={100} />
                                <h2 style={{ marginTop: '20px', fontSize: '1.5rem' }}>Success!</h2>
                                <p>{resultMessage}</p>
                            </div>
                        )}
                    </div>
                )}

            </div>

            <div style={{ marginTop: '30px', fontSize: '0.9rem', color: '#94a3b8' }}>
                System Ready • {new Date().toLocaleDateString()}
            </div>
        </div>
    );
};

export default AttendanceScanner;
```

frontend/src/pages/hr/Attendance.tsx

```
// frontend/src/pages/hr/Attendance.tsx
import { motion } from 'framer-motion';
import React, { useCallback, useEffect, useState } from 'react';
import {
    FaBriefcase, FaCalendarAlt, FaCheck, FaClock,
    FaFingerprint,
    FaSearch, FaSync, FaTimes,
    FaUserClock
} from 'react-icons/fa';

interface Employee {
    id: number;
    name: string;
    designation?: string;
    salary_type?: string;
}

interface AttendanceRecord {
    id?: number;
    employee_id: number;
    employee_name?: string;
    date: string;
    status: 'PRESENT' | 'ABSENT' | 'OD' | 'LEAVE' | 'HALF_DAY';
    check_in_time?: string;
    check_out_time?: string;
    work_assigned?: string;
    method?: string;
}

const Attendance: React.FC = () => {
    const [employees, setEmployees] = useState<Employee[]>([]);
    const [attendance, setAttendance] = useState<Record<string, AttendanceRecord>>({});
    const [selectedDate, setSelectedDate] = useState(new Date().toISOString().split('T')[0]);
    const [loading, setLoading] = useState(true);
    const [search, setSearch] = useState('');
    const [autoRefresh, setAutoRefresh] = useState(true);

    useEffect(() => {
        const load = async () => {
            const token = localStorage.getItem('erp-token');
            const res = await fetch('/api/employees', { headers: { 'Authorization': `Bearer ${token}` } });
            const data = await res.json();
            setEmployees(data || []);
            setLoading(false);
        };
        load();
    }, []);

    const fetchAttendanceForDate = useCallback(async (date: string) => {
        try {
            const token = localStorage.getItem('erp-token');
            const res = await fetch(`/api/hr/attendance?date=${date}`, {
                headers: { 'Authorization': `Bearer ${token}` }
            });
            const data = await res.json();
            const attendanceMap: Record<string, AttendanceRecord> = {};
            (data || []).forEach((record: AttendanceRecord) => {
                attendanceMap[`${record.employee_id}_${date}`] = record;
            });
            setAttendance(attendanceMap);
        } catch (err) {}
    }, []);

    useEffect(() => {
        fetchAttendanceForDate(selectedDate);
    }, [selectedDate, fetchAttendanceForDate]);

    useEffect(() => {
        if (!autoRefresh) return;
        const interval = setInterval(() => fetchAttendanceForDate(selectedDate), 15000);
        return () => clearInterval(interval);
    }, [autoRefresh, selectedDate, fetchAttendanceForDate]);

    const markAttendance = async (employeeId: number, status: string) => {
        const token = localStorage.getItem('erp-token');
        await fetch('/api/hr/attendance/manual', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${token}` },
            body: JSON.stringify({
                employee_id: employeeId,
                date: selectedDate,
                status: status,
                check_in_time: ['PRESENT', 'OD', 'HALF_DAY'].includes(status) ? new Date().toLocaleTimeString('en-US', { hour12: false }) : null
            })
        });
        fetchAttendanceForDate(selectedDate);
    };

    const stats = {
        total: employees.length,
        present: Object.values(attendance).filter(a => a.status === 'PRESENT').length,
        absent: employees.length - Object.values(attendance).length,
        onDuty: Object.values(attendance).filter(a => a.status === 'OD').length,
    };

    const statusConfig: any = {
        'PRESENT': { label: 'PRESENT', color: '#10b981', bg: '#dcfce7', icon: <FaCheck /> },
        'ABSENT': { label: 'ABSENT', color: '#ef4444', bg: '#fee2e2', icon: <FaTimes /> },
        'OD': { label: 'ON DUTY', color: '#3b82f6', bg: '#dbeafe', icon: <FaBriefcase /> },
        'LEAVE': { label: 'LEAVE', color: '#f59e0b', bg: '#fef3c7', icon: <FaUserClock /> },
        'HALF_DAY': { label: 'HALF DAY', color: '#8b5cf6', bg: '#ede9fe', icon: <FaClock /> }
    };

    return (
        <div className="employees-container">
            <header className="employees-header">
```

```
    <div className="employees-title">
        <motion.h1 initial={{ opacity: 0, x: -30 }} animate={{ opacity: 1, x: 0 }}>Presence Registry</motion.h1>
        <motion.p initial={{ opacity: 0 }} animate={{ opacity: 1 }} transition={{ delay: 0.2 }}>
            Real-time workforce monitoring and authentication matrix.
        </motion.p>
    </div>
    <div className="employees-actions" style={{ alignItems: 'center' }}>
        <div style={{ display: 'flex', alignItems: 'center', gap: '12px', background: 'white', padding: '10px 20px', borderRadius: '14px', b
            <FaCalendarAlt color="var(--primary)" />
            <input
                type="date"
                className="premium-input"
                value={selectedDate}
                onChange={e => setSelectedDate(e.target.value)}
                style={{ border: 'none', background: 'transparent', padding: 0, fontWeight: 800, width: '130px' }}
            />
        </div>
        <motion.button
            whileHover={{ scale: 1.05 }}
            whileTap={{ scale: 0.95 }}
            className={`btn-secondary ${autoRefresh ? 'active' : ''}`}
            onClick={() => setAutoRefresh(!autoRefresh)}
            style={{ height: '52px', border: autoRefresh ? '2px solid var(--primary)' : '1px solid var(--border-color)' }}
        >
            <FaSync className={autoRefresh ? 'fa-spin' : ''} style={{ animationDuration: '4s' }} />
            {autoRefresh ? 'Real-time Linked' : 'Stream Paused'}
        </motion.button>
    </div>
</header>

<div className="stats-matrix">
    {[
        { label: 'Total Personnel', value: stats.total, icon: <FaUserClock />, color: 'var(--text-muted)' },
        { label: 'Verified Present', value: stats.present, icon: <FaCheck />, color: 'var(--success)' },
        { label: 'Registry Absence', value: stats.absent, icon: <FaTimes />, color: 'var(--error)' },
        { label: 'On-Field Operatives', value: stats.onDuty, icon: <FaBriefcase />, color: 'var(--primary)' }
    ].map((stat, i) => (
        <motion.div initial={{ opacity: 0, y: 20 }} animate={{ opacity: 1, y: 0 }} transition={{ delay: i * 0.1 }} key={i} className="matrix
            <div className="matrix-icon" style={{ color: stat.color, opacity: 0.1, position: 'absolute', right: '20px', top: '20px', fontSi
            <span className="matrix-label">{stat.label}</span>
            <h3 className="matrix-value" style={{ color: i > 0 ? stat.color : 'inherit' }}>{stat.value}</h3>
        </motion.div>
    ))}
</div>

<div className="search-orb">
    <FaSearch style={{ color: 'var(--text-muted)' }} size={20} />
    <input
        placeholder="Identify workforce member via name or ID..."
        value={search}
        onChange={e => setSearch(e.target.value)}
    />
</div>

<div className="employee-table-container">
    <table className="emp-table">
        <thead>
            <tr>
                <th>Personnel Profile</th>
                <th style={{ textAlign: 'center' }}>Clock In</th>
                <th style={{ textAlign: 'center' }}>Clock Out</th>
                <th>Status Signature</th>
                <th>Methods</th>
                <th style={{ textAlign: 'center' }}>Interface Override</th>
            </tr>
        </thead>
        <tbody>
            {employees.filter(e => e.name.toLowerCase().includes(search.toLowerCase())).map((emp, idx) => {
                const record = attendance[`${emp.id}_${selectedDate}`];
                const cfg = record ? statusConfig[record.status] : null;
                return (
                    <motion.tr
                        key={emp.id}
                        initial={{ opacity: 0, x: -10 }}
                        animate={{ opacity: 1, x: 0 }}
                        transition={{ delay: idx * 0.03 }}
                        className="emp-row"
                    >
                        <td>
                            <div style={{ display: 'flex', alignItems: 'center', gap: '15px' }}>
                                <div className="profile-orb" style={{ background: record ? 'var(--primary-glow)' : '#f1f5f9' }}>
                                    {emp.name.charAt(0)}
                                </div>
                                <div>
                                    <div style={{ fontWeight: 800 }}>{emp.name}</div>
                                    <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', fontWeight: 600 }}>{emp.designation || '
                                </div>
                            </div>
                        </td>
                        <td style={{ textAlign: 'center', fontWeight: 900, color: record?.check_in_time ? 'var(--success)' : '#cbd5e1' }}>
                            {record?.check_in_time || '--:--'}
                        </td>
                        <td style={{ textAlign: 'center', fontWeight: 900, color: record?.check_out_time ? 'var(--error)' : '#cbd5e1' }}>
                            {record?.check_out_time || '--:--'}
                        </td>
                        <td>
                            {cfg ? (
                                <span className="status-pill" style={{ background: cfg.bg, color: cfg.color }}>
                                    {cfg.label}
                                </span>
                            ) : (
                                <span style={{ fontSize: '0.7rem', color: 'var(--text-muted)', fontWeight: 800, opacity: 0.5, letterSpacing
                            )}
                        </td>
                        <td>
                            <div style={{ display: 'flex', alignItems: 'center', gap: '8px', fontSize: '0.75rem', fontWeight: 800, color: '
                                <FaFingerprint size={14} style={{ opacity: record ? 1 : 0.2 }} />
                                {record ? (record.method || 'MANUAL') : 'WAITING'}
                            </div>
                        </td>
                        <td style={{ textAlign: 'center' }}>
                            <div style={{ display: 'flex', gap: '6px', justifyContent: 'center' }}>
```

```
                                                {Object.entries(statusConfig).map(([key, c]: any) => (
                                                    <motion.button
                                                        key={key}
                                                        whileHover={{ scale: 1.15 }}
                                                        whileTap={{ scale: 0.9 }}
                                                        style={{
                                                            width: 32, height: 32, borderRadius: 10, border: 'none',
                                                            color: record?.status === key ? 'white' : 'var(--text-muted)',
                                                            background: record?.status === key ? c.color : '#f1f5f9',
                                                            cursor: 'pointer', display: 'flex', alignItems: 'center', justifyContent: 'center'
                                                        }}
                                                        onClick={() => markAttendance(emp.id, key)}
                                                        title={c.label}
                                                    >
                                                        {c.icon}
                                                    </motion.button>
                                                ))}
                                            </div>
                                        </td>
                                    </motion.tr>
                                );
                            })}
                        </tbody>
                    </table>
                </div>
            </div>
        );
};

export default Attendance;
```

frontend/src/pages/hr/AdvanceSalaryModal.tsx

```
import React, { useState } from 'react';
import { FaTimes } from 'react-icons/fa';
import { apiFetch } from '../../utils/api';

interface Props {
    employeeId: number;
    employeeName: string;
    onClose: () => void;
    onSuccess: () => void;
}

const AdvanceSalaryModal: React.FC<Props> = ({ employeeId, employeeName, onClose, onSuccess }) => {
    const [amount, setAmount] = useState('');
    const [date, setDate] = useState(new Date().toISOString().substring(0, 10));
    const [type, setType] = useState('ONE_TIME');
    const [installment, setInstallment] = useState('');
    const [reason, setReason] = useState('');
    const [loading, setLoading] = useState(false);

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setLoading(true);
        try {
            await apiFetch('/hr/advance', {
                method: 'POST',
                body: {
                    employee_id: employeeId,
                    amount: Number(amount),
                    date,
                    reason,
                    repayment_type: type,
                    // Send 0 for manual/one-time types
                    installment_amount: (type === 'ONE_TIME' || type === 'MANUAL') ? 0 : Number(installment)
                }
            });
            onSuccess();
            onClose();
        } catch (err) {
            alert("Failed to record advance");
        } finally {
            setLoading(false);
        }
    };

    return (
        <div style={{ position: 'fixed', top: 0, left: 0, width: '100%', height: '100%', background: 'rgba(0,0,0,0.5)', display: 'flex', alignItems: 'c
            <div style={{ background: 'white', width: '500px', borderRadius: '12px', padding: '25px' }}>
                <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '20px' }}>
                    <h3 style={{ margin: 0 }}>Advance for {employeeName}</h3>
                    <button onClick={onClose} style={{ border: 'none', background: 'none', cursor: 'pointer' }}><FaTimes /></button>
                </div>

                <form onSubmit={handleSubmit}>
                    <div style={{ marginBottom: '15px' }}>
                        <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600, marginBottom: '5px' }}>Amount (₹)</label>
                        <input type="number" required value={amount} onChange={e => setAmount(e.target.value)} style={{ width: '100%', padding: '10px',
                    </div>

                    <div style={{ marginBottom: '15px' }}>
                        <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600, marginBottom: '5px' }}>Repayment Type</label>
                        <select value={type} onChange={e => setType(e.target.value)} style={{ width: '100%', padding: '10px', border: '1px solid #cbd5e
                            <option value="ONE_TIME">Deduct Full in Next Salary</option>
                            <option value="INSTALLMENT">Fixed Monthly Installment</option>
                            <option value="DAILY">Deduct Daily (Per Working Day)</option>
                            {/* ✅ NEW OPTION ADDED HERE */}
                            <option value="MANUAL">Flexible / Any Time (Manual)</option>
                        </select>
                    </div>

                    {/* Show installment input only for INSTALLMENT or DAILY types */}
                    {(type === 'INSTALLMENT' || type === 'DAILY') && (
                        <div style={{ marginBottom: '15px', background: '#eff6ff', padding: '10px', borderRadius: '6px' }}>
                            <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600, marginBottom: '5px' }}>
                                {type === 'DAILY' ? 'Daily Deduction Amount' : 'Monthly Installment Amount'}
                            </label>
                            <input type="number" required value={installment} onChange={e => setInstallment(e.target.value)} style={{ width: '100%', pa
                        </div>
                    )}

                    {type === 'MANUAL' && (
                        <div style={{ marginBottom: '15px', background: '#f0fdf4', padding: '10px', borderRadius: '6px', fontSize: '0.85rem', color: '#
                            ℹ️ No automatic deduction. Admin will deduct manually during payroll or via cash receipt when employee wishes to pay.
                        </div>
                    )}

                    <div style={{ marginBottom: '20px' }}>
                        <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600, marginBottom: '5px' }}>Reason / Notes</label>
                        <textarea value={reason} onChange={e => setReason(e.target.value)} style={{ width: '100%', padding: '10px', border: '1px solid #
                    </div>

                    <button type="submit" disabled={loading} style={{ width: '100%', padding: '12px', background: '#2563eb', color: 'white', border: 'no
                        {loading ? 'Processing...' : 'Grant Advance'}
                    </button>
                </form>
            </div>
        </div>
    );
};

export default AdvanceSalaryModal;
```

frontend/src/pages/finance/LoanScheduleViewer.tsx

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import './Finance.css';

const LoanScheduleViewer: React.FC<{ loanId: number, onClose: () => void }> = ({ loanId, onClose }) => {
    const [schedule, setSchedule] = useState<any[]>([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const fetchSchedule = async () => {
            try {
                const res = await axios.get(`http://localhost:5001/api/finance/loans/${loanId}/schedule`);
                setSchedule(res.data);
            } catch (err) {
                console.error(err);
            } finally {
                setLoading(false);
            }
        };
        fetchSchedule();
    }, [loanId]);

    return (
        <div className="modal-overlay">
            <div className="modal-content card max-w-4xl">
                <div className="flex justify-between items-center mb-4">
                    <h2>Repayment Schedule - Loan #{loanId}</h2>
                    <button className="btn btn-sm btn-secondary" onClick={onClose}>Close</button>
                </div>
                {loading ? <p>Loading schedule...</p> : (
                    <div className="table-wrapper">
                        <table className="modern-table">
                            <thead>
                                <tr>
                                    <th>Due Date</th>
                                    <th>Principal</th>
                                    <th>Interest</th>
                                    <th>Total Due</th>
                                    <th>Status</th>
                                </tr>
                            </thead>
                            <tbody>
                                {schedule.map((item, i) => (
                                    <tr key={i}>
                                        <td>{item.due_date}</td>
                                        <td>₹ {item.principal_component.toLocaleString()}</td>
                                        <td>₹ {item.interest_component.toLocaleString()}</td>
                                        <td class="font-bold">₹ {item.total_due.toLocaleString()}</td>
                                        <td><span class={`status-pill ${item.status.toLowerCase()}`}>{item.status}</span></td>
                                    </tr>
                                ))}
                            </tbody>
                        </table>
                    </div>
                )}
            </div>
        </div>
    );
};

export default LoanScheduleViewer;
```

frontend/src/pages/finance/LoanManagement.tsx

```
import React, { useState, useEffect } from 'react';
import { financeApi } from './financeApi';
import './Finance.css';

const LoanManagement: React.FC = () => {
    const [loans, setLoans] = useState<any[]>([]);
    const [showModal, setShowModal] = useState(false);
    const [loading, setLoading] = useState(false);
    const companyId = 1; // Real app would get this from Auth context/props

    const [formData, setFormData] = useState({
        party_name: '',
        party_type: 'EMPLOYEE',
        loan_direction: 'GIVEN',
        principal_amount: 0,
        interest_rate: 12,
        interest_type: 'EMI',
        start_date: new Date().toISOString().split('T')[0],
        duration_months: 12
    });

    const fetchLoans = async () => {
        try {
            const res = await financeApi.getLoans(companyId);
            setLoans(res.data);
        } catch (err) {
            console.error("Failed to fetch loans", err);
        }
    };

    useEffect(() => {
        fetchLoans();
    }, [companyId]);

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setLoading(true);
        try {
            await financeApi.createLoan({ ...formData, company_id: companyId });
            setShowModal(false);
            fetchLoans(); // Refresh real data
            alert('Loan created successfully!');
        } catch (err) {
            console.error(err);
        } finally {
            setLoading(false);
        }
    };

    const stats = {
        totalGiven: loans.filter(l => l.loan_direction === 'GIVEN').reduce((acc, curr) => acc + curr.principal_amount, 0),
        interestAccrued: loans.reduce((acc, curr) => acc + (curr.principal_amount * curr.interest_rate / 100), 0) / 12, // Simple monthly est for UI
        overdue: loans.filter(l => l.status === 'OVERDUE').reduce((acc, curr) => acc + curr.outstanding_amount, 0)
    };

    return (
        <div className="finance-container">
            <div className="finance-header">
                <h1>Loan & Advance Management</h1>
                <button className="btn btn-primary" onClick={() => setShowModal(true)}>
                    <i className="fas fa-plus"></i> New Loan
                </button>
            </div>

            <div className="stats-grid">
                <div className="stat-card">
                    <h3>Total Loans Given</h3>
                    <p className="stat-value">₹ {stats.totalGiven.toLocaleString()}</p>
                    <span className="stat-change positive">Real-time data</span>
                </div>
                <div className="stat-card">
                    <h3>Interest Earned (Est.)</h3>
                    <p className="stat-value">₹ {stats.interestAccrued.toLocaleString()}</p>
                    <span className="stat-change">Accrued from ledger</span>
                </div>
                <div className="stat-card">
                    <h3>Overdue Outstanding</h3>
                    <p className="stat-value overdue">₹ {stats.overdue.toLocaleString()}</p>
                    <span className="stat-change negative">{loans.filter(l => l.status === 'OVERDUE').length} cases</span>
                </div>
            </div>

            <div className="table-wrapper card">
                <h3>Active Loans</h3>
                <table className="modern-table">
                    <thead>
                        <tr>
                            <th>Party Name</th>
                            <th>Type</th>
                            <th>Principal</th>
                            <th>Rate</th>
                            <th>Duration</th>
                            <th>Outstanding</th>
                            <th>Status</th>
                        </tr>
                    </thead>
                    <tbody>
                        {loans.map(loan => (
                            <tr key={loan.id}>
                                <td>{loan.party_name}</td>
                                <td><span className={`badge badge-${loan.party_type === 'BANK' ? 'danger' : 'info'}`}>{loan.party_type}</span></td>
                                <td>₹ {loan.principal_amount.toLocaleString()}</td>
                                <td>{loan.interest_rate}%</td>
                                <td>{loan.duration_months} Mo</td>
                                <td>₹ {loan.outstanding_amount.toLocaleString()}</td>
                                <td><span className={`status-pill ${loan.status.toLowerCase()}`}>{loan.status}</span></td>
```

```
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>

        {showModal && (
            <div className="modal-overlay">
                <div className="modal-content card">
                    <h2>Register New Loan</h2>
                    <form onSubmit={handleSubmit}>
                        <div className="form-group">
                            <label>Party Name</label>
                            <input type="text" required onChange={e => setFormData({...formData, party_name: e.target.value})} />
                        </div>
                        <div className="form-row">
                            <div className="form-group">
                                <label>Principal Amount</label>
                                <input type="number" required onChange={e => setFormData({...formData, principal_amount: Number(e.target.value)})} /
                            </div>
                            <div className="form-group">
                                <label>Interest Rate (%)</label>
                                <input type="number" step="0.1" defaultValue="12" onChange={e => setFormData({...formData, interest_rate: Number(e.t
                            </div>
                        </div>
                        <div class="form-row">
                            <div class="form-group">
                                <label>Type</label>
                                <select onChange={e => setFormData({...formData, interest_type: e.target.value})}>
                                    <option value="EMI">EMI Based</option>
                                    <option value="SIMPLE">Simple Interest</option>
                                </select>
                            </div>
                            <div class="form-group">
                                <label>Duration (Months)</label>
                                <input type="number" defaultValue="12" onChange={e => setFormData({...formData, duration_months: Number(e.target.val
                            </div>
                        </div>
                        <div class="modal-actions">
                            <button type="button" class="btn btn-secondary" onClick={() => setShowModal(false)}>Cancel</button>
                            <button type="submit" class="btn btn-primary" disabled={loading}>
                                {loading ? 'Processing...' : 'Create Loan'}
                            </button>
                        </div>
                    </form>
                </div>
            </div>
        )}
    </div>
  );
};

export default LoanManagement;
```

frontend/src/pages/finance/FinancialReports.tsx

```
import React, { useState, useEffect } from 'react';
import { financeApi } from './financeApi';
import './Finance.css';

const FinancialReports: React.FC = () => {
    const [trialBalance, setTrialBalance] = useState<any>(null);
    const [pnL, setPnL] = useState<any>(null);
    const [loading, setLoading] = useState(true);
    const companyId = 1;

    useEffect(() => {
        const fetchReports = async () => {
            setLoading(true);
            try {
                const [tb, pl] = await Promise.all([
                    financeApi.getTrialBalance(companyId),
                    financeApi.getProfitLoss(companyId, '2020-01-01', new Date().toISOString().split('T')[0])
                ]);
                setTrialBalance(tb.data);
                setPnL(pl.data);
            } catch (err) {
                console.error("Error fetching reports", err);
            } finally {
                setLoading(false);
            }
        };
        fetchReports();
    }, [companyId]);

    if (loading) return <div className="loading">Generating Automated Financial Reports...</div>;

    return (
        <div className="finance-container">
            <div className="finance-header">
                <h1>Automated Financial Reports</h1>
                <div className="report-controls">
                    <button className="btn btn-secondary" onClick={() => window.print()}>
                        <i className="fas fa-print"></i> Print Reports
                    </button>
                </div>
            </div>

            <div className="report-grid">
                {/* Profit & Loss Statement */}
                <div className="report-card card">
                    <h2>Profit & Loss Statement (Real-time)</h2>
                    <div className="report-section">
                        <h3>Incomes</h3>
                        <ul className="report-list">
                            {pnL?.income?.map((item: any, i: number) => (
                                <li key={i}><span>{item.name}</span> <span>₹ {item.amount.toLocaleString()}</span></li>
                            ))}
                        </ul>
                        <div className="report-total">Total Income: ₹ {pnL?.total_income?.toLocaleString()}</div>
                    </div>
                    <div className="report-section mt-4">
                        <h3>Expenses</h3>
                        <ul className="report-list">
                            {pnL?.expenses?.map((item: any, i: number) => (
                                <li key={i}><span>{item.name}</span> <span className="negative">₹ {item.amount.toLocaleString()}</span></li>
                            ))}
                        </ul>
                        <div className="report-total">Total Expense: ₹ {pnL?.total_expense?.toLocaleString()}</div>
                    </div>
                    <div className="net-profit mt-4">
                        <h3>Net Profit: <span className={pnL?.net_profit >= 0 ? 'positive' : 'negative'}>₹ {pnL?.net_profit?.toLocaleString()}</span></h
                    </div>
                </div>

                {/* Trial Balance */}
                <div className="report-card card">
                    <h2>Trial Balance</h2>
                    <table className="modern-table">
                        <thead>
                            <tr>
                                <th>Account</th>
                                <th class="num">Debit (₹)</th>
                                <th class="num">Credit (₹)</th>
                            </tr>
                        </thead>
                        <tbody>
                            {trialBalance?.data?.map((item: any, i: number) => (
                                <tr key={i}>
                                    <td>{item.name}</td>
                                    <td class="num">{item.debit > 0 ? item.debit.toLocaleString() : '-'}</td>
                                    <td class="num">{item.credit > 0 ? item.credit.toLocaleString() : '-'}</td>
                                </tr>
                            ))}
                        </tbody>
                        <tfoot>
                            <tr class="total-row">
                                <td>TOTAL</td>
                                <td class="num">{trialBalance?.total_debit?.toLocaleString()}</td>
                                <td class="num">{trialBalance?.total_credit?.toLocaleString()}</td>
                            </tr>
                        </tfoot>
                    </table>
                </div>
            </div>
        </div>
    );
};

export default FinancialReports;
```

frontend/src/pages/finance/FinanceDashboard.tsx

```
import React, { useState, useEffect } from 'react';
import { financeApi } from './financeApi';
import './Finance.css';

const FinanceDashboard: React.FC = () => {
    const [summary, setSummary] = useState({
        totalCash: 0,
        totalBank: 0,
        totalReceivables: 0,
        totalPayables: 0,
        netProfit: 0
    });
    const companyId = 1;

    useEffect(() => {
        const fetchDashboardData = async () => {
            try {
                const [accountsRes, plRes] = await Promise.all([
                    financeApi.getAccounts(companyId),
                    financeApi.getProfitLoss(companyId, '2020-01-01', new Date().toISOString().split('T')[0])
                ]);

                const accounts = accountsRes.data;
                const getBal = (name: string) => accounts.find((a: any) => a.name === name)?.balance || 0;

                setSummary({
                    totalCash: getBal('Cash'),
                    totalBank: getBal('Bank'),
                    totalReceivables: getBal('Accounts Receivable'),
                    totalPayables: getBal('Accounts Payable'),
                    netProfit: plRes.data.net_profit
                });
            } catch (err) {
                console.error(err);
            }
        };
        fetchDashboardData();
    }, [companyId]);

    return (
        <div className="finance-container">
            <div className="finance-header">
                <h1>Finance Command Center</h1>
                <div className="company-badge">Company ID: {companyId}</div>
            </div>

            <div className="stats-grid">
                <div className="stat-card">
                    <h3>Liquid Cash</h3>
                    <p className="stat-value">₹ {summary.totalCash.toLocaleString()}</p>
                    <span className="stat-change positive">In Hand</span>
                </div>
                <div className="stat-card">
                    <h3>Bank Balance</h3>
                    <p className="stat-value">₹ {summary.totalBank.toLocaleString()}</p>
                    <span className="stat-change">Across Accounts</span>
                </div>
                <div className="stat-card">
                    <h3>Net Profit (YTD)</h3>
                    <p className={`stat-value ${summary.netProfit >= 0 ? 'positive' : 'negative'}`}>
                        ₹ {summary.netProfit.toLocaleString()}
                    </p>
                    <span className="stat-change">Automated Posting</span>
                </div>
            </div>

            <div className="dashboard-grid mt-4">
                <div className="card">
                    <h3>Outstanding Overview</h3>
                    <div className="progress-list">
                        <div className="progress-item">
                            <span>Receivables</span>
                            <span className="num">₹ {summary.totalReceivables.toLocaleString()}</span>
                        </div>
                        <div className="progress-item">
                            <span>Payables</span>
                            <span className="num negative">₹ {summary.totalPayables.toLocaleString()}</span>
                        </div>
                    </div>
                </div>

                <div class="card">
                    <h3>Recent Automated Postings</h3>
                    <ul class="report-list">
                        <li><span>Sales Invoice #1024</span> <span class="positive">+₹ 12,000</span></li>
                        <li><span>Purchase Bill #SB-99</span> <span class="negative">-₹ 4,500</span></li>
                        <li><span>Salary Batch Feb 26</span> <span class="negative">-₹ 2,40,000</span></li>
                    </ul>
                </div>
            </div>
        </div>
    );
};

export default FinanceDashboard;
```

frontend/src/pages/finance/CashReceipts.tsx

```
import React, { useState, useRef } from 'react';
import { financeApi } from './financeApi';

const CashReceipts: React.FC = () => {
    const [partyName, setPartyName] = useState('');
    const [amount, setAmount] = useState(0);
    const [purpose, setPurpose] = useState('');
    const [loading, setLoading] = useState(false);
    const canvasRef = useRef<HTMLCanvasElement>(null);
    const companyId = 1;

    const clearSignature = () => {
        const canvas = canvasRef.current;
        if (canvas) {
            const ctx = canvas.getContext('2d');
            ctx?.clearRect(0, 0, canvas.width, canvas.height);
        }
    };

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setLoading(true);
        const canvas = canvasRef.current;
        const signatureBase64 = canvas ? canvas.toDataURL() : null;

        try {
            const res = await financeApi.createCashReceipt({
                company_id: companyId,
                party_name: partyName,
                amount,
                purpose,
                signature_base64: signatureBase64,
                created_by: 1
            });
            alert('Cash Receipt Generated!');
            window.open(financeApi.getReceiptPdfUrl(res.data.id), '_blank');
        } catch (err) {
            console.error(err);
        } finally {
            setLoading(false);
        }
    };

    // Simple canvas drawing logic
    let drawing = false;
    const startDrawing = (e: any) => {
        drawing = true;
        draw(e);
    };
    const endDrawing = () => {
        drawing = false;
        const canvas = canvasRef.current;
        if (canvas) canvas.getContext('2d')?.beginPath();
    };
    const draw = (e: any) => {
        if (!drawing) return;
        const canvas = canvasRef.current;
        const ctx = canvas?.getContext('2d');
        if (!canvas || !ctx) return;

        const rect = canvas.getBoundingClientRect();
        const x = (e.clientX || e.touches[0].clientX) - rect.left;
        const y = (e.clientY || e.touches[0].clientY) - rect.top;

        ctx.lineWidth = 2;
        ctx.lineCap = 'round';
        ctx.strokeStyle = '#000';

        ctx.lineTo(x, y);
        ctx.stroke();
        ctx.beginPath();
        ctx.moveTo(x, y);
    };

    return (
        <div className="finance-container">
            <div className="finance-header">
                <h1>Cash Receipt Voucher</h1>
            </div>

            <div className="receipt-form-card card">
                <form onSubmit={handleSubmit}>
                    <div className="form-group">
                        <label>Party Name</label>
                        <input type="text" className="form-control" required value={partyName} onChange={e => setPartyName(e.target.value)} />
                    </div>
                    <div class="form-row">
                        <div class="form-group">
                            <label>Amount (₹)</label>
                            <input type="number" class="form-control" required value={amount} onChange={e => setAmount(Number(e.target.value))} />
                        </div>
                        <div class="form-group">
                            <label>Purpose</label>
                            <input type="text" class="form-control" required value={purpose} onChange={e => setPurpose(e.target.value)} />
                        </div>
                    </div>

                    <div class="signature-section">
                        <label>Digital Signature (Draw below)</label>
                        <div class="canvas-container">
                            <canvas
                                ref={canvasRef}
                                width={400}
                                height={200}
                                onMouseDown={startDrawing}
                                onMouseUp={endDrawing}
```

```
                                onMouseMove={draw}
                                onTouchStart={startDrawing}
                                onTouchEnd={endDrawing}
                                onTouchMove={draw}
                            />
                        </div>
                        <button type="button" className="btn btn-sm btn-secondary" onClick={clearSignature}>Clear Signature</button>
                    </div>

                    <div className="form-actions mt-4">
                        <button type="submit" className="btn btn-lg btn-success w-100" disabled={loading}>
                            {loading ? 'Processing...' : 'Generate Receipt & Post Entry'}
                        </button>
                    </div>
                </form>
            </div>
        </div>
    );
};

export default CashReceipts;
```

frontend/src/pages/finance/BankReconciliation.tsx

```
import React, { useState, useEffect } from 'react';
import { financeApi } from './financeApi';
import './Finance.css';

const BankReconciliation: React.FC = () => {
    const [bankAccounts, setBankAccounts] = useState<any[]>([]);
    const [selectedBank, setSelectedBank] = useState<number | null>(null);
    const [transactions, setTransactions] = useState<any[]>([]);
    const [loading, setLoading] = useState(false);
    const companyId = 1;

    useEffect(() => {
        const fetchBanks = async () => {
            try {
                const res = await financeApi.getBankAccounts(companyId);
                setBankAccounts(res.data);
                if (res.data.length > 0) setSelectedBank(res.data[0].id);
            } catch (err) {
                console.error(err);
            }
        };
        fetchBanks();
    }, [companyId]);

    const fetchTransactions = async () => {
        if (!selectedBank) return;
        setLoading(true);
        try {
            const res = await financeApi.getBankTransactions(companyId, selectedBank);
            setTransactions(res.data);
        } catch (err) {
            console.error(err);
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        fetchTransactions();
    }, [selectedBank]);

    const handleAutoReconcile = async () => {
        if (!selectedBank) return;
        setLoading(true);
        try {
            const res = await financeApi.reconcileBank(companyId, selectedBank);
            alert(`Auto-reconciliation complete! Matched ${res.data.matched_count} transactions.`);
            fetchTransactions();
        } catch (err) {
            console.error(err);
        } finally {
            setLoading(false);
        }
    };

    return (
        <div className="finance-container">
            <div className="finance-header">
                <h1>Bank Automation & Reconciliation</h1>
                <div className="actions">
                    <button className="btn btn-success" onClick={handleAutoReconcile} disabled={loading}>
                        <i className="fas fa-magic"></i> Run Auto-Matching
                    </button>
                    <button className="btn btn-primary ml-2">
                        <i className="fas fa-file-import"></i> Import Statement (CSV)
                    </button>
                </div>
            </div>

            <div className="bank-select card mb-4">
                <label>Select Bank Account:</label>
                <select value={selectedBank || ''} onChange={e => setSelectedBank(Number(e.target.value))}>
                    {bankAccounts.map(b => (
                        <option key={b.id} value={b.id}>{b.bank_name} - {b.account_number} (₹ {b.balance.toLocaleString()})</option>
                    ))}
                </select>
            </div>

            <div className="table-wrapper card">
                <h3>Bank Transactions vs Ledger</h3>
                <table className="modern-table">
                    <thead>
                        <tr>
                            <th>Date</th>
                            <th>Description</th>
                            <th>Amount</th>
                            <th>Type</th>
                            <th>Status</th>
                            <th>Action</th>
                        </tr>
                    </thead>
                    <tbody>
                        {transactions.map(tx => (
                            <tr key={tx.id}>
                                <td>{tx.date}</td>
                                <td>{tx.description}</td>
                                <td class="num">₹ {tx.amount.toLocaleString()}</td>
                                <td><span class={`badge ${tx.type === 'CR' ? 'badge-success' : 'badge-danger'}`}>{tx.type}</span></td>
                                <td>
                                    {tx.reconciled ?
                                        <span class="status-pill active">Reconciled</span> :
                                        <span class="status-pill pending">Unmatched</span>
                                    }
                                </td>
                                <td>
                                    {!tx.reconciled && <button class="btn btn-sm btn-outline-primary">Manual Match</button>}
```

```
                                </td>
                            </tr>
                        ))}
                        {transactions.length === 0 && (
                            <tr><td colSpan={6} style={{ textAlign: 'center' }}>No transactions found for this bank account.</td></tr>
                        )}
                    </tbody>
                </table>
            </div>
        </div>
    );
};

export default BankReconciliation;
```

frontend/src/pages/customer/MyLedger.tsx

```
import React, { useEffect, useState } from 'react';
import { FaCalendarAlt, FaCheckCircle, FaDownload, FaExclamationCircle, FaFileInvoiceDollar, FaReceipt } from 'react-icons/fa';
import { apiFetch } from '../../utils/api';

const MyLedger: React.FC = () => {
    const [data, setData] = useState<any>(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const load = async () => {
            try {
                // Fetches summary + transactions for logged-in user
                const res = await apiFetch('/portal/my-ledger');
                const json = await res.json();
                setData(json);
            } catch (e) {
                console.error("Ledger Load Error:", e);
            } finally {
                setLoading(false);
            }
        };
        load();
    }, []);

    const fmt = (n: any) => Number(n || 0).toLocaleString('en-IN', {
        style: 'currency',
        currency: 'INR',
        minimumFractionDigits: 0
    });

    if (loading) {
        return (
            <div style={{ padding: '80px 20px', textAlign: 'center' }}>
                <div className="flex-center" style={{ flexDirection: 'column', gap: '20px' }}>
                    <div className="loading-spinner" style={{ width: '48px', height: '48px', border: '5px solid var(--primary-glow)', borderTop: '5px so
                    <p style={{ color: 'var(--text-muted)', fontWeight: 600, fontSize: '1.1rem' }}>Preparing your financial statement...</p>
                </div>
            </div>
        );
    }

    const { summary, transactions } = data || { summary: {}, transactions: [] };

    return (
        <div className="page-transition" style={{ paddingBottom: '40px' }}>
            {/* Header Section */}
            <div style={{ marginBottom: '32px' }}>
                <h1 style={{ fontSize: '2.25rem', fontWeight: 900, color: 'var(--text-main)', letterSpacing: '-1px', margin: 0 }}>My Financial Ledger</h
                <p style={{ color: 'var(--text-muted)', fontSize: '1.1rem', marginTop: '4px', fontWeight: 500 }}>Track your billing history, payments, a
            </div>

            {/* Summary Cards with Premium Styling */}
            <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(280px, 1fr))', gap: '24px', marginBottom: '40px' }}>
                {/* Total Invoiced */}
                <div className="card" style={{ padding: '28px', borderLeft: '5px solid var(--primary)', boxShadow: 'var(--shadow-md)' }}>
                    <div style={{ color: 'var(--text-muted)', marginBottom: '12px', fontSize: '0.85rem', fontWeight: 700, textTransform: 'uppercase', le
                        <div style={{ background: 'var(--primary-glow)', padding: '8px', borderRadius: '8px', color: 'var(--primary)' }}><FaFileInvoiceD
                        Total Invoiced
                    </div>
                    <div style={{ fontSize: '2.25rem', fontWeight: 900, color: 'var(--text-main)', lineHeight: 1 }}>
                        {fmt(summary?.total_billed)}
                    </div>
                </div>

                {/* Total Paid */}
                <div className="card" style={{ padding: '28px', borderLeft: '5px solid var(--success)', boxShadow: 'var(--shadow-md)' }}>
                    <div style={{ color: 'var(--success)', marginBottom: '12px', fontSize: '0.85rem', fontWeight: 700, textTransform: 'uppercase', lette
                        <div style={{ background: 'rgba(16, 185, 129, 0.1)', padding: '8px', borderRadius: '8px' }}><FaCheckCircle /></div>
                        Settled Amount
                    </div>
                    <div style={{ fontSize: '2.25rem', fontWeight: 900, color: 'var(--success)', lineHeight: 1 }}>
                        {fmt(summary?.total_paid)}
                    </div>
                </div>

                {/* Balance Pending */}
                <div className="card" style={{
                    padding: '28px',
                    borderLeft: '5px solid var(--warning)',
                    background: 'linear-gradient(135deg, #ffffff 0%, #fffbf0 100%)',
                    boxShadow: 'var(--shadow-md)'
                }}>
                    <div style={{ color: 'var(--warning)', marginBottom: '12px', fontSize: '0.85rem', fontWeight: 700, textTransform: 'uppercase', lette
                        <div style={{ background: 'rgba(245, 158, 11, 0.1)', padding: '8px', borderRadius: '8px' }}><FaExclamationCircle /></div>
                        Outstanding Balance
                    </div>
                    <div style={{ fontSize: '2.25rem', fontWeight: 900, color: '#c2410c', lineHeight: 1 }}>
                        {fmt(summary?.balance_pending)}
                    </div>
                </div>
            </div>

            {/* Transactions Section */}
            <div style={{ padding: 0, overflow: 'hidden', border: '1px solid var(--border-color)', background: 'white', boxShadow: 'va
                <div style={{ padding: '20px 24px', borderBottom: '1px solid var(--border-color)', background: '#f8fafc', display: 'flex', alignItems:
                    <FaReceipt style={{ color: 'var(--primary)', opacity: 0.8 }} />
                    <h3 style={{ margin: 0, fontSize: '1.1rem', fontWeight: 800, color: 'var(--text-main)' }}>Transaction History</h3>
                </div>

                <div style={{ overflowX: 'auto' }}>
                    <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: '0.95rem', minWidth: '850px' }}>
                        <thead>
                            <tr style={{ background: '#f8fafc', borderBottom: '1px solid var(--border-color)' }}>
                                <th style={{ padding: '16px 24px', textAlign: 'left', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.75rem',
                                <th style={{ padding: '16px 24px', textAlign: 'left', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.75rem',
                                <th style={{ padding: '16px 24px', textAlign: 'right', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.75rem'
```

```
                        <th style={{ padding: '16px 24px', textAlign: 'right', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.75rem'
                        <th style={{ padding: '16px 24px', textAlign: 'center', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.75rem
                        <th style={{ padding: '16px 24px', textAlign: 'center', color: 'var(--text-muted)', fontWeight: 700, fontSize: '0.75rem
                    </tr>
                </thead>
                <tbody>
                    {!transactions || transactions.length === 0 ? (
                        <tr>
                            <td colSpan={6} style={{ padding: '80px 40px', textAlign: 'center' }}>
                                <div style={{ opacity: 0.4 }}>
                                    <FaReceipt size={48} style={{ color: 'var(--text-light)', marginBottom: '16px' }} />
                                    <p style={{ margin: 0, fontWeight: 700, fontSize: '1.25rem', color: 'var(--text-main)' }}>No Transactions Fo
                                    <p style={{ margin: '4px 0 0', color: 'var(--text-muted)' }}>You haven't been invoiced for any services yet
                                </div>
                            </td>
                        </tr>
                    ) : transactions.map((inv: any) => (
                        <tr key={inv.id} style={{ borderBottom: '1px solid var(--bg-body)', transition: 'background-color 0.2s' }} className="ro
                            <td style={{ padding: '20px 24px' }}>
                                <div style={{ display: 'flex', alignItems: 'center', gap: '10px' }}>
                                    <FaCalendarAlt style={{ color: 'var(--text-light)', fontSize: '0.85rem' }} />
                                    <span style={{ fontWeight: 700, color: 'var(--text-main)' }}>
                                        {new Date(inv.invoice_date).toLocaleDateString('en-GB', { day: '2-digit', month: 'short', year: 'numeric
                                    </span>
                                </div>
                            </td>
                            <td style={{ padding: '20px 24px' }}>
                                <span style={{ color: 'var(--primary)', fontWeight: 800, padding: '4px 10px', background: 'var(--primary-glow)'
                                    {inv.invoice_number}
                                </span>
                            </td>
                            <td style={{ padding: '20px 24px', textAlign: 'right', fontWeight: 800, color: 'var(--text-main)', fontSize: '1rem'
                                {fmt(inv.total_amount)}
                            </td>
                            <td style={{ padding: '20px 24px', textAlign: 'right', color: 'var(--success)', fontWeight: 700 }}>
                                {fmt(inv.paid_amount)}
                            </td>
                            <td style={{ padding: '20px 24px', textAlign: 'center' }}>
                                <span style={{
                                    background: inv.status === 'Paid' ? 'rgba(16, 185, 129, 0.1)' : 'rgba(245, 158, 11, 0.1)',
                                    color: inv.status === 'Paid' ? 'var(--success)' : 'var(--warning)',
                                    padding: '6px 14px', borderRadius: '20px', fontSize: '0.75rem', fontWeight: 800, textTransform: 'uppercase'
                                }}>
                                    {inv.status || 'PENDING'}
                                </span>
                            </td>
                            <td style={{ padding: '20px 24px', textAlign: 'center' }}>
                                {inv.file_url ? (
                                    <button
                                        onClick={() => window.open(`http://localhost:3001${inv.file_url}`, '_blank')}
                                        style={{ border: 'none', background: 'var(--bg-body)', color: 'var(--primary)', width: '38px', height:
                                        className="icon-btn-hover"
                                        title="View/Download Receipt"
                                    >
                                        <FaDownload />
                                    </button>
                                ) : (
                                    <span style={{color: 'var(--border-color)', fontWeight: 800}}>•</span>
                                )}
                            </td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    </div>

    <style>{`
        .row-hover:hover { background-color: #f8fafc; }
        .icon-btn-hover:hover { background: var(--primary) !important; color: white !important; transform: scale(1.1); }
        @keyframes spin { 0% { transform: rotate(0deg); } 100% { transform: rotate(360deg); } }
    `}</style>
  </div>
);
};

export default MyLedger;
```

frontend/src/pages/customer/Marketplace.tsx

```
import React, { useEffect, useState } from 'react';
import { FaBox, FaShoppingCart, FaTag } from 'react-icons/fa';
import { apiFetch } from '../../utils/api';

const Marketplace: React.FC = () => {
    const [products, setProducts] = useState<any[]>([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const load = async () => {
            try {
                // Fetch catalog specifically designed for customers (no cost price)
                const res = await apiFetch('/portal/catalog');
                const data = await res.json();
                setProducts(Array.isArray(data) ? data : []);
            } catch (e) {
                console.error("Catalog Load Error:", e);
            } finally {
                setLoading(false);
            }
        };
        load();
    }, []);

    if (loading) return <div style={{textAlign: 'center', padding: '50px', color: '#64748b'}}>Loading Catalog...</div>;

    return (
        <div>
            <div style={{ marginBottom: '30px' }}>
                <h1 style={{ fontSize: '2rem', fontWeight: 700, color: '#1e293b', marginBottom: '10px' }}>Product Catalog</h1>
                <p style={{ color: '#64748b' }}>Browse our latest inventory and place orders directly.</p>
            </div>

            {products.length === 0 ? (
                <div style={{ textAlign: 'center', padding: '60px', background: 'white', borderRadius: '12px', border: '1px dashed #e2e8f0' }}>
                    <FaBox size={40} style={{ marginBottom: '15px', color: '#cbd5e1' }} />
                    <p style={{ color: '#94a3b8' }}>No active products available at the moment.</p>
                </div>
            ) : (
                <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(280px, 1fr))', gap: '25px' }}>
                    {products.map(product => (
                        <div key={product.id} style={{
                            background: 'white', borderRadius: '16px', overflow: 'hidden',
                            border: '1px solid #e2e8f0', boxShadow: '0 4px 6px -1px rgba(0,0,0,0.05)',
                            display: 'flex', flexDirection: 'column', transition: 'transform 0.2s',
                            cursor: 'default'
                        }}>
                            {/* Product Image */}
                            <div style={{ height: '200px', background: '#f8fafc', display: 'flex', alignItems: 'center', justifyContent: 'center', colo
                                {product.image_url ? (
                                    <img src={product.image_url} style={{width:'100%', height:'100%', objectFit:'cover'}} alt={product.name} />
                                ) : (
                                    <FaBox size={48} />
                                )}
                                {/* Stock Tag */}
                                {product.current_stock > 0 ? (
                                    <span style={{ position: 'absolute', top: 10, right: 10, background: '#dcfce7', color: '#166534', fontSize: '0.75rem
                                        In Stock
                                    </span>
                                ) : (
                                    <span style={{ position: 'absolute', top: 10, right: 10, background: '#fee2e2', color: '#991b1b', fontSize: '0.75rem
                                        Out of Stock
                                    </span>
                                )}
                            </div>

                            {/* Details */}
                            <div style={{ padding: '20px', flexGrow: 1, display: 'flex', flexDirection: 'column' }}>
                                <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'start', marginBottom: '8px' }}>
                                    <h3 style={{ fontSize: '1.1rem', fontWeight: 600, color: '#1e293b', margin: 0, lineHeight: 1.4 }}>
                                        {product.name}
                                    </h3>
                                </div>

                                {product.sku && (
                                    <div style={{ display: 'flex', alignItems: 'center', gap: '6px', fontSize: '0.8rem', color: '#64748b', marginBottom
                                        <FaTag size={10} /> SKU: {product.sku}
                                    </div>
                                )}

                                <p style={{ color: '#64748b', fontSize: '0.9rem', marginBottom: '20px', flexGrow: 1, lineHeight: 1.5 }}>
                                    {product.description || 'No detailed description available.'}
                                </p>

                                {/* Footer: Price & Action */}
                                <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', borderTop: '1px solid #f1f5f9', pa
                                    <div>
                                        <div style={{ fontSize: '0.8rem', color: '#94a3b8' }}>Price</div>
                                        <div style={{ fontSize: '1.2rem', fontWeight: 700, color: '#0f172a' }}>
                                            ₹{Number(product.selling_price).toLocaleString('en-IN')}
                                        </div>
                                    </div>
                                    <button
                                        onClick={() => alert("Order placement will be enabled soon. Please contact admin for manual orders.")}
                                        style={{
                                            background: '#2563eb', color: 'white', border: 'none',
                                            padding: '10px 16px', borderRadius: '8px', cursor: 'pointer',
                                            fontWeight: 600, display: 'flex', alignItems: 'center', gap: '8px',
                                            transition: 'background 0.2s', opacity: product.current_stock > 0 ? 1 : 0.5
                                        }}
                                        disabled={product.current_stock <= 0}
                                    >
                                        <FaShoppingCart /> Order
                                    </button>
                                </div>
                            </div>
```

```
                        </div>
                    ))}
                </div>
            )}
        </div>
    );
};

export default Marketplace;
```

frontend/src/context/TenantContext.tsx

```
import React, { createContext, useContext, useEffect, useState } from 'react';
import { useAuthUser } from '../hooks/useAuthUser';

interface Branch {
    id: number;
    branch_name: string;
    branch_code: string;
}

interface TenantContextType {
    activeBranch: Branch | null;
    branches: Branch[];
    setActiveBranch: (branch: Branch) => void;
    refreshBranches: () => Promise<void>;
    isLoadingBranches: boolean;
}

const TenantContext = createContext<TenantContextType | undefined>(undefined);

export const TenantProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
    const { user } = useAuthUser();
    const [branches, setBranches] = useState<Branch[]>([]);
    const [activeBranch, setActiveBranchState] = useState<Branch | null>(null);
    const [isLoadingBranches, setIsLoadingBranches] = useState(false);

    const refreshBranches = async () => {
        if (!user) return;
        setIsLoadingBranches(true);
        try {
            const token = localStorage.getItem('erp-token');
            const baseUrl = import.meta.env.VITE_API_URL || "http://localhost:3001";
            const res = await fetch(`${baseUrl}/api/branches`, {
                headers: { 'Authorization': `Bearer ${token}` }
            });
            if (res.ok) {
                const data = await res.json();
                setBranches(data);

                // Set default branch if none selected
                const savedBranchId = localStorage.getItem('active-branch-id');
                if (savedBranchId) {
                    const found = data.find((b: Branch) => b.id.toString() === savedBranchId);
                    if (found) {
                        setActiveBranchState(found);
                    } else if (data.length > 0) {
                        setActiveBranch(data[0]);
                    }
                } else if (data.length > 0) {
                    setActiveBranch(data[0]);
                }
            }
        } catch (err) {
            console.error("Failed to load branches:", err);
        } finally {
            setIsLoadingBranches(false);
        }
    };

    const setActiveBranch = (branch: Branch) => {
        setActiveBranchState(branch);
        localStorage.setItem('active-branch-id', branch.id.toString());
        // Trigger a custom event so apiFetch can update its headers if needed
        window.dispatchEvent(new Event('branchChanged'));
    };

    useEffect(() => {
        if (user) {
            refreshBranches();
        } else {
            setBranches([]);
            setActiveBranchState(null);
        }
    }, [user]);

    return (
        <TenantContext.Provider value={{ activeBranch, branches, setActiveBranch, refreshBranches, isLoadingBranches }}>
            {children}
        </TenantContext.Provider>
    );
};

export const useTenant = () => {
    const context = useContext(TenantContext);
    if (!context) throw new Error('useTenant must be used within a TenantProvider');
    return context;
};
```

frontend/src/components/TransactionHistoryModal.tsx

```
import React, { useEffect, useState } from 'react';
import { FaCalendarAlt, FaFileInvoiceDollar, FaHistory, FaTimes } from 'react-icons/fa';
import { apiFetch } from '../utils/api';

interface Transaction {
    id: number;
    date: string;
    type: string;
    category: string;
    amount: number;
    description: string;
    lender_name?: string;
    user_name?: string;
}

interface TransactionHistoryModalProps {
    isOpen: boolean;
    onClose: () => void;
    entityType: 'supplier' | 'customer';
    entityId: number;
    entityName: string;
}

const TransactionHistoryModal: React.FC<TransactionHistoryModalProps> = ({
    isOpen,
    onClose,
    entityType,
    entityId,
    entityName
}) => {
    const [transactions, setTransactions] = useState<Transaction[]>([]);
    const [loading, setLoading] = useState(false);
    const [isMobile, setIsMobile] = useState(window.innerWidth <= 768);

    useEffect(() => {
        const handleResize = () => setIsMobile(window.innerWidth <= 768);
        window.addEventListener('resize', handleResize);
        return () => window.removeEventListener('resize', handleResize);
    }, []);

    useEffect(() => {
        if (isOpen) {
            fetchTransactions();
        }
    }, [isOpen, entityId]);

    const fetchTransactions = async () => {
        setLoading(true);
        try {
            const queryParam = entityType === 'supplier' ? `lender_id=${entityId}` : `user_id=${entityId}`;
            const url = `/transactions?${queryParam}`;
            const response = await apiFetch(url);
            const data = await response.json();
            setTransactions(data);
        } catch (error: any) {
            console.error('❌ Failed to fetch transactions:', error);
        } finally {
            setLoading(false);
        }
    };

    const formatDate = (dateString: string) => {
        return new Date(dateString).toLocaleDateString('en-GB', {
            day: '2-digit',
            month: 'short',
            year: 'numeric'
        });
    };

    const fmt = (amount: number) => {
        return new Intl.NumberFormat('en-IN', {
            style: 'currency',
            currency: 'INR',
            minimumFractionDigits: 0
        }).format(amount);
    };

    const getTotalAmount = () => {
        return transactions.reduce((sum, t) => {
            const amt = parseFloat(t.amount.toString());
            if (entityType === 'customer') {
                return t.type === 'INVOICE' || (t.type === 'ADJUSTMENT' && amt >= 0) ? sum + amt : sum - Math.abs(amt);
            } else {
                return t.type === 'BILL' ? sum + amt : sum - amt;
            }
        }, 0);
    };

    if (!isOpen) return null;

    const netBalance = getTotalAmount();

    return (
        <div style={{
            position: 'fixed',
            top: 0,
            left: 0,
            right: 0,
            bottom: 0,
            backgroundColor: 'rgba(15, 23, 42, 0.4)',
            backdropFilter: 'blur(8px)',
            display: 'flex',
            alignItems: 'center',
            justifyContent: 'center',
            zIndex: 1000,
            padding: isMobile ? '0' : '20px',
```

```
    animation: 'fadeIn 0.3s ease-out'
}}>
    <div className="card" style={{
        backgroundColor: 'white',
        borderRadius: isMobile ? '0' : '24px',
        width: isMobile ? '100%' : '95%',
        maxWidth: '900px',
        height: isMobile ? '100%' : 'calc(100vh - 40px)',
        display: 'flex',
        flexDirection: 'column',
        boxShadow: '0 25px 50px -12px rgba(0, 0, 0, 0.25)',
        overflow: 'hidden',
        border: 'none'
    }}>
        {/* Header */}
        <div style={{
            padding: '32px 32px 24px',
            borderBottom: '1px solid var(--border-color)',
            display: 'flex',
            justifyContent: 'space-between',
            alignItems: 'flex-start',
            background: 'linear-gradient(to right, #ffffff, #f8fafc)'
        }}>
            <div style={{ display: 'flex', gap: '20px', alignItems: 'center' }}>
                <div style={{
                    width: '56px', height: '56px', borderRadius: '16px',
                    background: 'var(--primary-glow)', color: 'var(--primary)',
                    display: 'flex', alignItems: 'center', justifyContent: 'center',
                    fontSize: '1.5rem', boxShadow: '0 8px 16px rgba(37, 99, 235, 0.1)'
                }}>
                    <FaHistory />
                </div>
                <div>
                    <h2 style={{ margin: 0, fontSize: '1.75rem', fontWeight: 900, color: 'var(--text-main)', letterSpacing: '-0.5px' }}>
                        Account Ledger
                    </h2>
                    <div style={{ display: 'flex', alignItems: 'center', gap: '8px', marginTop: '4px' }}>
                        <span style={{ fontWeight: 800, color: 'var(--text-main)' }}>{entityName}</span>
                        <span style={{
                            fontSize: '0.7rem', textTransform: 'uppercase', fontWeight: 800, color: 'var(--primary)',
                            background: 'var(--primary-glow)', padding: '2px 8px', borderRadius: '6px'
                        }}>
                            {entityType}
                        </span>
                    </div>
                </div>
            </div>
            <button
                onClick={onClose}
                className="icon-btn-hover"
                style={{
                    background: 'white', border: '1px solid var(--border-color)', cursor: 'pointer',
                    padding: '12px', borderRadius: '14px', color: 'var(--text-muted)',
                    boxShadow: 'var(--shadow-sm)'
                }}
            >
                <FaTimes size={18} />
            </button>
        </div>

        {/* Content */}
        <div className="custom-scrollbar" style={{ flex: 1, overflow: 'auto', padding: '32px' }}>
            {loading ? (
                <div style={{ textAlign: 'center', padding: '100px 0' }}>
                    <div className="loading-spinner" style={{ margin: '0 auto 20px' }}></div>
                    <p style={{ color: 'var(--text-muted)', fontWeight: 600 }}>Deciphering transaction logs...</p>
                </div>
            ) : transactions.length === 0 ? (
                <div style={{ textAlign: 'center', padding: '100px 0', opacity: 0.5 }}>
                    <FaFileInvoiceDollar size={64} style={{ color: 'var(--text-light)', marginBottom: '20px' }} />
                    <p style={{ margin: 0, fontSize: '1.25rem', fontWeight: 800, color: 'var(--text-main)' }}>No Transactions Recorded</p>
                    <p style={{ margin: '8px 0 0', fontWeight: 500 }}>This account has zero financial activity.</p>
                </div>
            ) : (
                <>
                    {/* Summary Cards */}
                    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(200px, 1fr))', gap: '24px', marginBottom: '32p
                        <div className="card" style={{ padding: '20px', background: 'white', borderLeft: '4px solid var(--primary)' }}>
                            <div style={{ fontSize: '0.7rem', fontWeight: 800, color: 'var(--text-muted)', textTransform: 'uppercase', letterSpa
                            <div style={{ fontSize: '1.5rem', fontWeight: 900, color: 'var(--text-main)', marginTop: '4px' }}>{transactions.leng
                        </div>
                        <div className="card" style={{ padding: '20px', background: 'white', borderLeft: '4px solid ' + (netBalance > 0 ? 'var(
                            <div style={{ fontSize: '0.7rem', fontWeight: 800, color: 'var(--text-muted)', textTransform: 'uppercase', letterSpa
                            <div style={{ fontSize: '1.5rem', fontWeight: 900, color: netBalance > 0 ? 'var(--danger)' : 'var(--success)', marg
                                {fmt(netBalance)}
                            </div>
                        </div>
                    </div>

                    {/* Ledger Table */}
                    <div style={{ border: '1px solid var(--border-color)', borderRadius: '20px', overflow: 'hidden', boxShadow: 'var(--shadow-sm
                        <table style={{ width: '100%', borderCollapse: 'collapse', fontSize: '0.9rem', minWidth: '800px' }}>
                            <thead style={{ backgroundColor: '#f8fafc', borderBottom: '1px solid var(--border-color)' }}>
                                <tr>
                                    <th style={{ textAlign: 'left', padding: '16px 20px', color: 'var(--text-muted)', fontWeight: 700, fontSize
                                    <th style={{ textAlign: 'left', padding: '16px 20px', color: 'var(--text-muted)', fontWeight: 700, fontSize
                                    <th style={{ textAlign: 'right', padding: '16px 20px', color: 'var(--text-muted)', fontWeight: 700, fontSize
                                    <th style={{ textAlign: 'right', padding: '16px 20px', color: 'var(--text-muted)', fontWeight: 700, fontSize
                                    <th style={{ textAlign: 'right', padding: '16px 20px', color: 'var(--text-muted)', fontWeight: 700, fontSize
                                </tr>
                            </thead>
                            <tbody style={{ backgroundColor: 'white' }}>
                                {(() => {
                                    let runningBalance = 0;
                                    const sorted = [...transactions].sort((a, b) => new Date(a.date).getTime() - new Date(b.date).getTime());

                                    return sorted.map((t) => {
                                        const amt = parseFloat(t.amount.toString());
                                        let dr = 0; let cr = 0;

                                        if (entityType === 'customer') {
                                            if (t.type === 'INVOICE' || t.type === 'ADJUSTMENT') {
```

```
                                                    if (amt >= 0) { dr = amt; runningBalance += amt; }
                                                    else { cr = Math.abs(amt); runningBalance -= Math.abs(amt); }
                                                } else if (t.type === 'RECEIPT') { cr = amt; runningBalance -= amt; }
                                            } else {
                                                if (t.type === 'BILL') { cr = amt; runningBalance += amt; }
                                                else if (t.type === 'PAYMENT' || t.type === 'RECEIPT' || t.type === 'DR') { dr = amt; runningBalance
                                            }

                                            return (
                                                <tr key={t.id} className="row-hover" style={{ borderBottom: '1px solid var(--bg-body)', transition:
                                                    <td style={{ padding: '16px 20px' }}>
                                                        <div style={{ display: 'flex', alignItems: 'center', gap: '8px' }}>
                                                            <FaCalendarAlt size={12} style={{ color: 'var(--text-light)' }} />
                                                            <span style={{ fontWeight: 650, color: 'var(--text-main)' }}>{formatDate(t.date)}</span>
                                                        </div>
                                                    </td>
                                                    <td style={{ padding: '16px 20px' }}>
                                                        <div style={{ fontWeight: 800, color: 'var(--text-main)', fontSize: '0.95rem' }}>{t.descrip
                                                        <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', marginTop: '2px', fontWeight
                                                    </td>
                                                    <td style={{ padding: '16px 20px', textAlign: 'right', fontWeight: 700, color: dr > 0 ? 'var(--
                                                        {dr > 0 ? fmt(dr) : '-'}
                                                    </td>
                                                    <td style={{ padding: '16px 20px', textAlign: 'right', fontWeight: 700, color: cr > 0 ? 'var(--
                                                        {cr > 0 ? fmt(cr) : '-'}
                                                    </td>
                                                    <td style={{ padding: '16px 20px', textAlign: 'right', fontWeight: 900, color: 'var(--text-main
                                                        {fmt(runningBalance)}
                                                    </td>
                                                </tr>
                                            );
                                        });
                                    })()}
                                </tbody>
                            </table>
                        </div>
                    </>
                )}
            </div>

            {/* Footer */}
            <div style={{
                padding: '24px 32px',
                borderTop: '1px solid var(--border-color)',
                display: 'flex',
                justifyContent: 'flex-end',
                background: '#f8fafc'
            }}>
                <button onClick={onClose} className="btn-primary" style={{ padding: '12px 32px', fontSize: '1rem', fontWeight: 800 }}>
                    Done
                </button>
            </div>
        </div>

        <style>{`
            .row-hover:hover { background-color: #f8fafc; }
            .loading-spinner { width: 40px; height: 40px; border: 4px solid var(--primary-glow); border-top: 4px solid var(--primary); border-radius
            @keyframes spin { 0% { transform: rotate(0deg); } 100% { transform: rotate(360deg); } }
            @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
        `}</style>
    </div>
  );
};

export default TransactionHistoryModal;
```

frontend/src/components/OfflineIndicator.tsx

```
import React, { useEffect, useState } from 'react';

const OfflineIndicator: React.FC = () => {
    const [isOffline, setIsOffline] = useState(!navigator.onLine);

    useEffect(() => {
        const handleOnline = () => setIsOffline(false);
        const handleOffline = () => setIsOffline(true);

        window.addEventListener('online', handleOnline);
        window.addEventListener('offline', handleOffline);

        return () => {
            window.removeEventListener('online', handleOnline);
            window.removeEventListener('offline', handleOffline);
        };
    }, []);

    if (!isOffline) return null;

    return (
        <div className="offline-bar">
            📡 You are currently offline. Some features may be limited.
        </div>
    );
};

export default OfflineIndicator;
```

frontend/src/components/AddTransactionModal.tsx

```
// frontend/src/components/AddTransactionModal.tsx
import React, { useEffect, useState } from 'react';
import {
    FaExchangeAlt,
    FaSave,
    FaTimes
} from 'react-icons/fa';
import { fetchLedgers, Ledger } from '../api/ledgerApi';
import { createTransaction } from '../api/transactionApi';
import { Customer, fetchCustomers } from '../api/userApi';
// Assuming fetchSuppliers exist, if not I'll check.
// I'll check lenders/suppliers API.

const styles = {
    overlay: {
        position: 'fixed' as const, top: 0, left: 0, right: 0, bottom: 0,
        backgroundColor: 'rgba(0, 0, 0, 0.65)', backdropFilter: 'blur(8px)',
        display: 'flex', justifyContent: 'center', alignItems: 'center', zIndex: 1000
    },
    modal: {
        backgroundColor: '#ffffff', width: '100%', maxWidth: '550px',
        borderRadius: '16px', boxShadow: '0 25px 50px -12px rgba(0, 0, 0, 0.25)',
        overflow: 'hidden', display: 'flex', flexDirection: 'column' as const,
        maxHeight: '90vh'
    },
    header: {
        padding: '20px 24px', borderBottom: '1px solid #f1f5f9',
        display: 'flex', justifyContent: 'space-between', alignItems: 'center',
        backgroundColor: '#fff'
    },
    body: { padding: '24px', overflowY: 'auto' as const, flexGrow: 1 },
    footer: {
        padding: '16px 24px', borderTop: '1px solid #f1f5f9',
        backgroundColor: '#f8fafc', display: 'flex', justifyContent: 'flex-end',
        gap: '12px'
    },
    title: { fontSize: '1.25rem', fontWeight: 800, color: '#0f172a', display: 'flex', alignItems: 'center', gap: '12px' },
    inputGroup: { marginBottom: '20px' },
    label: { display: 'block', marginBottom: '8px', fontSize: '0.9rem', fontWeight: 600, color: '#475569' },
    input: {
        width: '100%', padding: '12px 16px', borderRadius: '10px',
        border: '1px solid #e2e8f0', fontSize: '1rem', outline: 'none',
        transition: 'all 0.2s', boxSizing: 'border-box' as const
    },
    select: {
        width: '100%', padding: '12px 16px', borderRadius: '10px',
        border: '1px solid #e2e8f0', fontSize: '1rem', backgroundColor: '#fff',
        outline: 'none', cursor: 'pointer'
    }
};

interface Props {
    onClose: () => void;
    onSuccess: () => void;
}

const AddTransactionModal: React.FC<Props> = ({ onClose, onSuccess }) => {
    const [ledgers, setLedgers] = useState<Ledger[]>([]);
    const [customers, setCustomers] = useState<Customer[]>([]);
    const [loading, setLoading] = useState(false);
    const [formData, setFormData] = useState({
        date: new Date().toISOString().slice(0, 10),
        amount: '',
        description: '',
        type: 'RECEIPT', // RECEIPT, PAYMENT
        category: 'General',
        ledger_id: '',
        user_id: '', // Linked Customer
    });

    useEffect(() => {
        const loadInitialData = async () => {
            try {
                const [lRes, cRes] = await Promise.all([
                    fetchLedgers(),
                    fetchCustomers()
                ]);
                setLedgers(lRes);
                setCustomers(cRes);

                // Pick default ledger if available
                if (lRes.length > 0) {
                    setFormData(prev => ({ ...prev, ledger_id: String(lRes[0].id) }));
                }
            } catch (err) {
                console.error("Failed to load modal data:", err);
            }
        };
        loadInitialData();
    }, []);

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        if (!formData.amount || !formData.ledger_id) {
            alert("Amount and Ledger are required.");
            return;
        }

        setLoading(true);
        try {
            await createTransaction({
                ...formData,
                amount: parseFloat(formData.amount),
                ledger_id: parseInt(formData.ledger_id),
                user_id: formData.user_id ? parseInt(formData.user_id) : null
            });
```

```
        onSuccess();
        onClose();
    } catch (err) {
        alert("Failed to save transaction.");
    } finally {
        setLoading(false);
    }
};

return (
    <div style={styles.overlay}>
        <div style={styles.modal} className="page-transition">
            <div style={styles.header}>
                <h2 style={styles.title}>
                    <div style={{ background: '#f0fdf4', padding: '8px', borderRadius: '8px', color: '#10b981', display: 'flex' }}>
                        <FaExchangeAlt />
                    </div>
                    New Entry
                </h2>
                <button onClick={onClose} style={{ background: 'none', border: 'none', cursor: 'pointer', color: '#94a3b8' }}>
                    <FaTimes size={20} />
                </button>
            </div>

            <form onSubmit={handleSubmit}>
                <div style={styles.body}>
                    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '16px' }}>
                        <div style={styles.inputGroup}>
                            <label style={styles.label}>Entry Type</label>
                            <select
                                style={styles.select}
                                value={formData.type}
                                onChange={e => setFormData({ ...formData, type: e.target.value })}
                            >
                                <option value="RECEIPT">Receipt (Inward)</option>
                                <option value="PAYMENT">Payment (Outward)</option>
                            </select>
                        </div>
                        <div style={styles.inputGroup}>
                            <label style={styles.label}>Date</label>
                            <input
                                type="date"
                                style={styles.input}
                                value={formData.date}
                                onChange={e => setFormData({ ...formData, date: e.target.value })}
                            />
                        </div>
                    </div>

                    <div style={styles.inputGroup}>
                        <label style={styles.label}>Account / Ledger (Bank/Cash)</label>
                        <select
                            style={styles.select}
                            value={formData.ledger_id}
                            onChange={e => setFormData({ ...formData, ledger_id: e.target.value })}
                            required
                        >
                            <option value="">Select Account</option>
                            {ledgers.map(l => (
                                <option key={l.id} value={l.id}>{l.name} ({l.group_name})</option>
                            ))}
                        </select>
                    </div>

                    <div style={styles.inputGroup}>
                        <label style={styles.label}>Amount (₹)</label>
                        <input
                            type="number"
                            step="0.01"
                            placeholder="0.00"
                            style={{ ...styles.input, fontWeight: 700, fontSize: '1.25rem', color: '#1e293b' }}
                            value={formData.amount}
                            onChange={e => setFormData({ ...formData, amount: e.target.value })}
                            required
                        />
                    </div>

                    <div style={styles.inputGroup}>
                        <label style={styles.label}>Linked Party (Optional)</label>
                        <select
                            style={styles.select}
                            value={formData.user_id}
                            onChange={e => setFormData({ ...formData, user_id: e.target.value })}
                        >
                            <option value="">None / Cash Transaction</option>
                            {customers.map(c => (
                                <option key={c.id} value={c.id}>{c.username}</option>
                            ))}
                        </select>
                    </div>

                    <div style={styles.inputGroup}>
                        <label style={styles.label}>Description / Reference</label>
                        <textarea
                            placeholder="e.g., Payment for Invoice #123"
                            style={{ ...styles.input, height: '80px', resize: 'none' }}
                            value={formData.description}
                            onChange={e => setFormData({ ...formData, description: e.target.value })}
                            required
                        />
                    </div>
                </div>

                <div style={styles.footer}>
                    <button type="button" onClick={onClose} className="btn-secondary" style={{ padding: '10px 20px', borderRadius: '10px' }}>
                        Cancel
                    </button>
                    <button
                        type="submit"
                        disabled={loading}
                        style={{
                            padding: '10px 24px',
```

```
                                borderRadius: '10px',
                                backgroundColor: '#10b981',
                                color: 'white',
                                border: 'none',
                                fontWeight: 700,
                                display: 'flex',
                                alignItems: 'center',
                                gap: '10px',
                                cursor: 'pointer',
                                opacity: loading ? 0.7 : 1
                            }}
                        >
                            <FaSave />
                            {loading ? 'Saving...' : 'Save Entry'}
                        </button>
                    </div>
                </form>
            </div>
        </div>
    );
};

export default AddTransactionModal;
```

frontend/src/components/Layout/Sidebar.tsx

```
import { AnimatePresence, motion } from 'framer-motion';
import React, { JSX, useEffect, useState } from 'react';
import {
    FaBox,
    FaBrain,
    FaBuilding,
    FaChartPie,
    FaChevronRight,
    FaCog,
    FaIndent,
    FaLandmark,
    FaOutdent,
    FaShoppingCart,
    FaSignOutAlt,
    FaTachometerAlt,
    FaTruck,
    FaUsers
} from "react-icons/fa";
import { NavLink, useLocation, useNavigate } from 'react-router-dom';
import { useAuthUser } from '../../hooks/useAuthUser';
import "./Sidebar.css";

// --- Types ---
interface MenuItem {
    name: string;
    path?: string;
    icon?: React.ReactNode;
    module?: string;
    permission?: string;
    subItems?: MenuItem[];
}

interface SidebarProps {
    isOpen: boolean; // Mobile toggle
    setIsOpen: (isOpen: boolean) => void;
    isMobile: boolean;
    mode: 'HOST' | 'ADMIN' | 'USER';
}

const getMenuItems = (mode: string, user: any): MenuItem[] => {
    if (mode === 'HOST') return [
        { name: "Platform Hub", path: "/platform-admin", icon: <FaTachometerAlt /> },
        { name: "Tenants", path: "/platform-admin/tenants", icon: <FaBuilding /> },
        { name: "Global Config", path: "/platform-admin/config", icon: <FaCog /> },
    ];
    if (mode === 'ADMIN') return [
        { name: "Admin Dashboard", path: "/dashboard", icon: <FaTachometerAlt /> },
        { name: "Employees", path: "/admin/employees", icon: <FaUsers /> },
        { name: "Branch Assets", path: "/admin/branches", icon: <FaBuilding /> },
        { name: "Analytics", path: "/admin/reports", icon: <FaChartPie /> },
    ];

    const enabledModules = user?.enabled_modules ? String(user.enabled_modules).toLowerCase() : "";
    // If no modules info available (e.g. token doesn't carry it yet), show everything
    const modulesKnown = enabledModules.length > 0;
    const hasModule = (modName: string) => !modulesKnown || enabledModules.includes(modName);

    const baseItems: MenuItem[] = [
        { name: "Command Center", path: "/dashboard", icon: <FaTachometerAlt /> }
    ];

    if (hasModule("sales")) {
        baseItems.push({
            name: "Revenue Stream",
            icon: <FaShoppingCart />,
            module: "sales",
            subItems: [
                { name: "Sales Orders", path: "/invoices" },
                { name: "Stakeholders", path: "/customers" }
            ]
        });
    }

    if (hasModule("procurement") || hasModule("inventory")) {
        baseItems.push({
            name: "Procurement",
            icon: <FaTruck />,
            module: "procurement",
            subItems: [
                { name: "Suppliers", path: "/suppliers" },
                { name: "Purchase Bills", path: "/purchase-bills" }
            ]
        });
    }

    if (hasModule("inventory")) {
        baseItems.push({ name: "Inventory Matrix", path: "/products", icon: <FaBox />, module: "inventory" });
    }

    if (hasModule("finance") || hasModule("accounting")) {
        baseItems.push({
            name: "Fiscal Logic",
            icon: <FaLandmark />,
            module: "finance",
            subItems: [
                { name: "Finance Hub", path: "/finance/dashboard" },
                { name: "Loan Portfolio", path: "/finance/loans" },
                { name: "Cash Receipts", path: "/finance/receipts" },
                { name: "Auto Reconcile", path: "/finance/reconciliation" },
                { name: "Financial Intel", path: "/finance/reports" },
                { name: "Global Ledgers", path: "/ledgers" },
                { name: "Automated Log", path: "/transactions" },
            ]
        });
    }
```

```
    if (hasModule("hr")) {
        baseItems.push({
            name: "Workforce",
            icon: <FaUsers />,
            module: "hr",
            subItems: [
                { name: "Staff Registry", path: "/employees" },
                { name: "Presence Portal", path: "/attendance" }
            ]
        });
    }

    if (hasModule("ai")) {
        baseItems.push({ name: "Cognitive AI", path: "/ai-insights", icon: <FaBrain />, module: "ai" });
    }

    if (user?.role === 'admin') {
        baseItems.push({ name: "Admin Setup", path: "/admin/branches", icon: <FaCog /> });
    }

    return baseItems;
};

const Sidebar: React.FC<SidebarProps> = ({ isOpen, setIsOpen, isMobile, mode }): JSX.Element => {
    const { user } = useAuthUser();
    const [isCollapsed, setIsCollapsed] = useState(false);

    const location = useLocation();
    const navigate = useNavigate();

    const MENU_ITEMS = getMenuItems(mode, user);

    const [openMenu, setOpenMenu] = useState<string | null>(null);

    // Auto-open menu on mount/location change
    useEffect(() => {
        let currentMenuName = null;
        for (const item of MENU_ITEMS) {
            if (item.subItems) {
                if (item.subItems.some(sub => location.pathname.startsWith(sub.path!))) {
                    currentMenuName = item.name;
                    break;
                }
            }
        }
        if (currentMenuName && openMenu !== currentMenuName) {
            setOpenMenu(currentMenuName);
        }
    }, [location.pathname, MENU_ITEMS]);

    const handleLogout = () => {
        if (window.confirm("Terminate secure session?")) {
            localStorage.removeItem('erp-token');
            navigate(mode === 'HOST' ? '/host-login' : '/company-login');
        }
    };

    return (
        <aside className={`sidebar-container ${isCollapsed ? 'sidebar-collapsed' : 'sidebar-expanded'} ${isOpen ? 'sidebar-open' : ''}`}>
            <div className="sidebar-header">
                <div className="logo-icon">
                    <motion.div initial={{ rotate: -10 }} animate={{ rotate: 0 }} transition={{ type: 'spring' }}>
                        {mode === 'HOST' ? 'Ω' : 'Σ'}
                    </motion.div>
                </div>
                {!isCollapsed && (
                    <motion.div initial={{ opacity: 0, x: -10 }} animate={{ opacity: 1, x: 0 }} className="logo-content">
                        <span className="logo-text">{mode === 'HOST' ? 'QUANTUM' : 'ENTERPRISE'}</span>
                        <span className="logo-sub">{mode === 'HOST' ? 'OS CORE' : 'SYNTHESIS ERP'}</span>
                    </motion.div>
                )}
            </div>

            <div className="sidebar-nav">
                <div className="nav-scroll">
                    {MENU_ITEMS.map((item) => {
                        const hasActiveSub = item.subItems?.some(sub => location.pathname.startsWith(sub.path!));
                        const isActive = item.path === location.pathname || hasActiveSub;
                        return (
                            <div key={item.name} className="menu-group">
                                <NavLink
                                    to={item.path || '#'}
                                    onClick={(e) => {
                                        if (item.subItems) {
                                            e.preventDefault();
                                            setOpenMenu(openMenu === item.name ? null : item.name);
                                        }
                                    }}
                                    className={typeof isActive === 'boolean' && isActive ? 'menu-item menu-item-active' : 'menu-item'}
                                >
                                    <span className="menu-icon">{item.icon}</span>
                                    {!isCollapsed && (
                                        <motion.span initial={{ opacity: 0 }} animate={{ opacity: 1 }} className="menu-item-text">
                                            {item.name}
                                        </motion.span>
                                    )}
                                    {!isCollapsed && item.subItems && (
                                        <FaChevronRight size={10} style={{ marginLeft: 'auto', transform: openMenu === item.name ? 'rotate(90deg)' : 'no
                                    )}
                                </NavLink>

                                <AnimatePresence>
                                    {!isCollapsed && item.subItems && openMenu === item.name && (
                                        <motion.div
                                            initial={{ height: 0, opacity: 0 }}
                                            animate={{ height: 'auto', opacity: 1 }}
                                            exit={{ height: 0, opacity: 0 }}
                                            transition={{ duration: 0.3, ease: [0.16, 1, 0.3, 1] }}
                                            className="sub-menu"
                                            style={{ overflow: 'hidden' }}
                                        >
                                            {item.subItems.map(sub => (
```

```
                                                <NavLink key={sub.name} to={sub.path!} className={({isActive: isSubActive}) => `sub-menu-item ${isSubAc
                                                    <div className="dot" />
                                                    {sub.name}
                                                </NavLink>
                                            ))}
                                        </motion.div>
                                    )}
                                </AnimatePresence>
                            </div>
                        );
                    })}
                </div>
            </div>

            <div className="sidebar-footer">
                <motion.button
                    whileHover={{ scale: 1.05 }}
                    whileTap={{ scale: 0.95 }}
                    onClick={() => setIsCollapsed(!isCollapsed)}
                    className="footer-btn action-toggle"
                >
                    {isCollapsed ? <FaOutdent /> : <FaIndent />}
                </motion.button>
                <motion.button
                    whileHover={{ scale: 1.05, backgroundColor: 'rgba(239, 68, 68, 0.1)', color: '#ef4444' }}
                    whileTap={{ scale: 0.95 }}
                    onClick={handleLogout}
                    className="footer-btn logout-btn"
                >
                    <FaSignOutAlt />
                    {!isCollapsed && <span className="btn-text">TERMINATE</span>}
                </motion.button>
            </div>
        </aside>
    );
};

export default Sidebar;
```

frontend/src/components/Layout/CustomerLayout.tsx

```
import React from 'react';
import { FaBox, FaFileInvoiceDollar, FaSignOutAlt, FaUserCircle } from 'react-icons/fa';
import { NavLink, Outlet, useNavigate } from 'react-router-dom';
import { useAuthUser } from '../../hooks/useAuthUser';

const CustomerLayout: React.FC = () => {
    const { user } = useAuthUser();
    const navigate = useNavigate();

    const handleLogout = () => {
        localStorage.removeItem('erp-token');
        window.location.href = '/login';
    };

    return (
        <div style={{ minHeight: '100vh', backgroundColor: '#f8fafc', fontFamily: "'Inter', sans-serif" }}>
            {/* Top Navigation Bar */}
            <header style={{
                background: 'white',
                height: '70px',
                borderBottom: '1px solid #e2e8f0',
                display: 'flex',
                alignItems: 'center',
                justifyContent: 'space-between',
                padding: '0 40px',
                position: 'sticky',
                top: 0,
                zIndex: 100,
                boxShadow: '0 2px 4px rgba(0,0,0,0.02)'
            }}>
                {/* Logo */}
                <div style={{ display: 'flex', alignItems: 'center', gap: '10px' }}>
                    <div style={{ width: '32px', height: '32px', background: '#2563eb', borderRadius: '8px' }}></div>
                    <h2 style={{ fontSize: '1.2rem', fontWeight: 700, color: '#1e293b', margin: 0 }}>
                        My Portal
                    </h2>
                </div>

                {/* Navigation Links */}
                <nav style={{ display: 'flex', gap: '30px' }}>
                    <NavLink to="/shop" style={({ isActive }) => ({
                        textDecoration: 'none',
                        color: isActive ? '#2563eb' : '#64748b',
                        fontWeight: isActive ? 600 : 500,
                        display: 'flex', alignItems: 'center', gap: '8px'
                    })}>
                        <FaBox /> Products
                    </NavLink>
                    <NavLink to="/my-ledger" style={({ isActive }) => ({
                        textDecoration: 'none',
                        color: isActive ? '#2563eb' : '#64748b',
                        fontWeight: isActive ? 600 : 500,
                        display: 'flex', alignItems: 'center', gap: '8px'
                    })}>
                        <FaFileInvoiceDollar /> My Ledger & Orders
                    </NavLink>
                </nav>

                {/* User Profile */}
                <div style={{ display: 'flex', alignItems: 'center', gap: '20px' }}>
                    <div style={{ textAlign: 'right' }}>
                        <div style={{ fontWeight: 600, fontSize: '0.9rem', color: '#1e293b' }}>{user?.username}</div>
                        <div style={{ fontSize: '0.75rem', color: '#94a3b8' }}>Valued Customer</div>
                    </div>
                    <FaUserCircle size={36} color="#cbd5e1" />
                    <button
                        onClick={handleLogout}
                        style={{
                            background: '#fee2e2', color: '#ef4444', border: 'none',
                            padding: '8px 12px', borderRadius: '6px', cursor: 'pointer',
                            display: 'flex', alignItems: 'center', gap: '6px', fontWeight: 600, fontSize: '0.85rem'
                        }}
                    >
                        <FaSignOutAlt /> Logout
                    </button>
                </div>
            </header>

            {/* Main Content Area */}
            <main style={{ maxWidth: '1200px', margin: '0 auto', padding: '40px 20px' }}>
                <Outlet />
            </main>
        </div>
    );
};

export default CustomerLayout;
```

frontend/src/components/Layout/AppLayout.tsx

frontend/src/components/Layout/Forms/AddCustomerModal.tsx

```
import React, { useEffect, useState } from 'react';
import { FaBuilding, FaEnvelope, FaIdCard, FaLock, FaMapMarkerAlt, FaMoneyCheckAlt, FaPhone, FaSave, FaTag, FaTimes, FaUser } from 'react-icons/fa';
import { createCustomer, updateCustomer } from '../../../api/userApi';

interface Props {
    onClose: () => void;
    onSuccess: () => void;
    customerToEdit?: any;
}

const styles = {
    overlay: {
        position: 'fixed' as const, top: 0, left: 0, right: 0, bottom: 0,
        backgroundColor: 'rgba(0, 0, 0, 0.65)', backdropFilter: 'blur(4px)',
        display: 'flex', justifyContent: 'center', alignItems: 'center', zIndex: 1000
    },
    modal: {
        backgroundColor: '#ffffff', width: '100%', maxWidth: '600px',
        borderRadius: '12px', boxShadow: '0 25px 50px -12px rgba(0, 0, 0, 0.25)',
        overflow: 'hidden', display: 'flex', flexDirection: 'column' as const,
        maxHeight: '90vh', fontFamily: 'Inter, sans-serif'
    },
    header: {
        padding: '16px 24px', borderBottom: '1px solid #e5e7eb',
        display: 'flex', justifyContent: 'space-between', alignItems: 'center',
        backgroundColor: '#f8fafc', flexShrink: 0
    },
    body: {
        padding: '24px',
        overflowY: 'auto' as const, // Enable vertical scrolling
        flexGrow: 1, // Allow body to take available space
        maxHeight: 'calc(90vh - 140px)' // Ensure space for header/footer
    },
    footer: {
        padding: '16px 24px', borderTop: '1px solid #e5e7eb',
        backgroundColor: '#f8fafc', display: 'flex', justifyContent: 'flex-end',
        gap: '12px', flexShrink: 0
    },
    // ... other styles same as before
    title: { fontSize: '1.15rem', fontWeight: 700, color: '#1e293b', display: 'flex', alignItems: 'center', gap: '10px' },
    closeBtn: { background: 'none', border: 'none', cursor: 'pointer', color: '#94a3b8', fontSize: '1.2rem', display: 'flex', alignItems: 'center' },
    sectionTitle: { fontSize: '0.75rem', fontWeight: 700, color: '#64748b', textTransform: 'uppercase' as const, letterSpacing: '0.05em', marginBottom:
    inputGroup: { marginBottom: '16px' },
    label: { display: 'block', marginBottom: '6px', fontSize: '0.85rem', fontWeight: 600, color: '#475569' },
    inputWrapper: { position: 'relative' as const },
    icon: { position: 'absolute' as const, left: '12px', top: '50%', transform: 'translateY(-50%)', color: '#9ca3af', pointerEvents: 'none' as const, fo
    input: { width: '100%', padding: '10px 12px 10px 38px', borderRadius: '8px', border: '1px solid #cbd5e1', fontSize: '0.9rem', color: '#1e293b', back
    row: { display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '16px' },
    btnSecondary: { padding: '10px 20px', borderRadius: '8px', border: '1px solid #cbd5e1', backgroundColor: '#fff', color: '#475569', fontSize: '0.9rem
    btnPrimary: { padding: '10px 24px', borderRadius: '8px', border: 'none', backgroundColor: '#2563eb', color: '#fff', fontSize: '0.9rem', fontWeight:
};

const InputField = ({ label, icon: Icon, value, onChange, placeholder, required = false, type = "text", disabled = false }: any) => (
    <div style={styles.inputGroup}>
        <label style={styles.label}>{label} {required && <span style={{color: '#ef4444'}}>*</span>}</label>
        <div style={styles.inputWrapper}>
            <div style={styles.icon}><Icon /></div>
            <input type={type} required={required} disabled={disabled} style={{...styles.input, opacity: disabled ? 0.7 : 1, backgroundColor: disabled
        </div>
    </div>
);

const AddCustomerModal: React.FC<Props> = ({ onClose, onSuccess, customerToEdit }) => {
    const [formData, setFormData] = useState({
        username: '', nickname: '', email: '', phone: '', gstin: '',
        address_line1: '', city_pincode: '', state: '', state_code: '',
        bank_name: '', bank_account_no: '', bank_ifsc_code: '',
        opening_balance: 0,
        password: ''
    });
    const [loading, setLoading] = useState(false);

    useEffect(() => {
        if (customerToEdit) {
            setFormData({
                username: customerToEdit.username || '', nickname: customerToEdit.nickname || '',
                email: customerToEdit.email || '', phone: customerToEdit.phone || '',
                gstin: customerToEdit.gstin || '', address_line1: customerToEdit.address_line1 || '',
                city_pincode: customerToEdit.city_pincode || '', state: customerToEdit.state || '',
                state_code: customerToEdit.state_code || '',
                bank_name: customerToEdit.bank_name || '',
                bank_account_no: customerToEdit.bank_account_no || '',
                bank_ifsc_code: customerToEdit.bank_ifsc_code || '',
                opening_balance: customerToEdit.initial_balance || 0,
                password: ''
            });
        }
    }, [customerToEdit]);

    const handlePhoneChange = (e: any) => {
        const val = e.target.value;
        setFormData(prev => ({
            ...prev,
            phone: val,
            password: prev.password ? prev.password : val
        }));
    };

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setLoading(true);
        try {
            if (customerToEdit) {
                await updateCustomer(customerToEdit.id, formData);
                alert("Success! Customer profile updated.");
            } else {
```

```
                await createCustomer(formData);
                alert(`Success! Customer created.\n\nPortal Login Details:\nUsername: ${formData.username}\nPassword: ${formData.password || formData.ph
            }
            onSuccess();
            onClose();
        } catch (err: any) {
            alert(err.message || "Operation failed");
        } finally {
            setLoading(false);
        }
    };

    return (
        <div style={styles.overlay}>
            <div style={styles.modal}>
                <div style={styles.header}>
                    <h2 style={styles.title}>
                        <div style={{background:'#eff6ff', padding:'8px', borderRadius:'6px', color:'#2563eb', display:'flex'}}><FaUser /></div>
                        {customerToEdit ? 'Edit Customer' : 'Add New Customer'}
                    </h2>
                    <button onClick={onClose} style={styles.closeBtn}><FaTimes /></button>
                </div>

                <div style={styles.body}>
                    <form id="customer-form" onSubmit={handleSubmit}>

                        {/* 1. BUSINESS NAME */}
                        <div style={{...styles.sectionTitle, marginTop: 0}}>Business Details</div>
                        <InputField label="Customer / Business Name" icon={FaBuilding} value={formData.username} onChange={(e: any) => setFormData({...f

                        {/* 2. PORTAL LOGIN (Moved to Top) */}
                        <div style={{background: '#f0fdf4', padding: '15px', borderRadius: '8px', border: '1px solid #bbf7d0', marginBottom: '20px', ma
                            <div style={{fontSize: '0.85rem', fontWeight: 700, color: '#166534', marginBottom: '10px', display: 'flex', alignItems: 'cen
                                <FaLock /> Customer Portal Access
                            </div>
                            <div style={styles.row}>
                                <InputField label="Login Username" icon={FaUser} value={formData.username} disabled={true} placeholder="Auto-filled" />
                                <InputField
                                    label="Set/Reset Password"
                                    icon={FaLock}
                                    type="text"
                                    value={formData.password}
                                    onChange={(e: any) => setFormData({...formData, password: e.target.value})}
                                    placeholder={formData.phone || "Enter password"}
                                />
                            </div>
                            <p style={{fontSize: '0.75rem', color: '#166534', margin: 0, marginTop: '5px'}}>
                                * The customer can login using these details to view orders and ledger.
                            </p>
                        </div>

                        {/* 3. OTHER DETAILS */}
                        <div style={styles.row}>
                            <InputField label="Nickname / Alias" icon={FaTag} value={formData.nickname} onChange={(e: any) => setFormData({...formData,
                            <InputField label="Phone (Default Password)" icon={FaPhone} value={formData.phone} onChange={handlePhoneChange} placeholder=
                        </div>
                        <div style={styles.row}>
                            <InputField label="GSTIN" icon={FaIdCard} value={formData.gstin} onChange={(e: any) => setFormData({...formData, gstin: e.ta
                            <InputField label="Email" icon={FaEnvelope} value={formData.email} onChange={(e: any) => setFormData({...formData, email: e
                        </div>

                        {/* 4. BANK */}
                        <div style={styles.sectionTitle}>Bank Account Details</div>
                        <InputField label="Bank Name" icon={FaMoneyCheckAlt} value={formData.bank_name} onChange={(e: any) => setFormData({...formData,
                        <div style={styles.row}>
                            <InputField label="Account Number" icon={FaTag} value={formData.bank_account_no} onChange={(e: any) => setFormData({...formD
                            <InputField label="IFSC Code" icon={FaTag} value={formData.bank_ifsc_code} onChange={(e: any) => setFormData({...formData, b
                        </div>

                        {/* 5. LOCATION */}
                        <div style={styles.sectionTitle}>Location</div>
                        <InputField label="Address Line 1" icon={FaMapMarkerAlt} value={formData.address_line1} onChange={(e: any) => setFormData({...fo
                        <div style={styles.row}>
                            <InputField label="City - Pincode" icon={FaMapMarkerAlt} value={formData.city_pincode} onChange={(e: any) => setFormData({.
                            <InputField label="State" icon={FaMapMarkerAlt} value={formData.state} onChange={(e: any) => setFormData({...formData, state
                        </div>

                        {/* 6. OPENING BALANCE */}
                        <div style={{marginTop: '10px', background: '#f0f9ff', padding: '15px', borderRadius: '8px', border: '1px solid #bae6fd'}}>
                            <label style={{...styles.label, color: '#0369a1', marginBottom: '8px'}}>Opening Balance (Receivable)</label>
                            <div style={{display: 'flex', alignItems: 'center'}}>
                                <span style={{fontSize: '1.2rem', color: '#0369a1', fontWeight: 'bold', marginRight: '10px'}}>₹</span>
                                <input type="number" disabled={!!customerToEdit} style={{...styles.input, paddingLeft: '12px', textAlign: 'right', fontW
                            </div>
                        </div>
                    </form>
                </div>

                <div style={styles.footer}>
                    <button onClick={onClose} style={styles.btnSecondary} type="button">Cancel</button>
                    <button form="customer-form" type="submit" disabled={loading} style={{...styles.btnPrimary, opacity: loading ? 0.7 : 1}}>
                        {loading ? 'Saving...' : <><FaSave /> {customerToEdit ? 'Update Customer' : 'Save Customer'}</>}
                    </button>
                </div>
            </div>
        </div>
    );
};

export default AddCustomerModal;
```